

Programação Centrada em Objetos

Isabel Nunes, 2019

(revisto e aumentado em Julho 2021)

FCUL

ÍNDICE

1. Introdução.....	2
2. Escrever programas.....	2
2.1. Linguagens de programação.....	2
2.2. Programas Java.....	3
2.3. Exemplos usados neste livro.....	5
2.4. Erros.....	6
2.5. O que vai precisar.....	6
3. Representação e manipulação de dados.....	7
3.1. Expressões.....	7
3.2. Literais. Memória. Variáveis. Instrução de atribuição.....	8
3.3. Tipos de dados primitivos e (um cheirinho de) <code>String</code>	11
3.4. Revisitando variáveis. Instrução de atribuição.....	13
3.5. Ainda as expressões.....	15
3.6. Comentários, indentação e outros promotores de legibilidade.....	19
4. Abstração procedimental.....	21
4.1. Alguns métodos já nossos conhecidos.....	23
4.2. Procedimentos.....	24
4.3. Funções.....	29
4.4. Memória e âmbito das variáveis.....	31
4.5. Documentação.....	33
4.6. Repositórios de métodos úteis.....	34
5. O tipo de dados <code>boolean</code> e comandos condicionais.....	35
5.1. O tipo primitivo <code>boolean</code>	35
5.2. Execução condicional.....	38
5.3. Ainda a documentação.....	49
5.4. Comparação de valores numéricos reais.....	50
6. Ciclos.....	52
6.1. Atalhos para operações comuns.....	52
6.2. A instrução <code>for</code>	53
6.3. Ciclos <code>for</code> encaixados.....	64
6.4. Ainda a abstração procedimental.....	66
6.5. Ainda o âmbito das variáveis.....	73
6.6. A instrução <code>while</code>	75
6.7. A instrução <code>do-while</code>	79
7. Abstração de dados.....	80
7.1. Tipos de dados não primitivos.....	80
7.2. Revisitando o tipo <code>String</code>	81
8. Algumas classes da biblioteca Java.....	87
8.1. Pacotes.....	88
8.2. Geração de valores aleatórios.....	88
8.3. Construindo uma <code>String</code> passo a passo.....	92

8.4. A classe <code>StringBuilder</code>	93
8.5. <i>Input</i> e <i>output</i> em Java.....	94
8.6. <i>Aliasing</i>	99
8.7. <code>null</code>	100
8.8. <i>Garbage collection</i>	100
8.9. Ficheiros de texto – leitura e escrita.....	101
8.10. Exceções.....	103
9. Arrays.....	112
9.1. O tipo de dados não primitivo <i>array</i>	114
9.2. Acesso aos elementos de um <i>array</i>	116
9.3. Número de elementos de um <i>array</i>	118
9.4. Copiar <i>arrays</i>	118
9.5. Funções cujo resultado é um <i>array</i>	119
9.6. Métodos com parâmetros do tipo <i>array</i>	122
10. Criar novos tipos de dados.....	126
10.1. O tipo de dados <code>Jogador</code>	127
10.2. Atributos de instância.....	131
10.3. Construtores.....	132
10.4. Métodos que revelam o estado.....	134
10.5. Métodos que alteram o estado.....	136
10.6. Mais construtores.....	138
10.7. Imprimir, duplicar e comparar objetos - como NÃO fazer.....	139
10.8. Agora sim - Imprimir, duplicar e comparar objetos.....	141
10.9. A classe <code>Jogador</code> completa.....	144
11. Um novo tipo de dados, cliente de <code>Jogador</code>.....	145
11.1. A classe <code>Jogo</code>	148
11.2. Registrar jogadores no jogo.....	150
11.3. Registrar pontuação - 1ª versão.....	155
11.4. Outros métodos.....	164
11.5. Atributos de classe (<code>static</code>), constantes e <code>public</code>	173
11.6. O método <code>toString</code> na classe <code>Jogo</code>	177
11.7. Fim de jogo, vencedores, etc.....	178
11.8. A classe <code>Jogo</code> completa.....	189
11.9. Representação em UML.....	194
12. Tipos enumerados.....	195
13. Interfaces.....	200
13.1. Definição e implementação.....	200
13.2. Representação em UML.....	202
13.3. Subtipos.....	202
13.4. O interface <code>Ganhavel</code>	205
13.5. O interface genérico <code>List</code>	209
14. Herança.....	212
14.1. Uma subclasse de <code>Jogo</code>	220
14.2. A relação subclasse/superclasse.....	221

14.3. Subtipos.....	223
14.4. Herança e construtores.....	224
14.5. Invocação de métodos herdados.....	226
14.6. Redefinição de métodos.....	227
14.7. Outra subclasse de <code>Jogo</code>	230
14.8. Ainda a redefinição de métodos.....	233
14.9. Redefinição versus atributos.....	235
15. Polimorfismo e ligação dinâmica.....	236
15.1. Polimorfismo.....	237
15.2. Verificação estática de tipos.....	241
15.3. Ligação dinâmica.....	242
15.4. Invocação indireta de métodos redefinidos.....	244
15.5. Herança de contratos versus redefinição.....	245
15.6. Tipos genéricos e subtipos	247
15.7. Métodos genéricos	256
16. Classes abstratas.....	259
16.1. A classe abstrata <code>Jogo</code>	261
16.2. Métodos <i>template</i>	262

1. Introdução

Este não é um texto “para aprender Java”. Este é um texto para aprender a resolver problemas de forma computacional. Aplicando um raciocínio algorítmico/computacional, vamos encontrar soluções que possam ser usadas por um computador. Esta tarefa exige capacidades variadas como, por exemplo:

- saber dividir um problema complexo em sub-problemas mais simples;
- saber juntar componentes com utilidades variadas para formar sistemas que cumprem objetivos de maior ou menor complexidade;
- avaliar a qualidade de soluções alternativas.

E tudo isto usando linguagens com expressividade limitada, definidas através de regras estritas.

As soluções encontradas para os problemas vão ser expressas através de **programas** ou seja, textos escritos nalguma **linguagem de programação** (Java no nosso caso) que, quando executados num computador, permitem obter os resultados desejados.

Capacidades (*skills*) a adquirir com este livro:

- saber usar ferramentas básicas de raciocínio computacional para escrever pequenos programas;
- ter capacidade de compreender programas Java escritos por outros;
- mapear a descrição de um problema em algo computável;
- reutilizar código Java feito por outros.

2. Escrever programas

O paradigma de programação que vamos abordar é o da **Programação Orientada a Objetos**, em que um programa é uma sequência de instruções que comandam a forma do computador atuar, estruturadas através de várias componentes chamadas **classes**.

2.1. Linguagens de programação

Uma linguagem de programação define o vocabulário e as regras de construção de frases que nos permitem escrever instruções para o computador executar.

Embora as ações que nós, como utilizadores de um computador, pedimos ao computador para executar nos pareçam altamente complexas, na verdade os computadores só estão preparados para receber instruções extremamente simples. Essas instruções são em número reduzido e mandam o computador fazer tarefas muito, muito básicas – constituem aquilo a que se chama uma *linguagem de programação de baixo nível*, também chamada *linguagem máquina* ou *linguagem assembly*. Cada comando que damos ao computador (por exemplo, mover um ficheiro entre pastas, imprimir um texto) é na verdade um programa contendo muitas dessas instruções de baixo nível. Esse programa foi feito por alguém.

Ao longo do tempo foram-se inventando linguagens de programação de mais alto nível, ou seja, mais próximas da língua que falamos, para tornar mais fácil a tarefa de programar.

Porque as instruções escritas numa linguagem de alto nível não são entendidas por um computador, elas têm que ser traduzidas para instruções de baixo nível. Esse é o papel dos *compiladores*, que (também eles) são programas que pegam num ficheiro contendo o texto de um programa numa linguagem de alto nível e geram um ficheiro com as instruções correspondentes numa linguagem de baixo nível. São estas últimas que o computador executa. Claro que cada linguagem tem o seu próprio compilador.

Os *interpretadores* também são programas que fazem a tradução de programas para instruções de baixo nível, mas fazem-no á medida que vão executando o programa, instrução a instrução.

O compilador de programas Java traduz um programa escrito na linguagem Java (guardado num ficheiro com extensão `.java`) para um programa escrito numa linguagem de baixo-nível a que se chama Java *bytecode*; essas instruções são guardadas num novo ficheiro com a extensão `.class`.

Este *bytecode* é independente do sistema operativo específico onde o programa vai ser executado – qualquer computador capaz de executar programas Java será capaz de interpretar/traduzir este ficheiro e executá-lo.

Para que um computador específico consiga executar o programa em *bytecode*, tem que ter instalada uma Java Virtual Machine (JVM), que é um programa que consegue ler e executar esse *bytecode*.



2.2. Programas Java

No paradigma de programação que este livro aborda, Programação Orientada a Objetos (POO),

um *programa* é composto por uma ou mais *classes*, cada uma definindo um ou mais *métodos* que por sua vez contêm *instruções*.

A *classe* é a unidade de estruturação dos programas OO,

- cada programa, por mais simples que seja, tem pelo menos uma classe
- o texto que define uma classe é guardado num ficheiro com extensão `.java`
- um programa Java é composto por um ou mais ficheiros com extensão `.java`.

O processo de *criar e executar* um programa Java envolve a seguinte sequência de ações:

1. *escrever* o programa usando um editor de texto e guardar o texto do programa num ficheiro com extensão `.java`
2. *compilar* o programa
3. *executar* o programa.

2.2.1. Escrever o programa

Ao longo deste livro veremos, passo a passo, como abordar computacionalmente os problemas que queremos resolver e a forma de escrever em Java os programas que os resolvem.

Para já, ficamos com este primeiro programa Java, cujo texto escrevemos com a ajuda de um editor de texto e guardámos num ficheiro de nome `MyFirstProgram.java`.

```
public class MyFirstProgram {

    public static void main (String [] args) {

        System.out.println("Olah turma!!");

    }

}
```

Durante o curso, a pouco e pouco, irá aprendendo qual o papel que cada palavra deste programa tem. E não se preocupe agora em memorizar nada – este é só um exemplo inicial.

2.2.2. Compilar o programa

Para compilar o programa vai precisar do *Java SE Development Kit (JDK)* – pode descarregá-lo a partir do url <http://www.oracle.com/technetwork/java/javase/downloads/index.html>

Certifique-se de que escolhe o *JDK* e não o *JRE*.

Já com o *JDK* instalado no seu computador, escrever na linha de comando:

```
javac MyFirstProgram.java
```

`javac` é o nome do comando, no sistema operativo, que executa o compilador de Java; segue-se o nome do ficheiro que queremos compilar (incluindo a extensão `.java`).

Se o diretório corrente não for o que contém o ficheiro `MyFirstProgram.java` então (a) deve adicionar o *path*, ou caminho, necessário para identificar o ficheiro ou (b) deve mudar o diretório corrente para que seja o diretório que contém o ficheiro.

Como este programa não tem erros, um ficheiro com o mesmo nome e extensão `.class` será criado. Se o compilador detetasse algum erro, esse ficheiro não seria criado e seriam emitidas mensagens para o ecrã indicando os erros detetados.

Se o sistema operativo responder com a mensagem *'javac' is not recognized as an internal or external command, operable program or batch file*, isso significa que não consegue encontrar o compilador. Para resolver o problema deve atualizar a variável *PATH* do seu sistema operativo. Para resolver este ou outros problemas, ver <https://docs.oracle.com/javase/tutorial/getStarted/problems/index.html>

2.2.3. Executar o programa

Se o compilador não tiver detetado erros, escrever na linha de comando:

```
java MyFirstProgram
```

`java` é o nome do comando, no sistema operativo, que executa a JVM; o nome do ficheiro que queremos executar não inclui a extensão `.class`.

O programa será executado, ou seja, as instruções contidas no método `main` da classe `MyFirstProgram` serão executadas por ordem e até ao fim. Neste caso é uma única instrução, cujo efeito é o de escrever no *standard output* (normalmente o ecrã) a frase `Olah turma!!`

Um programa pode ser constituído por uma ou mais classes.

Para que possa ser executado, tem que existir numa dessas classes um método de nome `main`.

É o nome da classe que contém o `main` que é indicado no comando `java`.

No nosso exemplo, as linhas que definem esse método `main` são:

```
public static void main (String [] args) {  
  
    System.out.println("Olah turma!!");  
  
}
```

Uma vez compilado, um programa pode ser executado quantas vezes se quiser sem ter que se compilar de novo.

2.2.4. Instruções

As *instruções* são diferentes em diferentes linguagens de programação, mas há um pequeno conjunto de operações básicas comum à maior parte das linguagens de programação:

- *Cálculo* – executar cálculos matemáticos básicos;
- *Teste* – verificar se determinadas condições são verdadeiras e executar instruções apropriadas conforme o resultado do teste;
- *Repetição* – executar ações repetidamente, com eventuais variações a cada repetição;
- *Input* – “ler” informação a partir do teclado, de um ficheiro ou de outro dispositivo, para poder ser usada pelo programa;
- *Output* – “escrever” informação no ecrã, num ficheiro ou noutro dispositivo.

Ao longo deste livro vamos aprender como instruir o computador para fazer estas operações.

2.3. Exemplos usados neste livro

Há muitas formas de um programa transmitir ao seu utilizador – aquele que manda executar o programa – o efeito da sua execução: escrita de valores no ecrã, comandos dirigidos a máquinas variadas como, por exemplo, a impressão de textos numa impressora ou a alimentação intravenosa dada a um doente de forma automática.

Para nós, que vamos começar por fazer programas muito simples, uma boa maneira de vermos o efeito que certas instruções do nosso programa provocam, é fazendo o programa mostrar valores relevantes no ecrã.

Para isso vamos utilizar instruções semelhantes à instrução `System.out.println("Olah turma!!");` do método `main` da classe `MyFirstProgram`:

Veremos que basta variar o texto entre parentesis para conseguirmos escrever informação variada no ecrã. Para já é tudo o que precisa de saber.

Ao longo destas páginas ser-lhe-ão apresentados inúmeros exemplos de programas.

Na versão online deste livro, alguns dos programas poderão ser "executados ao vivo": para isso, foram embebidas nessas páginas código gerado pela aplicação *Python Tutor* (<http://pythontutor.com/java.html>), na sua versão para programas Java. Por vezes o servidor usado pelo Python Tutor está sobrecarregado, provocando lentidão no carregamento dos blocos *html* correspondentes.

Experimente agora "executar ao vivo" o programa da classe `MyFirstProgram` na secção correspondente da versão online deste livro.

2.4. Erros

Programar implica errar! Por mais experiente que um programador seja, há sempre um erro “à espreita”, que aproveita a mais pequena distração para se instalar no programa.

Há erros de vários tipos e mais ou menos difíceis de detetar. Encontrar erros pode ser um desafio altamente interessante.

2.4.1. Erros de compilação (sintáticos)

A sintaxe de uma linguagem define as regras de construção de frases dessa linguagem. Um erro de sintaxe numa frase em língua natural pode não ser grave – as pessoas que lêem ou ouvem essa frase conseguem entendê-la, pois fazem como que uma correção automática no seu cérebro. Claro que isto pode trazer alguma ambiguidade, pois essa correção pode não ser no sentido exato da frase original.

Quando o “interlocutor” é um computador e a linguagem é uma linguagem de programação, as regras têm que ser muito mais restritas e completas pois o computador não pode ter dúvidas acerca de como traduzir as instruções do programa.

O compilador verifica todas as instruções do programa e só as traduz se estiverem corretas do ponto de vista sintático e coerentes do ponto de vista das componentes que o formam.

Exemplos de erros de sintaxe:

- Falta de algum `{`, `(` ou `;`
- Palavra desconhecida onde devia estar uma palavra reservada. Exemplo: `public clast MyFirstProgram {`
- Ficheiro onde é guardado o texto de uma classe ter um nome diferente. Exemplo: classe de nome `MyProgram` guardada num ficheiro chamado `YourChocolate.java`

2.4.2. Erros semânticos

Há erros que o compilador não consegue detetar pois são erros que não têm a ver com a sintaxe mas com a semântica do programa, ou seja, com o significado das instruções e do programa (uma exceção a esta regra são os erros de tipos, como veremos no próximo capítulo, que, apesar de serem erros semânticos, são detetados pelo compilador).

Uma instrução pode estar correta do ponto de vista sintático ou dos tipos, mas o seu significado ser errado do ponto de vista do programa onde está incluída. No entanto, o compilador gera o ficheiro de *bytecode* e o programa pode ser executado.

Erros de execução. Alguns destes erros semânticos provocam situações em que o computador não consegue continuar a executar o programa. Como exemplo temos uma divisão por zero. Pode ser complicado detetar estes erros porque podem aparecer só em algumas execuções do programa, quando estão envolvidos valores específicos que os provocam.

Erros lógicos. Este tipo de erros abrange as situações em que as instruções do programa não fazem exatamente o pretendido, ou seja, implementam de forma errada aquilo que foi especificado. Exemplo: o programa aplica mal as regras de cálculo de um imposto. Estes erros são os mais difíceis de detetar pois o programa é executado sem problemas do início ao fim; é aquilo que ele faz que está errado relativamente ao que foi especificado.

2.5. O que vai precisar

No início não vai precisar de nenhuma ferramenta de desenvolvimento (IDE – *Integrated Development Environment*) especial. Um IDE poderia “esconder” alguns detalhes bastante importantes para uma boa compreensão das fases de compilação e execução, neste momento inicial da sua aprendizagem.

Recomendamos vivamente o uso de um editor de texto básico (p.ex., *Notepad++* para *MS Windows*, *jEdit* ou *TextWrangler* para *MacOS*, *Atom* ou *gedit* para *Linux*) até completar as matérias (da primeira parte) deste livro.

Precisará de instalar o *Java SE Development Kit (JDK)* que pode obter, como já referido atrás, no url <http://www.oracle.com/technetwork/java/javase/downloads/index.html>

3. Representação e manipulação de dados

Os programas manipulam informação (ou dados) de variados tipos: nomes de pessoas e de coisas, textos de artigos e livros, valores numéricos inteiros representando idades, classificações nas disciplinas de um curso, valores numéricos com parte decimal representando alturas, salários, distâncias, etc.

As operações que se podem fazer sobre esses valores também variam: os textos podem ser cortados, acrescentados, etc; os valores numéricos podem ser multiplicados, somados, etc.

Chama-se *tipo de dados* a

- um conjunto de valores e
- as operações que se podem fazer sobre esses valores.

O Java fornece vários tipos de dados base – os *tipos de dados primitivos* – com os quais é possível representar todos os valores de interesse.

O Java permite ainda, como veremos mais adiante neste livro, a criação de novos tipos de dados (*não primitivos*) pelo programador. Tem também uma extensa biblioteca de tipos de dados não primitivos – classes – que qualquer programador pode usar para simplificar a sua tarefa.

3.1. Expressões

Num programa podemos escrever *expressões* juntando valores de um ou mais tipos de dados e operações sobre esses valores.

É óbvio que a articulação de valores e operações numa dada expressão tem que ser coerente, tanto do ponto de vista sintático como do dos tipos. Não faz sentido, por exemplo, multiplicar o nome de uma pessoa pela distância da Terra à Lua.

Qualquer expressão pode ser avaliada,

- qualquer expressão representa um *valor*;
- qualquer expressão tem um *tipo*, que é o tipo do valor resultante da sua avaliação.

Por exemplo, uma expressão que adicione dois valores de tipo inteiro é também ela do tipo inteiro e o seu valor é o resultado da soma dos dois números.

Para vermos que o computador consegue fazer somas podemos fazer o seguinte programa que calcula a soma de 245 e 455 e imprime o resultado no ecrã:

```
public class Expressions1 {  
  
    public static void main (String [] args) {  
  
        System.out.println(245 + 455);  
  
    }  
}
```

Se executarmos este programa, o computador calculará o valor da expressão $245 + 455$ e depois imprimirá esse valor no ecrã. O utilizador verá no ecrã o valor `700`.

Se quisermos dar mais alguma informação ao utilizador, para ele ver, por exemplo, uma linha com o texto `245 + 455 = 700`, fazemos:

```
public class Expressions2 {  
  
    public static void main (String [] args) {  
  
        System.out.print("245 + 455 = ");  
        System.out.println(245 + 455);  
  
    }  
}
```

Repare na primeira instrução: a instrução `System.out.print("245 + 455 = ")` escreve o **texto** `245 + 455 =` no ecrã; não faz o cálculo pois a expressão está entre aspas.

Usamos `System.out.print` em vez de `System.out.println` para que não seja feita mudança de linha no ecrã após a escrita de `245 + 455 =`

Experimente "executar ao vivo" este programa na secção correspondente da versão online deste livro.

3.2. Literais. Memória. Variáveis. Instrução de atribuição

Os valores que usamos num programa podem ser representados de formas variadas. Já vimos que uma expressão representa um valor. Uma expressão é composta por sub-expressões unidas por um operador. Por exemplo, $(245 + 455) * (17 + 3)$ é uma expressão que se pode decompor nas sub-expressões $245 + 455$ e $17 + 3$ unidas pelo operador `*`. Por sua vez, cada uma destas sub-expressões pode decompor-se em duas sub-expressões mais simples unidas pelo operador `+` (245 e 455 na primeira, e 17 e 3 na segunda).

A expressão mais simples, que não se pode decompor, pode ter duas formas: um *literal* ou uma *variável*.

3.2.1. Literais

Chamamos *literal* a um valor que representamos explicitamente em instruções do nosso programa (por ex., 245 , 455 , 17 e 3 no exemplo acima). Qualquer literal tem um tipo e, por isso, o conjunto de operações em que pode participar como operando está bem definido.

3.2.2. Memória

À medida que o computador avalia uma expressão, os valores das sub-expressões intermédias têm que ser guardados nalgum sítio para poderem ser recuperados quando forem precisos, para o cálculo do valor da expressão global. O programa

```
public class Literals1 {  
  
    public static void main (String [] args) {  
  
        System.out.println((245 + 455) * (17 + 3));  
  
    }  
}
```

vai ter que ser capaz de guardar valores intermédios quando avalia a expressão, passo a passo, antes de imprimir o seu valor no ecrã. Por exemplo, antes de avaliar o resultado da multiplicação, as somas $245 + 455$ e $17 + 3$ têm que ser calculadas e os seus valores têm que ser guardados até serem usados na multiplicação. A forma como isso é feito é transparente para o programador que escreveu o programa, ou seja, ele não se apercebe de que isso acontece.

Para isso é usada a *memória*, que podemos imaginar dividida em células que conseguem guardar valores. O número de células de memória que é necessário usar para guardar um valor depende do tipo desse valor – uma única letra ocupa menos espaço de memória que um valor representando a distância da Terra à Lua, por exemplo.

3.2.3. Variáveis

Independentemente da forma como o computador avalia as expressões e usa a memória, o programador pode ter que guardar explicitamente um valor a certa altura do programa para o poder usar nalguma instrução mais à frente.

Por exemplo, no programa seguinte, que escreve os valores *700*, *20*, *14000*, *28000* e *56000* no ecrã, precisamos de usar o valor de algumas expressões mais do que uma vez.

```
public class Literals2 {

    public static void main (String [] args) {

        System.out.println("Valor do operando esquerdo da multiplicacao: ");
        System.out.println(245 + 455);
        System.out.println("Valor do operando direito da multiplicacao: ");
        System.out.println(17 + 3);
        System.out.println("Valor da multiplicacao: ");
        System.out.println((245 + 455) * (17 + 3));
        System.out.println("E agora o dobro do valor da multiplicacao: ");
        System.out.println(2 * (245 + 455) * (17 + 3));
        System.out.println("E agora o quadruplo do valor da multiplicacao: ");
        System.out.println(4 * (245 + 455) * (17 + 3));

    }

}
```

O ideal era podermos guardar o valor das expressões que precisamos de usar mais do que uma vez para não termos que estar sempre a repeti-las. Vamos fazer isso usando variáveis.

Uma *variável* é um nome que damos, no contexto de um programa, a uma ou várias células de memória.

Podemos *escrever* um valor nessas células e *ler* o valor que têm.

O programa seguinte faz exatamente o mesmo que o anterior, mas usa variáveis. Desse modo evita a repetição de cálculos. Não se preocupe se não percebe ainda as instruções do programa; é natural, pois só as abordaremos mais à frente.

```
public class Variables1 {

    public static void main (String [] args) {

        int leftOperand;
        leftOperand = 245 + 455;

        int rightOperand = 17 + 3;
```



```

        System.out.println("Valor do operando esquerdo da multiplicacao: ");
        System.out.println(leftOperand);

        System.out.println("Valor do operando direito da multiplicacao: ");
        System.out.println(rightOperand);

        int completeExpression = leftOperand * rightOperand;

        System.out.println("Valor da multiplicacao: ");
        System.out.println(completeExpression);

        System.out.println("E agora o dobro do valor da multiplicacao: ");
        System.out.println(2 * completeExpression);
        System.out.println("E agora o quadruplo do valor da multiplicacao: ");
        System.out.println(4 * completeExpression);

    }
}

```

3.2.4. Declaração de variáveis

Sempre que usamos uma variável num programa temos que a *declarar*.

A *declaração* de uma variável

- reserva espaço em memória e
- dá um nome a esse espaço.

A instrução `int leftOperand` é uma declaração de variável; tem o efeito de reservar espaço em memória para guardar um valor do tipo inteiro (`int` – mais adiante neste texto) e dar-lhe o nome de `leftOperand`.

3.2.5. Escrever um valor na memória – Instrução de atribuição

Para *escrever* um valor em memória temos que indicar qual o valor que queremos escrever e onde o queremos escrever.

O sinal `=` representa a *instrução de atribuição*. O efeito da instrução `var = expr` é:

- a expressão `expr` que está à direita de `=` é avaliada e
- o valor resultante é escrito na célula de memória representada pela variável `var`.

A instrução `leftOperand = 245 + 455` faz o computador calcular o valor da expressão `245 + 455` e de seguida escrever esse valor (700) na posição de memória representada por `leftOperand`.

A instrução `int rightOperand = 17 + 3` exemplifica uma *declaração de variável com inicialização*.

Faz a declaração da variável `rightOperand`, de seguida avalia a expressão `17 + 3` e finalmente atribui esse valor (20) à variável `rightOperand`.

O efeito é exatamente o mesmo que se obteria com uma declaração de variável seguida de uma instrução de atribuição, como feito no caso de `leftOperand`.

NOTA: O lado esquerdo de uma instrução de atribuição tem que ser uma expressão que representa um sítio na memória. Na maior parte dos casos, essa expressão será somente o nome de uma variável. Isto entende-se bem: o lado esquerdo da atribuição deve indicar o sítio na memória onde queremos que seja escrito o resultado da expressão que está à direita da atribuição. A instrução seguinte é ilegal: `leftOperand + 3 = 200;` pois a expressão `leftOperand + 3` não representa um sítio de memória, mas sim um valor.

Uma forma sugestiva de representar uma variável e o seu conteúdo é a seguinte:

leftOperand	700
rightOperand	20

3.2.6. Ler um valor da memória

Se declaramos uma variável e lhe atribuímos um valor é porque, mais cedo ou mais tarde, vamos querer utilizar esse valor. Para isso, temos que conseguir aceder ao valor contido na variável, ou seja, "ler" o seu valor.

Para ler o valor contido numa variável basta utilizar o seu nome.

Para que a instrução `int completeExpression = leftOperand * rightOperand;` seja executada, a expressão `leftOperand * rightOperand` tem que ser avaliada. As sub-expressões `leftOperand` e `rightOperand` representam os valores contidos nas variáveis correspondentes.

A instrução reserva espaço em memória para a variável `completeExpression`, de seguida avalia a expressão `leftOperand * rightOperand` fazendo a multiplicação dos conteúdos das variáveis `leftOperand` e `rightOperand` (700 e 20, respetivamente), e finalmente escreve o resultado (14000) na variável `completeExpression`.

3.2.7. Imprimir o valor de uma variável

Já sabemos escrever no *standard output* (o ecrã, normalmente) – usamos `System.out.println(expr);` em que `expr` é uma expressão (mais à frente nestas páginas falaremos mais sobre isto).

A expressão `expr` é avaliada e de seguida o seu valor é apresentado no *standard output*.

A expressão `leftOperand` é formada somente pelo nome de uma variável; logo, o seu valor é o valor que a variável tem no momento dessa avaliação. A instrução `System.out.println(leftOperand);` escreve no *standard output* o valor 700, que é o valor que foi escrito na posição de memória representada por essa variável.

Já a instrução `System.out.println(2 * completeExpression);` escreve no *standard output* o valor 28000, que é o valor que resulta da avaliação da expressão `2 * completeExpression`.

Experimente agora "executar ao vivo" o programa completo na versão online deste livro, observando, passo a passo, o conteúdo das variáveis.

3.3. Tipos de dados primitivos e (um cheirinho de) String

Mais atrás neste texto chamámos tipo de dados a um conjunto de valores e às operações que se podem fazer sobre esses valores.

O Java fornece uma série de tipos de dados base, prontos a usar – os *tipos de dados primitivos* –, com os quais é possível representar todos os valores de interesse.

Vamos aqui falar sobre dois desses tipos e acerca de outro tipo de dados – `String` – que, embora não primitivo, é extremamente útil desde já.

3.3.1. Inteiros

O tipo `int` representa em Java os valores inteiros no intervalo entre -231 e 231 - 1. Um valor deste tipo ocupa 32 bits de memória.

Os literais do tipo `int` são todos os números, naquele intervalo, explicitamente representados (exemplos: 2, 567 e -47).

As operações que suportam operandos do tipo `int` são a soma (+), o produto (*), a diferença (-), a divisão inteira (/), o resto da divisão inteira (%) e o simétrico (-). Este último operador é unário, ou seja, tem um só operando.

Os operadores relacionados com a divisão merecem uma explicação. A divisão inteira de um número *a* por um número *b* tem como resultado o maior inteiro contido na divisão (também chamado de quociente). Para obtermos este valor, usamos em Java o operador /. Quando *a* não é divisível por *b*, o resto desta divisão inteira é diferente de zero. Para obtermos o valor do resto da divisão, usamos em Java o operador %.

As instruções seguintes escrevem no ecrã os valores 2, 2, 0, 0, 3, 1, -2, -2, 2, -3, 3 e -3 respetivamente.

```
System.out.println(10 / 5);
System.out.println(13 / 5);
System.out.println(1 / 3);
System.out.println(10 % 5);
System.out.println(13 % 5);
System.out.println(1 % 3);

System.out.println(-13 / 5);
System.out.println(13 / -5);
System.out.println(-13 / -5);
System.out.println(-13 % 5);
System.out.println(13 % -5);
System.out.println(-13 % -5);
```

3.3.2. Números com parte não inteira

O tipo `double` representa em Java os valores numéricos com parte fracionária decimal no intervalo entre -10^{308} e 10^{308} , aproximadamente. Um valor deste tipo ocupa 64 bits de memória, representado em vírgula flutuante.

Os literais do tipo `double` são todos os números representáveis naquele intervalo que contenham um ponto (ex: 2.0, 5.67, -478903.11, etc); pode também ser usado E ou e para notação científica.

Muitas vezes os resultados das operações têm mais algarismos do que os que o computador consegue representar (basta pensar numa dízima infinita). Convém saber quantos algarismos são representáveis (algarismos significativos) para percebermos qual a precisão que podemos esperar nos cálculos que fazemos. No tipo `double` a precisão é de 15 a 16 algarismos significativos, aproximadamente (exemplo: se tiver o número 12345.67890123456789 os últimos dígitos à direita serão ignorados).

As operações que suportam operandos do tipo `double` são a soma (+), o produto (*), a diferença (-), a divisão (/), neste caso não inteira, o resto da divisão inteira (%) e o operador unário simétrico (-).

Podemos “misturar” `int` e `double` em operações. Basta haver um `double` numa operação destas para que o resultado da expressão seja `double`. Em relação à divisão, quando um dos operandos é `double`, a divisão é feita com a maior precisão possível.

As instruções seguintes escrevem no ecrã os valores 2.0, 2.6, 0.3333333333333333, 1.1500000000000001, 0.0 e 3.0, respetivamente.

```
System.out.println(10.0 / 5);
System.out.println(13.0 / 5);
System.out.println(1.0 / 3);
System.out.println(3.45 / 3);
System.out.println(10.0 % 5);
System.out.println(13.0 % 5);
```

3.3.3. String

O tipo `String` representa cadeias de caracteres em Java. O espaço que um valor do tipo `String` ocupa é proporcional ao número de caracteres que tem.

Os literais do tipo `String` são todas as cadeias de caracteres entre aspas (exemplos: `"Ola"`, `"67"`, `"a Maria"` e `"12?*&& "`).

Para já, a única operação que abordamos sobre valores do tipo `String` é a operação de concatenação (+), que junta duas *strings*. Este é o mesmo sinal usado para a soma, mas quando os operandos são do tipo `String` o resultado é a concatenação dessas cadeias de caracteres.

As instruções seguintes escrevem no ecrã *OlahTurma* e *Olah Turma*, respetivamente.

```
System.out.println("Olah" + "Turma");
System.out.println("Olah" + " Turma");
```

3.4. Revisitando variáveis

Agora que já sabemos como usar valores de três tipos diferentes, vamos aprender mais um pouco sobre declaração de variáveis, atribuição de valores a variáveis, acesso aos seus valores e utilização de expressões.

3.4.1. Declaração de variáveis

Para declarar uma variável num programa Java escrevemos uma instrução com a forma:

```
Tipo nome-da-variavel;
```

O *Tipo* pode ser um qualquer tipo de dados reconhecido pelo Java.

O *nome-da-variavel* deve ser sugestivo do significado que damos ao seu valor e deve seguir as seguintes regras de construção:

- Deve começar por uma letra, um `$` ou `_`; a convenção é, no entanto, começar os nomes das variáveis sempre por uma letra, sendo os outros dois caracteres usados só em casos muito específicos (por ex., código gerado automaticamente);
- Os caracteres seguintes podem ser letras e algarismos *Unicode* e também `$` ou `_` (embora por convenção se evitem estes dois últimos);
- O nome escolhido não pode ser uma palavra reservada do Java, ou seja, não pode ser nenhuma das [palavras reservadas do Java](#);
- Se o nome escolhido consistir de uma só palavra, deve ser escrito todo em minúsculas; se consistir de mais que uma palavra, devem ficar todas juntas e deve ser capitalizada a primeira letra de cada palavra a seguir à primeira (relembre os nomes das variáveis `leftOperand`, `rightOperand` e `completeExpression`; outros exemplos: `impostoMedioAnual` e `distanciaEntreEixos`)
- Os nomes das variáveis são *case sensitive* ou seja, os nomes `Ab` e `ab` são diferentes; evite criar variáveis em que os nomes só diferem na capitalização de uma ou mais letras.

3.4.2. Atribuição de valor a uma variável

Para atribuir um valor a uma variável que já foi declarada escrevemos uma instrução com a forma:

```
nome-da-variavel = expressao;
```

Podemos escolher fazer a declaração da variável e a inicialização do seu conteúdo numa só instrução:

```
Tipo nome-da-variavel = expressao;
```

Em qualquer dos casos, a expressao deve ser uma expressão bem construída e o seu tipo (que é o tipo do seu resultado) deve ser compatível com o tipo da variável – não podemos, por exemplo, atribuir um valor do tipo `String` a uma variável do tipo `int`. O compilador não deixa, originando um erro de compilação.

Vários exemplos:

```
String nome = "Maria";  
int idade = 12;  
double altura, peso;  
altura = 1.79;  
peso = 61.2;
```

Repare na terceira instrução; é uma declaração múltipla. Quando queremos declarar mais do que uma variável do mesmo tipo, podemos fazê-lo numa única instrução. Devemos ter cuidado para que não diminuam a legibilidade – por exemplo, declarar várias variáveis que não têm qualquer relação, todas na mesma declaração só porque são do mesmo tipo, diminui a legibilidade.

Podemos também ter declarações com inicialização múltiplas. Por exemplo, a instrução

```
double altura = 1.79, peso = 61.2;
```

tem o mesmo efeito que as três últimas instruções acima.

Experimente executar "ao vivo" este programa na versão online deste livro.

3.4.3. Atribuições múltiplas

Podemos fazer mais que uma atribuição de valor a uma variável durante um programa. O efeito dessas novas atribuições é a substituição do valor que a variável tinha pelo novo valor.

```
int idade = 12;  
System.out.println(idade);  
idade = 19;  
System.out.println(idade);
```

O efeito destas instruções seria a impressão dos valores 12 e 19 no ecrã.

Podemos olhar para as variáveis como contentores de valores. Sempre que atribuímos um valor a uma variável estamos na verdade a alterar-lhe o conteúdo, ou seja, ao atribuímos um novo valor a uma variável destruímos o valor que lá estava.

Experimente executar "ao vivo" na versão online deste livro.

Embora as atribuições múltiplas sejam muito frequentes e úteis em programação, devemos usá-las com cuidado, pois o código pode ficar difícil de ler e os erros podem ser mais difíceis de detetar.

Uma variável num programa deve representar um determinado conceito e por isso não devemos mudar o seu significado ao longo do programa. Se declaramos uma variável para representar uma idade, não devemos, quando já não precisamos de manipular idades, usar a mesma variável para representar, por exemplo, pesos ou outro conceito qualquer.

3.5. Ainda as expressões

3.5.1. Avaliação de expressões. Precedências.

Devemos ter em atenção a *ordem* pela qual as operações são avaliadas numa expressão. O produto e a divisão, por exemplo, têm prioridade sobre a soma e a subtração.

Nível	Operador	Descrição	Associatividade
16	[] . ()	aceder elemento array aceder membro de objeto parentesis	esquerda para direita
15	++ --	pós-incremento unário pós-decremento unário	não associativo
14	++ -- + - ! ~	pré-incremento unário pré-decremento unário positivo (unário) simétrico (unário) NOT lógico (unário) NOT bitwise (unário)	direita para esquerda
13	() new	cast criação de objeto	direita para esquerda
12	* / %	multiplicativo	esquerda para direita
11	+ - +	aditivo concatenação de strings	esquerda para direita
10	<< >> >>>>	shift	esquerda para direita
9	< <= > >= instanceof	relacional	não associativo
8	== !=	igualdade	esquerda para direita
7	&	AND bitwise	esquerda para direita
6	^	XOR bitwise	esquerda para direita
5		OR bitwise	esquerda para direita
4	&&	AND lógico	esquerda para direita
3		OR lógico	esquerda para direita
2	?:	condicional (ternário)	direita para esquerda
1	= += -= *= /= %= &= ^= = <<= >>= >>>=	atribuição	direita para esquerda

Se queremos contrariar as prioridades definidas para as operações no Java, temos que usar parentesis.

Quando numa expressão temos vários operadores associativos com igual precedência (por exemplo, `+` e `-` e `*`, `/` e `%`), a ordem de avaliação é da esquerda para a direita – por exemplo, `6 + 4 - 3` é equivalente a `(6 + 4) - 3`. Considere o seguinte programa:

```
public class Expressions3 {  
  
    public static void main (String [] args) {  
  
        int i = 2;  
        int j = 6;  
        double d1 = 3.5;  
        double d2 = 10;
```

```

int expr1 = 5 + j - i / 4;
int expr2 = 5 + (j - i) / 4;
int expr3 = (5 + (j - i)) / 4;

double d3 = j / 4;
double d4 = j / 4.0;

String s1 = "Alcibiades", s2 = "Briolanjo";
String s3 = "Oh " + s1 + ", jah viste o " + s2 + " hoje? ";

}
}

```

Na quarta instrução temos uma atribuição do valor inteiro `10` a uma variável do tipo `double`. Isto é possível – o que o computador faz é uma conversão automática do literal inteiro `10` para `double` e a sua atribuição à variável `d2`. O contrário não é possível, ou seja, atribuir um valor do tipo `double` a uma variável do tipo `int` (o que é lógico, se pensarmos bem).

No cálculo da expressão a atribuir à variável `expr1`, a primeira operação a ser feita é a divisão de `i` (cujo valor é 2) por 4 (que dá 0); repare que a divisão que é feita é a divisão inteira, porque tanto `i` como 4 são do tipo `int`; a seguir, dado que a soma e a subtração têm a mesma prioridade e as operações são associativas à esquerda, é feita a soma de 5 com `j` (que dá 11) e, finalmente, a subtração de 11 e 0 que dá 11.

No cálculo da expressão a atribuir à variável `expr2`, porque o uso de parentesis muda as prioridades, a primeira operação a ser feita é a subtração `j - i` (que dá 4); a seguir, uma vez que a divisão tem maior prioridade que a soma, é feita a divisão inteira entre 4 e 4 (que dá 1) e, finalmente, a soma de 5 e 1 que dá 6.

No cálculo da expressão a atribuir à variável `expr3`, a primeira operação a ser feita é também `j - i` (que dá 4) mas de seguida é feita a soma desse valor com 5, o que dá 9. Finalmente, é avaliada a divisão inteira de 9 por 4 que dá 2.

Qual o valor atribuído à variável `d3`? `1.5`? `1.0`? Se respondeu `1.0` acertou! Embora a variável seja do tipo `double`, a expressão `j / 4` é uma divisão entre dois valores do tipo `int` (`j` é `int` e 4 é um literal do tipo `int`), logo a divisão que é feita é a divisão inteira, que resulta no inteiro 1, o qual é de seguida automaticamente convertido para `double` e atribuído à variável `d3`.

A variável `d4` já ficará com o valor `1.5` pois a divisão que é efetuada entre `j` e `4.0` já é a divisão real – um dos operandos não é inteiro (`4.0` é um literal do tipo `double`).

A variável `s3` ficará com o valor `"Oh Alcibiades, jah viste o Briolanjo hoje?"`; quando a expressão `"Oh " + s1 + ", jah viste o " + s2 + " hoje? "` é avaliada, é feita a concatenação de todas as *strings* nela presentes; `s1` e `s2`, por serem variáveis, representam o seu conteúdo e é esse conteúdo que vai ser usado na avaliação da expressão.

NOTA: Como já deve ter percebido, uma expressão não é uma instrução! Uma expressão representa um valor que é o resultado da sua avaliação. E esse valor deve ser usado para algum objetivo concreto como, por exemplo, ser atribuído a uma variável ou ser escrito no ecrã. Não faz sentido, então, ter uma expressão sózinha no lugar de uma instrução pois, desse modo, nada é feito com o seu valor, depois de avaliada.

3.5.2. Conversão de `double` para `int`

O Java converte `int` para `double` automaticamente sempre que necessário, como já vimos acima, pois não se perde nenhuma informação no processo. Por outro lado, para converter `double` para `int` é necessário fazer algo com a parte decimal, a qual não é representável num inteiro.

O Java não faz isto automaticamente porque é importante que o programador fique ciente da perda dessa parte decimal e também para que seja ele a decidir o que fazer a essa parte decimal – se arredondar ou se truncar (cortar).

Há várias formas de fazer esta conversão, mas para já vamos só ver a mais simples e direta: usar *cast* de tipos. A forma de obter o *cast* de tipos é colocar o nome do tipo alvo da conversão entre parêntesis à frente da expressão que se pretende converter.

A primeira das instruções seguintes declara e inicializa uma variável `pi` do tipo `double`; a segunda instrução converte o valor da variável `pi` para `int` truncando a parte decimal e atribui esse valor inteiro à variável `k` – a variável `k` ficará com o valor 3. O conteúdo da variável `pi` não é afetado!

```
double pi = 3.14159;
int k = (int) pi;
double x = (int) pi * 20.3;
```

Na terceira instrução a variável `x` fica com o valor 60.9.

O operador de *cast* tem maior precedência do que a multiplicação (na verdade, maior do que de todos os operadores binários) e, por isso, se quisermos converter o resultado de uma expressão, temos que aplicar o *cast* sobre essa expressão entre parêntesis.

Com a instrução `double y = (int) (pi * 20.3);`, a variável `y` fica com o valor 63.0 – a expressão `pi * 20.3`, quando avaliada, tem o valor 63.774277 que, truncado pelo *cast*, fica 63.0.

Experimente executar o programa completo na versão online deste livro, observando o conteúdo das variáveis.

3.5.3. Ainda a concatenação de `String`s

Já vimos que o operador `+`, quando usado com valores do tipo `String`, resulta na concatenação dos seus operandos. Quando este operador é usado com uma `String` e um valor de outro tipo, o resultado é uma `String`.

O seguinte excerto de programa:

```
int idade = 12;
double altura = 1.79;
String nome = "Maria";

String info1 = "Maria tem 12 anos";
String info2 = "Maria tem " + 12 + " anos";
String info3Parcial = "Maria tem " + idade;
String info3 = info3Parcial + " anos";
String info4 = nome + " tem " + idade + " anos";
String info5 = 12 + " anos tem a Maria";

System.out.println(nome + " tem " + idade + " anos");
System.out.println(info1);
System.out.println(info2);
System.out.println(info3);
System.out.println(info4);
System.out.println(info5);
System.out.println(nome + " tem " + altura + " metros de altura " );

int i = 2;
String s = "linha" + i * 10;
System.out.println(s);
```

produz o seguinte *output*:


```
Maria tem 12 anos
Maria tem 12 anos
Maria tem 12 anos
Maria tem 12 anos
Maria tem 12 anos
12 anos tem a Maria
Maria tem 1.79 metros de altura
linha20
```

Os valores atribuídos às variáveis `info1` a `info4` são iguais, embora tenham sido construídos de forma diferente.

Repare que a expressão à direita da atribuição à variável `info2` tem dois operadores iguais, de concatenação; por serem iguais, são avaliados da esquerda para a direita. Na primeira operação, como um dos operandos é do tipo `String`, o operador `+` é interpretado como concatenação; é feita então como que uma conversão do inteiro 12 em `String` antes da concatenação ser avaliada. O valor resultante dessa primeira concatenação é a `String` `"Maria tem 12"` que, de seguida, é concatenada com a string `"anos"`, resultando em `"Maria tem 12 anos"`.

Na atribuição à variável `info3Parcial` o esquema de avaliação é o mesmo, com a diferença de que o valor que é concatenado com a `String` `"Maria tem "` é dado por uma variável. É o valor da variável que é usado na concatenação.

A atribuição à variável `info3` é uma concatenação vulgar entre duas `Strings`, em que uma é dada pelo conteúdo da variável `info3Parcial`.

Experimente executar "ao vivo" o programa completo na versão online deste livro.

3.5.4. Funções matemáticas

À semelhança das expressões matemáticas, podemos incluir numa expressão Java algumas das funções matemáticas mais comuns. Para já, vamos ver só alguns exemplos de como fazer isso; uma explicação mais aprofundada das funções em geral fica para o próximo capítulo.

```
int i = 3;
double raiz = Math.sqrt(17.0 * i);
System.out.println( Math.sin(raiz));
int k = 3 * Math.round(Math.log10(raiz));
```

As funções `Math.sqrt`, `Math.sin`, `Math.log10` e `Math.round` calculam, respetivamente, a raiz quadrada, o seno, o logaritmo na base 10 e o valor arredondado do seu argumento.

No cálculo do valor de uma função, é feita, em primeiro lugar, a avaliação dos seus argumentos e, de seguida então, a avaliação da função sobre esses valores (na segunda instrução é calculado primeiro o valor de `17.0 * i` e de seguida então a raiz quadrada desse resultado).

Na expressão da última instrução temos composição de funções – como o resultado de uma função aplicada a um valor é ele próprio um valor, pode ser usado como argumento de outra função.

Quando queremos usar no nosso programa um valor (pseudo)aleatório, podemos usar a função `Math.random`, cujo resultado é um número aleatório do tipo `double`, no intervalo `[0, 1[`. Por exemplo, `double umQualquer = Math.random();`

Se quisermos um número aleatório num intervalo diferente desse, basta-nos fazer as operações necessárias para o efeito. Como exemplo, a instrução `double entre25e50 = (Math.random() + 1) * 25;` gera um valor aleatório no intervalo `[25, 50[` e atribui esse valor à variável `entre25e50`.

Ver em <https://docs.oracle.com/javase/9/docs/api/java/lang/Math.html> as funções disponíveis mais comuns.

3.6. Comentários, indentação e outros promotores da legibilidade

Ao longo destas páginas vamos estar preocupados com a questão da *legibilidade* que é, entre outras, uma propriedade muito importante dos programas, pois facilita muito a sua manutenção (correção de erros, alteração, etc), não só por quem os escreveu como por qualquer outro programador.

Quando escrevemos um programa, há variadíssimas opções que podemos tomar que promovem a legibilidade. Vamos ver aqui algumas técnicas simples.

3.6.1. Comentários

O compilador de Java permite-nos escrever texto nos programas que não é interpretado como instruções – os *comentários*.

Para um programador, os comentários servem para ajudar à compreensão das várias partes do programa, sejam elas o significado das variáveis, a explicação de alguma expressão mais complexa, etc.

- Podemos inserir *comentários de linha* – texto que se situa entre os caracteres `//` e o fim da linha onde se encontram.

```
double raiz = Math.sqrt(17.0 * i);           // raiz quadrada de 51

System.out.println( Math.sin(raiz));        // seno de um valor em radianos

// o triplo do valor arredondado do logaritmo na base 10 do valor de raiz
int k = 3 * Math.round(Math.log10(raiz));
```

- Podemos inserir *comentários de bloco* – texto que se situa entre `/*` e `*/`.

```
double raiz = Math.sqrt(17.0 * i);           /* raiz quadrada de 51 */

/*
  o triplo do valor arredondado do logaritmo na base 10
  do valor da variavel raiz
*/
int k = 3 * Math.round(Math.log10(raiz));
```

- Podemos inserir *comentários de documentação Javadoc* – texto que se situa entre `/**` e `*/`.
 - Estes são comentários a serem reconhecidos por uma aplicação que os usa para gerar documentação em formato *html* (mais sobre isto no próximo capítulo).

3.6.2. Espaços

Para o compilador Java, os espaços a mais são ignorados, o que nos permite usá-los para melhorar a legibilidade dos programas.

Devemos, por exemplo, utilizar um espaço antes e após qualquer operador binário (exceto o ponto).

Preferimos

```
System.out.println(4 * (245 + 455) * (17 + 3));
int expr3 = 5 + j - i / 4;
double raiz = Math.sqrt(17.0 * i);
```

a

```
System.out.println(4*(245+455)*(17+3));  
int expr3=5+j-i/4;  
double raiz=Math.sqrt(17.0*i);
```

Usamos também linhas em branco para separar partes do nosso programa. Gostamos disto:

```
public class Expressions3 {  
  
    public static void main (String [] args) {  
  
        int i;  
        i = 2;  
        int j = 6;  
        double d1 = 3.5;  
        double d2 = 10;  
  
        int expr1 = 5 + j - i / 4;  
        int expr2 = 5 + (j - i) / 4;  
        int expr3 = (5 + (j - i)) / 4;  
  
    }  
}
```

Linhas demasiado longas podem parecer bem no terminal, mas não quando impressas. Vamos limitar-nos a 80 caracteres por linha, partindo linhas demasiado longas.

Ao longo deste livro veremos outros casos em que devemos usar espaços.

3.6.3. Indentação

A indentação é uma forma de usar os espaços em branco que é fundamental para a legibilidade dos programas. Uma classe Java define um bloco entre chavetas. Um método Java também define um bloco entre chavetas. Como iremos ver nos próximos capítulos, as instruções do programa também definem blocos.

A ideia é *indentar* – “empurrar” para a direita, usando espaços – todas as linhas de um programa que constituam um novo bloco.

Preferimos:

```
public class Expressions1 {  
    public static void main (String [] args) {  
        System.out.println(245 + 455);  
    }  
}
```

a

```
public class Expressions1 {  
public static void main (String [] args) {  
System.out.println(245 + 455);}  
}
```

```
public class Expressions1{public static void main(String [] args)
{System.out.println(245+455);}}
```

4. Abstração procedimental

Até agora vimos como fazer cálculos, comunicar com o utilizador imprimindo informação no ecrã, guardar valores em memória e ler esses valores. Embora muito limitadas, estas ações já nos permitem construir programas bastante extensos.

O seguinte programa calcula notas finais de alguns alunos numa determinada disciplina. A nota é calculada a partir da nota do exame (com peso 75%) e da média das notas das três fases do projeto (com peso 25%).

A nota final é o valor arredondado do resultado anterior – para isso usamos *cast* para inteiro do valor, depois de lhe adicionar 0.5 (lembre-se que o *cast* para inteiro ignora a parte decimal). Exemplos:

- `(int) (12.4 + 0.5)` dá 12;
- `(int) (18.6 + 0.5)` dá 19.

```
public class NotasFinais {

    public static void main (String [] args) {

        // Nota final do aluno 12345
        double notaCalculada = 0.75 * 16.2 + 0.25 * ((12.3 + 14.4 + 15.1) / 3) ;
        int nota = (int)(notaCalculada + 0.5);

        System.out.println("O aluno 12345 teve " + nota + " valores");
        System.out.println("++++");

        // Nota final do aluno 54321
        notaCalculada = 0.75 * 12.0 + 0.25 * ((14.2 + 10.0 + 10.5) / 3) ;
        nota = (int)(notaCalculada + 0.5);

        System.out.println("O aluno 54321 teve " + nota + " valores");
        System.out.println("++++");

        // Nota final do aluno 50000
        notaCalculada = 0.75 * 18.0 + 0.25 * ((15.6 + 10.2 + 16.2) / 3) ;
        nota = (int)(notaCalculada + 0.5);

        System.out.println("O aluno 50000 teve " + nota + " valores");
        System.out.println("++++");

    }
}
```

Se compilarmos e executarmos este programa, o seguinte texto será escrito no *standard output* (o ecrã, por defeito):

```
O aluno 12345 teve 16 valores
+++++
O aluno 54321 teve 12 valores
+++++
O aluno 50000 teve 17 valores
+++++
```

Repare que no programa há uma série de (blocos) de ações que se repetem, tanto integralmente como com ligeiras variações. Se conseguíssemos abstrair essas ações que se repetem – dando-lhes um nome sugestivo, por exemplo – e usar esse nome em vez do conjunto de instruções, então o nosso programa poderia ficar mais compacto e legível.

Se comparar o método `main` anterior com o que se segue (mesmo ainda não entendendo algumas das suas instruções) verifica que este é mesmo mais compacto e legível. Pois bem, o efeito que tem a execução dos dois programas é exatamente o mesmo!

```
public class NotasFinaisLegivel {

    public static void main (String [] args) {

        // Nota final do aluno 12345
        int nota = notaFinal(16.2, 12.3, 14.4, 15.1);
        imprimeInformacao(12345, nota);

        // Nota final do aluno 54321
        nota = notaFinal(12.0, 14.2, 10.0, 10.5);
        imprimeInformacao(54321, nota);

        // Nota final do aluno 50000
        nota = notaFinal(18.0, 15.6, 10.2, 16.2);
        imprimeInformacao(50000, nota);

    } // fim do metodo main

    /*****
    /*****  METODOS  *****/
    /*****

    /*
    * Nota final arredondada ahs unidades
    */
    static int notaFinal(double exame,
        double proj1, double proj2, double proj3) {
        double notaCalculada =
            0.75 * exame + 0.25 * ((proj1 + proj2 + proj3) / 3) ;
        return (int)(notaCalculada + 0.5);
    }

    /*
    * Imprimir nota de aluno no standard output
    */
    static void imprimeInformacao(int nrAluno, int notaAluno) {
        System.out.println("O aluno " + nrAluno +
            " teve " + notaAluno + " valores");
        System.out.println("+++++");
    }
}
```

```
}  
  
}
```

Usámos *abstracção*, uma técnica fundamental na construção de programas, pois promove uma série de fatores de qualidade, como aprenderá mais tarde.

Abstracção – ignorar certos pormenores acerca de um assunto, focando-nos somente no que é importante num dado contexto.

A abstracção deve ser aplicada aos vários níveis do desenvolvimento dos programas, como irá aprender durante o seu curso.

Para já vamos estudar a

abstracção procedimental – dar um nome a um conjunto de ações, abstraindo assim as instruções concretas que o implementam.

No programa, sempre que se quer executar esse conjunto de instruções refere-se o nome que o representa.

Obtemos um mecanismo muito poderoso para construção de programas quando lhe juntamos a

abstracção de parâmetro – identificação de pontos de variação no conjunto de instruções, juntamente com a capacidade de indicar valores específicos para esses pontos aquando de cada execução

Como exemplos, no programa acima, temos `notaFinal(...)` e `imprimeInformacao(...)`.

No Java chamamos *métodos* aos blocos de instruções que têm um nome. Podemos ainda distinguir os métodos que são *procedimentos* dos que são *funções*.

Mais adiante veremos como complementar a informação dos métodos para mais fácil compreensão e utilização.

4.1. Alguns métodos já nossos conhecidos

Nas secções anteriores já vimos vários métodos – as funções matemáticas ilustradas na secção 3.5.4. são métodos, bem como o procedimento `System.out.println` que temos utilizado sempre que queremos escrever algo no *standard output*.

Estes métodos foram criados por alguém, e estão definidos nas classes `Math` e `System`, respetivamente, classes acessíveis a todos os que programam em Java. Nestas classes estão definidos mais uma série de métodos úteis, para serem usados pelos programadores nos seus programas.

Podemos identificar uma característica comum a todos os métodos da classe `Math`. Todos eles calculam um valor – a que chamamos o *resultado* do método. Pelo facto de terem um resultado, são chamados de *funções*.

O método `System.out.println` não tem um resultado definido. É um método que "faz algo", nomeadamente escrever algo no *standard output*, mas que não "devolve" nenhum resultado. Métodos com esta característica dizem-se *procedimentos*.

4.1.1. Fluxo de execução

Uma sequência de instruções num programa é executada, instrução a instrução, de forma sequencial. Este *fluxo de execução* pode ser alterado através de comandos específicos, os quais aprenderemos ao longo destas páginas.

A *invocação* de métodos é um dos mecanismos que altera a aparente sequencialidade desse fluxo.

Na instrução

```
double raiz = Math.sqrt(17.0 * i);
```

o produto entre 17.0 e o valor da variável `i` é calculado e de seguida a função `Math.sqrt` é *invocada*. Isto faz com que o fluxo de execução do programa "salte" para este método, ou seja, as instruções definidas no método `Math.sqrt` (que não fomos nós que criámos, nem precisamos de saber quais são) são executadas. Quando o método termina, o fluxo de execução volta de novo à instrução `double raiz = Math.sqrt(17.0 * i);`. Nessa altura, o resultado da invocação da função é atribuído à variável `raiz`.

A instrução

```
System.out.println(raiz);
```

provoca uma invocação do método `System.out.println` com o consequente "salto" do fluxo de execução para esse método. A execução das instruções nele definidas provoca a escrita do valor da variável `raiz` no *standard output*.

O mecanismo de invocação de métodos tem mais que se lhe diga e vai ser abordado em detalhe nas próximas secções.

Ainda sem se preocupar em perceber esse mecanismo, se já quiser "espreitar" a execução do programa `NotasFinaisLegivel`, pode fazê-lo na versão online deste livro.

4.1.2. O método `main`

O bloco de instruções que se chama `main` que temos vindo a escrever até agora é, na verdade, um **método** cujo nome é *main*.

É um método especial, pois é ele que é executado quando mandamos executar o programa a partir do sistema operativo – por exemplo, escrever na linha de comando

```
java NotasFinais
```

provoca a execução do método `main` contido no ficheiro `NotasFinais.class`.

Isto significa que sempre que escrevemos um programa, criamos pelo menos uma classe contendo pelo menos um método, o `main`, que é um procedimento. As instruções que escrevemos nesse método variam muito, pois dependem do que queremos que o programa faça.

No método `main`, tal como em qualquer outro, podemos *invocar* outros métodos, sejam eles funções ou procedimentos. Esses métodos podem, por sua vez, invocar outros, e assim sucessivamente.

4.2. Procedimentos

Chamamos *procedimento* a um método que "faz" algo e que não devolve nenhum valor como resultado da sua execução. Imprimir algo no ecrã é um exemplo de um procedimento.

Distinguimos entre:

- *definição* do procedimento – indicar qual o conjunto de instruções que ele representa, e
- *invocação* do procedimento – indicar que queremos executar o conjunto de instruções por ele

representado.

4.2.1. Definição do procedimento

Para que possamos indicar qual o conjunto de instruções que compõe o procedimento, temos que o definir num local acessível pelo programa que o vai usar. Para já vamos considerar que o vamos definir na mesma classe que contém o nosso programa.

Para definir um procedimento que imprime no ecrã três linhas com cruces podemos escrever o seguinte no mesmo ficheiro onde o vamos usar:

```
static void imprimeTresLinhas() {  
    System.out.println("+++++");  
    System.out.println("+++++");  
    System.out.println("+++++");  
}
```

Podemos colocar esta definição do procedimento em qualquer parte do nosso ficheiro java, desde que não seja no meio de outro método!

A *definição* de um procedimento inclui o *corpo* e a *assinatura*.

- O corpo do procedimento é constituído por um bloco de instruções entre duas chavetas;
- A assinatura do método precede o corpo do método; nela se indica, entre outras coisas (mais sobre isto adiante), o nome do método.

No exemplo acima o corpo do procedimento compõe-se de três instruções iguais. O seu nome é `imprimeTresLinhas`. A palavra `void` precede o nome e significa “o método não produz nenhum resultado”.

A preceder as outras palavras, a palavra `static` vai ser usada, para já, sempre que definirmos métodos. Não a usaremos para sempre, mas explicaremos o seu significado mais tarde.

4.2.2. Definição de parâmetros

Até agora vimos como abstrair um conjunto de instruções dando-lhe um nome. O método `imprimeTresLinhas`, por exemplo, escreve três linhas com cruces no *standard output*.

Este procedimento terá sempre o mesmo efeito quando é executado (já veremos mais à frente como invocá-lo). Podemos executá-lo 100 vezes seguidas que o efeito será sempre o mesmo.

Há situações em que, num programa, queremos executar um mesmo conjunto de ações várias vezes mas com ligeiras variações. Para isso podemos definir um método em que o conjunto de instruções que o compõe tem pontos de variação que são desconhecidos na altura da definição, mas que serão conhecidos quando o método for invocado e executado.

Fazemos isso usando uma variável no ponto de variação e incluindo-a numa parte da assinatura do método que é a *lista de parâmetros*.

Um *parâmetro* representa um valor que é desconhecido na definição do método mas que vai ser indicado (instanciado) pelo programador quando invoca o método. Desta maneira podemos ter efeitos diferentes em várias execuções do mesmo método.

- Um parâmetro tem um *tipo* e um *nome*;
- Um parâmetro vai ter um valor quando o método é invocado e executado;
- O seu tipo restringe os valores que lhe podem ser dados;
- O seu nome serve para identificar, dentro do corpo do método, o seu valor;
- Os parâmetros representam "o que o método tem que conhecer para poder fazer o que precisa de

fazer".

Relembre o método `imprimeInformacao` da classe `NotasFinaisLegivel`:

```
static void imprimeInformacao(int nrAluno, int notaAluno) {  
    System.out.println("O aluno " + nrAluno + " teve " +  
        notaAluno + " valores");  
    System.out.println("+++++++");  
}
```

Neste exemplo definimos dois parâmetros na assinatura do método – `nrAluno` e `notaAluno`.

A forma como lemos a primeira instrução do corpo do método

```
System.out.println("O aluno " + nrAluno + " teve " +  
    notaAluno + " valores");
```

pode ser a seguinte:

Imprimir no ecrã a string resultante de concatenar as strings:

- `"O aluno "`,
- o valor em memória correspondente ao parâmetro `nrAluno` (definido aquando da invocação do método),
- `" teve "`,
- o valor em memória correspondente ao parâmetro `notaAluno` e
- `" valores"`.

A segunda instrução do corpo do método

```
System.out.println("+++++++");
```

não depende dos valores dos parâmetros e, como já sabemos, escreve `+++++++` no ecrã.

Então, se numa dada execução deste método os parâmetros `nrAluno` e `notaAluno` tiverem os valores `"58000"` e `15`, o efeito da execução do método é a impressão, no ecrã, da frase:

```
O aluno 58000 teve 15 valores  
+++++++
```

Falta-nos então ver como se faz para invocar um procedimento; é nessa altura que são definidos os valores que os seus parâmetros irão tomar na execução.

4.2.3. Invocação de procedimentos

Para executarmos um procedimento temos que o invocar. Já que um procedimento pode ter parâmetros, os valores desses parâmetros terão que ser conhecidos antes de iniciada a execução.

A *invocação* de um procedimento é feita através de uma instrução composta por

- o nome do procedimento
- os valores a atribuir aos parâmetros
 - na mesma ordem com que estão definidos na assinatura do método,
 - entre parentesis e separados por vírgulas,
 - estes valores são expressões, que, como já sabemos, podem ser literais, nomes de variáveis ou expressões mais complexas.

Como resultado da invocação, os valores indicados para os parâmetros são copiados para esses mesmos parâmetros e as instruções do corpo do método são executadas.

Qualquer referência ao nome de um parâmetro no corpo do método representa o valor “guardado” nesse parâmetro.

Por exemplo, a instrução

```
imprimeInformacao(12345, nota);
```

é uma invocação do procedimento `imprimeInformacao`. Os valores escolhidos para instanciar os valores dos parâmetros são o literal `12345` e o valor da variável `nota`.

Relembre de novo a definição do método `imprimeInformacao` da classe `NotasFinaisLegivel`:

```
static void imprimeInformacao(int nrAluno, int notaAluno) {  
    System.out.println("O aluno " + nrAluno + " teve " +  
        notaAluno + " valores");  
    System.out.println("+++++++");  
}
```

Se, no método `main` da classe `NotasFinaisLegivel`, tivéssemos, por exemplo, a instrução

```
imprimeInformacao(33333,10);
```

ela invocaria o método `imprimeInformação`, copiando os valores `33333` e `10` para os parâmetros `nrAluno` e `notaAluno`, respectivamente, e executaria as instruções que compõem o corpo desse método.

O efeito dessa invocação seria a impressão, no *standard output*, de:

```
O aluno 33333 teve 10 valores  
+++++++
```

É óbvio que se um procedimento não tem parâmetros, não se escreverá nada entre os parêntesis. O método `imprimeTresLinhas`, por exemplo, é invocado através da instrução

```
imprimeTresLinhas();
```

Já agora repare que `System.out.println(...)` é também uma invocação de um procedimento. A grande diferença é que esse procedimento não está definido no ficheiro `NotasFinaisLegivel.java` (mais tarde veremos onde).

NOTA: Um engano muito comum dos alunos é indicarem os tipos dos parâmetros aquando da invocação como, por exemplo, `imprimeInformacao(int 33333, int 10);`. Os tipos dos parâmetros são indicados na definição do método! Na invocação só temos que indicar os valores que os parâmetros devem tomar, sejam eles dados por literais, por nomes de variáveis ou por expressões mais complexas.

Vamos agora fazer uma variação do nosso programa `NotasFinaisLegivel`, começando por escrever três linhas de cruces e de seguida as informações de cada aluno, separadas também por três linhas de cruces:

```

public class NotasFinaisVariacao {

    public static void main (String [] args) {

        imprimeTresLinhas();

        // Nota final do aluno 12345
        int nota = notaFinal(16.2, 12.3, 14.4, 15.1);
        imprimeInformacaoB(12345, nota);

        // Nota final do aluno 54321
        nota = notaFinal(12.0, 14.2, 10.0, 10.5);
        imprimeInformacaoB(54321, nota);

        // Nota final do aluno 50000
        nota = notaFinal(18.0, 15.6, 10.2, 16.2);
        imprimeInformacaoB(50000, nota);

    } // fim do metodo main

    /*****
    /*****  METODOS  *****/
    /*****/

    /*
    * Nota final arredondada as unidades
    */
    static int notaFinal(double exame, double trab1,
                        double trab2, double trab3) {
        double notaCalculada =
            0.75 * exame + 0.25 * ((trab1 + trab2 + trab3) / 3) ;
        return (int)(notaCalculada + 0.5);
    }

    /*
    * Imprimir nota de aluno no standard output
    */
    static void imprimeInformacaoB(int nrAluno, int notaAluno) {
        System.out.println("O aluno " + nrAluno + " teve " +
            notaAluno + " valores");
        imprimeTresLinhas();
    }

    /*
    * Imprimir tres linhas com cruces no standard output
    */
    static void imprimeTresLinhas() {
        System.out.println("+++++++");
        System.out.println("+++++++");
        System.out.println("+++++++");
    }

}

```

Repare que o método `ImprimeInformaçãoB` invoca o método `imprimeTresLinhas`.

Este programa escreve o seguinte no *standard output*:

```
+++++
+++++
+++++
O aluno 12345 teve 16 valores
+++++
+++++
+++++
O aluno 54321 teve 12 valores
+++++
+++++
+++++
O aluno 50000 teve 17 valores
+++++
+++++
+++++
```

4.3. Funções

Até aqui falámos sobre procedimentos, que são métodos que fazem ações mas não calculam nenhum valor para dar como resposta à invocação ou, como se diz, não “devolvem” ou não “retornam” nenhum valor.

Vamos agora abordar os métodos que devolvem um valor, aos quais chamamos *funções*.

Como métodos que são, as funções têm muito em comum com os procedimentos. Explicamos em seguida as diferenças fundamentais.

4.3.1. Definição da função

Como uma função calcula e devolve um valor, há que indicar que tipo de valor é, para que quem invoca a função possa decidir o que fazer com ele – por exemplo, se precisamos de guardar o resultado da função numa variável, temos que saber de que tipo deve ser essa variável.

Na assinatura da função temos que indicar o *tipo* do valor que é devolvido.

O tipo de uma função, também chamado tipo de retorno, pode ser qualquer tipo reconhecido pelo Java (mais sobre isto adiante).

Tal como qualquer procedimento, uma função pode ter uma ou mais instruções (não há limite) e pode ter parâmetros.

Tem ainda que ter uma instrução especial que define o fim da execução do método e o valor a devolver como resultado da sua execução.

O corpo da função tem que ter (pelo menos) uma instrução da forma

```
return expressao;
```

A `expressão`, quando avaliada, deverá ter um valor do mesmo tipo que o tipo de retorno definido na assinatura da função.

No ficheiro `NotasFinaisLegivel` temos a definição de uma função:

```
static int notaFinal(double exame, double trab1,
                    double trab2, double trab3) {
    double notaCalculada =
        0.75 * exame + 0.25 * ((trab1 + trab2 + trab3) / 3) ;
    return (int)(notaCalculada + 0.5);
}
```

Repare que, na assinatura da função, em vez da palavra `void` que viu mais atrás (por exemplo, no método `imprimeInformacao`), tem a palavra `int` – com isto indicamos que o resultado que a função vai devolver é do tipo `int`.

Nesta função, o resultado é o valor arredondado da nota final calculada a partir da nota do exame – `exame` – e das notas parciais do projeto – `proj1`, `proj2` e `proj3`. Este valor é do tipo `int`, o que está de acordo com o tipo definido na assinatura da função para o valor de retorno.

4.3.2. Invocação da função

Já sabe que uma função, quando é executada, devolve um valor; quando queremos obter esse valor só temos que invocar a função.

É a expressão formada pela invocação da função que representa o resultado da função.

Enquanto que nos procedimentos a invocação é uma instrução (como se ordenássemos “faz o conjunto de ações que tem este nome”), nas funções a invocação constitui uma expressão; o valor desta expressão é o valor que é calculado e devolvido pela função.

Por isso, devemos fazer alguma coisa de útil com esse valor devolvido: por exemplo, imprimi-lo no ecrã, usá-lo como valor no cálculo de uma expressão mais elaborada, atribuir o seu valor a uma variável de tipo compatível ou usá-lo como valor a dar a um parâmetro na invocação de outro método.

No método `main` do ficheiro `NotasFinaisLegivel` temos a invocação da função `notaFinal` três vezes, com valores diferentes para os parâmetros. Em todas as vezes, atribuímos a uma variável o valor que é devolvido.

```
// Nota final do aluno 12345
int nota = notaFinal(16.2, 12.3, 14.4, 15.1);
imprimeInformacao(12345, nota);

// Nota final do aluno 54321
nota = notaFinal(12.0, 14.2, 10.0, 10.5);
imprimeInformacao(54321, nota);

// Nota final do aluno 50000
nota = notaFinal(18.0, 15.6, 10.2, 16.2);
imprimeInformacao(50000, nota);
```

Podíamos ter escrito diretamente o seu valor no ecrã:

```
System.out.println(notaFinal(16.2, 12.3, 14.4, 15.1)); OU
```

```
System.out.println("Nota do aluno 12345: " + notaFinal(16.2, 12.3, 14.4, 15.1) + " valores");
```

Podíamos ter usado o seu valor como termo numa expressão mais elaborada:

```
double percentagem = notaFinal(16.2, 12.3, 14.4, 15.1) / 20.0;
```

Podíamos ter usado diretamente o seu valor para instanciar um parâmetro na invocação de outro método, como em:

```
imprimeInformacao(12345, notaFinal(16.2, 12.3, 14.4, 15.1));
```

NOTA: Um erro algo comum dos alunos é fazerem a invocação de uma função como uma instrução por si só como, por exemplo,

```
notaFinal(16.2, 12.3, 14.4, 15.1);
```

Além de não ter sentido, isto é um desperdício de recursos computacionais, pois a função é invocada e executada e nada é feito com o seu resultado. É como se pedíssemos a um amigo para ele ir buscar uma coisa e, quando ele a trouxesse, lhe disséssemos que afinal não era necessária.

4.4. Memória e âmbito das variáveis

Qualquer variável declarada num método diz-se *local* a esse método.

Por exemplo, a variável `nota` é local ao método `main` pois é nele que é declarada.

Uma variável local a um método não está acessível fora desse método.

Por exemplo, a variável `nota` não pode ser usada nos outros métodos declarados na classe `NotasFinaisLegivel` – se usássemos a variável `nota` em qualquer outro método que não o `main`, o compilador daria o erro *“nota cannot be resolved to a variable”*.

Os parâmetros de um método funcionam como variáveis locais do método pois, por um lado, guardam valores e, por outro lado, só podem ser usadas dentro desse método.

Este carácter local das variáveis faz com que seja possível ter uma variável ou parâmetro declarados num dado método com um nome igual a outra declarada noutro método; elas são entidades diferentes!

Voltando ao exemplo acima, se tivéssemos declarado uma variável ou parâmetro também chamado `nota` no método `imprimeInformacao`, por exemplo, o compilador já não daria erro quando nos referíssemos a ela. Esta variável não teria nada a ver com a variável de igual nome no método `main`, ou seja, seriam duas variáveis completamente independentes.

Sempre que um método é invocado, é reservado espaço em memória para os seus parâmetros e variáveis locais. Logo que o método termina a sua execução, esse espaço deixa de estar reservado; logo, esses parâmetros e variáveis deixam de estar acessíveis.

Se voltar à secção 4.1.1., e simular a execução do programa `NotasFinaisLegivel`, confirmará a existência destas variáveis/parâmetros locais (repare na separação das variáveis e parâmetros de cada método através do uso de “caixas”).

Vamos então supor que o programa `NotasFinaisLegivel` está em execução e que a instrução que está a ser executada é a primeira instrução do método `imprimeInformacao` aquando da sua primeira invocação (provocada pela instrução `imprimeInformacao(12345,nota);` no `main`).

As variáveis locais a cada um dos dois métodos em execução estão agrupadas em caixas diferentes, cada uma etiquetada com o nome do método (à direita do nome do método é representado o número da instrução no programa; o que nos interessa agora é o conteúdo das variáveis e o método a que pertencem).

main:7	
nota	16
imprimeInformacao:35	
nrAluno	12345
notaAluno	16

Repare que os parâmetros `nrAluno` e `notaAluno` do método `imprimeInformacao` têm valores iguais aos usados na invocação, nomeadamente o literal `12345` e o valor da variável `nota` do `main`.

Andemos agora um pouco para a frente na execução, até à segunda instrução da função `notaFinal` aquando da sua segunda invocação (provocada por `nota = notaFinal(12.0,14.2,10.0,10.5);` no `main`).

main:10	
nota	16
notaFinal:28	
exame	12.0
proj1	14.2
proj2	10.0
proj3	10.5
notaCalculada	11.891666666666667

Repare que, como a execução da função ainda não terminou, a variável `nota` do `main` ainda tem o valor que lhe foi atribuído anteriormente. Logo que a instrução `return (int)(notaCalculada + 0.5);` seja executada, a função termina e devolve o valor 12, que será então atribuído à variável `nota` do `main`.

Continuemos a execução:

- imediatamente antes de a função terminar e retornar o valor calculado:

main:10	
nota	16
notaFinal:28	
exame	12.0
proj1	14.2
proj2	10.0
proj3	10.5
notaCalculada	11.891666666666667
Return value	12

- após a atribuição do valor da função à variável `nota` do `main`. Repare que o contexto de memória associado à função `notaFinal` já não está representado pois a função já terminou:

main:11

nota 12

4.5. Documentação

Quando um programador pondera invocar um método no seu código, deve ser capaz de perceber para que serve esse método e qual a forma de o usar (bem), sem ter que olhar para as instruções que o compõem.

Para este efeito, devemos adicionar a cada método alguma informação que ajude a clarificar

- o que faz, ou seja, qual o seu resultado e/ou os efeitos da sua execução;
- como se usa, ou seja, de que valores precisa (parâmetros) e que requisitos exige.

A *assinatura* do método – a parte que define o tipo do resultado, o nome do método, e o número, tipo e ordem dos seus parâmetros – já nos dá bastante informação.

Se um parâmetro é do tipo `int`, sabemos que temos que fornecer um inteiro quando invocamos o método; se o tipo de retorno é `String`, sabemos o que podemos fazer com o resultado, pois sabemos o seu tipo.

Além disso, se o nome do método e os nomes dos parâmetros forem sugestivos, a tarefa do programador que o vai usar fica facilitada.

Mas, na maior parte das vezes, só a assinatura não é suficiente: às vezes há muito mais a dizer acerca do resultado, dos parâmetros, etc.

Para este efeito, usamos comentários especiais e etiquetas (*tags*) *Javadoc*.

O *Javadoc* é uma aplicação que recebe um ficheiro de texto com extensão *java* (p. ex., `MyClass.java`) e gera um ficheiro *html* (p. ex., `MyClass.html`) contendo a documentação da classe, ao estilo da documentação das bibliotecas do Java.

O *Javadoc* interpreta alguns dos comentários que os programadores põem nas suas classes e métodos e usa-os para criar documentação que vai ser útil para todos os programadores que vão usar essas classes e métodos.

Os comentários que o *Javadoc* usa para a criação da documentação são aqueles que estão entre `/**` e `*/`.

Exemplo:

```
/**
 * Nota final arredondada ahs unidades, dadas as notas de exame e projeto
 * @param exame Nota do exame
 * @param proj1 Nota da primeira fase do projeto
 * @param proj2 Nota da segunda fase do projeto
 * @param proj2 Nota da terceira fase do projeto
 * @requires exame, proj1, proj2 e proj3 sao valores no intervalo [0,20]
 * @return A nota final, arredondada ahs unidades; o exame tem peso de 75% e
 *         a media de proj1, proj2 e proj3 tem peso de 25%
 */
static int notaFinal(double exame, double proj1,
                    double proj2, double proj3) {
    double notaCalculada =
        0.75 * exame + 0.25 * ((proj1 + proj2 + proj3) / 3) ;
    return (int)(notaCalculada + 0.5);
}
```

O texto que segue a tag `@param` é entendido como a descrição de um parâmetro.

O texto que segue a tag `@requires` é entendido como as restrições que existem à invocação do método.

O texto que segue a tag `@return` é entendido como a descrição do resultado da função.

O compilador de Java ignora os comentários, mas eles são muito úteis para todos os programadores:

- para o programador que criou o método, porque aumenta a legibilidade do código;
- para os programadores que vão invocar o método, porque têm acesso a essa informação (em formato *html* criada pela aplicação *Javadoc*).

Para gerar a documentação em formato *html* de uma classe `MyClass` contida no ficheiro `MyClass.java`, pode escrever na linha de comando:

```
javadoc -package -tag requires:m:''Requires:'' MyClass.java
```

Abaixo um *screenshot* da parte correspondente ao método `notaFinal` na página *html* gerada com este comando para a nossa classe `NotasFinaisLegivel`:

notaFinal

```
static int notaFinal(double exame,  
                    double proj1,  
                    double proj2,  
                    double proj3)
```

Nota final arredondada ahs unidades, dadas as notas de exame e projeto

Parameters:

exame - Nota do exame

proj1 - Nota da primeira fase do projeto

proj2 - Nota da segunda fase do projeto

proj2 - Nota da terceira fase do projeto

Returns:

A nota final, arredondada ahs unidades; o exame tem peso de 75% e a media de proj1, proj2 e proj3 tem peso de 25%

Requires:

exame, proj1, proj2 e proj3 sao valores no intervalo [0,20]

4.6. Repositórios de métodos úteis

As funções matemáticas que vimos na Secção 3.5.4 estão todas “arrumadas” numa classe de nome `Math`. É por isso que, quando as invocamos, temos que preceder o seu nome de `Math` e um ponto. Relembre o seu uso:

```
int i = 3;  
double raiz = Math.sqrt(17.0 * i);  
System.out.println( Math.sin(raiz));  
int k = 3 * Math.round(Math.log10(raiz));
```

A classe `Math` pertence à biblioteca base do Java, e por isso está disponível para todos os programadores que instalem o *JDK*.

A ideia que levou à criação desta classe – juntar várias funções úteis da mesma natureza numa mesma classe que todos os programadores podem usar – pode ser aproveitada por nós. À medida que vamos desenvolvendo procedimentos e funções que pensamos serem úteis para vários programas, vamos “arrumando-os” numa classe especial que pode depois ser utilizada por outros programas, feitos por nós ou pelo nosso grupo de trabalho.

Por exemplo, se acharmos que os métodos `imprimeTresLinhas()` e `notaFinal(...)` podem ser úteis noutros contextos, então podemos criar uma classe chamada, por exemplo, `MinhasUtilidades` – e nela definir estes métodos.

Desta maneira, nos programas em que quisermos usar alguma destas funções só teremos que fazer a sua invocação indicando o nome da classe onde estão definidos.

No programa `NotasFinaisLegivel`, por exemplo,

- retiramos as definições desses métodos, porque já estarão definidas na classe `MinhasUtilidades` e
- precedemos as suas invocações de `MinhasUtilidades.` como em:

```
MinhasUtilidades.imprimeTresLinhas();
int nota = MinhasUtilidades.notaFinal(16.2, 12.3, 14.4, 15.1);
```

Quando quisermos fazer alguma melhoria a algum desses métodos, basta alterarmos a sua definição na classe `MinhasUtilidades`. Não temos que mexer nas classes que o usam.

ATENÇÃO: Para que os métodos definidos na classe `MinhasUtilidades` possam ser usados a partir de qualquer outra classe, a palavra `public` deve ser adicionada imediatamente antes da palavra `static` (tal como temos no método `main`). Mais adiante aprenderemos mais sobre este assunto.

5. O tipo de dados `boolean` e comandos condicionais

É muito importante para qualquer programador ter a possibilidade de definir comportamentos alternativos no seu programa como, por exemplo, “se *A* for verdade, faz *X*; caso contrário (isto é, se *A* não for verdade), faz *Y*”.

Para isso há que conseguir determinar se uma dada expressão é verdadeira ou falsa e ter uma instrução que permita definir comportamento condicional.

5.1. O tipo primitivo `boolean`

O tipo das expressões cujo valor é verdade ou falso é o tipo de dados `boolean`.

Enquanto que os conjuntos de valores definidos pelos tipos `int` e `double`, por exemplo, são muito grandes, o conjunto de valores do tipo `boolean` tem somente dois elementos `{true, false}`.

Os literais deste tipo são, portanto, somente `true` e `false`.

As operações que podemos fazer com operandos do tipo `boolean` são: *conjunção*, com o operador `&&`, *disjunção*, com o operador `||`, e *negação*, com o operador `!`. Estes três operadores chamam-se *operadores lógicos* e têm operandos e resultado do tipo `boolean`.

O significado, ou semântica, dos operadores lógicos é o mesmo que aprendeu na lógica matemática:

- a *conjunção* só é verdadeira se ambos os operandos forem verdadeiros (nos outros casos é falsa);
- a *disjunção* só é falsa se ambos os operandos forem falsos (nos outros casos é verdadeira);

- a negação é falsa se o operando é verdadeiro e vice-versa.

As seguintes instruções imprimem os valores `false`, `true` e `false` no ecrã:

```
boolean a = true;
boolean b = false;
System.out.println(a && b);
System.out.println(a || b);
System.out.println(!a);
```

Temos ainda os seguintes *operadores relacionais* que, embora não tenham operandos do tipo `boolean`, têm resultado do tipo `boolean`. São eles `>`, `<`, `>=`, `<=`, `==` e `!=`.

Sendo `expr1` e `expr2` duas expressões de um mesmo tipo numérico (`int` ou `double`, para já), a semântica dos seguintes operadores relacionais é a esperada:

```
expr1 > expr2      // o valor de expr1 eh maior que o de expr2
expr1 < expr2      // o valor de expr1 eh menor que o de expr2
expr1 >= expr2     // o valor de expr1 eh maior ou igual que o de expr2
expr1 <= expr2     // o valor de expr1 eh menor ou igual que o de expr2
```

Os dois operadores seguintes são também operadores relacionais que podem ter operandos de tipos numéricos, `boolean` ou de outros tipos. O resultado é do tipo `boolean`.

```
expr1 == expr2     // o valor de expr1 eh igual ao de expr2
expr1 != expr2     // o valor de expr1 eh diferente do de expr2
```

Estes operadores `==` e `!=` também operam sobre *strings* mas o resultado não é o esperado (mais sobre isto adiante).

Estas expressões, sendo do tipo `boolean`, podem constituir operandos para os operadores lógicos.

Por exemplo,

- a expressão `x > 0 && x < 10` é `true` só se `x` é maior que zero e `x` é menor que 10
- a expressão `x > 0 || x % 3 == 0` é `true` sempre que `x` é maior que zero ou `x` é divisível por 3
- a expressão `!(x <= 0)` é `true` se `x <= 0` é `false`, ou seja, se `x` é positivo
- dada uma variável `x` do tipo `double`, a expressão `x >= 3.5 && x < 4.8` tem o valor `true` se `x` está no intervalo `[3.5,4.8[`.

O seguinte programa imprime os valores `true`, `false`, `true`, `true`, `false`, `false`, `true` e `false` no ecrã:

```
public class Expressions4 {

    public static void main (String [] args) {

        int i = 2;
        int j = 6;
        System.out.println(i + 3 < i * 3);
        System.out.println(i == j);
        System.out.println(i + 4 == j);

        // o resultado de um operador relacional eh boolean, portanto
        // pode ser guardado numa variavel de tipo boolean
        boolean expr = i <= j + 3;
        System.out.println(expr);
```

```

        System.out.println(expr && i != 2);
        System.out.println(i <= 2 && j <= 2);
        System.out.println(i <= 2 || j <= 2);
        System.out.println(!(i <= 2 || j <= 2));
    }
}

```

NOTA: um erro comum que se comete no início da aprendizagem é usar `=` em vez de `==` para exprimir o operador de igualdade. Lembre-se que `=` é a instrução de atribuição!

No que diz respeito à prioridade das operações, os operadores relacionais têm todos a mesma prioridade e são associativos à esquerda (por isso quando há vários numa expressão, são avaliados da esquerda para a direita).

Como pode ver no quadro das prioridades do Capítulo 3, a prioridade dos operadores relacionais é menor que, por exemplo, a dos operadores aritméticos (por isso na expressão `i <= j + 3` a soma é avaliada antes do operador `<=`).

Os operadores lógicos não têm todos a mesma prioridade – a negação é o que tem maior prioridade, seguido da conjunção; a disjunção é, desses operadores, o que tem menor prioridade. Para que a negação não seja avaliada em primeiro lugar, na expressão `!(i <= 2 || j <= 2)` usamos parêntesis à volta da expressão que queremos negar. Se os parêntesis não estivessem lá, a primeira operação a ser avaliada seria `!i`, o que não é legal pois `i` não é do tipo `boolean`.

Como qualquer tipo de dados, também o tipo `boolean` pode servir para tipo de retorno de uma função. Por exemplo, o seguinte programa imprime os valores `true`, `false` e `true` no ecrã:

```

public class ExemploFuncaoBoolean {

    public static void main (String [] args) {

        int i = 2;
        int j = 7;
        System.out.println(ehPar(i));
        System.out.println(ehPar(j));
        System.out.println(ehPar(i * j));
    }

    /**
     * Um numero inteiro eh par?
     * @param num
     * @return true se num eh par; false caso contrario
     */
    static boolean ehPar(int num) {
        return num % 2 == 0;
    }
}

```

Repare na única instrução da função `ehPar`. É a instrução de retorno da função que, como já referido, calcula a expressão à direita da palavra `return` e termina a execução do método devolvendo como resultado o valor dessa expressão.

A expressão `num % 2 == 0` é do tipo `boolean` pois o seu resultado só poderá ser `true` ou `false`. Esta expressão tem o valor `true` quando o resto da divisão de `num` por 2 é zero, ou seja, quando `num` é par.

Então, o valor devolvido pela função é `true` se `num` é par e `false` se `num` não é par.

5.2. Execução condicional

Agora que já sabemos escrever expressões Java cujos valores são verdade ou falso, já podemos verificar se uma dada condição é verdadeira e, com isso, definir comportamentos condicionais nos nossos programas.

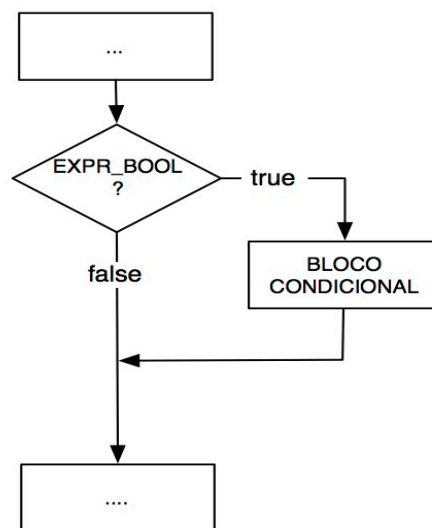
5.2.1. A instrução `if`

Uma forma de fazer isso é usando a instrução `if`, que determina um fluxo de execução condicionado por uma expressão que pode tomar valores verdade ou falso.

```
if (expr_bool)
    bloco_condicional;
```

A `expr_bool` é uma expressão cujo tipo é `boolean`, ou seja, cujo resultado é do tipo `boolean`.

O `bloco_condicional` é composto por uma sequência de instruções entre chavetas.



A *semântica*, ou significado, desta instrução, ilustrada na figura acima, é:

A expressão `expr_bool` é avaliada e,

- se o seu valor for `true`, então o `bloco_condicional` é executado e de seguida o fluxo do programa continua na instrução a seguir ao `bloco_condicional`;
- se o seu valor é `false`, o `bloco_condicional` não é executado e o fluxo do programa continua na instrução a seguir ao `bloco_condicional`.

Como a instrução `if` define um novo bloco de instruções, devemos usar indentação ao escrever esse bloco, para aumentar a legibilidade. Os espaços que escrevemos a mais para indentar o bloco de instruções não afetam em nada o que o programa realmente faz quando executado.

Voltando à classe `NotasFinaisLegivel` do Capítulo 4, como faríamos para imprimir uma dada mensagem no ecrã somente no caso em que a nota do aluno fosse maior ou igual a 15? As seguintes instruções, inseridas no método `main`, produzem esse efeito:

```
...
// nota maior ou igual a 15?
if (nota >= 15){
    System.out.println("Quadro de honra");
}
```

5.2.2. Blocos de uma só instrução

Se tivermos um bloco condicional composto por uma só instrução, então as chavetas são opcionais; no entanto, por uma questão metodológica, aconselhamos a incluir sempre as chavetas para delimitar o `bloco_condicional`, mesmo quando tem uma só instrução.

Quando decidimos não usar chavetas para delimitar blocos com uma só instrução, devemos ter presente que isso pode ser fonte de erros indesejáveis. Temos muitas vezes que alterar pedaços de código já feitos. Suponha que queremos, por exemplo, acrescentar uma ação num bloco de uma instrução `if`. Suponha ainda que o `bloco_condicional` dessa instrução `if` é composto por uma só instrução e que não é delimitado por chavetas. Se, ao acrescentar uma nova instrução ao `bloco_condicional`, nos esquecemos de acrescentar as chavetas, obtemos um efeito totalmente diferente do desejado.

Vamos ver um exemplo. Suponha que tem o seguinte pedaço de código, em que o `bloco_condicional` não está envolto em chavetas:

```
if (x < 6)
    System.out.println("Menor que 6");
    System.out.println("Adeus");
```

Se `x` tiver um valor menor que 6 as frases *Menor que 6* e *Adeus* serão escritas no ecrã. Se o valor de `x` for maior ou igual a 6, somente a frase *Adeus* será escrita.

Inspecionemos agora os seguintes casos, resultantes de adicionar uma instrução ao excerto anterior:

```
if (x < 6) {
    System.out.println("Menor que 6");
    System.out.println("Eu bem dizia!");
}
System.out.println("Adeus");
```

Se `x` tiver um valor menor que 6 as frases *Menor que 6*, *Eu bem dizia!* e *Adeus* serão escritas no ecrã. Se o valor de `x` for maior ou igual a 6, somente a frase *Adeus* será escrita.

```
if (x < 6)
    System.out.println("Menor que 6");
    System.out.println("Eu bem dizia!");
    System.out.println("Adeus");
```

Se `x` tiver um valor menor que 6 as frases *Menor que 6*, *Eu bem dizia!* e *Adeus* serão escritas no ecrã. Se o valor de `x` for maior ou igual a 6, as frases *Eu bem dizia!* e *Adeus* serão escritas no ecrã.

Repare que, não havendo chavetas, o bloco do `if` é composto somente pela instrução `System.out.println("Menor que 6");`.

A indentação apresentada sugere, erradamente, que a instrução seguinte também pertence ao bloco. Mas (relembre Secção 3.6.3) a indentação não tem qualquer valor semântico em Java! O efeito do pedaço de código seguinte é exatamente o mesmo que no exemplo anterior.

```
if (x < 6)
    System.out.println("Menor que 6");
System.out.println("Eu bem dizia!");
System.out.println("Adeus");
```

Repare que a única diferença deste caso para o anterior é a indentação da instrução `System.out.println("Eu bem dizia!");` que já sugere que essa instrução não faz parte do `bloco_condicional`.

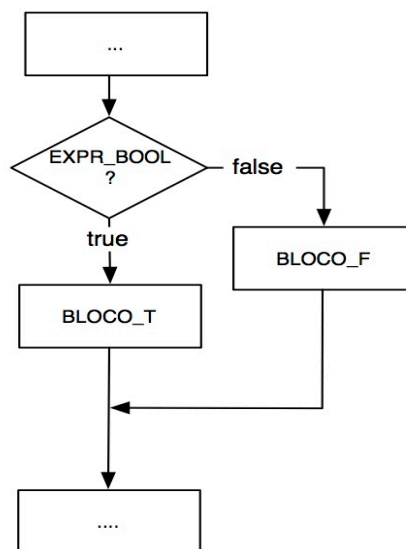
5.2.3. A instrução `if-else`

A instrução `if-else` também nos permite programar comportamento condicional, mais precisamente definindo duas alternativas mutuamente exclusivas:

```
if (expr_bool)
    bloco_T;
else
    bloco_F;
```

Como a `expr_bool` é uma expressão cujo tipo é `boolean`, então qualquer que seja o seu valor, exatamente um dos blocos é executado.

Cada um dos `bloco_T` e `bloco_F` é composto por uma sequência de instruções entre chavetas.



A semântica desta instrução, ilustrada na figura acima, é:

a expressão `expr_bool` é avaliada

- se o seu valor for `true`, então o `bloco_T` é executado;
- caso contrário o `bloco_F` é executado;
- em qualquer dos casos, o fluxo de execução do programa continua na instrução que vem a seguir ao `bloco_F`.

Repare que, se quisermos saber qual é exatamente a condição sob a qual é executado o bloco `bloco_F` basta calcularmos a negação da `expr_bool`. Por exemplo, se `expr_bool` for `x > 3` então o `bloco_F` será executado nos casos em que `x <= 3`.

Relembre que devemos usar indentação ao escrever os blocos condicionais, para aumentar a legibilidade, sabendo, no entanto, que os espaços que escrevemos a mais para indentar os blocos de instruções não afetam em nada o significado do programa.

Seguem-se alguns pedaços de código java que exemplificam o uso da instrução `if-else`.

```

if (x < 1) {
    System.out.println("Menor que 1");
} else {
    System.out.println("Maior ou igual a 1");
}
System.out.println("Adeus");

```

Se `x` tiver um valor menor que 1, as frases *Menor que 1* e *Adeus* serão escritas no ecrã. Se o valor de `x` não for menor que 1 (ou seja, se for maior ou igual a 1) as frases *Maior ou igual a 1* e *Adeus* serão escritas no ecrã.

Experimente executar o programa completo na versão online deste livro.

No seguinte excerto de programa,

```

if (x < 1) {
    System.out.println("Menor que 1");
    System.out.println("Eu bem dizia!");
} else {
    System.out.println("Maior ou igual a 1");
}
System.out.println("Adeus");

```

se `x` tiver um valor menor que 1 as frases *Menor que 1*, *Eu bem dizia!* e *Adeus* serão escritas no ecrã. Se o valor de `x` não for menor que 1, as frases *Maior ou igual a 1* e *Adeus* serão escritas no ecrã.

No seguinte,

```

if (x < 1) {
    System.out.println("Menor que 1");
} else {
    System.out.println("Maior ou igual a 1");
    System.out.println("Eu bem dizia!");
}
System.out.println("Adeus");

```

se `x` tiver um valor menor que 1 as frases *Menor que 1* e *Adeus* serão escritas no ecrã. Se o valor de `x` não for menor que 1, as frases *Maior ou igual a 1*, *Eu bem dizia!* e *Adeus* serão escritas no ecrã.

Finalmente, no seguinte,

```

if (x < 1) {
    System.out.println("Menor que 1");
    System.out.println("Eu bem dizia!");
} else {
    System.out.println("Maior ou igual a 1");
    System.out.println("Eu bem dizia!");
}
System.out.println("Adeus");

```

se `x` tiver um valor menor que 1 as frases *Menor que 1*, *Eu bem dizia!* e *Adeus* serão escritas no ecrã. Se o valor de `x` não for menor que 1, as frases *Maior ou igual a 1*, *Eu bem dizia!* e *Adeus* serão escritas no ecrã.

O seguinte excerto de programa tem um efeito equivalente ao anterior.


```

if (x < 1) {
    System.out.println("Menor que 1");
} else {
    System.out.println("Maior ou igual a 1");
}
System.out.println("Eu bem dizia!");
System.out.println("Adeus");

```

O que se fez foi extrair a instrução `System.out.println("Eu bem dizia!");` dos dois blocos alternativos e colocá-la fora dos blocos, pois ela seria executada qualquer que fosse o valor da condição do `if`.

Aqui, de novo, se decidirmos não usar chavetas para delimitar blocos com uma só instrução, devemos ter cuidado para não introduzirmos erros.

As instruções seguintes têm exatamente o mesmo efeito que o último excerto apresentado.

```

if (x < 1)
    System.out.println("Menor que 1");
else
    System.out.println("Maior ou igual a 1");
System.out.println("Eu bem dizia!");
System.out.println("Adeus");

```

Já que os blocos alternativos só tinham uma instrução cada um, pôde-se prescindir das chavetas.

Atente no bloco seguinte de instruções:

```

if (x < 1)
    System.out.println("Menor que 1");
else
    System.out.println("Maior ou igual a 1");
    System.out.println("Eu bem dizia!");
System.out.println("Adeus");

```

Aqui, não havendo chavetas no bloco do `else`, ele é composto somente pela instrução `System.out.println("Maior ou igual a 1");` mesmo que a indentação apresentada sugira que a instrução seguinte também pertence ao bloco. Por isso, se `x` tiver um valor menor que 1, as frases *Menor que 1*, *Eu bem dizia!* e *Adeus* serão escritas no ecrã. Se o valor de `x` não for menor que 1, as frases *Maior ou igual a 1*, *Eu bem dizia!* e *Adeus* serão escritas no ecrã.

Veja a execução "ao vivo" na versão online deste livro.

Considere agora o seguinte pedaço de código:

```

if (x < 1)
    System.out.println("Menor que 1");
    System.out.println("Eu bem dizia!");
else
    System.out.println("Maior ou igual a 1");
System.out.println("Adeus");

```

Neste caso ocorreria um erro de compilação pois o bloco do `if` é composto somente por uma instrução (não há chavetas), que é `System.out.println("Menor que 1");`. De seguida vem a instrução `System.out.println("Eu bem dizia!");`. Até aqui tudo bem para o compilador. O pior é quando deteta a palavra `else` “desgarrada”, ou seja, sem estar “emparelhada” com nenhum `if`...

5.2.4. Condição alternativa

A `expr_bool` nas instruções `if` e `if-else` pode ser uma expressão do tipo `boolean` tão complexa quanto desejemos.

Na instrução `if-else` temos que ter cuidado, no entanto, para percebermos bem quais são as situações em que é executado o `bloco_F` ou, o que é o mesmo, o que significa a negação da `expr_bool`. Para isso devemos conhecer bem a semântica dos operadores lógicos e relacionais.

Por exemplo, na seguinte instrução `if-else`, se `x` e `y` forem variáveis dos tipos `int` e `double`, respetivamente, em que condições será executada a instrução `System.out.println("Adeus")`?

```
if ((x > 1 && x <= 3) || x > 100) && y < 2.3){
    System.out.println("Olah");
} else {
    System.out.println("Adeus");
}
```

A resposta a essa pergunta é: quando a negação de `(x > 1 && x <= 3) || x > 100) && y < 2.3` for verdadeira. Sabendo que, pelas leis de De Morgan, a negação da conjunção é a disjunção das negações e a negação da disjunção é a conjunção das negações, temos que:

`!((x > 1 && x <= 3) || x > 100) && y < 2.3` é equivalente a

`!(x > 1 && x <= 3) || x > 100 || !(y < 2.3)` que, por sua vez, é equivalente a

`(!(x > 1 && x <= 3) && !(x > 100)) || (y >= 2.3)` por sua vez equivalente a

`((!(x > 1) || !(x <= 3)) && (x <= 100)) || (y >= 2.3)` por sua vez equivalente a

`((x <= 1) || (x > 3)) && (x <= 100) || (y >= 2.3)`

De qualquer forma, devemos ter cuidado para não usarmos condições muito complexas na instrução `if-else` para não introduzirmos erros lógicos acidentalmente e mantermos o programa legível.

5.2.5. Instruções condicionais “encaixadas”

Uma instrução `if-else` permite-nos tratar de situações em que temos duas ações alternativas. Sempre que precisamos de trabalhar com mais do que duas alternativas, podemos “encaixar” instruções condicionais.

Se quisermos, por exemplo, saber se um dado número é positivo, nulo ou negativo, quantas perguntas temos que fazer? Duas no pior caso, aquele em que só acertamos no fim – se as respostas forem ambas negativas então já sabemos que é a terceira hipótese que é a correta.

Vamos supor que queremos escrever numa variável `resultado` as strings “Positivo”, “Negativo” ou “Nulo”, conforme o valor de uma variável `n`. Podemos começar por perguntar se o valor de `n` é positivo:

```
if (n > 0){
    resultado = "Positivo";
} else
    ?????
```

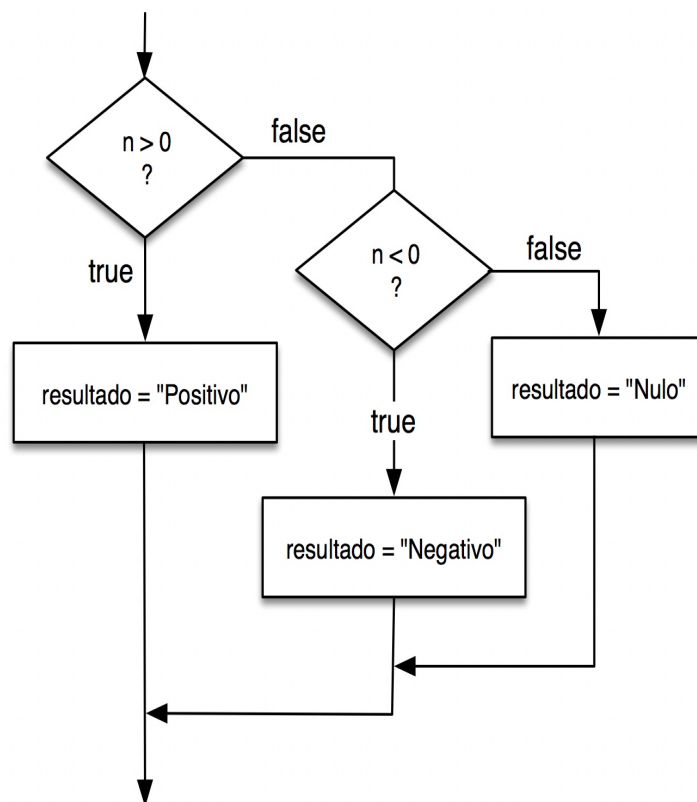
Que instrução devemos ter no lugar dos `?????`?

Depende! Se `n` tiver o valor zero queremos ter `resultado = "Nulo"`; . Se `n` for negativo queremos ter `resultado = "Negativo"`;

Então, a instrução que devemos ter no lugar dos `????` é uma outra instrução condicional, que nos permita decidir entre as duas alternativas que restam.

Obtemos o excerto de código seguinte (ao qual corresponde o fluxo de execução ilustrado em baixo) :

```
if (n > 0){
    resultado = "Positivo";
} else {
    if (n < 0){
        resultado = "Negativo";
    } else {
        resultado = "Nulo";
    }
}
```



Outra solução para o mesmo caso poderia ser:

```
if (n >= 0){
    if (n > 0){
        resultado = "Positivo";
    } else {
        resultado = "Nulo";
    }
} else {
    resultado = "Negativo";
}
```

Considere este último bloco de `ifs` encaixados; é importante saber responder às questões que se seguem:

- *Questão:* Sob que condição é executado o bloco `{resultado = "Negativo";}` do `else` correspondente ao primeiro `if`?

Resposta: Sob a negação de $n \geq 0$, ou seja, $n < 0$.

- *Questão:* Sob que condição é executado o bloco `{resultado = "Positivo";}` do segundo `if`?

Resposta: Basta fazer a conjunção de $n \geq 0$ com $n > 0$, o que dá $n > 0$.

- *Questão:* Sob que condição é executado o bloco `{resultado = "Nulo";}` do `else` correspondente ao segundo `if`?

Resposta: Basta fazer a conjunção de $n \geq 0$ com a negação de $n > 0$, o que dá $n \geq 0 \ \&\& \ n \leq 0$, que é equivalente a $n == 0$.

Tenha em atenção o facto de que um ramo `else` emparelha sempre com o `if` imediatamente anterior. Por isso, num caso como o seguinte:

```
int n = 100;
String resultado = "Fora";
if (n >= 0){
    if (n < 100){
        resultado = "No intervalo [0,99]";
    } else {
        resultado = "Negativo";
    }
}
```

a variável `resultado` ficaria com o valor `"Negativo"` pois o `else` corresponderia à condição $n \geq 0 \ \&\& \ n \geq 100$. Se quisermos que este `else` emparelhe com o `if` exterior, temos que usar as chavetas de modo a delimitar bem o fim do `if` interior.

Com o código seguinte a variável `resultado` ficaria com o valor `"Fora"`:

```
int n = 100;
String resultado = "Fora";
if (n >= 0) {
    if (n < 100){
        resultado = "No intervalo [0,99]";
    }
} else {
    resultado = "Negativo";
}
```

Imagine agora mais níveis de “encaixe” – a certa altura teríamos pouco espaço à direita para escrevermos as nossas instruções além de que a legibilidade seria seriamente afetada.

É muito comum não usar chavetas nos casos em que temos um `if` como única instrução do bloco de um `else`. Se adotarmos essa simplificação e, além disso, alinharmos os `if-else` encaixados, obtemos uma série de instruções mais compacta e legível.

A primeira versão do exemplo que determina se um número é positivo, negativo ou nulo (acima) poderia então ser escrito da seguinte forma:

```

if (n > 0){
    resultado = "Positivo";
} else if (n < 0){
    resultado = "Negativo";
} else {
    resultado = "Nulo";
}

```

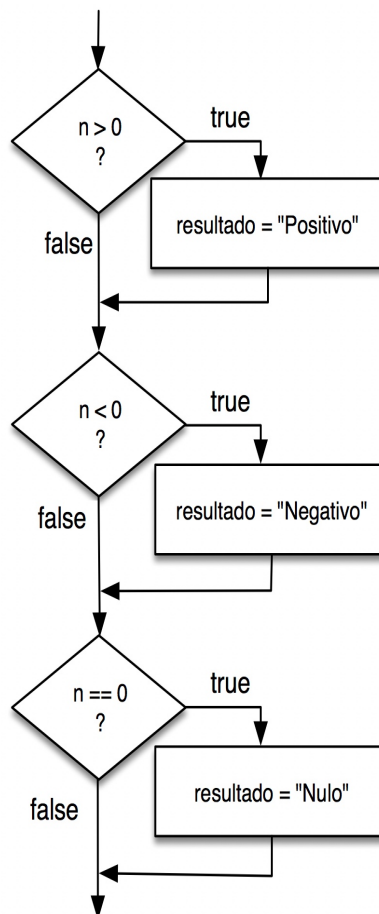
Será que para obter mais do que duas ações alternativas temos que ter sempre instruções `if` encaixadas?

Não necessariamente. Podemos usar instruções `if` sequenciais como as que se seguem (ver também fluxo de execução ilustrado a seguir) embora seja pior solução:

```

if (n > 0){
    resultado = "Positivo";
}
if (n < 0){
    resultado = "Negativo";
}
if (n == 0){
    resultado = "Nulo";
}

```



Porque é que é pior? Esta opção obriga a testar sempre as três condições, mesmo que alguma das anteriores já tenha sido verdadeira.

Mas por outro lado é mais legível pois estão explícitas todas as condições relevantes. De qualquer forma, tem que se ter cuidado, pois se as condições não forem mutuamente exclusivas, não obtemos o mesmo comportamento da versão que usa os `else-if`.

Finalmente, o programa seguinte imprime as palavras *Positivo*, *Negativo* e *Nulo* no ecrã.

```
public class TresEscolhas {

    public static void main (String [] args) {

        int i = 2;
        int j = -7;
        System.out.println(positividade(i));
        System.out.println(positividade(j));
        System.out.println(positividade(i - 2));
    }

    /**
     * Um numero eh positivo, nulo ou negativo?
     * @param num
     * @return "Positivo" se num > 0;
     *         "Nulo" se num == 0
     *         "Negativo" se num < 0
     */
    static String positividade(int num) {
        String resultado;
        if (num > 0){
            resultado = "Positivo";
        } else if (num < 0){
            resultado = "Negativo";
        } else {
            resultado = "Nulo";
        }
        return resultado;
    }
}
```

Experimente executá-lo "ao vivo" na versão online deste livro.

Retomemos agora a função `ehPar` definida na Secção 5.1. O seguinte programa imprime as frases *2 eh par* e *5 nao eh par* no ecrã:

```
public class BooleanFunctionExample2 {

    public static void main (String [] args) {
        int i = 2;
        imprimeParidade(i);
        imprimeParidade(i + 3);
    }

    /**
     * Imprime mensagem acerca da paridade de um dado numero
     * @param num
     */
    static void imprimeParidade(int num) {
        if (ehPar(num)){
            System.out.println(num + " eh par");
        } else {
            System.out.println(num + " nao eh par");
        }
    }
}
```

```

    }

    /**
     * Um numero inteiro eh par?
     * @param num
     * @return true se num eh par; false caso contrario
     */
    static boolean ehPar(int num) {
        return num % 2 == 0;
    }
}

```

Repare na condição da instrução `if` no método `imprimeParidade`. É uma invocação da função `ehPar`. Como a função `ehPar` tem tipo `boolean`, então a expressão `ehPar(num)` tem tipo `boolean` e por isso pode ser usada como condição de uma instrução `if`.

Experimente executá-lo ao vivo na versão online deste livro.

5.2.6. Retorno de funções

A função `ehPar` podia ter sido codificada assim (caso 1):

```

static boolean ehPar(int num) {
    boolean result;
    if (num % 2 == 0){
        result = true;
    } else {
        result = false;
    }

    return result;
}

```

Nesta versão estamos a ser redundantes pois a instrução `if` está a fazer o seguinte: avalia a expressão `num % 2 == 0` e testa o seu valor: se for verdadeiro, escreve o valor `true` na variável `result`; se for falso, escreve o valor `false` em `result`. Então, mais vale escrever diretamente o valor da expressão `num % 2 == 0` na variável `result` e terminar a função devolvendo esse valor.

```

static boolean ehPar(int num) {
    boolean result = num % 2 == 0;
    return result;
}

```

Mas agora a variável `result` é desnecessária, pois o seu valor só é usado uma vez – como resultado da função. Então mais vale escolhermos a versão original da Secção 5.1:

```

static boolean ehPar(int num) {
    return num % 2 == 0;
}

```

Podíamos, ainda, ter codificado a função `ehPar` assim (caso 2):

```
static boolean ehPar(int num) {
    if (num % 2 == 0){
        return true;
    } else {
        return false;
    }
}
```

Nesta versão estamos também a ser redundantes pois a instrução `if` avalia a expressão `num % 2 == 0` e testa o seu valor: se for verdadeiro, termina a função devolvendo o valor `true`; se for falso, termina a função devolvendo o valor `false`. Então, mais vale terminar a função devolvendo diretamente o valor da expressão `num % 2 == 0`. Que é precisamente a versão da Secção 5.1!

```
static boolean ehPar(int num) {
    return num % 2 == 0;
}
```

Já vimos no capítulo anterior que as funções têm que ter uma instrução `return`. Podem ter várias, aliás, como exemplificado numa das versões da função `ehPar`.

No entanto, quantas mais instruções `return` tiver uma função, mais complicado se torna percebê-la, ou seja, a legibilidade da função pode ficar comprometida.

Vamos então, sempre que possível, ter uma única instrução `return` nas nossas funções.

Se o valor devolvido depender de uma ou mais instruções condicionais, então podemos sempre definir uma variável local à função, à qual será atribuído um valor que depende das diversas condições; no fim da função retornamos o valor dessa variável: lembre o exemplo da função positividade:

```
static String positividade(int num) {
    String resultado;
    if (num > 0){
        resultado = "Positivo";
    } else if (num < 0){
        resultado = "Negativo";
    } else {
        resultado = "Nulo";
    }
    return resultado;
}
```

5.3. Ainda a documentação

O seguinte método serve para verificar se um inteiro tem um só algarismo:

```
static boolean isSingleDigit(int x) {
    return x >= 0 && x < 10;
}
```

O método retorna `false` sempre que invocado com um valor negativo para o parâmetro, mesmo que esse valor tenha só um dígito (p.ex. -6). Não está preparado para trabalhar com valores negativos do parâmetro.

Em casos como este, os programadores que vão usar o método devem ser informados que o método não faz o que é esperado quando é invocado com valores negativos no parâmetro.

Fazemos isso adicionando alguns comentários à definição do método:

```
/**
 * Um dado numero natural tem um soh algarismo?
 * @param x O numero a investigar
 * @return true se x tem um soh algarismo; false caso contrario
 * @requires x >= 0
 */
static boolean isSingleDigit(int x) {
    return x < 10;
}
```

Assim, os programadores que usem este método já ficam a saber que só o devem invocar com um valor positivo ou nulo para o parâmetro. E já que avisamos os potenciais invocadores deste método que o valor tem que ser positivo ou nulo, não precisamos de testar essa condição dentro do método (por isso só verificamos que é menor que 10).

Também podíamos ter feito assim:

```
/**
 * Um dado numero tem um soh algarismo?
 * @param x O numero a investigar
 * @return true se x >=0 e x tem um soh algarismo; false caso contrario
 */
static boolean isSingleDigit(int x) {
    return x >= 0 && x < 10;
}
```

Neste caso o programador fica a saber que não há qualquer restrição sobre o valor do parâmetro (para além de ter que ser do tipo `int`) e que se for negativo ou tiver mais que um dígito o método retorna `false`.

5.4. Comparação de valores numéricos reais

Quando comparamos valores inteiros podemos usar o operador relacional `==`. Quando comparamos valores com parte decimal (`double` ou `float`) temos que ter cuidado pois, devido ao número limitado de algarismos significativos nestes tipos, pequeníssimos erros podem ser introduzidos que fazem com que a comparação não produza o efeito desejado.

Estes erros podem ser introduzidos não só na representação dos números em decimais vírgula flutuante, mas também nos cálculos que vamos fazendo ao longo do programa.

O operador `==` só resulta num valor `true` quando os valores comparados são exatamente iguais. A execução do seguinte pedaço de código escreveria *diferentes!* no ecrã:

```
double x = 10.1;
double y = x - 7;    // y fica igual a 3.1
double w = x / y;    // divide x por 3.1
w = w * 3.1;         // multiplica w por 3.1 -- ficaria igual a 10.1

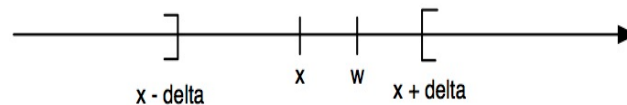
if (w == 10.1){
    System.out.println("iguais!");
} else {
    System.out.println("diferentes!");
}
```

E a execução da instrução

```
System.out.println("x = " + x + " w = " + w);
```

escreveria $x = 10.1$ $w = 10.100000000000001$ no ecrã:

Devemos então comparar números com parte decimal de forma não exata, ou seja, definindo uma tolerância máxima até à qual é aceitável considerar que os números são iguais:



Se chamarmos `delta` à tolerância, então a condição `w > x - delta && w < x + delta` define os valores `w` que consideramos iguais a `x`.

Esta condição é equivalente a esta outra `w - x > -delta && w - x < delta` que é exatamente a que resulta de desdobrar o módulo `|w - x| < delta`.

O seguinte método permite decidir se dois números com parte decimal são iguais a menos de uma tolerância `delta`.

```
/**
 * Dois numeros sao iguais a menos de uma dada tolerancia?
 * @param x1
 * @param x2
 * @param delta A tolerancia
 * @return true se a diferenca entre x1 e x2 eh menor que delta;
 *         false caso contrario
 */
static boolean iguais(double x1, double x2, double delta) {
    return Math.abs(x1 - x2) < delta;
}
```

A execução das instruções seguintes, por exemplo, inseridas no método `main` da classe contendo o método `iguais`, escreveria *iguais!* no ecrã.

```
double x = 10.1;
double y = x - 7;    // y fica igual a 3.1
double w = x / y;    // divide x por 3.1
w = w * 3.1;        // multiplica w por 3.1

if (iguais(w, 10.1, 0.0001){
    System.out.println("iguais!");
} else {
    System.out.println("diferentes!");
}
```

Aqui aceitámos considerar que os valores são iguais desde que a sua diferença, em valor absoluto, seja menor que 0.0001.

Sempre que queremos comparar valores numéricos não inteiros devemos fazê-lo desta maneira.

Experimente executar o programa "ao vivo" na versão online deste livro. Na execução deste programa dá bem para ver, através do conteúdo das variáveis, que a representação dos valores no tipo `double` não é necessariamente exata, devido ao número de algarismos significativos ser limitado.

6. Ciclos

Com o que aprendemos até agora, se quisermos repetir um conjunto de instruções várias vezes, não temos outra forma senão escrever essas instruções tantas vezes quantas as necessárias. Também podemos usar a abstração procedimental e escrever um pouco menos.

Como é óbvio, precisamos de uma melhor solução. Até porque, na maioria dos casos, é necessário repetir um conjunto de instruções um número de vezes que só é conhecido durante a execução do programa.

Por exemplo, imagine-se a programar um método que imprime um determinado número de vezes um *, sendo esse número de vezes dado por um parâmetro do método. Como esse número pode variar (é esse o objetivo de o definirmos como parâmetro) é impossível saber quantas instruções `System.out.print("*");` deveríamos escrever no método.

As instruções que permitem repetição são informalmente chamadas de *ciclos*. Vamos aqui aprender duas dessas instruções.

Antes disso, no entanto, vamos falar mais um pouco sobre atribuições de valores a variáveis e introduzir novos operadores aritméticos.

6.1. Atalhos para operações comuns

Usamos as palavras *incrementar* e *decrementar* para nos referirmos à adição de uma unidade e à subtração de uma unidade, respetivamente. Em Java podemos fazer isso da seguinte forma:

```
int i = 2;
int j = 5;
i = i + 1;           // incrementar i
                    // i fica com o valor 3
j = j - 1;           // decrementar j
                    // j fica com o valor 4
```

Repare na diferença entre estas instruções de atribuição e as correspondentes na Matemática. A expressão `i = i + 1` nunca seria válida na Matemática pois o sinal `=` representa a igualdade e um valor não pode ser igual seu próprio sucessor.

No Java, o sinal `=` é a instrução de atribuição e, por isso, na execução da instrução `i = i + 1;`

- a expressão `i + 1` é avaliada
- o seu valor é atribuído à variável `i`.

Se `i` vale 4, por exemplo, então após esta instrução fica com o valor 5.

Incrementar e decrementar são operações tão comuns, que a linguagem Java disponibiliza operadores especiais para elas:

- o operador `++` adiciona uma unidade ao valor corrente de uma variável;
- o operador `--` subtrai uma unidade ao valor corrente de uma variável.

Nenhum destes operadores trabalha com `boolean` ou `String`.

Repare que estes não são meros operadores, pois afetam a variável sobre a qual são aplicados, ou seja, têm efeitos colaterais.

Embora seja possível incluir estes operadores em expressões, nós não vamos fazer isso. Como queremos que a avaliação de expressões seja livre de efeitos colaterais, usamos `++` e `--` somente como instruções.

Quando executadas, as seguintes instruções escrevem 2, 5, 3 e 4 no ecrã, em linhas separadas.:

```
int i = 2;
int j = 5;
System.out.println(i);
System.out.println(j);
i++;
System.out.println(i);
j--;
System.out.println(j);
```

O Java também tem “atalhos” para outras instruções de atribuição comuns:

```
var += expr      // atribui a var o valor de var + expr
var -= expr      // atribui a var o valor de var - expr
var *= expr      // atribui a var o valor de var * expr
var /= expr      // atribui a var o valor de var / expr
var %= expr      // atribui a var o valor de var % expr
```

Por exemplo, quando executadas, as seguintes instruções escrevem 24 no ecrã, que é o resultado de multiplicar 2 por (5 + 7).

```
int i = 2;
int j = 5;
i *= j + 7;
System.out.println(i);
```

Experimente executar o programa completo na versão online deste livro, inspecionando o conteúdo das variáveis.

6.2. A instrução `for`

Em Java há várias instruções que podem ser usadas para repetir ações. São aquilo a que se chama *ciclos* (ou *loop* em inglês).

Começamos por estudar o ciclo `for`, que deve ser usado quando queremos exprimir repetição *limitada*, ou seja, quando o número máximo de vezes que o ciclo é executado está previamente determinado.

6.2.1. Sintaxe e semântica

A sintaxe geral do ciclo `for` pode ser descrita por:

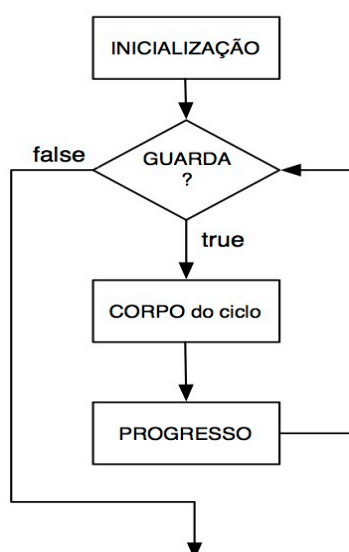
```
for (inicialização ; guarda ; progresso)
    corpo-do-ciclo
```

A `guarda` é uma expressão do tipo `boolean` (ou *condição*).

O `corpo-do-ciclo` é um bloco de instruções entre chavetas.

O `progresso` é uma instrução.

O fluxo de execução do ciclo `for` é representado pela seguinte figura.



Um exemplo:

```
int n = 4;
for (int i = 1 ; i <= n ; i++) {
    System.out.print("Blah");
}
System.out.println(".");
```

Aqui a inicialização é `int i = 1`, a guarda é `i <= n`, o progresso é `i++` e o corpo do ciclo é `System.out.print("Blah");`. Quando executadas, estas instruções escrevem *BlahBlahBlahBlah*. no ecrã.

Vamos “acompanhar” a execução deste ciclo, guiando-nos pelo fluxograma da figura acima.

A variável `i` é inicializada a 1. Porque 1 é menor ou igual ao valor de `n` (que é 4), então o corpo do ciclo é executado escrevendo *Blah* no ecrã (não há mudança de linha pois é `print` e não `println`). De seguida a instrução `i++` é executada, o que faz com que o valor da variável `i` seja incrementado de uma unidade. E agora volta-se a repetir o esquema anterior – guarda, corpo-do-ciclo, progresso – até uma altura em que `i` é incrementado para o valor 5 que, já não sendo `<= 4`, torna falsa a guarda, fazendo com que o ciclo termine. Neste processo foi escrita a palavra *Blah* mais 3 vezes, na mesma linha. Já fora do ciclo, é ainda escrito um ponto.

Chamamos *variável de progresso* à variável `i` pois as mudanças que ela sofre fazem a execução do ciclo progredir em direção ao fim do ciclo, o qual se dá quando a guarda `i <= n` deixa de se verificar.

Experimente agora "executar ao vivo" o programa contendo o ciclo acima descrito, na versão online deste livro.

6.2.2. Ciclos infinitos

Quando o progresso não garante que a guarda se torne falsa, então temos um *ciclo infinito*, isto é, um ciclo que nunca pára.

Na verdade, a guarda representa uma *condição de continuação*, pois enquanto for verdadeira, o ciclo continua a sua execução.

O seguinte pedaço de código ilustra um ciclo infinito.

```
for (int i = 1 ; i != 0 ; i++) {  
    System.out.print("Blah");  
}
```

Repare que a variável de progresso – `i` – é inicializada a 1 e que a guarda é verdadeira enquanto o valor de `i` for diferente de zero. A guarda é sempre verdadeira pois os valores que `i` vai tomando são os inteiros maiores que zero, os quais são sempre diferentes de zero.

NOTA: Na verdade, quando o valor de `i` se tornar tão grande quanto o valor máximo do conjunto de valores para o tipo `int`, a soma de uma unidade fará com que o valor de `i` se torne negativo (-231) devido aos mecanismos de representação e de adição em binário; assim, mais cedo ou mais tarde o ciclo terminará pois `i` tomará o valor zero inevitavelmente. Embora não se verifique nenhum erro de execução, no entanto essa é uma situação a evitar a todo o custo, pois conceptualmente este é um ciclo infinito.

6.2.3. Variações nas componentes do ciclo

Nos exemplos anteriores, o corpo do ciclo tem sempre o mesmo efeito, que é o de escrever a palavra *Blah* no ecrã. O próximo exemplo mostra um ciclo onde o corpo do ciclo tem um efeito diferente de cada vez que é executado porque contém uma parte que varia à medida que o ciclo progride.

```
int n = 4;  
for (int i = 1 ; i <= n ; i++) {  
    System.out.println(i);  
}
```

Quando executadas, estas instruções escrevem

```
1  
2  
3  
4
```

no ecrã. Tente certificar-se de que é isso que acontece, guiando-se pelo fluxograma que descreve o ciclo `for`, à semelhança do que foi feito no exemplo anterior.

Mudando ligeiramente as várias partes do ciclo, podemos alterar o seu efeito. Por exemplo, quando executadas, as seguintes instruções

```
int n = 4;
for (int i = 10 ; i <= n * 10 ; i += 10) {
    System.out.println(i);
}
```

escrevem o seguinte no ecrã

```
10
20
30
40
```

Repare que o corpo do ciclo é executado também quatro vezes, mesmo sendo diferentes as inicialização, guarda e progresso.

O ciclo seguinte tem um efeito exatamente igual no ecrã:

```
int n = 4;
for (int i = 1 ; i <= n ; i++) {
    System.out.println(i * 10);
}
```

Experimente agora "executar ao vivo" um programa contendo um dos ciclos acima descritos, na versão online deste livro.

6.2.4. Imprimir tabuadas

E se quisermos agora imprimir a tabuada do 7? Esta tabuada obtém-se multiplicando por 7 os números naturais de 1 a 10.

Para decidir como fazer isto em programação, devemos reparar que basta repetir uma mesma ação dez vezes. Essa ação pode-se descrever como *"imprimir o resultado de multiplicar 7 por um número natural"*, sabendo que a primeira vez esse número natural será 1, da segunda vez será 2, etc.

Então, se chamarmos `i` a esse número natural que varia, basta darmos valores sequenciais a `i`, partindo do valor 1, até chegar a 10, repetindo, de cada vez, a ação *"imprimir o resultado de multiplicar 7 por `i`"*.

Então, o programa seguinte:

```
public class EscreveTabuada {

    public static void main(String[] args) {

        for (int i = 1 ; i <= 10 ; i++) {
            System.out.println(i * 7);
        }

    }
}
```

quando executado, escreve no ecrã (abreviamos com ... para poupar espaço)

```
7
14
21
28
...
70
```

Experimente executá-lo "ao vivo" na versão online deste livro.

Podemos tornar o *output* mais informativo, explicitando a que expressões correspondem os vários valores:

```
public class EscreveTabuada {

    public static void main(String[] args) {

        for (int i = 1 ; i <= 10 ; i++) {
            System.out.println(i + " x 7 = " + i * 7);
        }
    }
}
```

Este programa, quando executado, escreve no ecrã

```
1 x 7 = 7
2 x 7 = 14
3 x 7 = 21
4 x 7 = 28
...
10 x 7 = 70
```

Note que, como a multiplicação (*) tem prioridade sobre a concatenação (+), na avaliação da expressão "Linha " + i * 10, o valor da sub-expressão i * 10 é calculado primeiro. De seguida, esse valor é concatenado com a string "Linha ", exatamente como pretendido.

Estes programas têm um interesse limitado pois, sempre que são invocados, escrevem no ecrã a mesma tabuada do 7.

Na secção 6.3. veremos como generalizar este cálculo e impressão de tabuada, construindo métodos que permitam fazê-lo para quaisquer valores, tanto do número da tabuada, como do número de valores apresentados.

6.2.5. Calcular somatórios

E se agora quisermos que o computador calcule e imprima a soma dos inteiros positivos menores que 10, por exemplo? Numa atitude de mínimo esforço mental, podemos fazê-lo avaliar a expressão 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9:

```
public class Somatorios {

    public static void main (String [] args) {

        System.out.println(1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9);
    }
}
```


Mas o que acontece quando queremos a soma dos inteiros positivos menores que 100? Ou menores que 1000? Pois... esta solução não é nada prática; é mesmo muito idiota do ponto de vista de programação.

Mas mesmo que quiséssemos adotar esta abordagem, há casos em que nem sequer conseguiríamos! Quais? Aqueles em que o limite superior só é decidido quando o código é executado, ou seja, o programador não o conhece quando escreve o programa.

Um exemplo típico (e que veremos mais adiante) é aquele em que esse valor é decidido pelo utilizador que executa o programa. Mas, para já, podemos focar-nos no caso em que esse valor é dado pelo parâmetro de um método.

Se quisermos construir um método que calcule e devolva a soma dos inteiros positivos menores que um dado número, que é dado por um parâmetro, teremos que adotar outra abordagem, mais esperta, pois não conhecemos esse número quando escrevemos a definição do método.

Supondo que a assinatura do método é:

```
/**
 * Soma dos inteiros positivos menores que um dado numero
 * @param n - o limite superior
 * @return A soma dos inteiros entre 1 (inclusivé) e n (exclusivé)
 */
static int somaPositivosMenoresQue (int n)
```

Como não conseguimos escrever e avaliar uma expressão que represente a soma completa dos inteiros no intervalo $[1, n - 1]$, então o corpo do método vai ter que conter instruções que digam ao computador para fazer essa soma inteiro a inteiro, até ter conseguido somar todos os valores no intervalo.

Para somar os inteiros um a um, vamos ter que ser capazes de adicionar cada novo inteiro à soma dos inteiros já processados. Por exemplo, se os inteiros entre 1 e 3 já tiverem sido processados, teremos que adicionar o 4 a 6 (que é a soma de 1, 2 e 3), de seguida o 5 a 10 e assim sucessivamente até termos somado todos os inteiros necessários.

Podemos descrever este comportamento repetitivo com a frase: *"adicionar cada novo número à soma já acumulada"*.

A *"soma já acumulada"* é o valor que temos que manter em memória até lhe adicionarmos um novo número. Vamos então usar uma variável para guardarmos a soma acumulada.

Embora o valor da soma acumulada vá mudando à medida que somamos cada novo número, só o último valor que ela toma nos interessa a cada instante. Então, basta-nos uma única variável, pois podemos atribuir-lhe novos valores tantas vezes quantas quisermos. Chamemos `soma` a essa variável.

A ação que queremos repetir pode ser codificada em Java pela instrução: `soma = soma + numero;` em que a variável `numero` representa o novo número a somar de cada vez. Esta instrução constituirá o corpo do nosso ciclo.

A instrução que queremos é, de forma ainda incompleta:

```
for (??? ; ??? ; ???) {
    soma = soma + numero;
}
```

Queremos que, a cada repetição da ação, a variável `numero` tome um valor diferente que, neste caso, é o de sucessor do valor que foi processado na repetição anterior.

Para obter o sucessor de um número inteiro basta acrescentar-lhe uma unidade; então o progresso da nossa repetição pode ser `numero++`; `numero` é a variável de progresso do ciclo.

Já podemos completar alguns dos "buracos" que nos faltam:

```
for (int numero = ?? ; ??? ; numero++) {
    soma = soma + numero;
}
```

Qual o valor inicial para a variável `numero`? É 1, pois queremos a soma desde 1 até $n - 1$.

E qual é a condição de continuação do ciclo? É `numero < n` porque queremos acumular os inteiros inferiores a `n`. Então será que o método seguinte está completo?

```
/**
 * Soma dos inteiros positivos menores que um dado numero
 * @param n - o limite superior
 * @return A soma dos inteiros entre 1 (inclusiv ) e n (exclusiv )
 */
static int somaPositivosMenoresQue (int n) {
    for (int numero = 1 ; numero < n ; numero++) {
        soma = soma + numero;
    }
}
```

Ainda n o... O que falta? Falta a instru  o `return`, que permite terminar o m todo devolvendo o resultado.

O resultado   o conte do da vari vel `soma`, que foi usada para ir acumulando os v rios inteiros positivos menores que `n`. No fim do ciclo o seu valor   precisamente o valor do somat rio que pretendemos.

Ent o, a  ltima instru  o do m todo deve ser `return soma;`.

N o falta mais nada? Falta! Falta a inicializa  o da vari vel `soma`. Com que valor?

Para que o primeiro inteiro a somar (neste caso o 1) possa ser processado da mesma maneira que os valores seguintes – *“adicionar cada novo n mero   soma j  acumulada”* –, a vari vel `soma` tem que ter um valor inicial que represente a situa  o em que nada foi acumulado ainda, ou seja zero.

NOTA: Repare que zero   precisamente o elemento neutro da adi  o. Se estivessemos a calcular o produto dos inteiros positivos menores que `n`, como seria o corpo do ciclo? E a inicializa  o da vari vel que guarda o produto acumulado?

Resumindo e concluindo, a vers o final do m todo  :

```
/**
 * Soma dos inteiros positivos menores que um dado valor
 * @param n - o limite superior
 * @return A soma dos inteiros entre 1 (inclusiv ) e n (exclusiv )
 */
static int somaPositivosMenoresQue (int n) {
    int soma = 0;
    for (int numero = 1 ; numero < n ; numero++) {
        soma = soma + numero;
    }
    return soma;
}
```

Experimente executar ao vivo um programa que imprime o somat rio dos inteiros positivos menores que 8, na vers o online deste livro.

O ciclo seguinte tem exatamente o mesmo efeito final que o ciclo do m todo `somaPositivosMenoresQue`.

```
for (int numero = n - 1 ; numero >= 1 ; numero--) {
    soma = soma + numero;
}
```

Qual a diferença? É somente a forma como são gerados os inteiros positivos menores que `n`. Nesta versão é do maior para o menor – a variável de progresso `numero` é inicializada com o maior dos números a somar ($n - 1$) e é decrementada de uma unidade a cada execução do corpo do ciclo. A guarda, ou condição de continuação, é, neste caso, `numero >= 1` pois enquanto a variável `numero` tiver um valor maior ou igual a 1, o seu valor deve ser acumulado na variável `soma`.

Experimente a execução passo a passo, observando o progresso do ciclo, na versão online deste livro.

6.2.6. Casos críticos

Sempre que construímos um ciclo, devemos pensar em resolver o caso geral. Quando estamos convencidos que funciona bem para o caso geral, devemos investigar se também funciona bem para os *casos críticos*.

No exemplo da soma dos inteiros, devemos investigar:

- os casos em que `n` é menor ou igual a 1 (casos em que não existem positivos menores que `n`, devendo a sua soma ser zero) e
- o caso em que `n` é 2 (caso em que só existe um positivo menor que `n`, que é 1 e, por isso, a soma deve ser 1).

Vamos estudar os casos em que `n` é menor ou igual a 1. Vamos ver se o corpo do método calcula bem o resultado, que deve ser zero:

```
int soma = 0;
for (int numero = 1 ; numero < n ; numero++) {
    soma = soma + numero;
}
return soma;
```

A variável `soma` começa por tomar o valor zero. O fluxo de execução do ciclo `for` determina que o primeiro bloco a ser executado é a inicialização, ou seja, neste caso a variável `numero` fica com o valor 1.

De seguida, a guarda é avaliada. Neste caso a guarda vale `false` pois como `n` é menor ou igual a 1, a variável `numero` não é menor que `n`.

Então o fluxo de execução segue imediatamente para a instrução a seguir ao ciclo, sem nunca executar o corpo do ciclo. A instrução `return soma` faz com que o método termine a sua execução devolvendo o resultado zero, como pretendido.

No caso de `n` ter o valor 2, então o resultado deverá ser 1. A variável `soma` começa por tomar o valor zero. A execução da inicialização atribui o valor 1 à variável `numero`.

De seguida a avaliação da guarda conclui que esta vale `true`. Então o fluxo de execução segue para o corpo do ciclo, acumulando o valor da variável `numero` na variável `soma`, ficando esta com o valor 1.

De seguida o fluxo de execução segue para a execução do progresso, incrementando a variável `numero` de uma unidade.

A guarda é de novo avaliada dando `false` desta vez. Segue-se a instrução `return soma` que termina a execução devolvendo o resultado 1, como pretendido.

Tente agora construir uma função para calcular o produto de um dado número. Comece por considerar o caso geral. De seguida identifique os casos críticos e investigue se a sua solução funciona bem. Recorde que o elemento neutro do produto é 1.

6.2.7. Reforço da guarda

Vamos programar uma função que calcule e devolva o menor divisor maior que 1 de um dado número `n` maior que 1, com a seguinte assinatura e documentação:

```
/**
 * Menor divisor maior que 1 de um dado numero
 * @param n - o numero
 * @requires n > 1
 * @return O menor divisor de n que eh maior que 1
 */
static int menorDivisorMaiorQueUm (int n)
```

Sabendo que um número `i` é divisor de um número `n` se o resto da divisão de `n` por `i` é zero, então podemos começar por testar se `n % i == 0`, para `i` igual a 2. Se for esse o caso, então 2 é o menor divisor de `n` maior que 1.

Se não for, fazemos o mesmo para `i` igual a 3 e assim sucessivamente; temos então um ciclo `for` com `i` como variável de progresso, inicializada a 2, progredindo de 1 em 1.

Logo que encontremos o divisor que procuramos, vamos guardá-lo na memória para depois podermos devolver o seu valor como resultado da função. Para isso usamos uma variável do mesmo tipo do resultado – `int` neste caso.

Qual poderá ser a guarda do ciclo? Não vale a pena testar o resto da divisão de `n` por valores maiores que ele próprio, pois um divisor de um número é sempre menor ou igual ao próprio número. Então podemos ter `i <= n` como guarda.

Esta análise já nos permitiu encontrar uma primeira abordagem para esboçar uma solução:

```
static int menorDivisorMaiorQueUm (int n) {
    int resultado;
    for (int i = 2 ; i <= n ; i++) {
        if (n % i == 0) {
            resultado = i;
        }
    }
    return resultado;
}
```

Esta solução tem vários problemas, uns mais graves que outros.

Qualquer que seja o número `n`, sabemos que não há divisores entre `n/2` e `n`. Então vamos otimizar o nosso ciclo evitando que sejam feitos cálculos e testes desnecessários. Vamos mudar a guarda para `i <= n/2`.

Chega? Não, pois sabemos que se `n` for número primo, o menor divisor de `n` maior que 1 é o próprio `n`. Então temos que nos certificar que, se a instrução `resultado = i;` nunca for executada, a variável `resultado` terá o valor de `n` no fim do ciclo. Como fazemos isso? Inicializando-a com o valor de `n` antes do ciclo. Então temos:

```
static int menorDivisorMaiorQueUm (int n) {
    int resultado = n;
    for (int i = 2 ; i <= n / 2 ; i++) {
        if (n % i == 0)
            resultado = i;
    }
    return resultado;
}
```

A inicialização da variável `resultado` teria que ser feita mesmo que não quiséssemos. Se não a inicializássemos, o compilador detetaria que o seu valor vai ser usado como retorno da função e não teria a certeza que um valor lhe seria atribuído – a única instrução que lhe atribui um valor está dentro do bloco de uma instrução condicional (cuja condição pode ser falsa) que, por sua vez, está dentro de um ciclo (que pode nunca ser efetivamente executado).

Para se aperceber de outro problema desta solução, experimente agora simular a execução do método para um valor de `n` de 15. A instrução `resultado = i;` é executada quando `i` tem o valor 3 e é esse o resultado que pretendemos.

Só que a execução do ciclo continua, pois o progresso incrementa o valor de `i` para 4 e a guarda continua verdadeira.

Então, quando `i` toma o valor 5, a instrução `resultado = i;` volta a ser executada, deixando a variável `resultado` com o valor 5. Oops! E o 3? Já o perdemos!

Execute este programa na versão online deste livro e verá que o resultado **não** é o correto.

Devíamos ter parado a execução do ciclo logo que encontrássemos o primeiro divisor maior que 1.

Para isso vamos fazer o que se chama de *reforço da guarda*, ou seja, vamos tornar a guarda mais restrita, de modo a ser satisfeita num número menor de situações. Neste caso só queremos executar o corpo do ciclo enquanto não encontrarmos o tal divisor que procuramos.

Quando é que sabemos que encontrámos o divisor que procurávamos? Repare que a variável `resultado`, cujo valor inicial é `n`, muda de valor a primeira vez logo que é encontrado o primeiro divisor.

Então, logo que a variável `resultado` muda de valor, ou seja, logo que deixa de ser igual a `n`, podemos parar.

Basta-nos então reforçar a guarda com uma condição que seja verdadeira enquanto a variável `resultado` tiver o valor de `n`.

Com isto podemos exprimir que não basta o `i` ser menor ou igual a `n/2` para continuarmos à procura do divisor. É preciso *também* que ainda não o tenhamos encontrado.

```
static int menorDivisorMaiorQueUm (int n) {
    int resultado = n;
    for (int i = 2 ; i <= n / 2 && resultado == n ; i++) {
        if (n % i == 0) {
            resultado = i;
        }
    }
    return resultado;
}
```

Repare no operador lógico que usámos para reforçar a guarda. Se queremos caracterizar as situações que satisfaçam ambas as condições `i <= n / 2` e `resultado == n`, então é a conjunção (&&) que devemos usar!

Verifique que com esta alteração já obtemos o resultado correto, na versão online deste livro.

Poderá agora perguntar-se: será que não bastaria a guarda `resultado == n`? Pense no caso em que `n` é um número primo, por exemplo 7, e verifique que o ciclo não teria fim pois a variável `resultado` teria sempre o valor `n` – a partir do momento em que fosse inicializada e mais tarde quando se verificasse que `n` é divisor dele próprio.

Só para exercitar a mente, se tivéssemos usado a disjunção (`||`), ficando a guarda `i <= n / 2 || resultado == n`, que situações estaríamos a caracterizar com essa guarda? Era isso que queríamos? Uma pista: teríamos obtido um ciclo infinito sempre que `n` fosse um número primo... Outra pista: obteríamos o maior divisor menor que `n`, se `n` não fosse primo...

Uma alternativa ao reforço da guarda

Em funções cujo principal objetivo é o de encontrar um dado valor e logo devolver esse valor, torna-se mais simples terminar a execução da função logo que o valor é encontrado, retornando-o como resultado.

Nestes casos, a guarda não precisa de ser reforçada pois a terminação da execução do ciclo (e da própria função) é provocada por uma instrução `return`.

O ciclo que usamos no método `menorDivisorMaiorQueUm` tem por único objetivo encontrar um dado valor e logo devolver esse valor; a seguinte versão, sem reforço explícito da guarda, é uma boa alternativa à solução estudada acima:

```
static int menorDivisorMaiorQueUm (int n) {
    for (int i = 2 ; i <= n / 2 ; i++) {
        if (n % i == 0) {
            return i;
        }
    }
    return n;
}
```

Experimente executar esta versão, na versão online deste livro, acompanhando o fluxo de execução e o conteúdo das variáveis.

Nesta versão, embora o trio *inicialização-guarda-progresso* indique que o corpo do ciclo vai ser executado $n / 2 - 1$ vezes, isso pode não acontecer; basta que `n` tenha mais divisores além de 1 e dele próprio para que o ciclo termine antes que a guarda se torne falsa, terminando com isso a própria função e tendo como resultado o valor encontrado.

Em termos de legibilidade dos programas isto pode ser negativo, mas é compensado pela forma extremamente natural como representa a ideia subjacente de terminar a procura logo que o valor é encontrado, devolvendo-o como resultado da função.

Encontramos este padrão de comportamento em muitas outras situações, para as quais uma abordagem deste tipo é também adequada.

6.2.8. Nunca fazer!

Uma solução que alguns (maus) programadores sugeririam, em alternativa ao reforço da guarda que fizemos na secção anterior, é a seguinte:

```
// BAD BAD code
for (int i = 2 ; i <= n / 2 ; i++) {
    if (n % i == 0) {
        resultado = i;
        i = n;
    }
}
```

O que se fez aqui? Em vez de reforçar a guarda testando se o valor de `resultado` mudou, mudou-se o valor de `i` para um valor que falsifica a guarda (`n` neste caso), de modo a que o ciclo termine logo a seguir.

Nunca se deve fazer este tipo de coisas, pois piora muito a legibilidade dos programas! A variável de progresso `i` representa, neste ciclo, o número “candidato” a divisor de `n` e o progresso diz-nos claramente que `i` progride de uma em uma unidade. Quando mudamos o valor da variável de progresso abruptamente no corpo do ciclo, estamos a alterar esta perceção e a dificultar a compreensão do programa.

Como regra de ouro do ciclo `for` temos:

“Não se deve alterar o valor da variável de progresso dentro do corpo do ciclo”.

6.2.9. O significado das variáveis

O significado que damos às variáveis tem que ser claro desde o início, a bem da legibilidade e facilidade de manutenção dos programas.

Por exemplo, posso “ler” o ciclo do método `menorDivisorMaiorQueUm`, de forma literal, da seguinte forma:

enquanto o valor de i for menor ou igual a $n/2$ e o valor de resultado for igual a n , fazer:

- *se o resto da divisão de n por i for zero, então*
 - *resultado fica com o valor de i*

Mas na verdade, dado o significado que atribuímos às variáveis, essa leitura seria mais clara se feita da seguinte forma:

enquanto não tivermos testado todos os potenciais valores para divisores de n nem encontrado o divisor que procurávamos, fazer:

- *se i for divisor de n , então*
 - *o divisor que procurávamos é i*

Como programadores, devemos ler as instruções do nosso programa de acordo com o significado que fomos atribuindo às variáveis e aos métodos, abstraindo assim dos detalhes irrelevantes das várias instruções que usamos; isto ajuda-nos muito a pensar sobre os problemas a resolver e a entender o que vamos fazendo.

6.3. Ciclos `for` encaixados

Tal como encaixámos `if`s dentro de `if`s, também podemos encaixar, ou aninhar, ciclos `for`. No pedaço de código seguinte, quantas vezes é escrita a palavra *Blah* no ecrã?

```
for (int i = 1 ; i <= 3 ; i++) {  
    for (int j = 1 ; j <= 2 ; j++) {  
        System.out.print("Blah");  
    }  
}
```

Relembre o fluxograma do ciclo `for` ao interpretar o código acima.

- A variável `i` é inicializada a 1. Como 1 é menor ou igual a 3, o corpo do ciclo é executado, ou seja, o ciclo `for` interno é iniciado.
- A variável `j` fica com o valor 1. Como 1 é menor ou igual a 2, a instrução `System.out.print("Blah")` é executada. De seguida `j` fica com o valor 2 e a mesma instrução é executada de novo. De seguida `j` fica com o valor 3 e o ciclo interno termina.
- Como este ciclo interno é a única instrução do corpo do ciclo externo, então a variável `i` é incrementada para o valor 2 que, por ser menor ou igual a 3, provoca uma nova execução do corpo do ciclo, ou seja, o ciclo interno é de novo executado e assim sucessivamente até `i` tomar o valor 4.

O que constatamos aqui? Que o corpo do ciclo externo (formado por uma única instrução, que é o ciclo `for` interno) é executado 3 vezes. Por sua vez, o corpo deste ciclo interno (a instrução `System.out.print("Blah")`) é executado 2 vezes.

Então, a instrução `System.out.print("Blah")` é executada 3 x 2 vezes, ou seja, a palavra "Blah" é escrita seis vezes.

Experimente executar o programa contendo os ciclos encaixados, na versão online deste livro.

Como provocar uma mudança de linha de cada vez que se escrevem dois `Blah`s, ou seja, de cada vez que o ciclo interno termina?

Basta adicionar a instrução `System.out.println()` logo após o ciclo interno. Desta forma o corpo do ciclo externo fica com duas instruções: o ciclo interno e a mudança de linha.

```
for (int i = 1 ; i <= 3 ; i++) {
    for (int j = 1 ; j <= 2 ; j++) {
        System.out.print("Blah");
    }
    System.out.println();
}
```

Execute a nova versão do programa com atenção às diferenças, na versão online deste livro.

6.3.1. Exemplos

Vamos supor que queremos imprimir uma tabela da multiplicação para as tabuadas dos naturais de `limInf` a `limSup`, com `m` valores cada. Se `limInf` for 1, `limSup` for 9 e `m` for 10 queremos o seguinte:

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90

Vamos dividir a tarefa da seguinte forma:

1. Como imprimir uma linha com a tabuada de um natural `n`, com `m` valores?
2. Como fazer isso para valores de `n` num dado intervalo [`limInf` , `limSup`]?

O método seguinte resolve a sub-tarefa 1:

```
/**
 * Escrever os primeiros multiplos de um dado numero no standard output
 * @param n - O numero
 * @param m - O numero de valores a imprimir
 * @requires m > 0 && n > 0
 */
static void imprimeMultiplos (int n, int m) {

    for (int i = 1 ; i <= m ; i++) {
        System.out.print(n * i + " ");
    }
    System.out.println();
}
```

A última instrução é para provocar a mudança de linha.

Para resolver a sub-tarefa 2, invocamos o método anterior para valores de `n` no intervalo `[ limInf , limSup ]`. O seguinte método realiza essa tarefa:

```
/**
 * Imprimir as tabuadas dos naturais de 1 até um dado valor, no standard output
 * @param limInf - O primeiro natural
 * @param limSup - O último natural
 * @param m - O numero de valores que as tabuadas devem ter
 * @requires limInf > 0 && limSup > 0 && m > 0
 */
static void imprimeTabuadas (int limInf, int limSup, int m) {

    for (int linha = limInf ; linha <= limSup ; linha++) {
        imprimeMultiplos(linha, m);
    }
}
```

O seguinte programa escreve no ecrã as tabuadas dos naturais de 1 a 9, cada uma com 10 valores:

```
public class TabelaDeTabuadas {

    public static void main (String [] args) {
        imprimeTabuadas(1, 9, 10);
    }
}
```

Experimente executá-lo "ao vivo" na versão online deste livro.

Repare que, embora divididos por dois métodos, o que temos aqui é um ciclo encaixado noutro – para cada valor de `linha` no ciclo do método `imprimeTabuadas`, é executado o ciclo definido no método `imprimeMultiplos`.

O seguinte programa é equivalente ao anterior:

```
public class TabelaDeTabuadasV2 {

    public static void main (String [] args) {
        imprimeTabuadas(1, 9, 10);
    }

    static void imprimeTabuadas (int limInf, int limSup, int m) {

        for (int linha = limInf ; linha <= limSup ; linha++) {
            for (int i = 1 ; i <= m ; i++) {
                System.out.print(linha * i + " ");
            }
            System.out.println();
        }
    }
}
```

6.4. Ainda a abstração procedimental

Ao longo deste capítulo reforçamos a ideia de que a abstração procedimental é extremamente útil na programação:

- Ao dar um nome a uma sequência de instruções, obtemos programas mais legíveis;
- Dividir um programa longo em métodos, permite a procura e correção de erros (depuração ou *debugging*) sobre partes mais simples do programa e posterior composição para obter o programa final;
- Os métodos facilitam a criação de ciclos encaixados; e
- Métodos bem desenhados são frequentemente úteis para se usarem em vários programas – uma vez escritos e depurados de erros, podemos reutilizá-los.

6.4.1. Retângulos de caracteres

Usando de novo a abstração procedimental, vamos desenvolver um programa para imprimir um retângulo de caracteres. O retângulo da figura abaixo é composto por 6 linhas com 15 asteriscos cada uma:

```
*****
*****
*****
*****
*****
*****
```

Vamos começar por construir um método que imprime uma linha com 15 asteriscos:

```
/**
 * Imprimir uma linha com 15 asteriscos no standard output
 */
static void imprimeAsteriscos () {

    for (int i = 1 ; i <= 15 ; i++) {
        System.out.print('*');
    }
    System.out.println();
}
```

A última instrução é para provocar a mudança de linha.

Agora é preciso invocar este método seis vezes para obter o retângulo desejado. O seguinte programa obtém o efeito pretendido:

```
public class EscreveRetanguloAsteriscos {

    public static void main(String[] args) {

        for (int i = 1 ; i <= 6 ; i++) {
            imprimeAsteriscos();
        }
    }
}
```

E se quisermos imprimir o seguinte retângulo, com 7 asteriscos em cada linha?

```
*****
*****
*****
*****
*****
*****
```

Ou ainda este, com 25 asteriscos em cada linha?

```
*****
*****
*****
*****
*****
*****
```

Na verdade, o método `imprimeAsteriscos` não serve para muita coisa... Teríamos que ter vários métodos quase iguais a este para podermos imprimir os dois retângulos acima, pois o número de caracteres em cada linha é diferente.

O ideal era ter um método que permitisse imprimir um asterisco um qualquer número de vezes. Como podemos fazer isso?

Usando *abstração de parâmetro* – adicionamos um parâmetro que representa o número de asteriscos a imprimir.

Sempre que o método é invocado, um valor específico deve ser indicado para esse parâmetro. Assim obtemos um método mais geral e, portanto, mais útil, pois pode ser usado em várias circunstâncias.

```
/**
 * Imprimir no standard output uma linha com um dado numero
 * de asteriscos
 * @param n - O numero de asteriscos a imprimir
 * @requires n > 0
 */
static void imprimeAsteriscos (int n) {

    for (int i = 1 ; i <= n ; i++) {
        System.out.print('*');
    }
    System.out.println();
}
```

O seguinte programa imprime os três retângulos pretendidos:

```
public class EscreveRetangulos {

    public static void main(String[] args) {

        for (int i = 1 ; i <= 6 ; i++) {
            imprimeAsteriscos(15);
        }

        for (int i = 1 ; i <= 6 ; i++) {
            imprimeAsteriscos(7);
        }
    }
}
```

```

        for (int i = 1 ; i <= 6 ; i++) {
            imprimeAsteriscos(25);
        }
    }
}

```

E se quisermos imprimir o seguinte retângulo?

0000000000
 0000000000
 0000000000
 0000000000

Ou ainda este?

€€€€€€€€€€€€€€€€€€€€€€€€

€€€€€€€€€€€€€€€€€€€€€€€€

€€€€€€€€€€€€€€€€€€€€€€€€

O ideal era ter um método que permitisse imprimir um qualquer carácter um qualquer número de vezes.

Usando outra vez *abstração de parâmetro*, adicionamos um segundo parâmetro que representa o caráter a escrever.

Quando o método é invocado, indicamos um valor específico para cada um desses parâmetros. Alteramos também o nome do método para ficar mais de acordo com a sua funcionalidade.

```
/**
 * Imprimir no standard output uma linha com um dado carater
 * repetido um dado numero de vezes
 * @param n - O numero de carateres a imprimir
 * @param carater - O carater a imprimir
 * @requires n > 0
 */
static void imprimeCarateres (int n, char carater) {

    for (int i = 1 ; i <= n ; i++) {
        System.out.print(carater);
    }
    System.out.println();
}
```

O seguinte programa imprime vários dos retângulos pretendidos:

```
public class EscreveRetangulos {

    public static void main(String[] args) {

        for (int i = 1 ; i <= 6 ; i++) {

            imprimeCarateres(15, '*');

        }

        for (int i = 1 ; i <= 4 ; i++) {
```

```

        imprimeCarateres(9, '@');
    }

    for (int i = 1 ; i <= 3 ; i++) {
        imprimeCarateres(25, '€');
    }
}
}

```

Se observarmos melhor o método `main`, vemos que a instrução `for` que permite repetir a invocação do método `imprimeCarateres` é praticamente igual nos três casos. As diferenças são:

- no valor que usamos como limite superior para a variável de progresso `i`, o qual define a altura do retângulo;
- na única instrução no corpo desses ciclos, que só varia nos valores com que instanciamos os parâmetros do método `imprimeCarateres`.

Além disso, cada um desses ciclos tem como objetivo a escrita de um retângulo no *standard output*.

Justifica-se então a criação de um método cujo objetivo é o de escrever no *standard output* um retângulo formado por um dado número de linhas, em que cada linha será formada por um dado número de um qualquer carácter.

O seguinte método faz o que pretendemos:

```

/**
 * Imprimir no standard output um retangulo com uma dada altura, em que
 * em cada linha e' repetido um carater dado, um dado numero de vezes
 * @param altura - O numero de linhas do retangulo
 * @param largura - O numero de carateres a imprimir em cada linha
 * @param carater - O carater a imprimir
 * @requires n > 0
 */
static void imprimeRetangulo (int altura, int largura, char carater) {

    for (int i = 1 ; i <= altura ; i++) {
        imprimeCarateres(largura, carater);
    }
}

```

O programa `EscreveRetangulos` fica então simplificado e com maior legibilidade:

```

public class EscreveRetangulos {

    public static void main(String[] args) {

        imprimeRetangulo(6, 15, '*');
        imprimeRetangulo(4, 9, '@');
        imprimeRetangulo(3, 25, '€');
    }
}

```

Experimente executar "ao vivo" o programa final, na versão online deste livro.

Na verdade, o método `imprimeRetangulo` poderia ter sido implementado da seguinte forma:

```
static void imprimeRetangulo (int altura, int largura, char carater) {

    for (int i = 1 ; i <= altura ; i++) {
        for (int j = 1 ; j <= largura ; j++) {
            System.out.print(carater);
        }
        System.out.println();
    }
}
```

Repare que nesta versão tivemos que substituir a variável de progresso do ciclo interno para `j` de modo a ficar diferente de `i`, pois não podemos ter dois ciclos encaixados declarando variáveis de progresso com nomes iguais.

Qual das duas versões é a melhor?

- A primeira produziu um método adicional – `imprimCarateres` – que pode ser útil por si só; além disso o código é mais legível e mais perto da nossa forma de raciocinar;
- A segunda versão, embora menos legível, também é perfeitamente aceitável se acharmos que não precisamos de um método para imprimir um dado número de carateres numa linha.

Agora imagine que queremos um triângulo retângulo de asteriscos em que a base contém `n` asteriscos. Se `n` for 6, queremos:

```
*
**
***
****
*****
*****
```

Como fazer? Repare que queremos escrever `n` linhas, cada uma com um número de asteriscos que varia conforme a linha. Ou seja, reaproveitando o método `imprimeAsteriscos` queremos algo parecido com:

```
/**
 * Imprimir um triangulo de carateres com um dado numero de linhas
 * @param n - o numero de linhas
 * @param carater - O carater a imprimir
 * @requires n > 0
 */
static void imprimeTriangulo (int n, char carater){

    for (int i = 1 ; i <= n ; i++) {
        imprimeCarateres(???, carater);
    }
}
```

Coloca-se então a questão: o que devemos escrever em vez dos `???`? Depende.

- Quando `i` é 1, significando que vamos escrever a 1ª linha do triângulo, queremos escrever *um* caráter;
- Quando `i` é 2 (para escrever a 2ª linha do triângulo) queremos escrever *dois* carateres;
- No geral, quando `i` tem um dado valor `x` (que corresponde à linha `x`), queremos escrever `x` carateres;

Na verdade queremos escrever tantos carateres quantos o valor de `i`!

Então, a nossa função deve ser:

```
static void imprimeTriangulo (int n, char carater){

    for (int i = 1 ; i <= n ; i++) {
        imprimeCarateres(i, carater);
    }
}
```

Se optarmos por outra versão sem a invocação do método `imprimeCarateres` (à semelhança do que fizemos acima com o retângulo), teremos:

```
static void imprimeTriangulo (int n, char carater) {

    for (int i = 1 ; i <= n ; i++) {
        for (int j = 1 ; j <= i ; j++) {
            System.out.print(carater);
        }
        System.out.println();
    }
}
```

Experimente executar o programa completo na versão online deste livro.

Descubra como imprimir um triângulo retângulo invertido (com a base em cima e o vértice em baixo).

E se agora quisermos imprimir `n` x `m` números sucessivos, organizados em `n` linhas com `m` números cada? Por exemplo, se `n` for 3 e `m` for 4 queremos isto:

```
1 2 3 4
5 6 7 8
9 10 11 12
```

Raciocinemos de forma semelhante ao que já fizemos: vamos imprimir uma linha de `m` números sucessivos, `n` vezes.

Então vamos ter dois ciclos `for` encaixados:

- o interior para imprimir cada linha de `m` números,
- o exterior para repetir essa tarefa `n` vezes.

```
/**
 * Imprimir uma tabela de numeros sucessivos
 * @param n - o numero de linhas da tabela
 * @param m - o numero de valores por linha
 * @requires n > 0 && m > 0
 */
static void imprimeTabelaNumeros (int n, int m) {

    for (int i = 1 ; i <= n ; i++) {
        for (int j = 1 ; j <= m ; j++) {
            System.out.print(???);
        }
        System.out.println();
    }
}
```

```
}
```

O que deverá estar no lugar dos `???`? Depende do número de vezes que cada ciclo já foi executado.

Como queremos imprimir um número que, começando em 1, resulta do anterior somando-lhe um, podemos utilizar uma variável para memorizar o número que deve ser impresso a seguir.

Então, é o valor dessa variável que deve ser impresso no corpo do ciclo interno. Essa variável deve ser inicializada com o valor 1, que é o primeiro número que queremos imprimir, e, para que o seu valor progrida, devemos incrementá-la de cada vez que executamos a instrução `System.out.print`.

```
static void imprimeTabelaNumeros (int n, int m) {
    int count = 1;
    for (int i = 1 ; i <= n ; i++) {
        for (int j = 1 ; j <= m ; j++) {
            System.out.print(count);
            count++;
        }
        System.out.println();
    }
}
```

Inspecione o fluxo de execução e o conteúdo das variáveis na execução de um programa que invoca o método `imprimeTabelaNumeros` na versão online deste livro.

Um pequeno exercício – o que imprimiria a seguinte versão?

```
static void imprimeTabelaNumerosA (int n, int m) {

    for (int i = 1 ; i <= n ; i++) {
        int count = 1;
        for (int j = 1 ; j <= m ; j++) {
            System.out.print(count);
            count++;
        }
        System.out.println();
    }
}
```

Se não conseguiu adivinhar, pode descobrir executando o programa na versão online deste livro.

6.5. Ainda o âmbito das variáveis

Repare que, na secção 6.3.1., em cada um dos métodos `imprimeTabuadas` e `imprimeMultiplos`, definimos um parâmetro chamado `m`.

Lembra-se do que dissemos na Secção 4.4. acerca do âmbito das variáveis (e parâmetros)?

- Qualquer variável/parâmetro declarada num método é local a esse método, ou seja, só é conhecida dentro desse método;
- Isto permite que se tenham duas variáveis/parâmetros com o mesmo nome, desde que sejam declaradas em métodos diferentes.

Então, em termos de memória, os dois parâmetros `m` não têm nada em comum, pois são declarados em métodos diferentes. São completamente independentes.

Se já experimentou executar os programas ao longo destas páginas, já está familiarizado com a forma como é usual representar o conteúdo da memória, compartimentada em blocos locais aos métodos que estão em execução.

O programa `TabelaDeTabuadas` da secção 6.3.1. é muito revelador dessa "localidade" das variáveis, pois em muitos dos seus passos de execução existem três blocos de memória, associados aos três métodos (`main` e `imprimeTabuadas` em *standby* e `imprimeMultiplos` em execução).

Para quem, por alguma razão, não experimentou executar esse programa, vamos supor que o programa `TabelaDeTabuadas` está em execução e que a instrução que está a ser executada é `System.out.print(n * i + " ")` do método `imprimeMultiplos` para valores 3 e 10 dos seus parâmetros e quando a variável `i` local ao método tem o valor 5.

Esta execução, em particular, deste método, terá resultado da invocação `imprimeMultiplos(3,10)` a partir do método `imprimeTabuadas`, quando a variável local `linha` tem o valor 3.

Podemos representar o estado da memória nesse momento da seguinte forma:

main:4	
imprimeTabuadas:10	
limInf	1
limSup	9
m	10
linha	3
imprimeMultiplos:16	
n	3
m	10
i	5

Repare nas duas variáveis de nome `m`, uma em cada bloco local a um método. Embora neste caso as duas refiram exatamente o mesmo valor, pois representam o mesmo conceito (só que em contextos diferentes), nem sempre isto acontece. Podemos ter duas variáveis com o mesmo nome, em contextos diferentes, que tenham significados e valores completamente diferentes.

Também os blocos das instruções `if` e `for` definem contextos locais para as variáveis neles definidas. Inspeccione o seguinte programa:

```
public class EscreveTabuada {  
  
    public static void main(String[] args) {  
  
        for (int i = 1 ; i <= 10 ; i++) {  
            System.out.println(i * 7);  
        }  
        System.out.println(i * 7);  
    }  
}
```

Como o ciclo termina quando `i` fica com o valor 11, poder-se-ia pensar que a última instrução (já fora do ciclo) imprimiria o valor 77. Mas isso não é verdade pois o programa tem um erro de compilação:

```
EscreveTabuada.java:8: error: cannot find symbol
    System.out.println(i * 7);
                       ^
symbol:   variable i
location: class EscreveTabuada
1 error
```

O problema aqui é que a variável `i` é local ao ciclo `for`, pois foi declarada no contexto do ciclo, na parte da inicialização. Por isso só é conhecida no âmbito do ciclo.

Se precisássemos mesmo de aceder à variável de progresso fora do ciclo, teríamos que a declarar fora do ciclo, para que o âmbito de uso dessa variável fosse alargado, neste caso ao método `main`.

```
public class EscreveTabuada {

    public static void main(String[] args) {

        int i;
        for (i = 1 ; i <= 10 ; i++) {
            System.out.println(i * 7);
        }
        System.out.println(i * 7);
    }
}
```

Nesta versão a variável `i` é declarada antes do ciclo e é inicializada já dentro do ciclo. A última instrução do `main` já estaria correta e teria como efeito a impressão do valor 77 no *standard output*.

A última instrução do pedaço de código seguinte também provocaria um erro semelhante. A variável `i` é local à instrução condicional pois é declarada dentro do bloco do `if`:

```
int n = 3;
if (n > 0) {
    int i;
    for (i = 1 ; i <= n ; i++) {
        System.out.println(i * 7);
    }
    System.out.println(i * 7);
}
System.out.println(i);
```

6.6. A instrução `while`

No início da Secção 6.2., quando introduzimos o ciclo `for`, dissemos que a instrução `for` deve ser usada em qualquer situação em que o número de vezes que se quer repetir uma dada tarefa é limitado.

Se no programa `TabelaDeTabuadas` tivéssemos invocado o método `imprimeTabuadas` com dois valores dados pelo utilizador, por exemplo, não haveria maneira de nós, programadores, sabermos quais seriam esses valores na altura em que escrevêssemos o programa. Seriam potencialmente diferentes de cada vez que o programa fosse executado. E isto é muito bom!

No que diz respeito aos ciclos, o que nos interessa é que, quaisquer que fossem os valores dados pelo utilizador, eles seriam conhecidos na altura em que os ciclos fossem executados; então, na verdade o número de vezes que aqueles ciclos são executados é sempre limitado.

Existem casos, no entanto, em que não é conhecido nenhum limite máximo para o número de vezes que um ciclo tem de ser executado. Considere, por exemplo, o caso em que, dado um valor `n`, queremos descobrir o menor número primo que é maior que `n`.

Uma estratégia possível seria investigar, para cada número `i` a partir de `n + 1`, se `i` é primo e parar a pesquisa logo que o primeiro número primo fosse encontrado.

A questão aqui é que não é conhecido, à partida, o número máximo de vezes que o ciclo vai ser executado pois não se sabe quantos números, no máximo, vão ter que ser investigados (não é conhecido, em geral, qual o intervalo entre primos sucessivos).

Então, o número máximo de vezes que o ciclo é executado não é conhecido e não está, *a priori*, limitado. Nestes casos em que queremos exprimir repetição não limitada, usamos outra instrução de repetição – o ciclo `while`.

6.6.1. Sintaxe e semântica

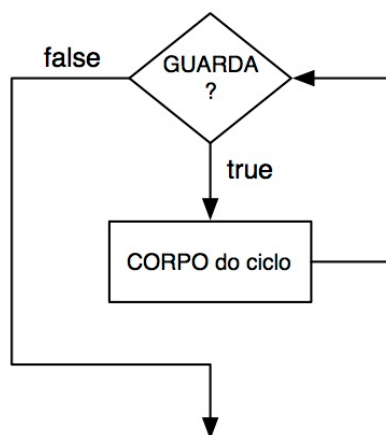
A sintaxe geral do ciclo `while` pode ser descrita por:

```
while (guarda)
    corpo-do-ciclo
```

A `guarda` é uma expressão do tipo `boolean` (ou *condição*).

O `corpo-do-ciclo` é um bloco de instruções entre chavetas.

O fluxo de execução do ciclo `while` é representado pela seguinte figura.



Um exemplo:

```
/**
 * O menor primo maior que um dado numero
 * @param n - O numero
 * @requires n > 0
 * @return O menor primo maior que n
 */
static int menorPrimoMaiorQue (int n) {

    int palpite = n + 1;

    while (menorDivisorMaiorQueUm(palpite) != palpite) {
        palpite++;
    }
}
```

```

    }

    return palpite;
}

```

Usamos o método `menorDivisorMaiorQueUm`, que fizemos na Secção 6.2.7, para testarmos se um número é primo – se a função, aplicada a um valor `v`, retorna o próprio `v`, é porque `v` é primo.

Vamos supor que a função `menorPrimoMaiorQue` é invocada com o valor 20 para o parâmetro `n`. A função começa por considerar o primeiro número maior que `n`, que é 21.

Esta primeira instrução funciona como que uma inicialização do ciclo pois a guarda depende do valor que a variável `palpite` vai tomando.

Acompanhando o fluxograma da figura da página anterior, vemos que a guarda é agora avaliada. Como a função `menorDivisorMaiorQueUm` aplicada a 21 dá 3, a guarda avalia para `true`. Segue-se a execução do corpo do ciclo, que incrementa a variável `palpite` para 22.

A guarda é de novo avaliada e de novo avalia para `true`. O corpo do ciclo é de novo executado e `palpite` fica com o valor 23.

Quando a guarda é de novo avaliada, o seu valor é `false` pois a função `menorDivisorMaiorQueUm` aplicada a 23 retorna 23, que não é diferente de 23. Então o ciclo termina e a função `menorPrimoMaiorQue` também, retornando o valor 23.

Esta função pode ser programada de forma mais eficiente. Tente encontrar passos desnecessários.

O corpo do ciclo deve mudar algo no estado do programa de modo a que a guarda, mais tarde ou mais cedo, se torne falsa e o ciclo termine. Se isso não acontecer temos um ciclo infinito.

Note que se a guarda é falsa logo à partida, o corpo do ciclo nunca chega a ser executado.

6.6.2. Mais exemplos

Se quisermos contar o número de algarismos de um dado inteiro, podemos dividi-lo por 10 (divisão inteira) sucessivamente obtendo, a cada divisão, um número com menos um algarismo. Se contarmos o número de vezes que conseguimos fazer isto, então ficamos a conhecer quantos algarismos tem o número original.

Seja `n` o número dado; queremos então repetir o seguinte bloco de instruções:

```

{
    n = n / 10;
    count++;
}

```

Quantas vezes, no máximo, temos que repetir este bloco? Não sabemos ao certo quantas vezes, só sabemos que devemos repetir até não conseguirmos reduzir mais o número. Então vamos usar um ciclo `while` pois trata-se de repetição não limitada.

Qual a pergunta que temos que fazer para saber se ainda conseguimos reduzir o número? Se considerarmos somente valores de `n` não negativos, se `n` for maior ou igual a 10 ainda conseguimos diminuir-lhe um algarismo.

```

while (n >= 10) {
    n = n / 10;
    count++;
}

```

Falta inicializar a variável `count`. Com que valor? Como o corpo do ciclo só é executado para valores maiores ou iguais a 10, devemos dar a `count` um valor inicial que represente o número de algarismos que os números menores que 10 têm. Devemos inicializá-la com o valor 1 pois todos os números não negativos menores que 10 têm um algarismo.

```
/**
 * O número de algarismos de um dado numero
 * @param n - O numero
 * @requires n >= 0
 * @return O numero de algarismos de n
 */
static int quantosAlgarismos (int n) {
    int count = 1;
    while (n >= 10) {
        n = n / 10;
        count++;
    }
    return count;
}
```

Quais os casos críticos que devemos analisar? Quando o parâmetro `n`:

- é zero
- está no intervalo $]0,10[$
- é 10

A forma como fomos construindo a solução desta função levou-nos a restringir a sua utilização. A pré-condição – condição descrita com a tag `@requires` – desta função diz-nos que só a devemos invocar para valores não negativos do seu parâmetro.

Se ignorarmos a pré-condição e invocarmos esta função com valores negativos obteremos resultados errados (exceto em algumas situações; sabe quais são?).

Se quisermos estender a utilização deste método a todos os valores negativos podemos trabalhar com o valor absoluto do parâmetro:

```
/**
 * O número de algarismos de um dado numero
 * @param n - O numero
 * @return O numero de algarismos de n
 */
static int quantosAlgarismos (int n) {

    int count = 1;
    int num = Math.abs(n);

    while (num >= 10) {
        num = num / 10;
        count++;
    }

    return count;
}
```

Experimente executar um programa que usa esta função, na versão online deste livro.

6.7. A instrução `do-while`

Outra instrução Java que nos permite obter repetição não limitada é a instrução `do-while`.

A grande diferença entre este e o ciclo `while` está na sequência de passos que são dados. Aqui o bloco é executado primeiro e só depois é testada a guarda.

6.7.1. Sintaxe e semântica

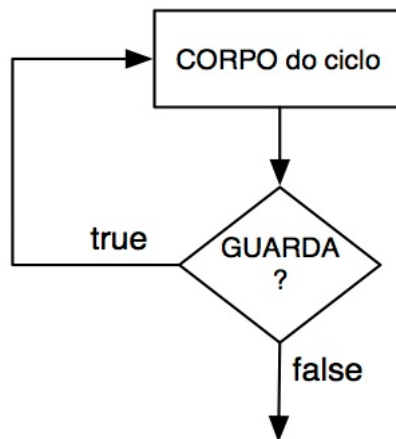
A sintaxe geral do ciclo `do-while` pode ser descrita por:

```
do
    corpo-do-ciclo
while(guarda)
```

A `guarda` é uma expressão do tipo `boolean` (ou *condição*).

O `corpo-do-ciclo` é um bloco de instruções entre chaves.

O fluxo de execução do ciclo `do-while` é representado pela seguinte figura.



Na instrução `do-while` o corpo do ciclo é executado uma ou mais vezes (lembre-se que no ciclo `while` o corpo do ciclo é executado zero ou mais vezes pois a guarda pode ser falsa logo à primeira).

Vamos exemplificar este ciclo reprogramando o último exemplo da secção anterior:

```
static int quantosAlgarismos (int n){

    int count = 0;
    int num = Math.abs(n);

    do {
        num = num / 10;
        count++;
    } while (num > 0);

    return count;
}
```

Analise as alterações feitas e tente compreender a sua razão. Verifique que funciona bem para os casos críticos.

Pode experimentar executar um programa que usa esta versão do método, na versão online deste livro.

7. Abstração de dados

No Capítulo 4 introduzimos a abstração procedimental e a de parâmetro que nos permitem,

- por um lado, facilitar a tarefa de criar programas
 - é mais fácil decidir como automatizar uma tarefa quando abstraímos os pormenores;
 - é mais fácil descobrir e corrigir erros quando os podemos delimitar
- e, por outro, obter código reutilizável
 - ao definirmos parâmetros, aumentamos o potencial de reutilização de um método.

Vamos agora introduzir uma nova forma de abstração – a *abstração de dados* – que potencia a nossa capacidade de construir programas *modulares*, ou seja,

- formados por módulos independentes,
- módulos estes possivelmente construídos por equipas diferentes e
- ligados de modo a alcançar um dado objetivo.

7.1. Tipos de dados não primitivos

No Capítulo 3 dissemos que um tipo de dados é definido por:

- um conjunto de valores e
- um conjunto de operações sobre esses valores.

Quando pensamos no tipo `int`, por exemplo, e no que podemos fazer com valores desse tipo, estamos na verdade a *abstrair* uma série de pormenores de implementação como, por exemplo, a forma como esses valores são guardados em memória, a forma como são feitas as operações sobre esses valores, etc.

Relembre que os tipos de dados `int`, `double` e `boolean` que usámos até agora são o que se chama de tipos de dados primitivos no Java porque estão incluídos na própria linguagem.

Podemos pensar em tipos de dados mais complexos, não só em termos dos valores que representam, mas também em termos das operações que podemos usar sobre esses valores.

Por exemplo, podemos pensar no conceito de *aluno* como um tipo de dados a que poderíamos chamar `Aluno`. Para isso, deveríamos definir:

- o conjunto de *valores* do tipo de dados `Aluno`: seria o conjunto de todos os alunos que pudessem existir (pessoas definidas, por exemplo, por um número, um nome e as notas obtidas nas disciplinas já finalizadas);
- o conjunto de *operações* sobre os valores do tipo de dados `Aluno`: poderia incluir, por exemplo, operações que permitissem conhecer a nota obtida pelo aluno a uma dada disciplina, conhecer o nome ou o número do aluno, registar uma dada nota numa nova disciplina, calcular a média do aluno, etc.

O tipo de dados `Aluno` seria um *tipo de dados não primitivo*, pois não está definido de base na linguagem Java. Ele seria definido através de uma **classe** de nome `Aluno`, onde se descreveria o conjunto de valores e de operações que se poderiam realizar sobre esses valores.

Quando, num programa, precisássemos de trabalhar com informações sobre alunos, poderíamos, por exemplo, criar uma variável do tipo `Aluno`, definir o seu valor ou *estado* (qual o seu nome, o seu número e as notas obtidas). Poderíamos de seguida aplicar sobre esse aluno todas as operações definidas no tipo de dados `Aluno`. Seríamos aquilo a que se chama *clientes* do tipo de dados `Aluno` porque o nosso código Java usaria esse tipo de dados.

A ideia é que os clientes de um dado tipo de dados usam-no sem precisarem de conhecer os pormenores de implementação desse tipo de dados, ou seja, *abstraindo* esses pormenores.

Por isso usamos a expressão *abstração de dados* para nos referirmos à separação entre

- as propriedades abstratas de um tipo de dados – aquelas que são visíveis para os clientes desse tipo e
- os detalhes concretos da sua implementação – o código contido na classe que define o tipo de dados.

Um tipo de dados não primitivo

- é definido por uma classe cujo nome é o nome do tipo de dados
- os seus valores chamam-se **objetos**, ou **instâncias da classe** que o define,
- as suas operações chamam-se **métodos**.

Prosseguindo a metáfora do “cliente”, podemos dizer que os objetos são *fornecedores* e as suas operações são os *serviços* que eles fornecem aos seus clientes.

7.2. Revisitando o tipo `String`

Já usámos um tipo de dados não primitivo! É o tipo `String` que é definido através da classe `String`.

Mas usámo-lo de forma muito básica, sem aproveitar o seu grande potencial. Vamos agora perceber porque é que o tipo `String` é tão importante na construção de programas.

Se as `Strings` são objetos, podemos perguntar

- “Quais os dados guardados num objeto do tipo `String`?” e
- “Que métodos podemos invocar sobre esse objeto?”.

Antes de respondermos a estas questões, vamos primeiro falar sobre mais um tipo de dados primitivo, fundamental para abordarmos as strings.

7.2.1. O tipo de dados primitivo `char`

Os componentes de um objeto do tipo `String` são letras, algarismos ou outros caracteres.

Cada um desses caracteres, separadamente, pode ser visto como um valor do tipo `char` – um outro tipo de dados primitivo que ainda não tínhamos visto e que representa caracteres individuais.

Os literais deste tipo são definidos entre pelicas, como em `'t'`, `'3'` e `'#'`.

Um exemplo de utilização de `char`:

```
char letra = 'w';

if (letra != '&') {
    System.out.println(letra);
}
```

Experimente executar o programa completo na versão online deste livro.

7.2.2. Métodos definidos no tipo `String`

Existem muitos métodos definidos no tipo `String`, mas vamos estudar só alguns.

Pode ver o resto em <https://docs.oracle.com/javase/9/docs/api/java/lang/String.html>, onde encontra a documentação completa da classe `String`.

O método `charAt` permite conhecer um carácter numa dada posição de uma `String`. Aplicado sobre uma dada `String`, devolve um valor do tipo `char`. A assinatura do método `charAt` é:

```
public char charAt(int index)
```

e a descrição sucinta é: "Returns the `char` value at the specified index."

Um exemplo de utilização do método `charAt`:

```
String animal = "tigre";
char letra = animal.charAt(1);
System.out.println(letra);
```

A expressão `animal.charAt(1)` significa que estamos a invocar o método `charAt` sobre a `String` contida na variável `animal`. O resultado é um carácter que é de seguida guardado numa variável do tipo `char` chamada `letra`.

Que letra pensa que iremos obter com a expressão `animal.charAt(1)`? Execute o programa com estas instruções na versão online deste livro.

Se calhar estava à espera que aparecesse a primeira letra de "tigre" e afinal aparece a letra `i`...

Se tivesse lido com atenção a documentação do método `charAt`, na API da classe `String`, teria percebido que:

- as posições numa `String` são numeradas a partir de zero e, por isso, a letra na posição 1 da `String` é a segunda letra da `String`.

No geral, o carácter na posição `n` de uma string é o seu `(n+1)`-ésimo carácter.

charAt

```
public char charAt(int index)
```

Returns the char value at the specified index. An index ranges from 0 to `length() - 1`. The first char value of the sequence is at index 0, the next at index 1, and so on, as for array indexing.

If the char value specified by the index is a [surrogate](#), the surrogate value is returned.

Specified by:

`charAt` in interface [CharSequence](#)

Parameters:

`index` - the index of the char value.

Returns:

the char value at the specified index of this string. The first char value is at index 0.

Throws:

[IndexOutOfBoundsException](#) - if the `index` argument is negative or not less than the length of this string.

Então, se queremos a primeira letra de uma `String`, temos que dar o valor zero ao parâmetro do método:

```
char letra = animal.charAt(0);
```

7.2.3. Métodos de instância versus métodos de classe

Repare bem na forma como invocámos o método `charAt` – está precedido por `animal.`, isto é,

- por uma expressão do tipo `String`
- seguida de um ponto.

É assim que dizemos que queremos que o método `charAt` “trabalhe” sobre a `String` guardada na variável `animal`. Se invocarmos o mesmo método sobre uma `String` diferente, o seu resultado é potencialmente diferente.

As instruções que se seguem exemplificam como o mesmo método, invocado com o mesmo valor para o parâmetro, sobre três `String`s diferentes, devolve três resultados diferentes.

```
char c1 = animal.charAt(0);
char c2 = "papagaio".charAt(0);
String outro = "baleia";
char c3 = outro.charAt(0);
```

Experimente executar um programa com estas e outras instruções na versão online deste livro.

A variável `c1` fica com valor `'t'`, `c2` com o valor `'p'` e `c3` com o valor `'b'`.

Lembra-se dos métodos da classe `Math`? A forma como invocamos esses métodos é escrevendo `Math.` seguido do nome do método e dos valores para os parâmetros (se for caso disso), como no caso do cálculo da raiz quadrada de 7:

```
double raiz = Math.sqrt(7);
```

Temos que fazer assim porque todos os métodos na classe `Math` são definidos como `static`, significando que são *métodos de classe*.

Os métodos de classe (`static`) não são invocados sobre nenhum objeto em particular – o seu resultado depende unicamente dos valores dados para os parâmetros.

De facto, ao contrário da classe `String`, a classe `Math` não tem objetos (ou instâncias).

Se inspecionar a API da classe `String`, verificará que o método `charAt` não é `static`, nem a maior parte dos métodos dessa classe.

Isto significa que têm que ser invocados sobre objetos específicos do tipo `String` e que o seu resultado depende não só dos valores dados para os parâmetros, mas também das características específicas desses objetos (o seu *estado*).

Os métodos que **não** são `static` dizem-se *métodos de instância* pois são invocados sobre um objeto ou instância da classe em que estão definidos.

Dizemos que o objeto sobre o qual o método é invocado é o *alvo da invocação*.

7.2.4. Comprimento de uma `String`

O método que, na classe `String`, permite conhecer o comprimento de uma `String` é o método de instância `length` que, quando invocado sobre uma dada `String` retorna um inteiro representando o comprimento dessa `String`. Não tem parâmetros. Exemplo:

```
int quantos = animal.length();
System.out.println( "papagaio".length() );
```

A variável `quantos` ficará com o valor 5 e o valor 8 será escrito no ecrã.

Para conhecer a última letra de uma string poderá sentir-se tentado a experimentar algo como:

```
char ultimo = animal.charAt( animal.length() );           // ERRADO!!!
```

Isto não funciona. A razão é que não existe uma letra na posição cinco em "tigre". Como o primeiro carácter de uma `String` é o que está na posição zero, então as cinco letras de "tigre" estão nas posições zero a quatro. Para obtermos o último carácter, temos que subtrair 1 de `length`.

```
char ultimo = animal.charAt( animal.length() - 1 );        // CERTO
```

7.2.5. Travessia de uma `String`

É muito comum precisarmos de percorrer, um a um, os caracteres de uma `String` usando-os para algum objetivo. Uma forma natural de fazer essa travessia é usando uma instrução `for`:

```
String palavra = "papagaio";

for (int index = 0 ; index < palavra.length() ; index++) {
    char letra = palavra.charAt(index);
    if (letra == 'a') {
        System.out.print('@');
    } else {
        System.out.print(letra);
    }
}
```

Este ciclo atravessa a `String` contida na variável `palavra` e imprime cada um dos seus caracteres no ecrã ou `@` se o carácter for `a`. Os caracteres seguintes serão escritos no ecrã:

```
p@p@g@io
```

Repare que a guarda é `index < palavra.length()`, o que significa que, quando a variável `index` é igual ao comprimento da `String`, a guarda é falsa e o corpo do ciclo já não é executado – o último carácter que acedemos é o que está na posição `palavra.length() - 1`, que é exatamente o que pretendemos.

Experimente executar o programa completo na versão online deste livro.

Escolhemos `index` para o nome da variável de progresso do ciclo. Um *índice* ou *index* é um valor usado para identificar um elemento de um conjunto ordenado, neste caso a `String` de caracteres. O índice indica (daí o nome) qual o elemento que pretendemos.

7.2.6. Erros de execução

Na Secção 2.4.2 falámos sobre erros de execução, erros que só aparecem quando um programa já está em fase de execução. Em alguns casos os erros provocam aquilo a que se chama, em Java, de *exceções*, que podem ajudar o programador a corrigir o seu programa.

O método `charAt` provoca uma exceção quando é invocado com um valor para o parâmetro que é negativo ou maior ou igual ao comprimento da `String` sobre o qual é invocado.

Quando isso acontece, o Java imprime uma mensagem de erro indicando o tipo da exceção que ocorreu e a lista dos métodos que estavam em execução nessa altura (*stack trace*).

Um exemplo:

```
public class ExemploErroExecucao {

    public static void main(String[] args) {
        imprimeUltima("papagaio");
        imprimeUltima("tigre");
    }

    public static void imprimeUltima(String s) {
        char c = ultimaLetra(s);
        System.out.println(c);
    }

    public static char ultimaLetra(String s) {
        int index = s.length();
        char c = s.charAt(index);    // ERRO!
        return c;
    }
}
```

Repare no erro no método `ultimaLetra`: o índice usado para ir buscar o último carácter devia ser `index - 1`.

O programa compila sem problemas mas, quando executado, imprime:

```
Exception in thread "main" java.lang.StringIndexOutOfBoundsException: String index out of
range: 8

    at java.lang.String.charAt(String.java:686)
    at ExemploErroExecucao.ultimaLetra(ExemploErroExecucao.java:15)
    at ExemploErroExecucao.imprimeUltima(ExemploErroExecucao.java:9)
    at ExemploErroExecucao.main(ExemploErroExecucao.java:4)
```

e de seguida termina a sua execução. Esta lista (*stack trace*) pode ser difícil de ler, mas contém muita informação útil.

Mais para a frente veremos que o efeito das exceções não tem que ser sempre este. Há casos em que o programa pode "recuperar" de uma exceção, sem necessariamente ter que terminar.

7.2.7. O método `indexOf`

O método `indexOf` é como que o inverso de `charAt`: dado um índice, o método `charAt` retorna o carácter que está na posição dada pelo índice; dado um carácter, o método `indexOf` devolve o índice (posição) onde o carácter aparece.

Já viu que o método `charAt` falha quando o índice está fora do intervalo permitido, lançando uma exceção. O método `indexOf` não consegue determinar o índice de um carácter que não aparece na `String` mas, em vez de lançar uma exceção, retorna o valor -1.

```
int index1 = "papagaio".indexOf('t');
int index2 = "papagaio".indexOf('a');
```

Este pedaço de código atribui à variável `index1` o valor -1 pois a letra `'t'` não existe na `String` "papagaio".

Atribui ainda à variável `index2` a posição da letra `'a'` na `String` "papagaio". Neste caso a letra aparece três vezes e por isso não é óbvio o resultado que `indexOf` produz. De acordo com a documentação, o método retorna o índice da *primeira* ocorrência do carácter (neste caso o valor 1, pois a letra `'a'` ocorre pela primeira vez na posição 1 da `String`).

Para encontrar posteriores ocorrências de uma letra, pode usar-se uma outra versão de `indexOf`, que, além do carácter a procurar, pede também a posição a partir da qual deve fazer a procura.

7.2.8. As `Strings` são imutáveis

Na documentação do método `toUpperCase` da classe `String` está escrito:

Converts all of the characters in this `String` to upper case...

Podemos ser levados a pensar que o método altera a `String` sobre a qual é chamado. Este é um engano comum a quem começa a usar esta classe.

Nenhum dos métodos da classe `String` altera as `Strings` sobre as quais são invocados

- os objetos do tipo `String` são *imutáveis*, isto é, o seu estado nunca muda.

No caso do método `toUpperCase`, a sua invocação devolve uma nova `String`. Por exemplo:

```
String animal = "papagaio";
String animalMaiusculas = animal.toUpperCase();
```

Após a execução destas duas instruções, a variável `animalMaiusculas` contém a `String` "PAPAGAI0", mas `animal` ainda contém "papagaio".

7.2.9. Comparação de `Strings`

É muitas vezes necessário comparar `Strings` de modo a verificar se são iguais. As instruções seguintes

```
String animal1 = "papagaio";
String animal2 = "PAPAGAI0".toLowerCase();

if (animal1 == animal2) {
    System.out.println("Iguais");
} else {
    System.out.println("Diferentes");
}
```

imprimem a palavra *Diferentes* embora os valores das variáveis `animal1` e `animal2` sejam textualmente iguais.

A razão para isso prende-se com o facto de as variáveis `animal1` e `animal2` serem do tipo `String`, que é um tipo não primitivo.

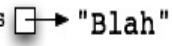
Numa atribuição a uma variável de um tipo não primitivo, a expressão à direita da atribuição denota um objeto.

O conteúdo dessa variável passa a ser uma *referência* para esse objeto.

Por exemplo, a instrução seguinte atribui um objeto do tipo `String` a uma variável do mesmo tipo (não primitivo):

```
String s = "Blah";
```

Embora o "simulador" de execução de programas que temos usado nestas páginas represente o conteúdo das variáveis do tipo `String` como se de um tipo primitivo se tratasse, no entanto, a forma *standard* de representar graficamente o conteúdo de variáveis de tipos não primitivos é:

s  "Blah"

Como usualmente, o nome da variável `s` aparece fora da caixa e o seu valor dentro da caixa. Neste caso, o valor é uma referência, representada graficamente por uma seta. A seta aponta para o objeto que a variável refere.

Agora percebe porque é que as instruções anteriores imprimem *Diferentes* em vez de *Iguais*?

Se não, repare: o conteúdo de cada uma das variáveis é uma referência para um objeto:

animal1  "papagaio"

animal2  "papagaio"

Embora os objetos sejam iguais, as referências não o são (pode imaginar uma referência a um objeto como a morada onde esse objeto "vive" em memória).

Como o operador relacional `==` compara o conteúdo das variáveis, o valor da expressão `animal1 == animal2` é `false`.

É por esta razão que não se devem comparar expressões de tipo não primitivo usando o operador `==`. A comparação só dá `true` quando se compara um objeto com ele próprio.

Então como podemos comparar `Strings`?

- usando o método `equals`, que recebe como parâmetro um objeto e o compara com a `String` sobre a qual o método é invocado;
- o resultado será `true` se as duas `Strings` forem iguais, ou seja, se forem formadas pelos mesmos caracteres, pela mesma ordem.

As seguintes instruções já imprimem a palavra *Iguais*:

```
String animal1 = "papagaio";
String animal2 = "PAPAGAI0".toLowerCase();
if (animal1.equals(animal2)) {
    System.out.println("Iguais");
} else {
    System.out.println("Diferentes");
}
```

Pode experimentar executar um programa que usa esta versão do método, na versão online deste livro.

8. Algumas classes da biblioteca do Java

As `Strings` são objetos, mas são objetos atípicos porque:

- são imutáveis;
- podem ser criadas a partir de um literal do mesmo tipo (uma cadeia de caracteres delimitada por aspas).

Neste capítulo introduzimos as seguintes três classes da biblioteca do Java:

- `Random`;
- `StringBuilder`;
- `Scanner`.

Todas elas representam classes cujas instâncias (objetos) são *mutáveis*, ou seja, podem mudar de estado como consequência da invocação de determinados métodos.

8.1. Pacotes

A biblioteca Java está dividida em *packages*, ou pacotes, incluindo o pacote `java.lang`, ao qual pertencem as classes que usamos até aqui – `String` e `Math`.

Outros pacotes incluem classes destinadas a utilizações várias como, por exemplo, o pacote `java.util`, que contém classes de utilidade variada, ou o pacote `java.io`, que contém classes para *input/output*, e muitos outros.

Para usarmos uma classe definida noutro pacote, a partir da nossa classe, temos que indicar a que pacote pertence:

- escrevendo o nome da classe precedido do nome do pacote a que pertence, sempre que nos referimos a ela ao longo do programa, ou
- usando uma declaração `import`
 - todas as declarações de `import` aparecem no início do ficheiro que contém a classe que estamos a escrever, antes da definição da classe;
 - por exemplo, a declaração `import java.io.FileReader;` numa classe `MyClass` diz ao compilador que, sempre que o nome `FileReader` aparece no texto de `MyClass`, ele refere-se à classe `FileReader` do pacote `java.io`.

Como exemplo, as classes `Random` e `Scanner` pertencem ao pacote `java.util`, por isso “importamo-las” da seguinte forma:

```
import java.util.Random;
import java.util.Scanner;
```

As classes que pertencem ao pacote `java.lang`, como `String` e `Math`, são importadas automaticamente, o que explica o porquê de nunca termos precisado de fazer nenhuma declaração `import` até agora.

8.2. Geração de valores aleatórios

A maior parte dos programas fazem a mesma coisa sempre que são executados com os mesmos dados, por isso dizemos que são deterministas. Normalmente é bom que um programa seja determinista, já que esperamos que o mesmo cálculo, feito com os mesmos valores, tenha o mesmo resultado. Mas, para algumas aplicações, gostaríamos de ter um programa imprevisível – por exemplo, os jogos.

Tornar um programa verdadeiramente imprevisível não é fácil mas há formas de o fazer *parecer* imprevisível. Uma delas é gerar valores aleatórios e usá-los para determinar o resultado do nosso programa.

A biblioteca do Java fornece uma classe que gera valores pseudo aleatórios que, apesar de não serem puramente aleatórios, servem o nosso objetivo.

8.2.1. A classe `Random`

Consulte a documentação da classe `Random` do pacote `java.util` em <https://docs.oracle.com/javase/9/docs/api/java/util/Random.html>

Uma instância desta classe é um objeto que sabe gerar valores aleatórios.

A classe define vários métodos de instância que retornam valores aleatórios de vários tipos. Por exemplo,

- o método `int nextInt(int max)` retorna um inteiro aleatório no intervalo $[0, max - 1]$;
- o método `int nextInt()` retorna um inteiro aleatório no domínio definido para o tipo `int`;
- o método `boolean nextBoolean()` retorna um `boolean` aleatório;
- e ainda outros que poderá consultar na documentação.

8.2.2. Criação de objetos

Antes de podermos usar um objeto do tipo `Random` para gerarmos valores aleatórios, temos que o criar.

- Criamos novos objetos usando o operador `new`.
 - Exemplo: `Random gerador = new Random();`
- O operador `new` instancia a classe indicada (i.e., cria um novo objeto desse tipo)
 - reservando espaço para um novo objeto e
 - retornando uma referência para esse local de memória.

Também invoca o construtor da classe (mais sobre isto adiante neste texto).

O operador `new` é um operador unário, ou seja, requer um só operando:

- o operando é a invocação de um *construtor*, ou seja,
 - o nome da classe a instanciar, seguido de parêntesis delimitando zero ou mais valores para parâmetros;
- é o nome desse construtor que indica o nome da classe a ser instanciada.

O operador `new` retorna uma referência para o objeto acabado de criar; esta referência pode ser atribuída a uma variável de tipo apropriado, como na instrução acima.

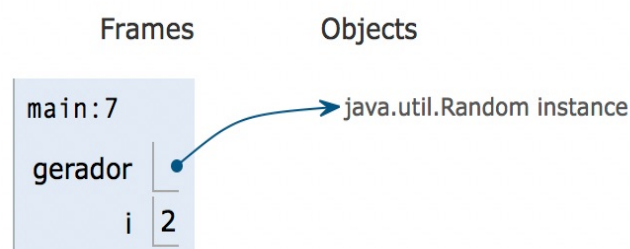
Como exemplo, as instruções seguintes geram dez inteiros aleatórios no intervalo $[0,5]$ e imprimem-nos no ecrã:

```
Random gerador = new Random();
for (int i = 1; i <= 10; i++) {
    System.out.println(gerador.nextInt(6));
}
```

Experimente executar o programa completo na versão online deste livro.

Se executar várias vezes o mesmo programa, os valores gerados não serão os mesmos, com toda a probabilidade. No entanto, o "simulador de execução de programas" que usamos acima não nos permite verificar isso, devido à forma como é embebido no código *html* destas páginas.

Se experimentou executar o programa acima, reparou na forma como o conteúdo da variável `gerador` foi representado.



O "simulador de execução de programas" representa os objetos no lado direito, numa coluna de título "*Objects*" e as variáveis numa coluna de título "*Frames*".

Como a variável `gerador` é de um tipo não primitivo, é usada uma seta para representar o seu conteúdo, que é uma referência a um objeto.

Esse objeto é representado aqui somente pelo texto "*java.util.Random instance*". Veremos mais para a frente exemplos mais ricos.

Para gerar números aleatórios no intervalo $[1,5]$:

```
Random gerador = new Random();
for (int i = 1; i <= 10; i++) {
    int n = gerador.nextInt(5) + 1;
    System.out.println(n);
}
```

Para gerar números aleatórios no intervalo $[-val, val]$ para um dado valor *val*:

```
Random gerador = new Random();
for (int i = 1; i <= 10; i++) {
    int n = gerador.nextInt(2 * val + 1) - val;
    System.out.println(n);
}
```

De forma mais geral, o método seguinte gera um número aleatório num intervalo $[min, max]$:

```
/**
 * Um numero aleatorio num dado intervalo, gerado por um dado gerador
 * @param min O limite inferior do intervalo
 * @param max O limite superior do intervalo
 * @param g O gerador de aleatorios que deve ser usado
 * @return Um inteiro aleatorio no intervalo [min,max], gerado pelo
 *         gerador g
 * @requires min <= max && g != null
 */
public static int aleatorioIntervalo(int min, int max, Random g) {
    return g.nextInt(max - min + 1) + min;
}
```

A sequência de instruções para gerar números aleatórios no intervalo $[1,5]$ apresentada acima tem o mesmo efeito que:

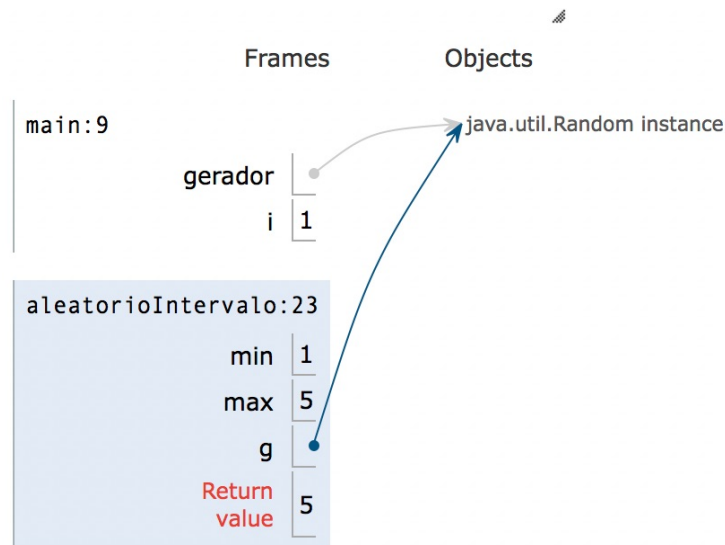
```
Random gerador = new Random();
for (int i = 1; i <= 10; i++) {
    System.out.println(aleatorioIntervalo(1,5,gerador));
}
```

Um objeto do tipo `Random` tem um estado; sempre que lhe pedimos um novo valor aleatório, ele muda de estado.

A ideia de definir o método `aleatorioIntervalo` com um parâmetro do tipo `Random` é boa, pois permite que seja o invocador do método que define qual o objeto `Random` que vai criar o valor aleatório, em vez de um novo objeto `Random` ser criado dentro do método, sempre que ele é invocado.

Experimente executar o programa na versão online deste livro.

Se executou o programa acima, reparou, de novo, na forma como foi representado o conteúdo das variáveis de tipo não primitivo:



Quando um método é invocado, o valor indicado para o parâmetro, na invocação, é copiado para o parâmetro do método. Neste caso particular, o valor da variável `gerador` é copiado para o parâmetro `g`.

Como estas duas variáveis são de um tipo não primitivo, já sabemos que o conteúdo de `gerador` é uma referência a um objeto. Então, o valor que é copiado para `g` é essa mesma referência.

Na representação da memória, acima, uma referência é representada por uma seta. Por isso, quando duas variáveis contêm referências iguais, as respectivas setas apontam para o mesmo objeto.

Se usarmos dois objetos `Random` criados da seguinte forma:

```
Random g1 = new Random();
Random g2 = new Random();
```

para gerar valores aleatórios sucessivos, exatamente da mesma maneira com cada um deles, como em:

```
for (int i = 1; i <= 10; i++) {
    int n1 = g1.nextInt();
    int n2 = g2.nextInt();
    System.out.println(n1 + " " + n2);
}
```

não só `g1` cria 10 valores pseudo aleatórios e `g2` cria 10 valores pseudo aleatórios, como os valores que `g1` cria são perfeitamente independentes dos valores criados por `g2`. Além disso, a cada nova execução deste conjunto de instruções são gerados valores potencialmente diferentes das execuções anteriores.

Experimente executar o programa com estas instruções na versão online deste livro.

Podemos criar objetos `Random` usando outro construtor – o construtor `Random(int seed)` – com o qual podemos definir o valor inicial que serve de base para a criação dos valores aleatórios – a *semente*. Se criarmos dois objetos `Random` com a mesma semente e os fizermos gerar dez valores cada, como nas instruções que se seguem:

```
Random g1 = new Random(1); // usamos uma semente 1
Random g2 = new Random(1); // aqui tambem

for (int i = 1; i <= 10; i++) {
    int n1 = g1.nextInt();
    int n2 = g2.nextInt();
    System.out.println(n1 + "    " + n2);
}
```

então `g1` e `g2` geram 10 valores pseudo aleatórios cada um, mas os valores que `g1` gera são iguais aos valores gerados por `g2` pois a semente usada na construção dos dois objetos foi a mesma. Repare que a única diferença entre este pedaço de código e o anterior é o construtor usado na criação dos objetos `Random`.

Experimente executar o programa com estas instruções na versão online deste livro.

De igual forma, um gerador criado com uma semente gerará sempre a mesma sequência de valores. Por isso, o seguinte programa escreve sempre os mesmos 5 valores no ecrã de cada vez que é executado.

```
public class GeracaoAleatoriosV2 {

    public static void main(String[] args) {

        Random gerador = new Random(1); // definimos a semente
        for (int i = 1; i <= 5; i++) {
            System.out.println(gerador.nextInt());
        }
    }
}
```

8.3. Construindo uma `String` passo a passo

Relembre que as `String`s são imutáveis, ou seja, não existem métodos que permitam modificar a sequência de caracteres que compõe uma `String`.

Por isso, quando queremos construir uma `String` a pouco e pouco, começando com uma dada sequência de caracteres e acrescentando, passo a passo, novos caracteres, podemos optar por fazer algo do estilo:

```
String s = "Blah";
for (int i = 1; i <= 10; i++) {
    s = s + " blah";
}
System.out.println(s);
```

A seguinte linha será escrita no *standard output*:

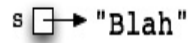
```
Blah blah blah blah blah blah blah blah blah
```

Mas esta não é a melhor forma de o fazer. Porquê?

Relembre que quando atribuímos um valor (objeto) a uma variável de tipo não primitivo, o conteúdo da variável não será o objeto propriamente dito mas uma referência para ele. O conteúdo da variável `s` após a execução da instrução

```
String s = "Blah";
```

é representado por

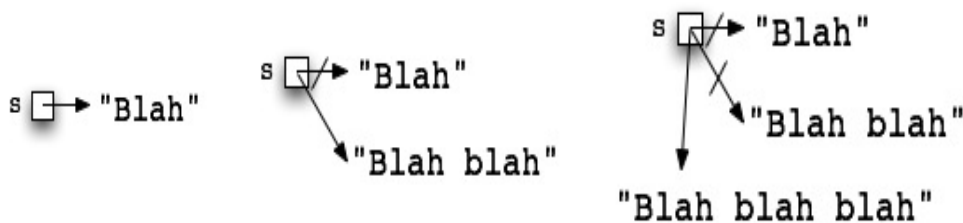


Como consequência, a execução das instruções acima

- não modifica o objeto referenciado por `s`;
- modifica, sim, o conteúdo da variável `s`

De cada vez que concatenamos `s` com a `String` `"blah"` estamos na verdade a criar um novo objeto do tipo `String`, e é esse novo objeto que passa a ser referenciado por `s`.

Ver imagem abaixo; uma seta com um traço significa que a referência contida na variável `s` já não é aquela.



Para evitar construir todos aqueles objetos intermédios (que não serão usados de novo), usamos um objeto que é especialista em construção de `Strings` – um objeto do tipo `StringBuilder`.

8.4. A classe `StringBuilder`

Consulte a documentação da classe `StringBuilder` do pacote `java.lang` em <https://docs.oracle.com/javase/9/docs/api/java/lang/StringBuilder.html>.

Uma instância desta classe é um objeto capaz de construir e manipular uma `String`.

As instruções seguintes exemplificam a utilização de um objeto do tipo `StringBuilder`.

```
StringBuilder texto = new StringBuilder("ola");
System.out.println(texto.toString());

texto.append(", bom dia");
System.out.println(texto.toString());

texto.insert(3, " Maria");
System.out.println(texto.toString());

texto.setCharAt(0, 'O');
System.out.println(texto.toString());
```

```
texto.delete(3,9);
System.out.println(texto.toString());
```

A invocação do método `toString()` sobre o `StringBuilder` permite obter a `String` que lhe está subjacente.

Tente perceber o que retorna cada método invocado sabendo que a execução destas instruções escreve no ecrã:

```
ola
ola, bom dia
ola Maria, bom dia
Ola Maria, bom dia
Ola, bom dia
```

Concluindo, as seguintes instruções ilustram uma forma mais correta (porque mais eficiente no uso da memória) de obter o efeito do exemplo da secção anterior:

```
StringBuilder sb = new StringBuilder("Blah");
for (int i = 1; i <= 10; i++) {
    sb.append(" blah");
}
System.out.println(sb.toString());
```

Aqui criamos um único objeto do tipo `StringBuilder` e modificamos o seu estado passo a passo.

No final invocamos o método `toString` sobre o `StringBuilder` para obter a `String` que lhe está subjacente e imprimimo-la no ecrã.

Experimente executar o programa na versão online deste livro, embora não consiga "espreitar" para dentro do objeto `StringBuilder`.

8.5. Input e Output em Java

`System.out` é um objeto. Este objeto sabe como escrever informação no *standard output* (em geral o ecrã). Quando invocamos os métodos `print` e `println`, invocamo-los sobre este objeto, ou seja, o `System.out` é o *alvo* da invocação destes métodos.

Também existe um objeto chamado `System.in` que permite ler valores do *standard input* (em geral o teclado). No entanto, este objeto não torna fácil essa leitura de valores a partir do teclado.

Vamos usar um outro objeto que nos vai facilitar a interpretação da informação obtida pelo `System.in`: uma instância da classe `Scanner` (do pacote `java.util`).

Este objeto vai-nos ajudar a “trazer para dentro do nosso programa” a informação que o utilizador digita no teclado.

Consulte a documentação da classe `Scanner` do pacote `java.util` em <https://docs.oracle.com/javase/9/docs/api/java/util/Scanner.html>.

8.5.1. Input do teclado – usando `Scanner`

Um objeto `Scanner` é um objeto que

- sabe como ir buscar um pedaço de informação – *token* (sequência de caracteres delimitada por um dado delimitador) – a uma dada fonte de caracteres, ou *canal de leitura*, e

- torna essa *token* acessível no programa (eventualmente convertida para um dado tipo de dados).

O objeto `System.in` representa um canal de leitura (em geral o teclado).

Como tal, podemos usá-lo como origem da informação, para que algum objeto `Scanner` possa “consumir” os *tokens* que o utilizador escreve.

Fazemos isso quando criamos o objeto do tipo `Scanner`, indicando que o canal de leitura que ele vai usar é o `System.in`:

```
Scanner leitor = new Scanner (System.in);
```

Agora já podemos invocar métodos que lêem informação a partir do teclado tendo como alvo da invocação este objeto.

Por exemplo, o método `String nextLine()` lê todos os caracteres até encontrar um carácter especial que representa um “fim de linha” (gerado, por exemplo, quando o utilizador carrega na tecla *Enter*) e retorna a `String` contendo esses caracteres.

Outros exemplos são o método `String next()` que lê a próxima *token* e retorna-a como uma `String`, e o método `int nextInt()` que lê a próxima *token* e retorna-a como um inteiro.

Por defeito, os delimitadores que definem até onde se estende uma *token* são o espaço, o *tab* ou o fim de linha.

As instruções seguintes:

```
Scanner leitor = new Scanner (System.in);
System.out.println("Qual o seu nome?");
String nome = leitor.nextLine();
System.out.println("Viva " + nome + "!");
System.out.println("E que idade tem?");
int idade = leitor.nextInt();
if (idade > 150) {
    System.out.println("Impossivel!");
}
```

produzem o seguinte efeito no ecrã caso o utilizador introduza os valores *Pai Natal* e *524*:

```
Qual o seu nome?
Pai Natal
Viva Pai Natal!
E que idade tem?
524
Impossivel!
```

Repare que o *input* do utilizador, embora escrito no teclado, é reproduzido pelo sistema operativo no ecrã.

Suponha que queremos garantir que o utilizador introduz um valor entre *1* e *20*. Se isso for importante, não basta dizer isso ao utilizador. Ele pode, mesmo sem querer, escrever um valor não aceitável.

É muito importante que o programa consiga recusar valores e repetir o pedido enquanto o utilizador não introduz um valor aceitável.

O programa seguinte pede uma nota entre *1* e *20* ao utilizador, só aceitando um valor que satisfaça essa condição.

```
import java.util.Scanner;
```

```

public class NotasInput {

    public static void main (String [] args) {

        Scanner leitor = new Scanner (System.in);
        System.out.println("Qual a nota? (valor entre 1 e 20)");
        double nota;
        boolean erro;
        do {
            nota = leitor.nextDouble();
            erro = nota < 1 || nota > 20;
            if ( erro ){
                System.out.println ("Tem que ser um valor entre 1 e 20!");
            }
        } while ( erro );

        System.out.println("Nota arredondada: " + Math.round(nota));
    }
}

```

Exemplo de execução deste programa:

```

Qual a nota? (valor entre 1 e 20)
-3.4
Tem que ser um valor entre 1 e 20!
25.4
Tem que ser um valor entre 1 e 20!
13.6
Nota arredondada: 14

```

Os métodos `int nextInt()` e `double nextDouble()`, que lêem a próxima *token* acessível no canal de leitura sobre o qual o `Scanner` trabalha, provocam um erro de execução no caso de a *token* lida não constituir um valor numérico do tipo correspondente.

Os valores que estes métodos conseguem manipular são

- uma sequência de algarismos no caso de `nextInt()`
- uma sequência de algarismos que pode ter um ponto no caso de `nextDouble()`.

Quando a próxima *token* no canal de leitura não tem estas características (por exemplo, porque contém uma letra ou carácter especial) o erro consequente é representado por uma exceção do tipo `InputMismatchException`.

Considere o seguinte pedaço de um programa:

```

Scanner leitor = new Scanner(System.in);
System.out.println("Escreva um inteiro");
int i = leitor.nextInt();
System.out.println("Escreva um numero com parte decimal");
double x = leitor.nextDouble();
System.out.println("Produto dos dois: " + i * x);

```

No caso em que o utilizador insere, por exemplo, 2 e 1.5, o efeito destas instruções é:

```
Escreva um inteiro
2
Escreva um numero com parte decimal
1.5
Produto dos dois: 3.0
```

Quando o utilizador insere, por exemplo, *1q3* em vez de um inteiro, o efeito é:

```
Escreva um inteiro
1q3

Exception in thread "main" java.util.InputMismatchException
    at java.util.Scanner.throwFor(Scanner.java:840)
    at java.util.Scanner.next(Scanner.java:1461)
    at java.util.Scanner.nextInt(Scanner.java:2091)
    at java.util.Scanner.nextInt(Scanner.java:2050)
    at Exemplo86.main(Exemplo86.java:63)
```

O mesmo aconteceria se o utilizador introduzisse qualquer carácter que não fosse um algarismo ou um ponto, quando lhe fosse pedido um número com parte decimal (através do método `nextDouble()`).

Se quisermos tornar o nosso programa robusto no sentido de não “rebentar” no caso de o utilizador não introduzir um número inteiro quando o programa quer consumir um número inteiro, podemos testar cada *token* que é lida do canal de leitura e só o aceitar se for realmente um inteiro.

Para isso usamos o método `hasNextInt()` que “espreita” para o canal de leitura e retorna `true` se a próximo *token* for um número inteiro e `false` caso contrário.

Conforme o resultado deste método (*true* ou *false*), podemos escolher

- consumir a *token* de forma útil invocando o método `nextInt()` e usando o seu resultado ou
- usar o método `next()`, que lê qualquer sequência de caracteres que constitui uma *token*, desprezando o seu resultado.

```
import java.util.Scanner;
public class LeituraRobusta {

    public static void main (String [] args) {
        Scanner leitor = new Scanner (System.in);
        System.out.println("Introduza um valor inteiro");
        int valor = 0;
        boolean erro = true;
        do {
            if (leitor.hasNextInt () ) {    // espreita a proxima token
                valor = leitor.nextInt (); // consome o inteiro
                erro = false;
            } else {
                leitor.next ();              // consome o que lah esteja
                System.out.println ("Tem que ser um inteiro!");
            }
        } while ( erro );
        System.out.println("O valor que introduziu eh: " + valor);
    }
}
```

Exemplo de execução deste programa:


```

Introduza um valor inteiro
1q3
Tem que ser um inteiro!
1.3
Tem que ser um inteiro!
123
O valor que introduziu eh: 123

```

Os seguintes métodos podem ser usados para fazer leitura robusta de inteiros num dado intervalo:

```

/**
 * Primeiro inteiro no canal de leitura que esta num dado intervalo
 * @param infLim - Limite inferior do intervalo
 * @param supLim - Limite superior do intervalo
 * @param errMess - Mensagem de erro a apresentar no System.out
 * @param sc - Canal de leitura
 * @return um valor entre infLim e supLim
 * @requires sc != null && infLim <= supLim && errMess != null
 */
static int lerValorNoIntervalo(int infLim, int supLim,
                               String errMess, Scanner sc) {
    int valor = 0;
    boolean erro;
    do {
        valor = lerInteiro ( errMess, sc );
        erro = valor < infLim || valor > supLim;
        if ( erro ){
            System.out.println ( errMess );
        }
    } while ( erro );
    return valor;
}

/**
 * Primeiro inteiro no canal de leitura
 * @param errMess - mensagem a escrever no System.out caso o valor
 *                  acessivel no canal de leitura nao seja um inteiro
 * @param sc - canal de leitura
 * @return valor inteiro
 * @requires errMess != null && sc != null
 */
private static int lerInteiro ( String errMess, Scanner sc ) {
    int valor = 0;
    boolean erro = true;
    do {
        if ( sc.hasNextInt () ) {
            valor = sc.nextInt (); // consome o inteiro
            erro = false;
        }
        else {
            sc.next ();           // consome o que lah esteja
            System.out.println ( errMess );
        }
    }
    while ( erro );
    return valor;
}

```

```
}
```

A seguinte instrução poderia então ser escrita num programa que tivesse acesso a estes métodos, para receber do utilizador um valor inteiro entre 1 e 20:

```
Scanner leitor = new Scanner (System.in);
System.out.println("Introduza um inteiro entre 1 e 20");
int nota = lerValorNoIntervalo(1,
                               20,
                               "Tem que ser um inteiro entre 1 e 20!",
                               leitor);
```

8.6. Aliasing

Lembre-se que o valor de uma expressão de tipo não primitivo é uma referência a um objeto. Então, quando atribuímos uma expressão de tipo não primitivo a uma variável, o valor da variável passa a ser uma referência.

É possível ter várias variáveis a referenciar o mesmo objeto. Por exemplo, as instruções seguintes

```
StringBuilder sb1 = new StringBuilder("Olah!");
StringBuilder sb2 = sb1;
```

geram um estado da memória que se pode representar assim:



As variáveis `sb1` e `sb2` contêm o mesmo valor: uma referência para o mesmo objeto. Por outras palavras,

podemos referir-nos a este objeto de duas formas: através de `sb1` e de `sb2`.

- Chama-se a isto *aliasing*.

Quando duas variáveis referenciam o mesmo objeto, quaisquer mudanças no objeto que uma delas refere, afetam também o objeto (o mesmo!) que a outra refere, pois na verdade existe um só objeto, que é partilhado pelas duas variáveis. Por exemplo:

```
System.out.println(sb2.toString());
sb1.append(" Adeus.");
System.out.println(sb2.toString());
```

A primeira instrução escreve *Olah!* no ecrã, que é o valor correspondente ao texto guardado em `sb2`.

A segunda instrução invoca o método `append` sobre `sb1`, que acrescenta " Adeus." ao `StringBuilder`.

A terceira linha imprime *Olah! Adeus.* no ecrã. Todas as alterações que são feitas sobre o objeto através de `sb1` afetam o objeto referenciado por `sb2` (o mesmo objeto) e vice-versa.

Embora muito útil, como veremos adiante, é preciso ter cuidado com a utilização de *aliasing* pois rapidamente pode levar a erros e confusões.

8.7. null

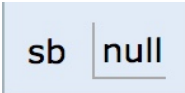
Uma variável de tipo não primitivo que não tenha sido inicializada, não referencia nenhum objeto. O seu conteúdo é indefinido até que lhe seja atribuído um valor.

Um valor que podemos atribuir a uma variável de qualquer tipo não primitivo é o valor `null`; este é um valor especial (e é uma palavra reservada do Java) e significa “nenhum objeto”.

A declaração com inicialização:

```
StringBuilder sb = null;
```

gera um estado da memória que se pode representar assim:



sb null

O valor `null` é representado por um pequeno quadrado contendo a palavra *null* em vez de uma seta.

Se tentarmos invocar um método sobre uma expressão cujo valor é `null`, o Java lança uma exceção `NullPointerException`, imprime uma mensagem de erro e termina o programa.

```
public class Surpresa {
    public static void main(String[] args) {
        StringBuilder sb = null;
        sb.append("Surpresa!");
    }
}
```

A instrução `sb.append("Surpresa!");` provoca a exceção:

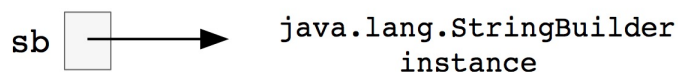
```
Exception in thread "main" java.lang.NullPointerException
    at Surpresa.main(Surpresa.java:4)
```

8.8. Garbage collection

O que acontece quando nenhuma variável referencia um dado objeto? Por exemplo:

```
StringBuilder sb = new StringBuilder();
sb.append("Olah!");
sb = null;
```

A primeira instrução cria um novo objeto do tipo `StringBuilder` e atribui a referência a esse objeto à variável `sb`.



sb → java.lang.StringBuilder instance

A segunda instrução acrescenta a `String` "Olah!" ao objeto referenciado por `sb`.

A terceira instrução muda o conteúdo da variável `sb` de modo a ficar `null`.

sb null java.lang.StringBuilder
instance

Repare que o objeto `StringBuilder` que foi criado deixou de estar acessível pois não é referenciado por nenhuma variável do programa.

Se um objeto não é referenciado por nenhuma variável do programa, então não é possível invocar um método sobre esse objeto, nem usá-lo para nenhum outro propósito. Na verdade, é como se deixasse de existir pois ninguém o conhece ou lhe pode aceder.

O mecanismo de recolha de lixo (*garbage collection*) do Java liberta automaticamente o espaço de memória que os objetos inacessíveis estão a ocupar, para que possa ser usado quando for necessário atribuir espaço de memória a outros objetos do programa.

Esta pesquisa por objetos “perdidos” é feita periodicamente pelo sistema, durante a execução de um programa; é por isso que, em Java, não temos que incluir instruções explícitas de destruição de objetos, pois sabemos que, mais cedo ou mais tarde, a memória que eles ocupam vai ser libertada.

8.9. Ficheiros de texto - leitura e escrita

Até agora, sempre que precisámos de obter alguma informação externa para o nosso programa, fizemo-lo através do `System.in`, que é um atributo *static* público da classe `System`, que é do tipo `java.io.InputStream`.

Um objeto do tipo `InputStream` fornece-nos um canal através do qual *bytes* de informação podem ser “lidos para dentro” do nosso programa.

Para podermos trabalhar estes *bytes* de informação de forma mais fácil, usamos um objeto do tipo `Scanner`, que converte sequências de *bytes* lidos do `InputStream` em valores de tipos vários (relembrar secção 8.5 neste livro).

8.9.1. Ler informação a partir de um ficheiro de texto qualquer

A classe `Scanner` fornece vários construtores, entre os quais:

```
Scanner(File source)
Scanner(InputStream source)
```

Já vimos como usar o segundo.

Para criarmos um *scanner* que nos ajude a ler o conteúdo de um ficheiro de texto qualquer, podemos usar o primeiro construtor da lista acima:

```
Scanner reader = new Scanner(new File("myInput.txt"));
```

Esta instrução cria um canal de leitura para o ficheiro `myInput.txt`. Depois é só usar os métodos da classe `Scanner` para ter acesso aos vários *tokens* nesse ficheiro.

Não é possível prever em tempo de compilação a existência de um ficheiro com o nome `myInput.txt`. Por essa razão, o método que contém aquela instrução pode lançar uma exceção em tempo de execução. Para já, não vamos ver como se trataria essa exceção (ver capítulo ??); vamos apenas avisar que tal exceção pode ocorrer, colocando na assinatura do método a expressão `throws FileNotFoundException`.

```
public static void main(String[] args) throws FileNotFoundException {
```

```
// abrir um canal de leitura
Scanner leitor = new Scanner(new File("myInput.txt"));

// enquanto existem linhas por ler no ficheiro
while (leitor.hasNextLine()){
    String linha = leitor.nextLine();
    if(linha.contains("x")){
        System.out.println(linha);
    }
}
leitor.close();
}
```

Se o conteúdo do ficheiro `myInput.txt` for:

```
Pontos
Pontuacao
Caixa
Tacha
Taxa
```

então o *output* do `main` será:

```
Caixa
Taxa
```

8.9.2. Escrever informação num ficheiro de texto

Até agora, sempre que precisámos de fazer sair alguma informação para o exterior do nosso programa, de modo a que o utilizador a possa ver, fizemo-lo através do `System.out`, que é outro atributo *static* público da classe `System`, mas do tipo `java.io.PrintStream`.

Uma outra classe da biblioteca do Java, no pacote `java.io`, é a classe `PrintStream` que fornece métodos `print` e `println` para escrever valores dos vários tipos primitivos e do tipo `String`, e o método `write` para escrever uma `String` ou um único carácter, dado um inteiro representando o seu código.

O seguinte método lê o conteúdo de um ficheiro cujo nome é dado pelo parâmetro `fileIn`, linha a linha, e escreve as linhas invertidas num ficheiro cujo nome é dado pelo parâmetro `fileOut`.

```
public static void inverteLinhas (String fileIn,
                                String fileOut) throws IOException {

    Scanner in = new Scanner (new File(fileIn));
    PrintWriter out = new PrintWriter(fileOut);

    while (in.hasNextLine()) {
        String line = in.nextLine();
        StringBuilder sb = new StringBuilder(line);
        out.write(sb.reverse().toString() + "\n");
    }
    in.close();
    out.close();
}
```

Se o conteúdo do ficheiro `myInput.txt` for:

```
Eles não sabem que o sonho  
é uma constante da vida  
tão concreta e definida  
como outra coisa qualquer
```

então a execução da seguinte instrução:

```
inverteLinhas("myInput.txt", "myOutput.txt");
```

criará um ficheiro de nome `myOutput.txt` (se não existir ainda) e escreverá nesse ficheiro o seguinte conteúdo:

```
ohnos o euq mebas oãn selE  
adiv ad etnatsnoc amu é  
adinifed e atercnoc oât  
reuqlauq asioc artuo omoc
```

8.10. Exceções

Já se deve ter deparado com situações em que a execução de um programa termina abruptamente e uma mensagem é escrita na consola. Esta mensagem é muito útil para quem a sabe decifrar, pois a partir dela conseguimos perceber qual o tipo de problema ocorrido e onde, no código do programa, foi detetado.

Se executarmos o seguinte programa,

```
import java.util.Scanner;  
  
public class ExemploExcecoes1 {  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
        System.out.println("Introduza um valor numerico");  
  
        double n = sc.nextDouble();  
        System.out.println("log_2 (" + n + ") = " + Math.log(n));  
  
        sc.close();  
    }  
}
```

e respondermos à sua solicitação escrevendo os caracteres `31.w2`, por exemplo, o programa terminará abruptamente com a seguinte mensagem:

```
Exception in thread "main" java.util.InputMismatchException  
    at java.base/java.util.Scanner.throwFor(Scanner.java:939)  
    at java.base/java.util.Scanner.next(Scanner.java:1594)  
    at java.base/java.util.Scanner.nextDouble(Scanner.java:2564)  
    at ExemploExcecoes1.main(ExemploExcecoes1.java:10)
```

Este programa não é robusto em relação à introdução errada de dados, pois o método `nextDouble` da classe `Scanner` só tolera sequências de caracteres que representam `double`, como se pode ver na API dessa classe:

nextDouble

```
public double nextDouble()
```

Scans the next token of the input as a double. This method will throw `InputMismatchException` if the next token cannot be translated into a valid double value. If the translation is successful, the scanner advances past the input that matched.

O mesmo se aplica aos métodos `nextInt`, `nextFloat`, etc.

Podemos sempre evitar este problema recorrendo a outros métodos da classe `Scanner` (por exemplo `hasNext`, `hasNextInt`, `hasNextDouble`) que permitem observar o canal de entrada antes de decidir ler o valor que lá está.

Outra abordagem possível é permitir que o erro ocorra e reagir de forma adequada, sem deixar que o programa termine abruptamente. Isso é feito através do tratamento das exceções, como iremos ver.

8.10.1. O que são exceções

“Exception” ou *exceção*, é uma abreviatura para “*Exceptional Event*”, que é um evento que ocorre durante a execução de um programa e que interrompe o seu fluxo normal.

Quando, durante a execução do programa, é detetado um erro:

- uma *exceção é lançada* (*throwing an exception*), ou seja, é criado um objeto Java que contém as informações relevantes sobre o erro ocorrido:
 - o seu tipo e
 - o estado do programa quando o erro ocorreu.
- o sistema tenta lidar com o problema
 - recorre à lista ordenada de métodos que foram invocados para chegar ao método onde o erro ocorreu (*pilha de chamadas*) e
 - vê se algum desses métodos é capaz de *tratar a exceção* (*handle the exception*).

Na figura da página anterior pode ver a pilha de chamadas que é apresentada na consola quando a exceção ocorre.

Na biblioteca do Java existem várias classes que representam exceções que se aplicam a uma série de situações mais ou menos comuns em execuções de programas.

Como exemplos bastante conhecidos, temos `NullPointerException`, que é lançada sempre que um programa tenta usar `null` numa situação em que é requerido um objeto (por exemplo, invocar um método sobre uma referência que é `null`), `ArrayIndexOutOfBoundsException`, que é lançada sempre que se tenta aceder um *array* usando um índice ilegal, `ArithmeticException`, que é lançada quando alguma condição aritmética excecional ocorre (por exemplo, divisão por zero), etc.

Existem várias alternativas para lidar com as exceções.

8.10.2. Lidar com uma exceção

Por norma, um método que pode lançar uma exceção tem de assumir o tratamento dessa mesma exceção. Existem duas formas de um método tratar uma exceção:

1. *Apanhar* a exceção, usando a instrução `try-catch` que permite tratar a exceção da forma que se ache adequada;
2. *Reencaminhar* a exceção para o método que o invocou, acrescentando a cláusula `throws` na assinatura do método. Também se chama a isto *especificar* a exceção.

Se todos os métodos na pilha de chamadas reencaminharem a exceção, incluindo o `main` onde tudo começou, o programa termina abruptamente indicando, como no exemplo anterior, qual a exceção ocorrida e a pilha de chamadas.

Vamos agora estudar estas duas formas de tratamento de exceções.

1. Apanhar uma exceção (try-catch)

A ideia é "isolar" as instruções que sabemos que podem provocar exceções, e definir ações alternativas a serem executadas como resposta a cada tipo possível de exceção.

Fazemos isso através de um bloco de instruções `try-catch`, composto por:

- um bloco `try`:
 - neste bloco colocamos as instruções que queremos executar e que sabemos que podem levantar exceções;
- um ou mais blocos `catch` a serem executados quando alguma das instruções no bloco `try` provoca o lançamento de uma exceção.

Um bloco `catch` recebe a exceção como parâmetro.

Em cada um dos blocos `catch` podemos ter, em alternativa:

- a. código que vai ser executado para resolver o erro (por exemplo, informando o utilizador de que aconteceu algo errado durante a execução do programa);
- b. propagação da exceção, eventualmente de forma diferente (outro tipo de exceção), para o método que invocou o método corrente.

1.a. Resolver a exceção

Como exemplo da alternativa **a.**, em reação à exceção `InputMismatchException` que pode ser provocada pela execução do método `nextDouble`, podemos "apanhar" essa exceção e informar o utilizador que o programa terminou sem o resultado esperado, porque um dos valores que o programa estava à espera de receber não é do tipo `double`.

Se executarmos esta nova versão do programa:

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class ExemploExcecoes2 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        System.out.println("Introduza um valor numerico");
        try {
            double n = sc.nextDouble();
```



```

        System.out.println("log_2 (" + n + ") = " + Math.log(n));
    } catch (InputMismatchException e) {
        System.out.println("Ocorreu um erro. Formato errado");
    }
    sc.close();
}
}

```

e voltarmos a introduzir os caracteres `31.w2`, a mensagem será:

```
Ocorreu um erro. Formato errado
```

1.b. Lançar uma exceção (`throw`)

Podemos querer substituir a exceção que foi lançada por outra que consideremos mais adequada. Podemos usar uma das exceções já definidas na biblioteca do Java, ou um novo tipo de exceção por nós criado.

Quando uma exceção é criada, é possível associar-lhe uma mensagem específica através de um parâmetro do construtor. Essa mensagem pode depois ser obtida através da invocação do método `getMessage` da exceção. Esta mensagem aparece na primeira linha da pilha de mensagens que é mostrada ao utilizador quando o programa termina devido a uma exceção não tratada.

- A criação de uma exceção é feita, como com qualquer outro objeto, com a instrução `new`.
- O construtor recebe uma *string*, que pode depois ser acedida através da invocação do método `getMessage` da exceção.
- O lançamento de uma exceção é feito usando a instrução `throw`.

Na seguinte versão do nosso programa, o bloco `catch` que trata a exceção `InputMismatchException` lança uma nova exceção, do tipo `NumberFormatException`, e associa-lhe uma mensagem elucidativa.

Se executarmos esta nova versão do programa:

```

import java.util.InputMismatchException;
import java.util.Scanner;

public class ExemploExcecoes3 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        System.out.println("Introduza um valor numerico");
        try {
            double n = sc.nextDouble();
            System.out.println("log_2 (" + n + ") = " + Math.log(n));
        } catch (InputMismatchException e) {
            throw new NumberFormatException("Formato do input invalido");
        }
        sc.close();
    }
}

```

e voltarmos a introduzir os caracteres `31.w2`, aparecerá o seguinte na consola:

```
Exception in thread "main" java.lang.NumberFormatException: Formato do input invalido
    at ExemploExcecoes3.main(ExemploExcecoes3.java:16)
```

As soluções apresentadas acima resolvem o problema quando algo corre mal com a aquisição do valor de `n`, contudo não resolvem todos os problemas.

Por exemplo, o que acontece quando o número `n` é um número não positivo (relembre que o logaritmo é indefinido para valores não positivos)? Apesar de o programa não dar erro nem lançar uma exceção (devolve `NAN` ou `Infinity`), pode-se interpretar esta situação como um comportamento excecional e que, portanto, deve dar origem a uma exceção.

No exemplo seguinte, o comportamento excecional é sinalizado lançando uma exceção do tipo `InputMismatchException`, que depois será apanhada no bloco `catch` correspondente.

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class ExemploExcecoes4 {

    public static final double MINIMO = 0.001;

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        System.out.println("Introduza um valor numerico");
        try {
            double n = sc.nextDouble();
            if(n <= MINIMO) {
                throw new InputMismatchException("Ocorreu um erro.\n" +
                    "O argumento não é positivo!");
            }
            System.out.println("log_2 (" + n + ") = " + Math.log(n));
        } catch(InputMismatchException e) {
            if (e.getMessage() == null) {
                System.out.println("Ocorreu um erro. Formato errado");
            } else {
                System.out.println(e.getMessage());
            }
        }
        sc.close();
    }
}
```

Se executarmos este programa e introduzirmos o valor `-1`, a mensagem será:

```
Ocorreu um erro.
O argumento não é positivo!
```

Como já foi referido, quando uma exceção é criada, é possível associar-lhe uma mensagem específica. Essa mensagem pode depois ser obtida através da invocação do método `getMessage` da exceção.

Como sabemos que o método `nextDouble` dá origem a uma exceção sem qualquer mensagem associada quando o formato do *input* não é o esperado, então, se compararmos o resultado do `getMessage` com `null`, ficamos a saber que é esse o caso.

O ramo `else` do `if` cobre as situações em que o resultado não é `null`, como por exemplo a exceção que é lançada quando `n <= MINIMO`.

2. Reencaminhar uma exceção para outro método (`throws`)

Se num método `m` não sabemos como devemos tratar uma exceção, porque desconhecemos as especificidades do contexto em que foi invocado, devemos deixar esse tratamento para o método que invocou `m`.

Quando não queremos apanhar uma exceção num método `m`,

- temos que “anunciar” que a execução de `m` pode levar ao seu lançamento (*especificar a exceção*);
- Para isso, acrescentamos à assinatura do método a palavra `throws` (notar o “s” no final da palavra) seguida do nome das exceções potencialmente geradas e não tratadas.

Quando um método gera uma exceção que não é tratada internamente, a sua execução termina de imediato.

No exemplo seguinte, o método `lerDeUmFicheiro`, que lê um valor a partir de um ficheiro, não trata as exceções do tipo `FileNotFoundException` e, por isso, tem na sua assinatura a declaração `throws FileNotFoundException`.

No caso de ocorrer uma exceção provocada por erros de acesso ao ficheiro, ela é passada para o método que invocou este método e assim sucessivamente, até encontrar um método que trate a exceção ou, em última instância, até chegar ao método `main`. Neste caso, se o `main` não trata a exceção (também tem `throws`), o programa termina a sua execução indicando uma mensagem de erro.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.InputMismatchException;
import java.util.Scanner;

public class ExemploExcecoes5 {

    public static final double MINIMO = 0.001;

    public static void main (String[] args) throws FileNotFoundException{

        try {
            double n = lerDeUmFicheiro("inteiros.txt");

            if(n <= MINIMO) {
                throw new InputMismatchException("Ocorreu um erro.\n" +
                    "O argumento nao eh positivo!");
            }

            System.out.println("log_2 (" + n + ") = " + Math.log(n));

        } catch (InputMismatchException e) {
            if(e.getMessage() == null) {
                System.out.println("Ocorreu um erro. Formato errado.");
            } else {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

```

    public static double lerDeUmFicheiro(String nomeFicheiro)
        throws FileNotFoundException {

        Scanner sc = new Scanner(new File(nomeFicheiro));

        double n = sc.nextDouble();

        sc.close();
        return n;
    }
}

```

Se executarmos este programa sem que exista acessível um ficheiro de nome `inteiros.txt`, a mensagem obtida será:

```

Exception in thread "main" java.io.FileNotFoundException: inteiros.txt (No such file or
directory)
    at java.base/java.io.FileInputStream.open0(Native Method)
    at java.base/java.io.FileInputStream.open(FileInputStream.java:211)
    at java.base/java.io.FileInputStream.<init>(FileInputStream.java:153)
    at java.base/java.util.Scanner.<init>(Scanner.java:639)
    at ExemploExcecoes5.lerDeUmFicheiro(ExemploExcecoes5.java:34)
    at ExemploExcecoes5.main(ExemploExcecoes5.java:13)

```

Se existir um ficheiro de nome `inteiros.txt`, o valor que for lido determinará a reação do programa, como já vimos nos exemplos anteriores.

8.10.3. Exceções *checked* e *unchecked*

Relembre o primeiro programa apresentado neste capítulo:

```

import java.util.Scanner;

public class ExemploExcecoes1 {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        System.out.println("Introduza um valor numerico");

        double n = sc.nextDouble();
        System.out.println("log_2 (" + n + ") = " + Math.log(n));

        sc.close();
    }
}

```

Embora o método `nextDouble` da classe `Scanner` possa provocar uma exceção `InputMismatchException` (além de outras), o compilador não nos obrigou nem a tratá-la nem a declarar no método `main` que isso podia acontecer (com `throws InputMismatchException`).

Em contraste, o seguinte programa tem um erro de compilação:

```
public class ExemploExcecoes6 {

    public static void main (String[] args) {

        Scanner sc = new Scanner(new File("inteiros.txt"));

        double n = sc.nextDouble();
        System.out.println("log_2 (" + n + ") = " + Math.log(n));

        sc.close();
    }
}
```

O erro de compilação é:

```
Unhandled exception type FileNotFoundException
```

Para resolvermos este erro temos que tratar a exceção com um bloco `try-catch` ou acrescentar `throws FileNotFoundException` à assinatura do método `main`.

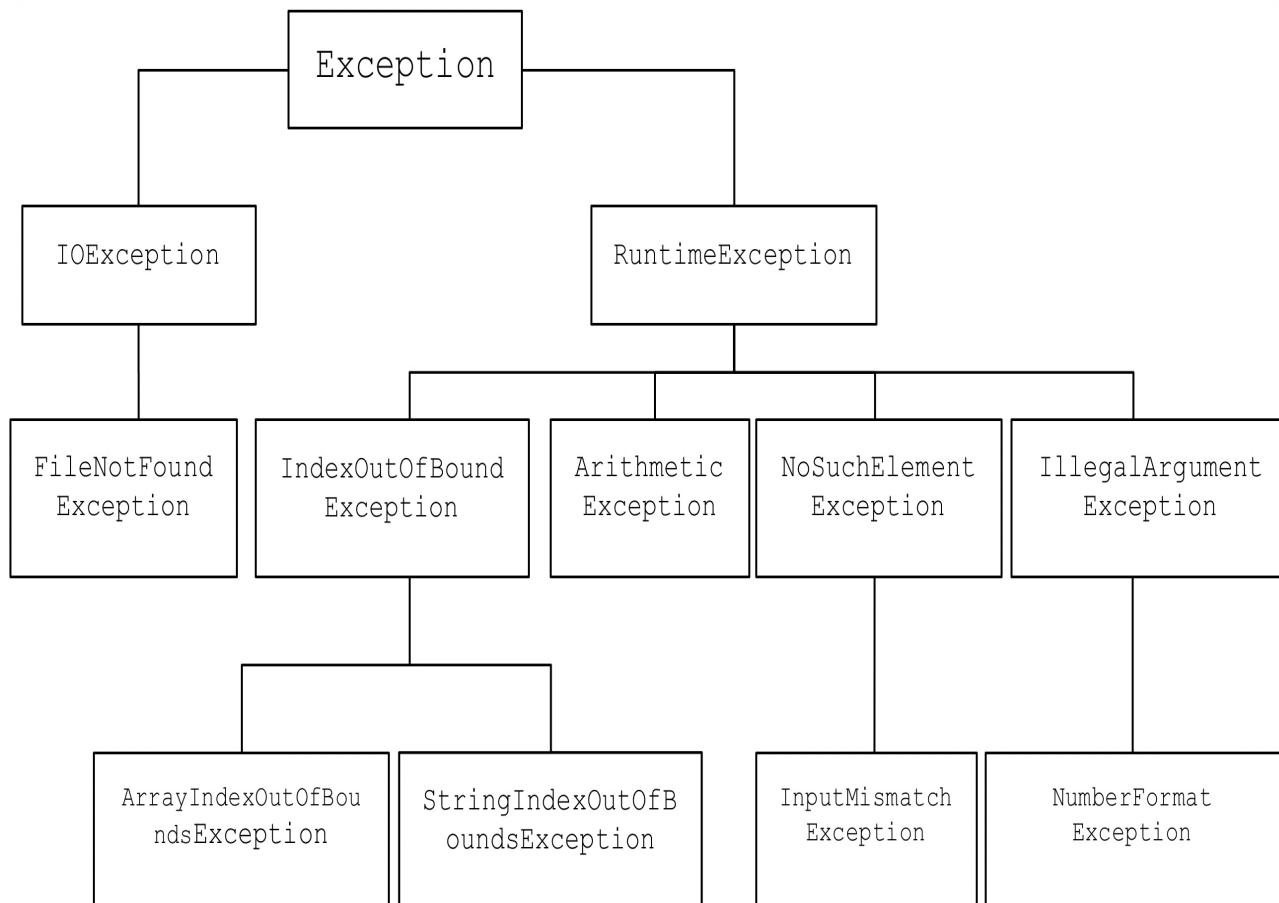
O que justifica esta diferença?

Em Java existem dois tipos de exceções, no que diz respeito ao seu tratamento:

- **Checked:** são as exceções que são verificadas em tempo de compilação. Se alguma parte do código de um método pode dar origem a uma exceção checked, então o método tem que:
 - tratar a exceção através de um bloco `try-catch` ou
 - anunciar que essa exceção pode acontecer, adicionando à sua assinatura a palavra `throws` seguida do nome da exceção.
- **Unchecked:** são as exceções que não são verificadas em tempo de compilação.

Claramente as exceções do tipo `InputMismatchException` são *unchecked* e as do tipo `FileNotFoundException` são *checked*.

A figura abaixo mostra parte da hierarquia de exceções do Java. Todas as `RuntimeException` são *unchecked*.



A ideia subjacente à existência destes dois tipos de exceções é a de que as exceções *checked* são aquelas das quais um método cliente tem hipótese de recuperar e as *unchecked* aquelas que um método cliente não consegue ou não deve tratar, por poderem configurar situações de programação deficiente.

Por exemplo, quando um pedaço de código pode lançar uma `NullPointerException` ou uma `ArrayIndexOutOfBoundsException`, o melhor que temos a fazer é corrigir o nosso código em vez de tratar essas exceções.

8.10.4. Try-with-resources em Java

Voltando ao exemplo do método `lerDeUmFicheiro`, da classe `ExemploExcecoes5`, o que acontece ao `Scanner` quando o método `lerDeUmFicheiro` termina lançando a exceção `InputMismatchException` causada pelo lançamento dessa mesma exceção pelo método `nextDouble`?

Como a execução termina abruptamente, as instruções `sc.close()` e `return n` não são executadas. Nesse caso o `Scanner` fica “aberto”. É necessário fechá-lo, isto é libertar este recurso para outras execuções.

De modo a simplificar a tarefa dos programadores, a instrução `try` pode ser reforçada com a indicação dos recursos que deverão ser fechados de forma automática no final do bloco (seja lançada uma exceção ou não):

```

public static double lerDeUmFicheiro(String nomeFicheiro)
    throws FileNotFoundException {

    try (Scanner sc = new Scanner(new File(nomeFicheiro))) {

        double n = sc.nextDouble();

        return n;
    }
}

```

Repare que assim já não precisa de invocar explicitamente o método `close()` do `Scanner`.

Num mesmo bloco `try` podem ser iniciados vários recursos. A indicação desses vários recursos é feita através de separação com `;` dentro do parêntesis que se segue à palavra reservada `try`.

8.10.5. O bloco finally

O bloco `finally` é usado sempre que se pretende garantir que um dado bloco de instruções é executado mesmo que seja lançada uma exceção. Antes do Java 7, o papel de fecho dos recursos cabia ao programador e as instruções necessárias eram realizadas dentro de um bloco `finally`.

A partir de então, essa tarefa é automática com o `try-with-resources`.

No entanto, há situações onde se justifica a utilização do bloco `finally` como, por exemplo, para finalizar tarefas importantes de escrita (por exemplo em bases de dados), ou eliminação de ficheiros que foram criados como meios auxiliares ou colocar estruturas num estado específico.

9. Arrays

O seguinte programa calcula a média de uma série de valores introduzidos pelo utilizador:

```

import java.util.Scanner;
public class MediaDeValores {

    public static void main (String [] args) {
        Scanner leitor = new Scanner (System.in);
        System.out.println("Quantos valores?");
        int quantos = lerInteiro("Tem que ser um inteiro!", leitor);
        double soma = 0.0;
        for (int i = 1 ; i <= quantos ; i++){
            int valor = lerInteiro("Tem que ser um inteiro!", leitor);
            soma += valor;
        }
        System.out.println("A media dos " + quantos + " valores eh: " +
                           soma / quantos);
    }

    /**
     * Primeiro inteiro no canal de leitura
     * @param errMess - mensagem a escrever no System.out caso o valor
     *                  acessivel no canal de leitura nao seja um inteiro
     */
}

```

```

* @param sc - canal de leitura
* @return valor inteiro
* @requires errMess != null && sc != null
*/
private static int lerInteiro ( String errMess, Scanner sc ) {
    int valor = 0;
    boolean erro = true;
    do {
        if ( sc.hasNextInt () ) {
            valor = sc.nextInt (); // consome o inteiro
            erro = false;
        }
        else {
            sc.next ();          // consome o que lah esteja
            System.out.println ( errMess );
        }
    } while ( erro );
    return valor;
}
}

```

Este programa pergunta primeiro ao utilizador quantos valores vai introduzir e, um a um, lê esses valores e vai adicionando-os ao valor da variável `soma`.

No fim, apresenta o valor da média, resultante de dividir o valor de `soma` pelo número de valores introduzidos.

Imagine agora que, em vez da média, quer calcular o desvio-padrão que, como se deve lembrar, é definido pela seguinte fórmula, em que $\bar{x}$ é a média dos n valores x_i :

$$\sqrt{\frac{1}{n} \sum_{i=1}^n (\bar{x} - x_i)^2}$$

Repare que, para o cálculo do desvio-padrão, precisa de calcular a diferença entre cada um dos valores e a média desses mesmos valores, ou seja, primeiro tem que calcular a média e só a seguir é que pode calcular a diferença para cada um dos valores.

Da forma como o programa está feito, cada valor introduzido é “consumido” e “descartado”, pois é usada uma só variável `valor` que, a cada instante, contém somente o último valor introduzido pelo utilizador.

Então, como fazer para calcular a média e, a seguir, o desvio-padrão? Não queremos pedir de novo todos os valores ao utilizador.

A solução seria guardar em memória todos os valores lidos para os podermos usar outra vez.

Como havemos de fazer isto? Declarar várias variáveis? Se sim, quantas? Não sabemos! Isso só é decidido em tempo de execução (o utilizador é que determina quantos valores vai introduzir).

Para estes casos em que queremos guardar em memória vários valores do mesmo tipo, usamos um novo tipo de dados – o *array* – que, pelas suas características, nos vai permitir resolver de forma elegante muitos e diversos problemas.

É esse importante tipo de dados que é apresentado no que resta deste capítulo.

NOTA: Poderíamos, em alternativa, usar uma outra fórmula para o cálculo do desvio-padrão, que não exige o cálculo prévio da média, e que pode ser feito com uma única “passagem” pelos valores dados. A fórmula alternativa é $\sqrt{(B - A^2/N)/N}$, onde N é o número de valores a considerar, A é a soma desses valores e B é a soma dos quadrados desses valores.

O método `main` para esta versão poderia ser:

```
public static void main (String [] args) {

    Scanner leitor = new Scanner (System.in);
    System.out.println("Quantos valores?");
    int quantos = lerInteiro("Tem que ser um inteiro!", leitor);
    double soma = 0.0;
    double somaQuadrados = 0.0;
    System.out.println("Introduza " + quantos + " valores inteiros");
    for (int i = 1 ; i <= quantos ; i++){
        int valor = lerInteiro("Tem que ser um inteiro!", leitor);
        soma += valor;
        somaQuadrados += valor * valor;
    }

    double desvio =
        Math.sqrt((somaQuadrados - soma * soma / quantos) / quantos);

    System.out.println("O desvio padrao dos " + quantos +
        " valores eh: " + desvio);
}
```

9.1. O tipo de dados não primitivo *array*

O tipo de dados *array* permite

- representar, num só objeto, vários valores do mesmo tipo e
- aceder a cada um desses valores diretamente, bastando para isso saber qual o *índice*, ou posição, em que se encontra no *array*.

Podemos ter *arrays* de `int`, de `boolean`, de `String`, etc; na verdade, os elementos de um *array* podem ser de qualquer tipo de dados, primitivo ou não primitivo.

Em termos sintáticos, representamos o tipo de dados *array* de elementos do tipo `T`, para algum tipo `T`, por `T[]`, ou seja, o nome do tipo `T` seguido de parêntesis retos.

Os exemplos seguintes são ilustrativos de declarações de variáveis do tipo *array*:

```
int[] notas;           // um array de inteiros
boolean[] permissoes;  // um array de boolean
String[] nomes;        // um array de String
```

Como sabe, até serem inicializadas, estas variáveis têm um valor indefinido.

Como com qualquer tipo não primitivo, os valores a atribuir a estas variáveis têm que ser referências para objetos; neste caso os objetos serão *arrays* cujos elementos têm um tipo específico (`int`, `boolean` e `String`, respetivamente).

A criação de um *array* faz-se

- usando o operador `new`
- seguido do tipo dos elementos do *array* e
- de uma expressão inteira cujo valor define o número de elementos que o *array* deverá ter.

```

notas = new int[4];
int quantos = 3;
permissoes = new boolean[quantos];
nomes = new String[quantos * 2];

```

As instruções acima têm como efeito:

- a criação de três novos *arrays* (de valores de tipo `int`, `boolean` e `String`, respectivamente) e
- a atribuição das referências para esses novos objetos às variáveis `notas`, `permissoes` e `nomes`.

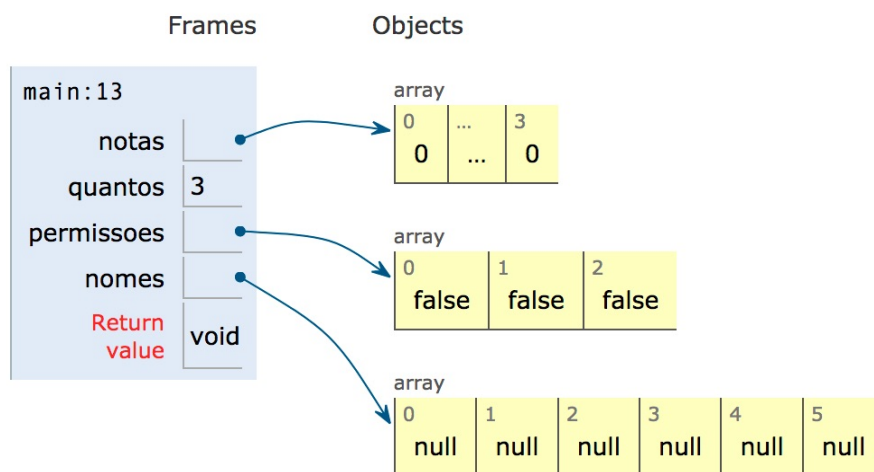
A primeira instrução tem como efeito a criação de um *array* de inteiros com 4 elementos e a atribuição à variável `notas` de uma referência a esse *array*.

A terceira instrução tem como efeito a criação de um *array* de `boolean` com 3 elementos e a atribuição à variável `permissoes` de uma referência a esse *array*.

Na última instrução é criado um *array* de `String`s com 6 elementos e uma referência a esse *array* é atribuída à variável `nomes`.

Se executar o programa com estas instruções, na versão online deste livro, poderá verificar, na parte da representação da memória, a criação dos três *arrays*.

Se não experimentou executar o programa, pode ver ilustrada na figura seguinte a representação das variáveis `notas`, `permissoes` e `nomes`, cujo conteúdo são referências a *arrays* dos tipos correspondentes às definições.



Na representação que adotámos e que também é adotada nesta execução *online*,

- os textos dentro das caixas representam os valores dos elementos do *array* (a representação do *array* `notas` está abreviada; os seus segundo e terceiro elementos não aparecem);
- os números mais pequenos no canto superior esquerdo de cada caixa representam os *índices*, ou posições, usados para identificar cada elemento do *array*.

Quando um *array* é criado, os seus elementos são inicializados com um valor que depende do seu tipo:

- zero para tipos numéricos (`int`, `double`, `long`, `float`, etc),
- `false` para `boolean`,
- `null` para tipos não primitivos.

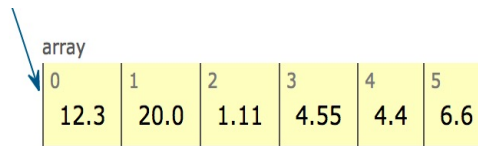
Repare que este programa cria os *arrays* mas não faz nada de útil com eles. Este é apenas um exemplo de declaração e inicialização de *arrays*. Tal como com qualquer variável, normalmente os programas usam *arrays* para armazenar valores em memória de modo a poderem ser usados sempre que necessário.

Podemos ainda criar um *array* indicando explicitamente quais os valores que cada um dos seus elementos vai tomar.

Exemplo:

```
double[] distancias = {12.3 , 20.0 , 1.11 , 4.55 , 4.4 , 6.6};
```

A seguinte figura ilustra a representação da variável `distancias`.



Por analogia com a Matemática, também nos referimos aos *arrays* unidimensionais (mais sobre isto mais à frente) usando a designação de *vetores*.

9.2. Acesso aos elementos de um *array*

Para aceder aos elementos de um *array* (tanto para **leitura** do valor que contêm, como para **escrita** de valores nos lugares de memória a eles atribuídos), usamos

- uma expressão cujo valor é uma referência ao *array* (a mais simples é o nome de uma variável, mais adiante veremos expressões mais complexas),
- seguida de uma expressão inteira, entre parêntesis retos, representando o índice do elemento a que queremos aceder.
 - o primeiro elemento de um *array* é o que tem índice zero,
 - o segundo elemento tem índice 1,
 - ...,
 - o último elemento tem um índice cujo valor é $n - 1$, sendo n o número de elementos do *array*.

As instruções seguintes permitem escrever valores em elementos de *arrays*:

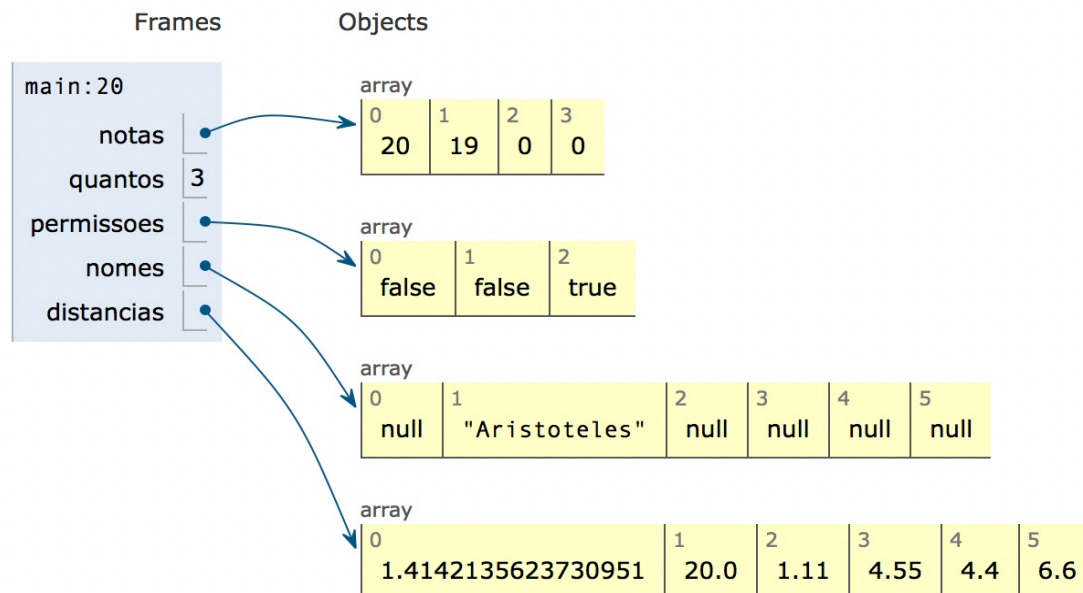
```
notas[0] = 20;
notas[1] = 19;
permissoes[quantos - 1] = true;
nomes[quantos / 3] = "Aristoteles";
distancias[0] = Math.sqrt(2);
```

Após a sua execução:

- o primeiro e o segundo elementos do *array* referenciado pela variável `notas` ficam com os valores 20 e 19, respetivamente;
- o último elemento do *array* referenciado pela variável `permissoes` fica com o valor `true`;
- o elemento cujo índice é `quantos/3` do *array* referenciado pela variável `nomes` fica a referenciar a `String` `"Aristoteles"` e
- o primeiro elemento do *array* referenciado pela variável `distancias` fica com o valor da raiz quadrada de 2.

Experimente executar o programa completo na versão online deste livro.

Se não experimentou executar o programa, pode ver ilustrada na figura seguinte a representação da memória após a execução do programa.



A partir daqui, para simplificar, desde que não introduza ambiguidades, passamos a referir “o *array* v” em vez de “o *array* referenciado pela variável v”.

Para ler o valor de um elemento de um *array*, fazemos de forma semelhante. As instruções seguintes:

```
int minhaNota = notas[1];
System.out.println(nomes[2]);
distancias[1] = 3 * distancias[0];
```

têm como consequência

- a atribuição do valor do segundo elemento do *array* `notas` à variável `minhaNota`,
- a escrita no *standard output* do valor do terceiro elemento do *array* `nomes` e
- a atribuição ao segundo elemento do *array* `distancias` de um valor igual ao triplo do valor do primeiro elemento do mesmo *array*, respetivamente.

É um erro tentarmos aceder a um elemento de um *array* usando um valor para o índice que esteja fora do intervalo legal $[0, \text{comprimento} - 1]$, onde *comprimento* denota o número de elementos do *array* em questão. O programa terminará lançando uma exceção `ArrayIndexOutOfBoundsException`.

Voltando agora ao cálculo do desvio padrão usando a fórmula da soma dos quadrados das diferenças entre os vários valores e a média, o seguinte método `main` cria e utiliza um *array* para guardar os valores dados pelo utilizador de modo a usá-los posteriormente:

```
public static void main (String [] args) {

    Scanner leitor = new Scanner (System.in);
    System.out.println("Quantos valores?");
    int quantos = lerInteiro("Tem que ser um inteiro!", leitor);
    int[] valores = new int[quantos];

    double soma = 0.0;
```

```

    for (int i = 0 ; i < quantos ; i++){
        valores[i] = lerInteiro("Tem que ser um inteiro!", leitor);
        soma += valores[i];
    }
    double media = soma / quantos;

    double somaQuadrados = 0.0;
    for (int i = 0 ; i < quantos ; i++){
        somaQuadrados += Math.pow(media - valores[i], 2);
    }

    System.out.println("Desvio padrao: " +
        Math.sqrt(somaQuadrados / quantos));
}

```

Em todos os exemplos que vimos, é conhecido o número de elementos dos *arrays*; esta não é, no entanto, uma situação comum, pois é muito frequente termos que trabalhar com *arrays* sobre os quais nada sabemos além de qual o tipo dos seus elementos.

9.3. Número de elementos de um *array*

Para conhecer o número de elementos de um *array* acedemos ao seu atributo `length`.

Como exemplo, a seguinte instrução tem exatamente o mesmo efeito que a correspondente na secção anterior, pois o valor da variável `quantos` é exatamente o número de elementos do *array*:

```

for (int i = 0 ; i < valores.length ; i++){
    somaQuadrados += Math.pow(media - valores[i], 2);
}

```

Esta outra resulta na escrita dos valores dos elementos do *array* `notas` no *standard output*, um em cada linha:

```

for (int i = 0 ; i < notas.length ; i++){
    System.out.println(notas[i]);
}

```

Esta ainda, atribui ao último elemento do *array* `nomes` uma referência à *string* "Platao":

```

nomes[nomes.length - 1] = "Platao";

```

9.4. Copiar *arrays*

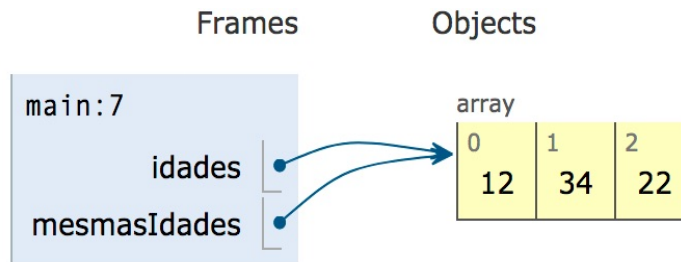
Como o conteúdo de uma variável de um tipo *array* é uma referência a um objeto (tal como acontece com qualquer variável de tipo não primitivo), quando copiamos o seu valor para outra variável é a referência que estamos a copiar e não o *array* propriamente dito.

```

int[] idades = {12, 34, 22};
int[] mesmasIdades = idades;

```

As instruções acima criam um único *array* de `int` e atribuem a duas variáveis diferentes a referência a esse objeto, como ilustrado na figura.



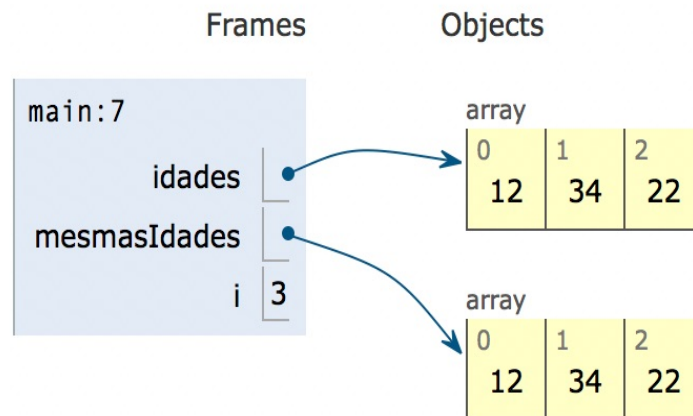
Esta situação é uma forma de *aliasing*. Quaisquer alterações aos elementos do *array* feitas através de uma das variáveis serão refletidas na outra.

Se não é isto que queremos, ou seja, se queremos uma cópia mesmo, no sentido em que queremos um objeto igual ao original mas independente dele, temos que construir um novo *array* e copiar os elementos do primeiro *array* para o novo, um a um.

```
int[] idades = {12, 34, 22};
int[] mesmasIdades = new int[3];
for (int i = 0 ; i < idades.length ; i++) {
    mesmasIdades[i] = idades[i];
}
```

Experimente executar o programa completo na versão online deste livro.

O resultado destas instruções é ilustrado na figura.



9.5. Funções cujo resultado é um *array*

Podemos ter métodos que criam um *array* e o devolvem como resultado, ou seja, funções cujo retorno é de tipo *array*.

Vamos ilustrar esta situação com um método que cria e devolve um *array* de inteiros aleatórios entre zero e um dado valor.

Precisamos que quem invoque o método defina o número de elementos que o *array* deverá ter, além do valor máximo para os aleatórios, por isso o método terá dois parâmetros do tipo `int`.

```
/**
 * Constroi e devolve um vetor de valores aleatorios
 * entre zero e um dado valor
 * @param n - Quantos elementos deverah ter o array
```

```

    * @param max - Valor maximo para os aleatorios
    * @return Um vetor de n valores aleatorios em [0,max]
    * @requires n > 0
    */
    static int[] vetorDeAleatorios (int n, int max) {
        int[] vetor = new int[n];
        Random gerador = new Random();
        for (int i = 0 ; i < n ; i++){
            vetor[i] = gerador.nextInt(max + 1);
        }
        return vetor;
    }
}

```

O seguinte programa invoca o método acima para obter um *array* de 7 números aleatórios no intervalo $[0,10]$ e de seguida imprime os seus valores no *standard output*:

```

import java.util.Random;
public class Aleatorios {

    public static void main (String [] args) {

        int[] aleatorios = vetorDeAleatorios(7, 10);

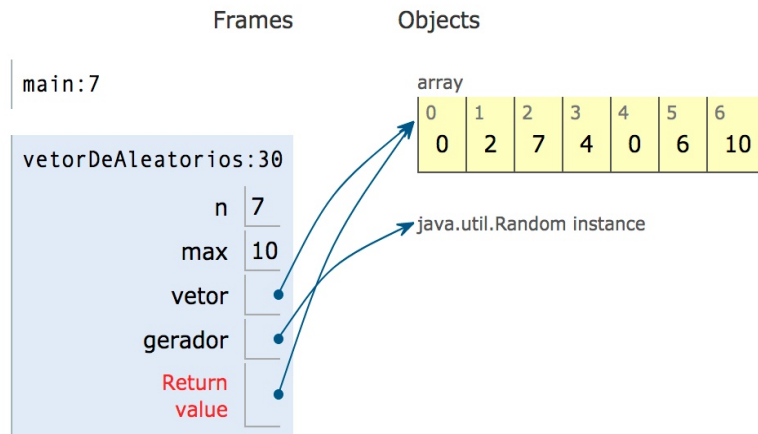
        for (int i = 0 ; i < 7 ; i++){
            System.out.println(aleatorios[i]);
        }
    }

    /**
     * Constroi e devolve um vetor de valores aleatorios
     * entre zero e um dado valor
     * @param n - Quantos elementos deverah ter o array
     * @param max - Valor maximo para os aleatorios
     * @return Um vetor de n valores aleatorios em [0,max]
     * @requires n > 0
     */
    static int[] vetorDeAleatorios (int n, int max) {
        int[] vetor = new int[n];
        Random gerador = new Random();
        for (int i = 0 ; i < n ; i++){
            vetor[i] = gerador.nextInt(max + 1);
        }
        return vetor;
    }
}

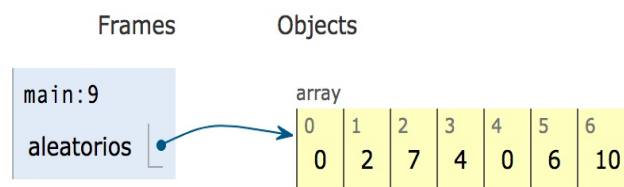
```

Experimente executar o programa na versão online deste livro.

Repare no estado da memória no momento em que o método termina a sua execução e vai retornar o resultado. Como o valor a retornar é do tipo *array*, o que é retornado é uma referência ao *array* que foi criado no método:



Esse valor é de seguida atribuído à variável `aleatórios`, que é local ao `main`. De seguida essa variável é usada no ciclo que escreve no ecrã os vários elementos do `array`.

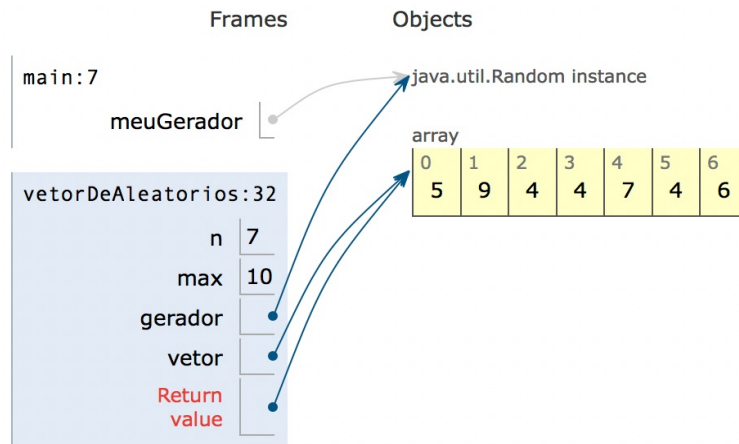


Se quiséssemos que o método usasse um gerador de aleatórios específico, em vez de criar um de cada vez que é executado, bastaria acrescentar um terceiro parâmetro, do tipo `Random`.

```
/**
 * Constroi e devolve um vetor de valores aleatorios
 * entre zero e um dado valor
 * @param n - Quantos elementos deverah ter o array
 * @param max - Valor maximo para os aleatorios
 * @param gerador - Gerador de aleatorios
 * @return Um vetor de n valores aleatorios em [0,max]
 * @requires n > 0
 */
static int[] vetorDeAleatorios (int n, int max, Random gerador) {
    int[] vetor = new int[n];
    for (int i = 0 ; i < n ; i++){
        vetor[i] = gerador.nextInt(max + 1);
    }
    return vetor;
}
```

Experimente executar esta nova versão na versão online deste livro.

Repare na diferença, em relação ao programa anterior, do estado da memória no momento em que o método termina a sua execução e vai retornar o resultado: a instância de `java.util.Random` que é usada no método é a mesma que foi criada no `main`, cuja referência foi guardada na variável `meuGerador`, local ao `main`; esta referência foi depois copiada para o parâmetro `gerador` do método `vetorDeAleatorios`, ficando o objeto a ser referenciado por essas duas variáveis.



9.6. Funções com parâmetros do tipo *array*

Na Secção 9.3. vimos um excerto de código que imprime os elementos de um *array* de inteiros.

Por ser uma ação muito usual nos programas que vamos fazer neste curso, pois é uma forma de irmos verificando o efeito das ações dos nossos programas, vamos criar um método que, dado um qualquer *array* de inteiros, imprime-o no *standard output*.

```
/**
 * Imprime os elementos de um dado vetor de inteiros no standard output
 * @param v - O vetor a imprimir
 * @requires v != null
 */
static void imprimeVetor(int[] v) {

    for (int i = 0 ; i < v.length ; i++) {
        System.out.print(v[i] + " ");
    }
}
```

Como não sabemos nada acerca do *array*, a não ser que os seus elementos são inteiros, temos que usar o seu atributo `length` para conhecermos quantos elementos tem.

O método `main` seguinte cria um *array* com oito inteiros aleatórios no intervalo $[0,10]$, usando a função `vetorDeAleatorios` da secção anterior, e imprimem esses valores no *standard output*, invocando o procedimento `imprimeVetor`.

```
public static void main (String [] args) {

    Random meuGerador = new Random();
    int[] valores = vetorDeAleatorios(8, 10, meuGerador);
    imprimeVetor(valores);
}
```

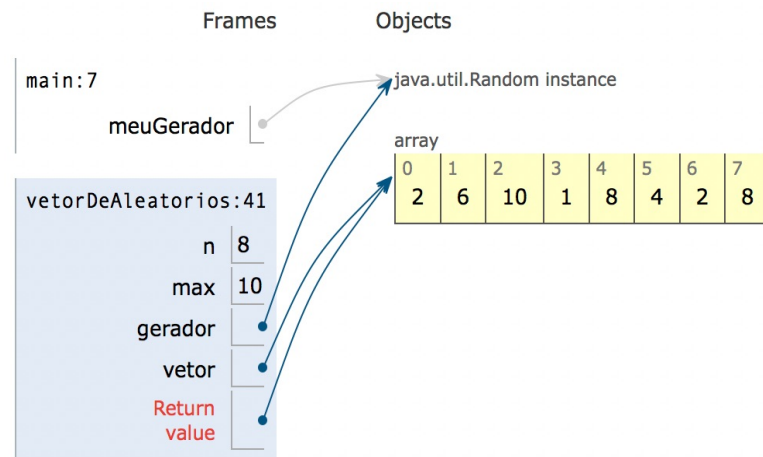
O método `imprimeVetor` não modifica os elementos do *array*.

Experimente executar o programa completo na versão online deste livro.

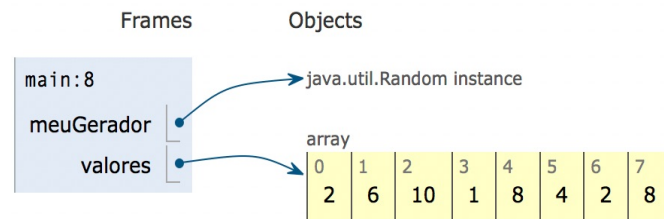
Repare no estado da memória em várias fases da execução:

- No fim da execução da função `vetorDeAleatorios`, na altura de executar a instrução `return vetor;`,

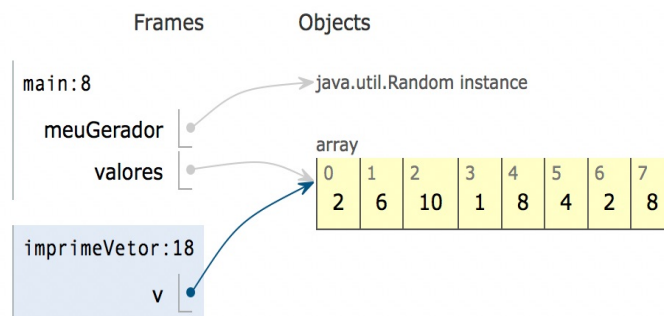
- o parâmetro `gerador` da função referencia o mesmo objeto que a variável `meuGerador` do `main`,
- o valor que vai ser retornado referencia o mesmo objeto que foi criado e guardado na variável local `vetor`.



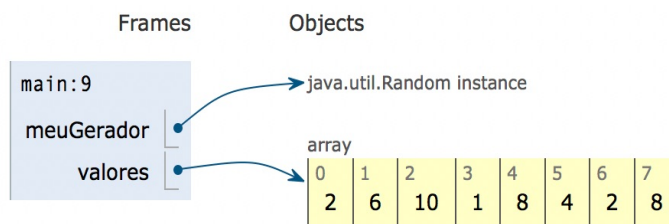
- Quando a instrução `int[] valores = vetorDeAleatorios(8, 10, meuGerador);` acaba de ser executada, o conteúdo da variável `valores`, do `main`, é uma referência ao mesmo `array` que foi criado e devolvido pela função `vetorDeAleatorios`



- No início da execução do método `imprimeVetor`, o seu parâmetro `v` contém uma cópia do conteúdo da variável `valores`, do `main`, que é uma referência ao `array` já existente.



- Já depois do método `imprimeVetor` ter sido executado, o conteúdo da variável local `valores`, do `main`, tem exatamente o mesmo conteúdo que tinha antes da invocação do método.



9.6.2. Calcular o máximo de um *array* de números

Em muitas ocasiões precisamos de calcular o maior ou menor valor de um dado conjunto de valores. Para calcular, por exemplo, o máximo valor contido num *array* de inteiros, temos que percorrer todos os elementos do *array* e determinar qual é o maior.

Como não conseguimos olhar para todos os elementos de um *array* ao mesmo tempo, temos que ir um a um e tentar perceber se o valor dele é importante ou não para o nosso objetivo. O valor de um elemento é importante se for maior do que todos os que já inspecionámos até lá chegar.

Se for maior que todos os outros que já inspecionámos, então ele passa a ser "o maior que vimos até agora" e é esse valor que deverá ser usado quando inspecionarmos o elemento seguinte do *array*.

No fim, quando já inspecionámos todos os elementos do *array*, o máximo será o valor do atual "o maior que vimos até agora".

Só falta decidir que valor terá esse "o maior que vimos até agora" logo no início, quando ainda não inspecionámos nenhum elemento do *array*. O que podemos decidir é que, quando inspecionamos o elemento de índice 1 do *array*, "o maior que vimos até agora" é o elemento de índice zero, o primeiro elemento do *array*.

Segue-se uma solução:

```
/**
 * O maximo de um dado vetor de inteiros
 * @param v - O vetor em questao
 * @requires v != null && v.length > 0
 */
static void maximo(int[] v) {
    int maiorAtehAgora = v[0];
    for (int i = 1 ; i < v.length ; i++) {
        if (v[i] > maiorAtehAgora){
            maiorAtehAgora = v[i];
        }
    }
    return maiorAtehAgora;
}
```

Repare na pré-condição. Além de exigir que o parâmetro `v` tenha um valor diferente de `null`, precisamos também de exigir que o *array* tenha pelo menos um elemento (`v.length > 0`) – não tem sentido calcular o máximo de um vetor vazio, além de que iria dar erro de execução quando, na primeira instrução do corpo do método tentássemos aceder ao elemento `v[0]`.

9.6.3. Modificar os elementos de um *array* dado como parâmetro

Como um parâmetro de tipo *array* recebe uma referência a um objeto, então é possível modificar esse objeto através dessa referência. Há casos em que isso é desejável.

Para um dado *array* `v` e um dado valor do seu parâmetro `val`, o método seguinte altera para o valor `val` cada elemento do *array* `v` que tenha um valor inferior a `val`.

```
/**
 * Altera os elementos de um dado vetor de inteiros
 * que sao menores que um dado valor para esse valor
 * @param v - O vetor a considerar
 * @param val - O valor de referencia
 * @requires v != null
 */
static void alteraValoresInferiores (int[] v, int val) {
```

```

for (int i = 0 ; i < v.length ; i++) {
    if(v[i] < val) {
        v[i] = val;
    }
}
}

```

A cada invocação do método para um dado vetor e um dado inteiro, é copiada para o parâmetro `v` a referência ao vetor dado e para o parâmetro `val` o valor do inteiro dado; de seguida, todos os elementos desse vetor que tinham valores inferiores ao dado inteiro ficam com um valor igual a esse inteiro.

As instruções seguintes criam um *array* com oito inteiros aleatórios no intervalo [0,20] e de seguida alteram os valores desse *array* que são menores que 10 para o valor 10. Imprimem os elementos do *array* antes e após essa alteração.

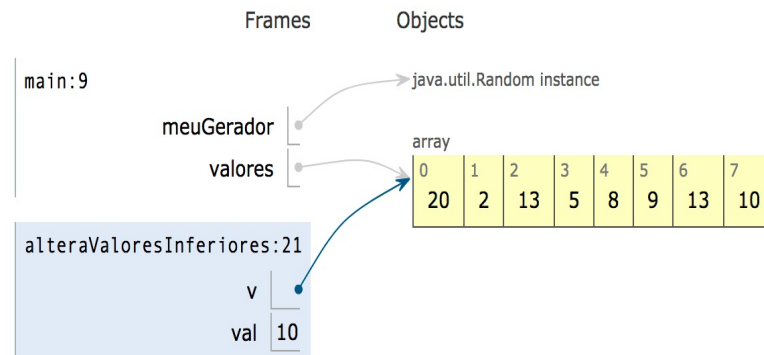
```

Random meuGerador = new Random();
int[] valores = vetorDeAleatorios(8, 20, meuGerador);
imprimeVetor(valores);
alteraValoresInferiores(valores, 10);
imprimeVetor(valores);

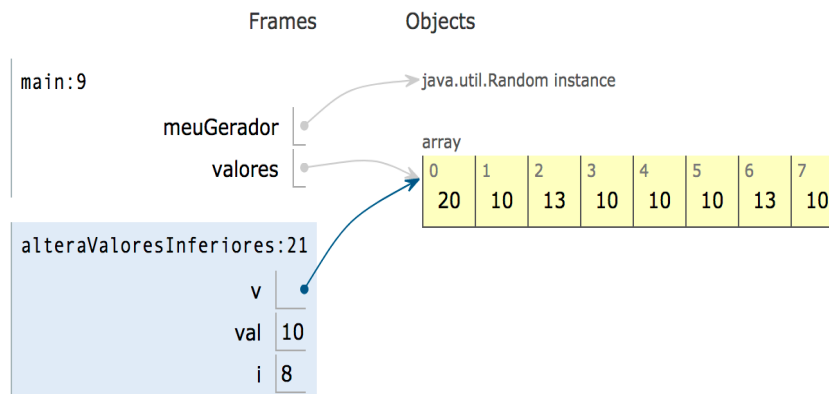
```

De seguida exemplificamos o estado da memória em várias fases de uma possível execução das instruções acima:

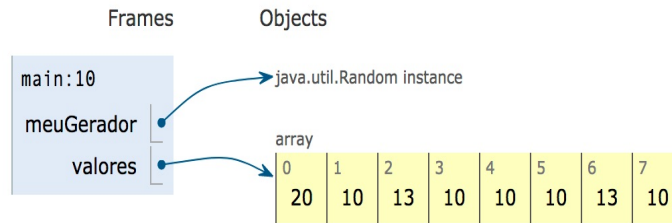
- Quando o método `alteraValoresInferiores` começa a ser executado, o seu parâmetro `v` contém uma cópia do conteúdo da variável `valores`, do `main`, que é uma referência ao *array* já existente.



- Quando o ciclo termina, no método `alteraValoresInferiores`, os elementos do *array* referenciado por `v` já estão alterados.



- Quando o controle de execução volta ao `main`, após a execução do método `alteraValoresInferiores`; o *array* referenciado pela variável `valores` já está alterado (claro! é o mesmo que foi usado no método).



10. Criar novos tipos de dados

Relembre de novo que um *tipo de dados não primitivo* define:

- um universo de objetos e
- um conjunto de métodos que podemos invocar sobre esses objetos.

A classe `Math` do pacote `java.lang` e a nossa classe `UtilidadesIP` onde fomos “guardando” os métodos úteis que fomos criando ao longo deste curso, são dois exemplos de classes que **não** definem tipos de dados.

Não faz sentido falar de objetos do tipo `Math` ou do tipo `UtilidadesIP`

- em ambas as classes só existem métodos `static` – métodos cujo alvo de invocação é a classe propriamente dita e não uma dada instância dessa classe;
- logo, o resultado/efeito de qualquer método dessas classes depende somente dos valores usados para os parâmetros aquando da invocação;
- entendemos estas classes como repositórios de métodos úteis e não como definições de novos tipos de dados.

Por outro lado, as classes `String` e `StringBuilder` do pacote `java.lang` e as classes `Random` e `Scanner` do pacote `java.util` são exemplos de tipos de dados não primitivos

- nos nossos programas podemos criar instâncias diversas dessas classes e
- atuar sobre cada um desses objetos através de *métodos de instância* (não `static`) cujos resultados/efeitos normalmente dependem
 - do *estado* de cada objeto (i.e., os valores das características específicas que o diferenciam de todos os outros objetos do mesmo tipo) e
 - dos valores usados para os parâmetros aquando da invocação.

Tal como os autores da biblioteca do Java criaram uma série de classes que definem tipos de dados, também nós podemos criar novos tipos de dados para representarmos a informação que queremos manipular nos nossos programas.

Classes que definem alunos, turmas, escolas, leilões, participantes em leilões, equipas de futebol, apostadores, jogos, jogadores, planetas, e muitas outras mais, dependendo dos dados que queremos manipular nos nossos programas.

Suponha que quer fazer um programa que trata informação sobre jogadores e jogos envolvendo jogadores que vão fazendo jogadas, acumulando pontos ao longo do jogo.

Suponha que a informação relevante acerca de um jogador é o seu nome, os pontos já acumulados e o número de pontos máximo ganho numa só jogada.

Uma abordagem **pouco esperta** seria criar três *arrays* com a informação sobre os vários jogadores participantes num jogo – um *array* de *strings* para guardar os nomes, um *array* de *int* que guardaria as pontuações dos vários jogadores e, finalmente, um outro *array* de *int* para guardar o máximo de cada um numa só jogada.

O problema com esta abordagem é que a informação de um dado jogador ficaria “espalhada” pelos três *arrays*. Uma mesma posição *p* em cada um dos três *arrays* teria que corresponder a informações acerca de um mesmo jogador, ou seja, se ordenássemos um dos *arrays* teríamos que ordenar também os outros para manter esta correspondência; se inseríssemos um novo jogador entre outros dois jogadores, teríamos que o fazer exatamente na mesma posição nos três *arrays*; se apagássemos um jogador teríamos que apagar exatamente a mesma posição dos três *arrays*. Este tipo de ações são geradoras de erros e, por isso, indesejáveis.

Vamos então ver qual a abordagem adequada para criar novos tipos de dados.

10.1. O tipo de dados **Jogador**

Uma abordagem muito mais esperta é considerar um jogador como uma entidade única,

- cujo *estado* é definido pelos valores do seu nome, pontuação atual e número máximo de pontos que obteve numa só jogada,
- que “sabe” coisas acerca dela própria (por ex., sabe o seu nome, a sua pontuação e o número máximo de pontos numa só jogada) e
- que sabe “fazer” coisas (por ex., registar uma nova pontuação parcial).

O tipo de dados que representa estes jogadores é definido por uma classe Java.

Um jogador em particular é uma *instância* dessa classe, um *objeto*.

A classe em questão deverá definir:

- o conjunto potencial de todos os jogadores (como combinações possíveis de valores permitidos para o nome, pontuação e número máximo de pontos numa só jogada) e
- os métodos que podemos invocar sobre esses objetos
 - métodos que revelam o estado do objeto, ou seja, os valores dos *atributos* que caracterizam o aluno – nome, pontuação e número máximo de pontos numa só jogada;
 - método que permite alterar o valor de um atributo – da pontuação;
 - outros que motivaremos mais à frente.

Depois de criada essa classe, se bem documentada, poder-se-ia gerar a documentação da sua API (*Application Programming Interface*) contendo, entre outras, a seguinte informação:

Constructors

Constructor and Description

Jogador(java.lang.String nomeJog)

Inicializa os atributos de uma nova instancia

Jogador(java.lang.String nomeJog, int pontos, int max)

Inicializa os atributos de uma nova instancia

Method Summary

All Methods

Instance Methods

Concrete Methods

Modifier and Type	Method and Description
Jogador	copia() Uma copia deste jogador
boolean	igualJogador(Jogador outro) Este jogador eh igual a outro?
int	maximoNumaJogada() O numero maximo de pontos que este jogador ganhou numa soh jogada
java.lang.String	nome() O nome deste jogador
int	pontuacao() A pontuacao deste jogador
void	registrarPontos(int pontos) Registrar uma nova pontuacao para este jogador
java.lang.String	toString() Representacao textual deste jogador

A partir do momento em que a classe `Jogador` estivesse criada e disponível, poderíamos criar uma outra classe que declarasse variáveis do tipo `Jogador`, construísse objetos desse tipo e usasse esses objetos, como é feito na classe seguinte.

```
public class ClienteDeJogador {  
  
    public static void main (String [] args) {  
  
        Jogador meuJogador = new Jogador("Maria");  
        System.out.println("O nome eh " + meuJogador.nome());  
  
        meuJogador.registrarPontos(5);  
        System.out.println("a pontuacao eh " + meuJogador.pontuacao());  
        meuJogador.registrarPontos(9);  
    }  
}
```

```

        System.out.println("agora a pontuacao eh " +
            meuJogador.pontuacao());
        System.out.println("o maximo de pontos feito numa jogada eh " +
            meuJogador.maximoNumaJogada());

        Jogador teuJogador = new Jogador("Pedro");
        System.out.print("O nome deste outro eh " + teuJogador.nome());

        // Um array com 7 elementos do tipo Jogador
        Jogador[] participantes = new Jogador[7];
        participantes[0] = new Jogador("Antonio");
        participantes[0].registarPontos(7);
        participantes[1] = new Jogador("Rui");

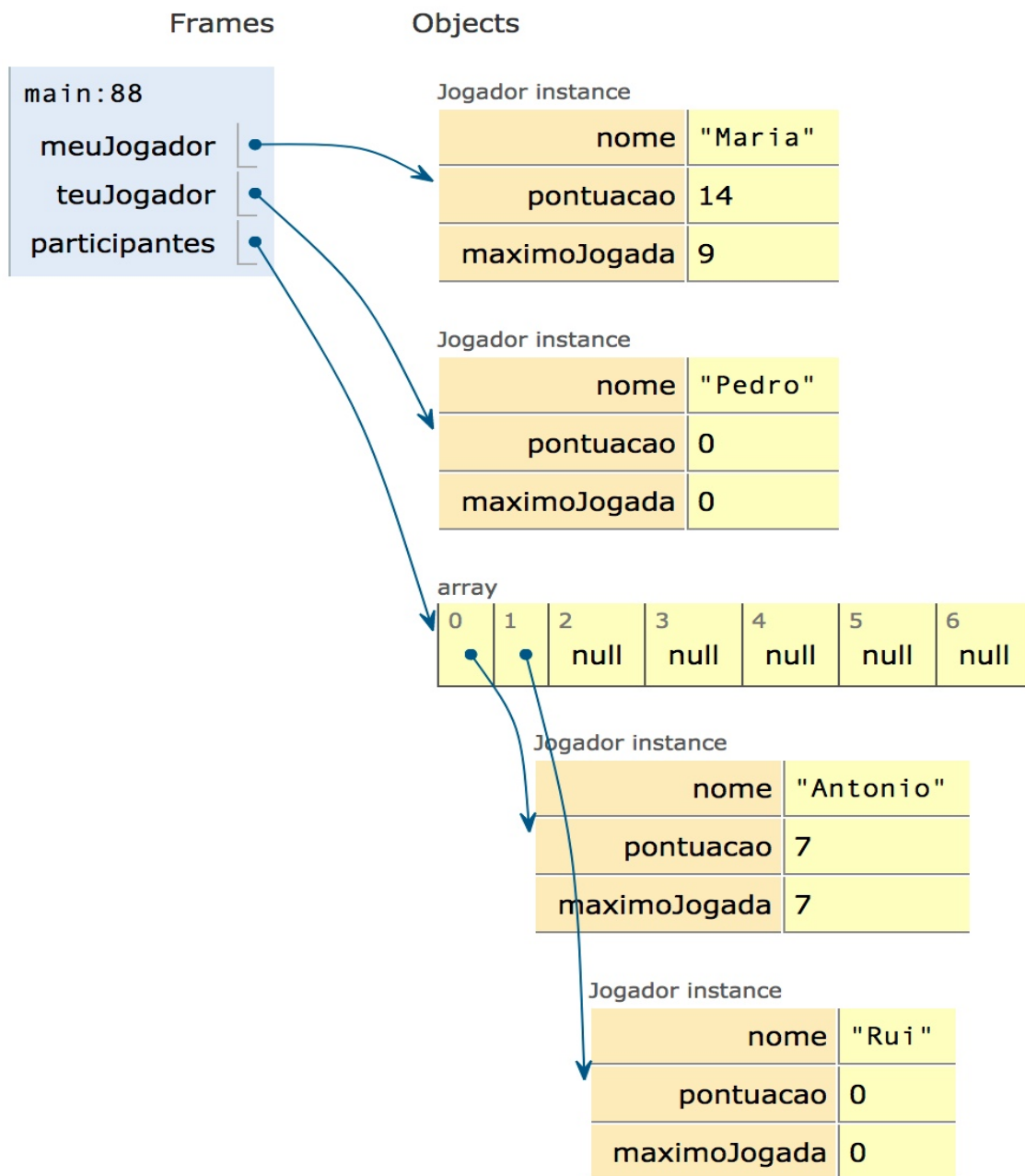
        System.out.print("O nome do primeiro jogador: " +
            participantes[0].nome());
        System.out.println(" e a sua pontuacao: " +
            participantes[0].pontuacao());

        System.out.print("O nome do segundo jogador: " +
            participantes[1].nome());
        System.out.println(" e a sua pontuacao: " +
            participantes[1].pontuacao());
    }
}

```

Esta classe é uma classe *cliente* da classe `Jogador` pois nela se usa o tipo de dados `Jogador`. A classe `Jogador` é uma classe *fornecedora* desta.

Embora ainda não tenhamos cosntruído a classe `Jogador`, podemos imaginar a representação do estado da memória após a execução da última instrução do programa acima (não se preocupe por ainda não conseguir perceber bem algumas partes da imagem):



E podemos também imaginar o *output* do programa:

```
O nome eh Maria
a pontuacao eh 5
agora a pontuacao eh 14
o maximo de pontos feito numa jogada eh 9
O nome deste outro eh Pedro
O nome do primeiro jogador: Antonio e a sua pontuacao: 7
O nome do segundo jogador: Rui e a sua pontuacao: 0
```

Nas próximas secções vamos ver, passo a passo, como se cria uma classe que define um tipo de dados.

Usaremos a classe `Jogador` para ilustrar o processo.

As várias partes que compõem a definição da classe são descritas no bloco da classe, ou seja, entre as chavetas:

```
public class Jogador {  
  
    // AQUI !!  
  
}
```

10.2. Atributos de instância

Já vimos que as características que consideramos relevantes acerca de um jogador são o seu nome, a sua pontuação e número máximo de pontos que obteve numa só jogada.

Não nos interessa, neste contexto, a altura do jogador, nem o seu NIF, nem se tem irmãos, etc. Noutros contextos poderá até haver interesse em representar também estas informações, mas aqui não.

O universo possível de objetos do tipo `Jogador` será então formado por todas as combinações de valores para aquelas características, ou atributos, ou seja, todas as formas possíveis que um jogador pode tomar.

A informação que permite caracterizar um dado objeto e, eventualmente, distingui-lo dos outros, constitui o que se chamam *atributos* do objeto.

São os valores que os atributos de um dado objeto têm que definem o *estado* desse objeto.

Porque cada instância da classe, ou objeto, tem os seus próprios valores para os atributos, estes dizem-se *atributos de instância* (em oposição aos atributos de classe de que falaremos mais adiante).

Vamos então começar

```
public class Jogador {  
  
    private String nome;  
    private int pontuacao;  
    private int maximoJogada;  
  
    // Ainda nao estah completa  
}
```

Os atributos de instância declaram-se dentro do bloco da classe, fora de qualquer método, pois eles não são locais a nenhum método; eles são acessíveis a partir de qualquer método da classe `Jogador`.

Colocamos a palavra `private` a preceder a declaração de cada atributo

- assim escondemos os pormenores de representação de cada objeto,
 - não aparecem na documentação;
 - não é possível aceder-lhes diretamente a partir de classes cliente;
- a manipulação indevida dos valores dos atributos é assim impossível,
 - há atributos cujos valores não devem mudar durante a vida de um objeto (por exemplo, o nome do jogador)
 - e outros em que os valores que tomam não podem ser quaisquer (por exemplo, a pontuação, que tem que ser não negativa; e o máximo de pontos numa só jogada depende das pontuações que o jogador vai tendo ao longo de uma ou mais jogadas);
- se em vez de `private` tivéssemos a palavra `public`, então em qualquer classe cliente de `Jogador` os atributos poderiam ser alterados diretamente
 - por exemplo, no `main` da classe `ClienteDeJogador` poderíamos ter a instrução `meuJogador.pontuacao = -47;`, que alteraria para -47 o valor do atributo `pontuacao` do objeto

referenciado pela variável `meuJogador`; isto não é desejável...

- se em vez de `private` não tivéssemos colocado nenhuma palavra, então qualquer classe cliente definida no mesmo pacote (*package*) que a classe `Jogador` poderia aceder diretamente aos atributos.

Representamos um objeto:

- como um retângulo dividido em tantas linhas quantas o número de atributos que tem;
- os nomes dos atributos são apresentados no lado esquerdo de cada linha;
- os valores dos atributos representam-se no lado direito de cada linha:
 - valores específicos no caso de atributos de tipos primitivos;
 - referências a objetos (setas) no caso de atributos de tipos não primitivos (com exceção de strings, para simplificar).

Exemplo:

nome	"Maria"
pontuacao	14
maximoJogada	9

10.3. Construtores

Relembre do capítulo 8, que para construir um objeto usamos o operador `new` seguido do nome da classe que queremos instanciar, seguido de zero ou mais expressões, entre parêntesis, para dar valores aos parâmetros de um dado *construtor* dessa classe.

A criação de uma instância de uma classe provoca as seguintes ações, executadas em três passos:

1. reserva e afetação de memória para o objeto (contando com o espaço a ocupar por cada um dos seus atributos);
2. inicialização desses atributos com *valores por omissão*:
 - zero para atributos de tipos primitivos numéricos,
 - `false` para `boolean`, e
 - `null` para atributos cujo tipo não é primitivo;
3. invocação do construtor indicado, sobre o objeto acabado de criar.

O objetivo principal dos construtores é o de inicializar os atributos de um objeto acabado de criar com valores diferentes dos valores por omissão.

Incluímos a definição do construtor que pretendemos, na classe que estamos a construir:

```
public class Jogador {  
  
    private String nome;  
    private int pontuacao;  
    private int maximoJogada;  
  
    /**
```

```

    * Inicializa os atributos de uma nova instancia
    * @param nomeJog O nome do novo jogador
    * @requires nomeJog != null
    */
    public Jogador (String nomeJog) {
        this.nome = nomeJog;
        this.pontuacao = 0;
        this.maximoJogada = 0;
    }

    // Ainda nao estah completa
}

```

10.3.1. O objeto corrente – `this`

No corpo de um método, a palavra reservada `this` representa o *objeto corrente* – o objeto alvo da invocação do método.

Neste caso, em que `this` aparece num construtor, ele representa o objeto acabado de criar, aquele sobre o qual o construtor foi invocado.

A palavra `this` seguida de um ponto e do nome de um atributo, representa o atributo em questão do objeto corrente.

10.3.2. Construindo um objeto

Tal como descrito atrás, a execução da instrução

```
Jogador meuJogador = new Jogador("Maria");
```

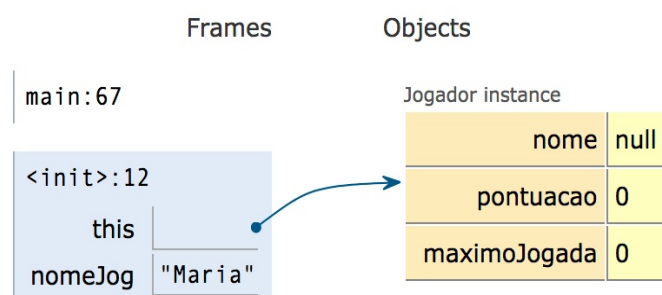
do método `main` da classe `ClienteDeJogador`, tem como efeito

1. a criação de uma instância de `Jogador`,
2. a inicialização dos valores dos seus atributos numéricos a zero e do seu nome a `null`,
3. a posterior atribuição, por execução do construtor, dos valores "Maria", 0 e 0 aos seus atributos `nome`, `pontuacao` e `maximoJogada`,
4. a atribuição, à variável `meuJogador`, de uma referência para esse objeto.

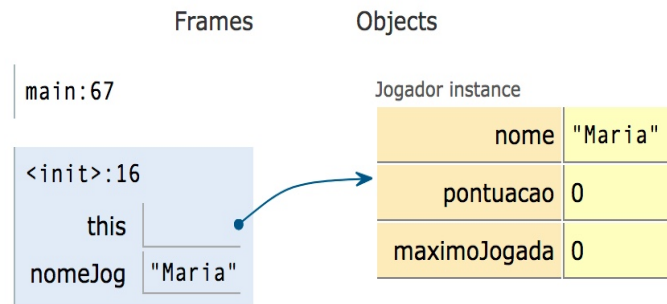
Repare nas duas últimas instruções do construtor: atribuem o valor zero aos atributos `pontuacao` e `maximoJogada`. Na verdade, estas instruções não são necessárias, pois no instante em que são executadas já esses atributos têm o valor zero (o valor que é dado por omissão aos atributos do tipo `int`). Escolhemos inclui-las para melhor percepção do estado inicial do objeto.

Representamos agora o estado da memória como resultado daquela instrução.

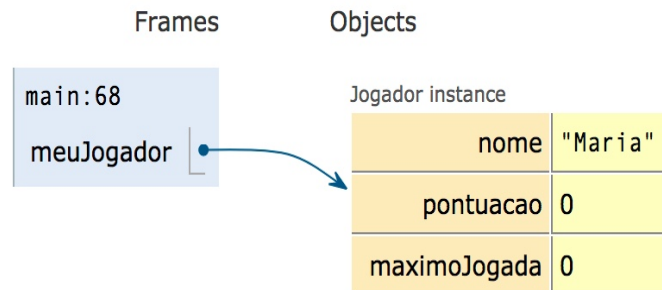
Após os passos 1 e 2:



Após o passo 3:



Após o passo 4:



10.4. Métodos que revelam o estado

Como os atributos foram declarados como `private`, não há forma de as classes cliente da classe `Jogador` conhecerem diretamente os valores dos objetos que criam.

Para os atributos cujos valores queremos poder revelar (neste caso todos), declaramos métodos que retornam o seu valor.

```
public class Jogador {

    private String nome;
    private int pontuacao;
    private int maximoJogada;

    /**
     * Inicializa os atributos de uma nova instancia
     * @param nomeJog O nome do novo jogador
     * @requires nomeJog != null
     */
    public Jogador (String nomeJog) {
        this.nome = nomeJog;
        this.pontuacao = 0;
        this.maximoJogada = 0;
    }

    /**
     * O nome deste jogador
     */
    public String nome () {
        return this.nome;
    }

    /**
     * A pontuacao deste jogador
     */
}
```

```

    */
    public int pontuacao () {
        return this.pontuacao;
    }

    /**
     * O numero maximo de pontos que este jogador ganhou numa
     * soh jogada
     */
    public int maximoNumaJogada () {
        return this.maximoJogada;
    }

    // Ainda nao estah completa
}

```

Não é obrigatório que os nomes dos métodos que revelam os valores dos atributos sejam iguais aos nomes que nós demos aos atributos, embora neste caso isso aconteça com os dois primeiros atributos.

A execução das instruções

```

Jogador meuJogador = new Jogador("Maria");
System.out.println("O nome eh " + meuJogador.nome());

Jogador teuJogador = new Jogador("Pedro");
System.out.print("O nome deste outro eh " + teuJogador.nome());

```

do método `main` da classe `ClienteDeJogador`, tem como efeito a escrita, no *standard output*, das frases

```

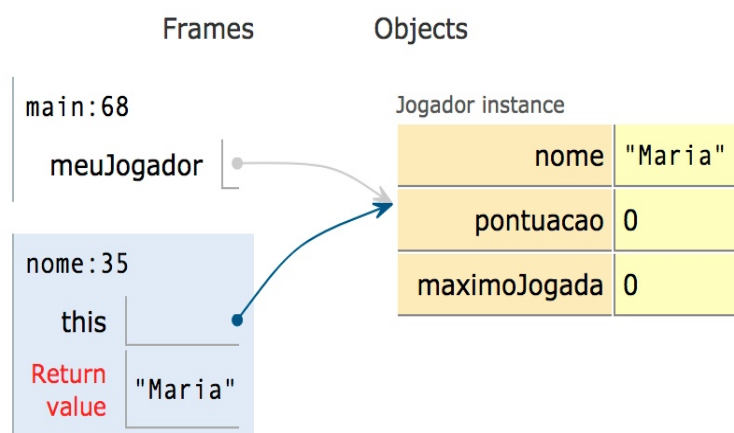
O nome eh Maria
O nome deste outro eh Pedro

```

Repare que os métodos de instância (não `static`), definidos atrás, retornam valores diferentes conforme o objeto alvo de uma específica invocação.

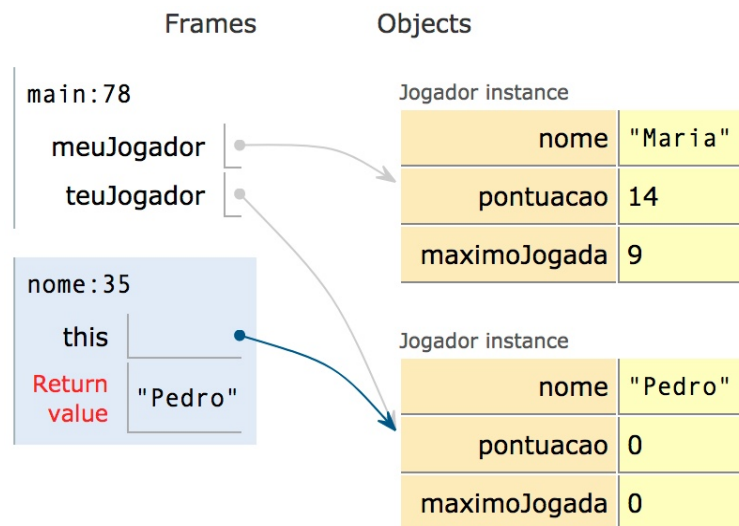
O objeto alvo da invocação é diferente nas invocações `meuJogador.nome()` e `teuJogador.nome()`, logo `this` refere-se a objetos diferentes de cada vez – na primeira invocação `this` refere o objeto cujo nome é Maria e na segunda invocação `this` refere o objeto cujo nome é Pedro.

A figura seguinte ilustra o estado da memória no fim da execução do método `nome()` aquando da invocação `meuJogador.nome()` na instrução `System.out.println("O nome eh " + meuJogador.nome());` acima:



Repare como a variável `this` referencia o mesmo objeto que foi alvo da invocação do método.

A figura seguinte ilustra o estado da memória no fim da execução do método `nome()` aquando da invocação `teuJogador.nome()` na instrução `System.out.println("O nome deste outro eh " + teuJogador.nome());` acima:



Volte a reparar no objeto referenciado por `this` desta vez. É o mesmo objeto referenciado pela variável `teuJogador`, exatamente aquele sobre o qual a invocação foi feita.

10.5. Métodos que alteram o estado

Podemos construir métodos que permitem alterar os valores de todos ou só de alguns atributos.

No caso dos jogadores, só vamos permitir alterar os atributos que dizem respeito às pontuações. Não vamos permitir que o nome seja alterado.

```
public class Jogador {

    private String nome;
    private int pontuacao;
    private int maximoJogada;

    // Escondemos aqui partes jah feitas
    ...

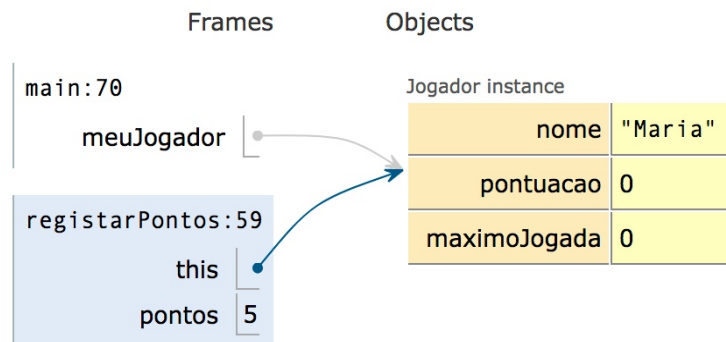
    /**
     * Registrar uma nova pontuacao para este jogador
     * @param pontos Os pontos a acumular ah pontuacao deste jogador
     * @requires pontos > 0
     */
    public void registrarPontos (int pontos) {
        this.pontuacao += pontos;
        if (pontos > this.maximoJogada) {
            this.maximoJogada = pontos;
        }
    }

    // Ainda nao estah completa
}
```

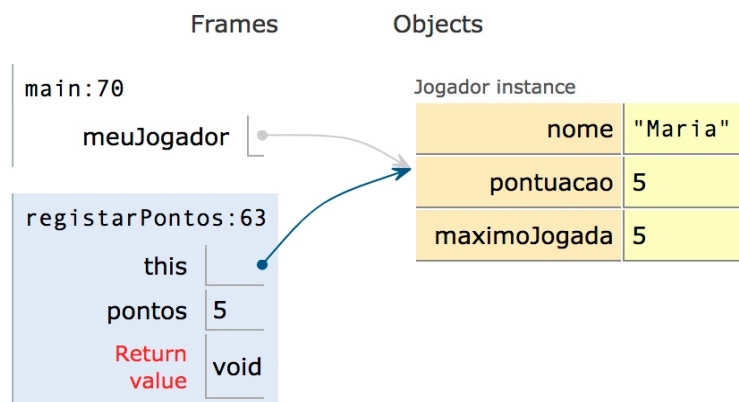
Repare que este método não se limita a acumular os novos pontos na pontuação total. Também verifica se este valor a acumular é maior que o máximo numa jogada registado até ao momento (guardado no atributo `maximoJogada`); no caso de ser superior, então esse valor passa a ser o novo máximo registado numa só jogada.

As três figuras seguintes ilustram o estado da memória aquando da execução da instrução `meuJogador.registarPontos(5)` no `main` da classe `ClienteDeJogador` das secções anteriores:

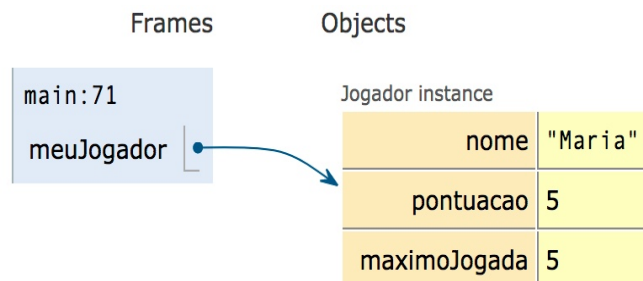
- no início da execução do método, repare a variável `this` a referenciar o objeto que foi alvo da invocação e o parâmetro `pontos` com o valor dado na invocação:



- já no fim da execução do método, mesmo antes de voltar ao `main`, repare que os valores dos atributos `pontuacao` e `maximoJogada` do objeto `meuJogador` já têm novos valores, que lhes foram atribuídos pelas instruções `this.pontuacao += pontos;` e `this.maximoJogada = pontos;` do método `registarPontos`:

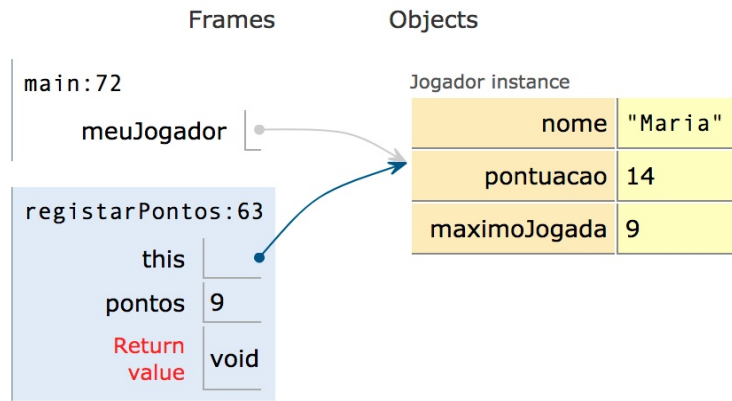


- já de volta ao `main` após a execução do método; repare que o jogador já tem a nova pontuação:



Quando, de seguida, a instrução `meuJogador.registarPontos(9)` é executada,

- os valores destes dois atributos voltam a ser atualizados (o valor 9 passa a ser o máximo de pontos obtido numa só jogada, como seria de esperar):



10.6. Mais construtores

Podemos querer criar um jogador inicializando-o logo com todos os valores para os seus atributos (por exemplo, para poder ser usado em contextos em que os jogadores já podem ter pontuações anteriormente acumuladas).

Apesar de podermos fazer isso usando o construtor que já definimos e invocando de seguida o método que altera a sua pontuação, preferimos definir um novo construtor que faz essa inicialização.

```
public class Jogador {

    private String nome;
    private int pontuacao;
    private int maximoJogada;

    // Escondemos aqui partes jah feitas
    ...

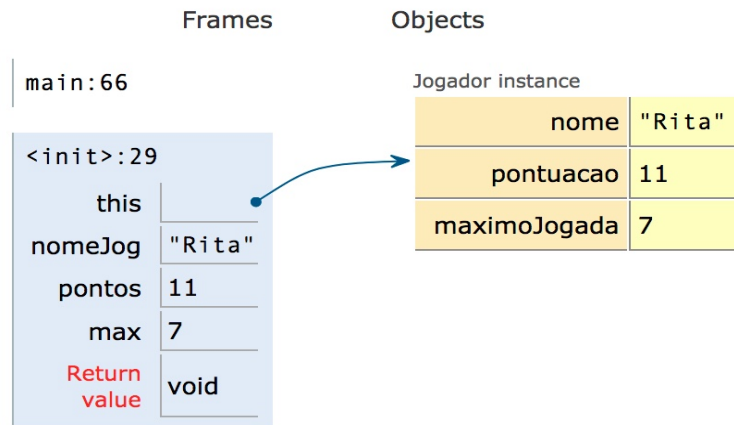
    /**
     * Inicializa os atributos de uma nova instancia
     * @param nomeJog O nome do novo jogador
     * @param pontos A pontuacao do novo jogador
     * @param max O numero maximo de pontos ganho numa soh jogada
     * @requires nomeJog != null && pontos >= 0 && max >= 0 && pontos >= max
     */
    public Jogador (String nomeJog, int pontos, int max) {
        this.nome = nomeJog;
        this.pontuacao = pontos;
        this.maximoJogada = max;
    }

    // Escondemos aqui partes jah feitas
    ...
}
```

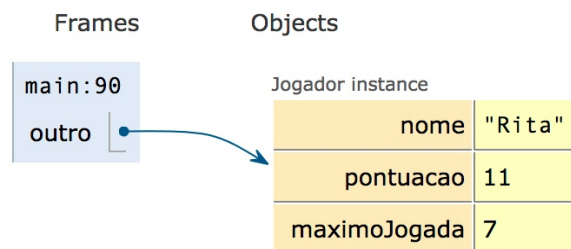
Podemos representar o estado da memória como resultado da execução desta instrução (inserida, por exemplo, no `main` da classe `ClienteDeJogador`):

```
Jogador outro = new Jogador("Rita", 11, 7);
```

Após executar o construtor, imediatamente antes de retornar ao `main`:



Já de volta ao `main`, após fazer a atribuição à variável `outro` da referência para o novo objeto:



10.7. Imprimir, duplicar e comparar objetos – como NÃO fazer!

Vamos começar por mostrar como **NÃO** se devem fazer as coisas, para depois, na próxima secção, fazermos da forma correta.

Imprimir um objeto – como NÃO fazer

Se adicionássemos a seguinte instrução

```
System.out.println(meuJogador);
```

ao método `main` da classe `ClienteDeJogador`, com o propósito de imprimir o estado do nosso jogador, obteríamos um *output* parecido com:

```
Jogador@677327b6
```

que não é, decididamente, o que nós queríamos...

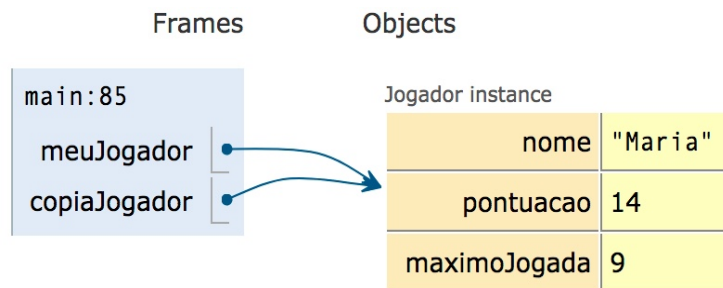
O que obtivemos foi o nome do tipo do objeto seguido de um código hexadecimal (base 16) especial que é único para cada objeto. Este código pode variar de máquina para máquina e até de execução para execução.

Copiar um objeto – como NÃO fazer

Se acrescentássemos a seguinte instrução, com a intenção de “copiar” um jogador:

```
Jogador copiaJogador = meuJogador;
```

iríamos ter o seguinte objeto e variáveis em memória:



o que mostra que não foi criada nenhuma cópia, ou seja,

- continuamos a ter somente um objeto;
- temos duas variáveis a referenciar esse objeto (relembrar secção 8.7).

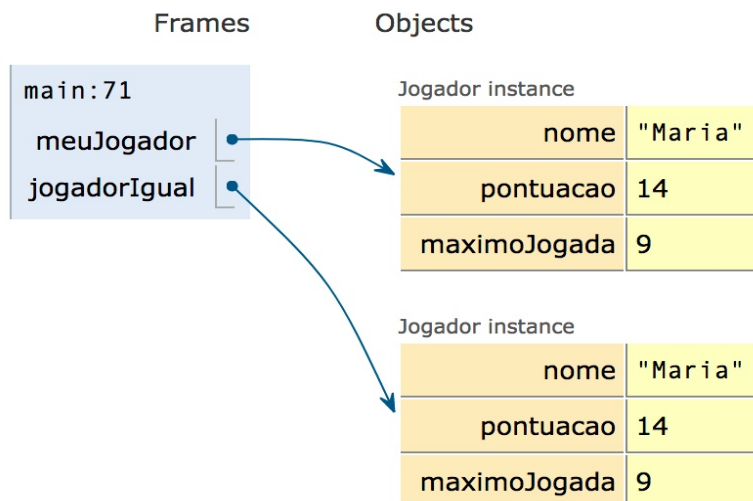
Por isso, se invocássemos o método para alterar a pontuação, por exemplo, com a instrução `copiaJogador.registarPontos(13);`, essa invocação afetaria o único objeto existente.

Comparar dois objetos – como NÃO fazer

Agora, se acrescentássemos as seguintes instruções:

```
Jogador jogadorIgual = new Jogador("Maria",14,9);
if(meuJogador == jogadorIgual){
    System.out.println("Sao iguais!");
} else {
    System.out.println("Sao diferentes!");
}
```

iríamos ter os seguintes objetos e variáveis em memória:



e, por isso, iríamos obter o *output*:

```
Sao diferentes!
```

que também não é o que poderíamos estar à espera, pois os objetos têm igual estado.

Mas, aqui, já devíamos saber! (relembrar secção 7.2.9).

O problema é que estamos a comparar os valores das variáveis `meuJogador` e `jogadorIgual`, os quais são referências para dois objetos distintos; logo, embora os objetos tenham valores iguais para os seus atributos, as referências para esses objetos são diferentes.

Nova tentativa – mas AINDA NÃO É ASSIM que devemos fazer

Já que a classe nos fornece métodos que revelam os valores dos atributos, poderíamos usar esses métodos para obter o que queremos...

```
// Imprimir o jogador
System.out.println("Nome: " + meuJogador.nome() +
    " Pontuacao: " + meuJogador.pontuacao());
System.out.println("Maximo numa soh jogada: " + meuJogador.maximoNumaJogada());

// Criar uma copia de jogador
Jogador copiaJogador = new Jogador(meuJogador.nome(),
    meuJogador.pontuacao(),
    meuJogador.maximoNumaJogada());

// Testar se dois objetos sao iguais
boolean iguais = meuJogador.nome().equals(copiaJogador.nome()) &&
    meuJogador.pontuacao() == copiaJogador.pontuacao() &&
    meuJogador.maximoNumaJogada() ==
        copiaJogador.maximoNumaJogada();

if(iguais){
    System.out.println("Sao iguais!");
} else {
    System.out.println("Sao diferentes!");
}
```

Um dos defeitos desta abordagem é que dificulta a evolução do *software*: por exemplo, se um dia quisermos aumentar o número de atributos que caracterizam um jogador, todas as classes cliente da classe `Jogador` que têm este tipo de instruções, terão que ser modificadas para acomodar essa modificação.

Na verdade **ESTA NÃO É UMA ABORDAGEM ORIENTADA A OBJETOS!!**

Raciocinemos então:

se um jogador encapsula toda a informação sobre ele próprio, ninguém melhor do que ele para

- fornecer a representação textual dele próprio,
- fornecer uma cópia dele próprio e
- dizer se é igual a um outro jogador.

Assim, as classes cliente só precisariam de invocar os métodos da classe `Jogador` que fornecem esses “serviços”.

É o que vamos ver na próxima secção.

10.8. Agora SIM! – Imprimir, duplicar e comparar objetos

Na classe `Jogador` vamos acrescentar 3 métodos que, quando invocados sobre um dado jogador, devolvem a sua representação textual, constroem e devolvem uma cópia sua e decidem se é igual a um dado jogador.

O método `toString`

Quando imprimimos um objeto usando `System.out.println` ou `System.out.print`, o método `toString` é invocado automaticamente sobre esse objeto.

Em cada classe há um método `toString` acessível, mesmo quando não está explicitamente definido na classe (como é o caso da classe `Jogador`).

Este método `toString`, quando invocado sobre um dado objeto, retorna a representação desse objeto em formato de *string*.

Este método está acessível em todas as classes porque é “herdado” da classe `Object` (este é um tópico mais avançado, que não exploraremos aqui).

A versão por omissão do método `toString`, herdada por toda e qualquer classe, retorna

- uma *string* contendo o tipo do objeto e
- um identificador único.

Já vimos o resultado deste método no início da secção 10.7.: instrução `System.out.println(meuJogador)` invoca automaticamente o método `toString` sobre o objeto `meuJogador` e é a *string* resultante dessa invocação que é escrita no ecrã – “Jogador@677327b6”.

Quando construímos uma classe, podemos *redefinir* o comportamento deste método, fazendo com que devolva uma representação textual do objeto mais apropriada ao que queremos.

Na classe `Jogador`, poderíamos ter o seguinte método `toString`:

```
/**
 * Representacao textual deste jogador
 */
public String toString () {
    StringBuilder sb = new StringBuilder();
    sb.append("Nome: " + this.nome);
    sb.append("    Pontuacao: " + this.pontuacao);
    sb.append("    Maximo numa soh jogada: " + this.maximoJogada);
    return sb.toString();
}
```

Agora, existindo este método na classe `Jogador`, o *output* resultante da instrução `System.out.println(meuJogador)`, na nossa classe `ClienteDeJogador`, seria:

```
Nome: Maria    Pontuacao: 14    Maximo numa soh jogada: 9
```

O método `igualJogador`

Existem duas noções de igualdade entre objetos:

- *identidade*, que significa que os objetos são o mesmo, e
- *equivalência*, que significa que os objetos são iguais, ou seja, têm o mesmo estado.

O operador `==` testa a identidade. Já estudámos este assunto.

No que diz respeito à equivalência, não há nenhum operador que a teste, porque o significado de equivalência depende do tipo específico dos objetos em questão.

Em vez disso, todas as classes têm acessível um método de nome `equals`, com o propósito de definir a equivalência.

Tal como com o método `toString`, o método `equals` deveria ser redefinido nas novas classes, porque

- o seu comportamento por omissão é o mesmo que a identidade (ou seja, só o próprio objeto é considerado

equivalente a si próprio),

- normalmente não é o que pretendemos.

No entanto, porque o conceito de equivalência envolve uma série de conceitos mais avançados, não trataremos aqui da redefinição do método `equals`.

Em vez disso, criaremos um método de nome `igualJogador` que tem um parâmetro do tipo `Jogador` e devolve `true` se os atributos do objeto corrente (`this`) e os do objeto dado como parâmetro têm valores iguais:

```
/**
 * Este jogador eh igual a outro?
 * @param outro O outro jogador
 * @return true se os atributos deste jogador sao iguais aos do outro
 * @requires outro != null
 */
public boolean igualJogador (Jogador outro) {
    boolean iguais = this.nome().equals(outro.nome()) &&
                    this.pontuacao() == outro.pontuacao() &&
                    this.maximoNumaJogada() == outro.maximoNumaJogada();

    return iguais;
}
```

As seguintes instruções, se acrescentadas à classe `ClienteDeJogador`, resultariam no output “Sao iguais!”

```
if(meuJogador.igualJogador(copiaJogador)){
    System.out.println("Sao iguais!");
} else {
    System.out.println("Sao diferentes!");
}
```

O método `copia`

A tarefa de criar uma cópia de um dado objeto deve ser dada ao próprio objeto, porque é ele que possui a informação necessária. Então adicionamos este método à classe `Jogador`:

```
/**
 * Uma copia deste jogador
 * @return Um novo jogador que eh uma copia deste jogador
 */
public Jogador copia () {
    return new Jogador(this.nome, this.pontuacao, this.maximoJogada);
}
```

Agora, sempre que uma classe cliente precisa de criar uma cópia de um objeto do tipo `Jogador`, basta usar este método, como por exemplo, na nossa classe `ClienteDeJogador`:

```
Jogador copiaJogador = meuJogador.copia();
```

Mais para a frente neste livro veremos outra forma de criar um *clone* de um objeto.

10.9. A classe Jogador completa

```
public class Jogador {

    private String nome;
    private int pontuacao;
    private int maximoJogada;

    /**
     * Inicializa os atributos de uma nova instancia
     * @param nomeJog O nome do novo jogador
     * @requires nomeJog != null.
     */
    public Jogador (String nomeJog) {
        this.nome = nomeJog;
        this.pontuacao = 0;
        this.maximoJogada = 0;
    }

    /**
     * Inicializa os atributos de uma nova instancia
     * @param nomeJog O nome do novo jogador
     * @param pontos A pontuacao do novo jogador
     * @param max O numero maximo de pontos ganho numa soh jogada
     * @requires nomeJog != null && pontos >= 0 && max >= 0 && pontos >= max
     */
    public Jogador (String nomeJog, int pontos, int max) {
        this.nome = nomeJog;
        this.pontuacao = pontos;
        this.maximoJogada = max;
    }

    /**
     * O nome deste jogador
     */
    public String nome () {
        return this.nome;
    }

    /**
     * A pontuacao deste jogador
     */
    public int pontuacao () {
        return this.pontuacao;
    }

    /**
     * O numero maximo de pontos que este jogador ganhou numa
     * soh jogada
     */
    public int maximoNumaJogada () {
        return this.maximoJogada;
    }

    /**
```

```

    * Registrar uma nova pontuacao para este jogador
    * @param pontos Os pontos a acumular ah pontuacao deste jogador
    * @requires pontos >= 0
    */
    public void registrarPontos (int pontos) {
        this.pontuacao += pontos;
        if (pontos > this.maximoJogada) {
            this.maximoJogada = pontos;
        }
    }

    /**
     * Representacao textual deste jogador
     */
    public String toString () {
        StringBuilder sb = new StringBuilder();
        sb.append("Nome: " + this.nome);
        sb.append("    Pontuacao: " + this.pontuacao);
        sb.append("    Maximo numa jogada: " + this.maximoJogada);
        return sb.toString();
    }

    /**
     * Este jogador eh igual a outro?
     * @param outro O outro jogador
     * @return true se os atributos deste jogador sao iguais aos do outro
     * @requires outro != null.
     */
    public boolean igualJogador (Jogador outro) {
        boolean iguais = this.nome().equals(outro.nome()) &&
                        this.pontuacao() == outro.pontuacao() &&
                        this.maximoNumaJogada() == outro.maximoNumaJogada();

        return iguais;
    }

    /**
     * Uma copia deste jogador
     * @return Um novo jogador que eh uma copia deste jogador
     */
    public Jogador copia () {
        return new Jogador(this.nome, this.pontuacao, this.maximoJogada);
    }
}

```

11. Um novo tipo de dados, cliente de **Jogador**

Neste capítulo construiremos programas que trabalham com jogos e jogadores de modo a ilustrar a utilização de uma coleção de objetos.

Se quisermos fazer um programa que ponha vários jogadores a jogarem até que um deles alcance uma dada pontuação, por exemplo, podemos sempre criar uma classe com um `main` que declare e crie um *array* de objetos do tipo `Jogador` e que use esses objetos para ir registando pontuações e verificando se o jogo deve acabar.

Podemos adotar outra abordagem mais estruturada e criar um tipo de dados, a que podemos chamar `Jogo`, que esconde (encapsula) a forma como o conjunto de jogadores é representado e permitisse operações sobre as suas instâncias como, por exemplo, registar jogadores, dar início ao jogo, registar pontuações para os jogadores, verificar se já terminou, determinar o(s) vencedor(es), etc..

Depois de criada essa classe, se bem documentada, poder-se-ia gerar a documentação da sua API (*Application Programming Interface*). Por exemplo:

Constructors

Constructor and Description

Jogo(int objetivo)

Construir um jogo.

Method Summary

All Methods

Instance Methods

Concrete Methods

Modifier and Type	Method and Description
boolean	emJogo (java.lang.String nomeJog) Um dado jogador estah em jogo?
void	juntarJogador (Jogador novo) Juntar um jogador a este jogo.
void	juntarJogador (java.lang.String nomeJog) Juntar um jogador com um dado nome a este jogo.
int	maiorPontuacao () A maior pontuacao dos jogadores deste jogo
int	mediaMaximosNumaJogada () A media das maximas pontuacoes numa soh jogada dos jogadores deste jogo
int	pontuacao (java.lang.String nomeJog) A pontuacao de um dado jogador deste jogo
int	pontuacaoObjetivo () A pontuacao objetivo deste jogo.
java.lang.String[]	pontuacaoSuperiorA (int limiteInf) Os nomes dos jogadores com pontuacao superior a um dado valor
int	quantosEmJogo () O numero de jogadores que estao em jogo.
void	registarPontosJogada (java.lang.String nomeJog, int pontuacao) Registar a pontuacao na jogada atual de um dado jogador.
boolean	terminou () Este jogo jah terminou?
java.lang.String	toString () Representacao textual deste jogo.
java.lang.String[]	vencedores () Vencedor(es) deste jogo

A partir do momento em que a classe `Jogo` estivesse criada e disponível, poderíamos criar uma outra classe que declarasse uma variável do tipo `Jogo`, construísse um objeto desse tipo e usasse esse objeto para simular a execução de um jogo em que as pontuações para os vários jogadores seriam geradas através de um objeto do tipo `Random`, como é feito na classe seguinte.

```

import java.util.Random;

public class PossivelClienteDeJogo {

    public static void main (String [] args) {

        Jogo meuJogo = new Jogo(30);
        System.out.println("O objetivo eh " + meuJogo.pontuacaoObjetivo());
        String[] participantes = {"Maria","Pedro","Rui","Margarida"};
        for (int i = 0 ; i < participantes.length ; i++) {
            meuJogo.juntarJogador(participantes[i]);
        }

        Random gerador = new Random();
        int max = Jogo.MAX_PONTUACAO_JOGADA;
        int min = Jogo.MIN_PONTUACAO_JOGADA;

        // O jogo eh jogado em rondas.
        // Em cada ronda jogam todos os jogadores.
        while(!meuJogo.terminou()) {

            for (int j = 0 ; j < participantes.length ; j++) {
                // gera um valor no intervalo [min,max]
                int pontuacao = gerador.nextInt(max - min + 1) + min;
                // regista a pontuacao do proximo jogador
                meuJogo.registarPontosJogada(participantes[j], pontuacao);

                System.out.println(participantes[j] + " ganhou mais " +
                                   pontuacao + " pontos");
            }
        }

        System.out.println("=====");
        System.out.println(meuJogo.toString());
        System.out.println("Media dos maximos numa soh jogada: " +
                            meuJogo.mediaMaximosNumaJogada());
        System.out.println("=====");
        String[] vencedores = meuJogo.vencedores();
        for (int i = 0 ; i < vencedores.length ; i++) {
            System.out.println(vencedores[i] + " venceu com " +
                               meuJogo.pontuacao(vencedores[i]) + " pontos");
        }
        System.out.println("=====");
        System.out.println("Parabens aos vencedores!");
    }
}

```

Vamos então construir a classe `Jogo` nas secções seguintes.

11.1. A classe `Jogo`

Para o objetivo que temos em mente, consideraremos que um jogo é totalmente caracterizado pela coleção de jogadores que nele se registam e pela pontuação que um jogador tem que ter para poder ganhar o jogo.

Começamos então por declarar os atributos da classe, um construtor que permita inicializar esses atributos e um método de instância que revela a pontuação objetivo do jogo.

```
/**
 * Classe cujos objetos representam jogos
 * @author Isabel Nunes
 */
public class Jogo {

    // A colecao dos jogadores
    private Jogador [] jogadores;
    // Pontuacao objetivo
    private int objetivo;

    /**
     * Construir um jogo.
     * @param objetivo A pontuacao com que se ganha este jogo
     * @requires objetivo > 0
     */
    public Jogo (int objetivo) {
        this.objetivo = objetivo;
        this.jogadores = new Jogador [5];
    }

    /**
     * A pontuacao objetivo deste jogo.
     */
    public int pontuacaoObjetivo () {
        return this.objetivo;
    }

    ...

}
```

O construtor recebe o valor de pontuação objetivo para o novo jogo como parâmetro e usa-o para inicializar o atributo `objetivo` do objeto em construção. Também inicializa o atributo `jogadores` com um novo *array* de 5 elementos do tipo `Jogador`. Como se deve lembrar, todos os elementos deste *array* são inicializados com o valor `null`, o que é perfeitamente aceitável, pois quando um jogo é criado, nenhum jogador se registou ainda.

Esta classe é uma classe *cliente* da classe `Jogador` pois nela se usa o tipo de dados `Jogador`. Então, a classe `Jogador` é uma classe *fornecedora* desta.

O programa seguinte cria e inicializa um objeto do tipo `Jogo` e imprime o valor da sua pontuação objetivo no *standard output*:

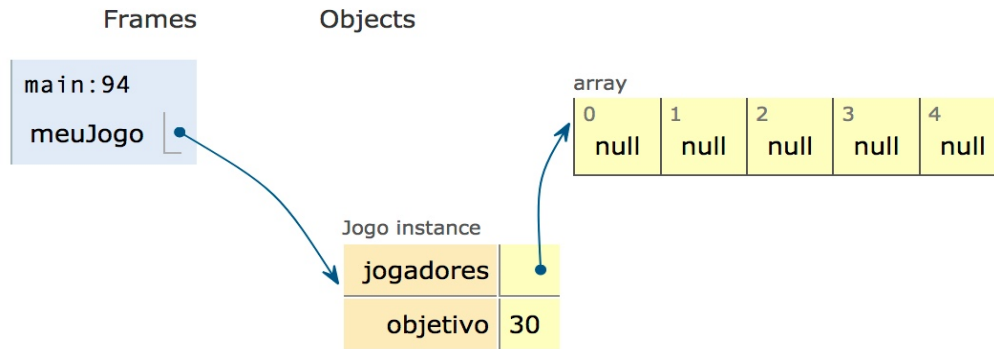
```
public class ClienteDeJogo {

    public static void main (String [] args) {

        Jogo meuJogo = new Jogo(30);
        System.out.println("O objetivo eh " + meuJogo.pontuacaoObjetivo());
    }

}
```

Após a execução das duas instruções, o estado de memória para o método `main` pode ser representado como na figura:



E a linha seguinte é impressa no *standard output*:

```
O objetivo eh 30
```

11.2. Registrar jogadores no jogo

Agora precisamos de um método que permita registrar um jogador no jogo. Adicionamos o seguinte método à classe `Jogo`:

```
/**
 * Juntar um jogador com um dado nome a este jogo.
 * @param nomeJog O nome do jogador.
 * @requires O jogador com o nome nomeJog ainda não estah
 *           registado neste jogo &&
 *           este jogo ainda comporta mais jogadores
 */
public void juntarJogador (String nomeJog) {
    this.jogadores[??] = new Jogador (nomeJog);
}
```

O método cria um novo jogador, com o nome dado pelo parâmetro, e adiciona-o ao conjunto de jogadores do jogo.

Que índice devemos usar para indicar o elemento do *array* `jogadores` ao qual deve ser atribuído o novo jogador? A primeira vez que o método é chamado deve ser zero, a segunda vez deve ser um e assim sucessivamente.

Repare que o índice a usar tem o mesmo valor que o número de jogadores registados no jogo no momento em que se vai registar um novo jogador (quando há zero jogadores em jogo, o próximo jogador deve ser atribuído ao elemento de índice zero do *array*, quando há um jogador em jogo, o próximo jogador deve ser atribuído ao elemento de índice um do *array* e assim sucessivamente).

Então, para que se possa guardar, para um jogo, quantos jogadores estão inscritos a cada momento, vamos adicionar um novo atributo de instância.

Adicionamos o atributo `quantosEmJogo`, o qual é inicializado a zero, e acrescentamos também um método que revela o seu valor. Mudamos também parte da pré-condição do método `juntarJogador` de forma a que as classes cliente saibam como podem verificar se o jogo não comporta mais jogadores (definimos um máximo de 5 jogadores).

```
/**
```

```

* Classe cujos objetos representam jogos
* @author Isabel Nunes
*/
public class Jogo {

    // A colecao dos jogadores
    private Jogador [] jogadores;
    // Pontuacao objetivo
    private int objetivo;
    // Numero de jogadores em jogo
    private int quantosEmJogo;

    /**
     * Construir um jogo.
     * @param objetivo A pontuacao com que se ganha este jogo
     * @requires objetivo > 0
     */
    public Jogo (int objetivo) {
        this.objetivo = objetivo;
        this.jogadores = new Jogador [5];
        this.quantosEmJogo = 0;
    }

    /**
     * A pontuacao objetivo deste jogo.
     */
    public int pontuacaoObjetivo () {
        return this.objetivo;
    }

    /**
     * O numero de jogadores que estao em jogo.
     */
    public int quantosEmJogo () {
        return this.quantosEmJogo;
    }

    /**
     * Juntar um jogador com um dado nome a este jogo.
     * @param nomeJog O nome do jogador.
     * @requires O jogador com o nome nomeJog ainda não estah
     *          registado neste jogo &&
     *          este jogo ainda comporta mais jogadores
     */
    public void juntarJogador (String nomeJog) {
        this.jogadores[this.quantosEmJogo] = new Jogador (nomeJog);
        this.quantosEmJogo++;
    }

    ...
}

```

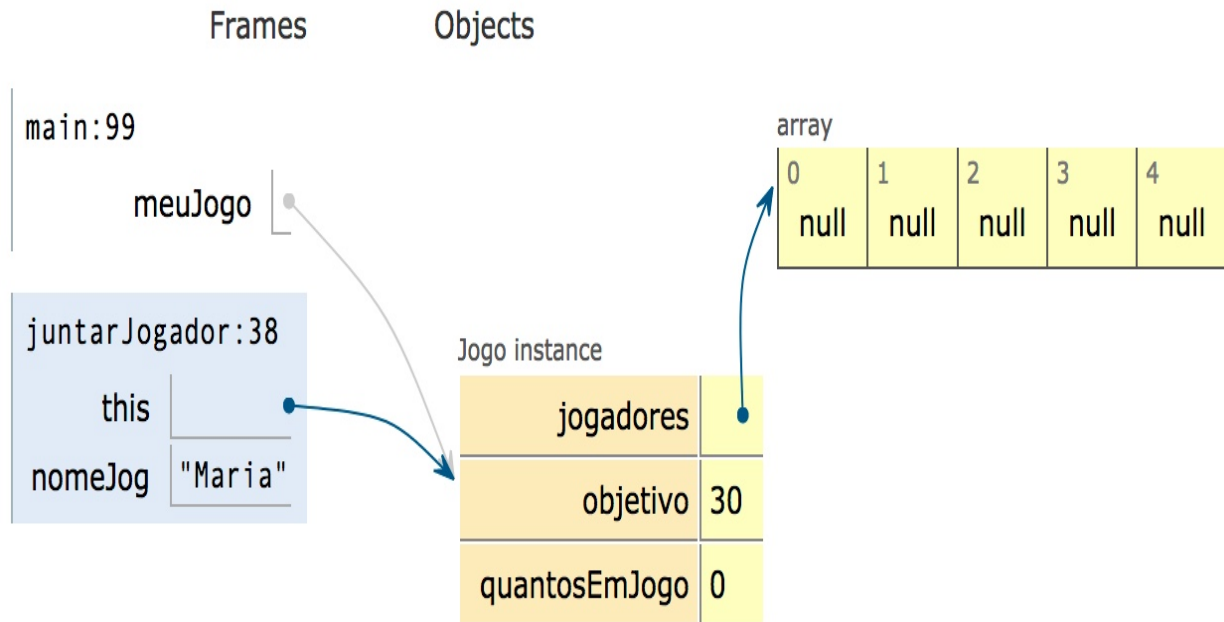
Repare que usámos este novo atributo como o índice que faltava no método `juntarJogador`. Quando um jogador é acrescentado ao jogo, o valor desse atributo `quantosEmJogo` tem que ser incrementado, pois passa a existir mais um jogador registado no jogo.

Vamos agora adicionar a instrução seguinte

```
meuJogo.juntarJogador("Maria");
```

ao método `main` da classe `ClienteDeJogo`. As figuras seguintes ilustram o estado da memória em várias fases da execução desta instrução:

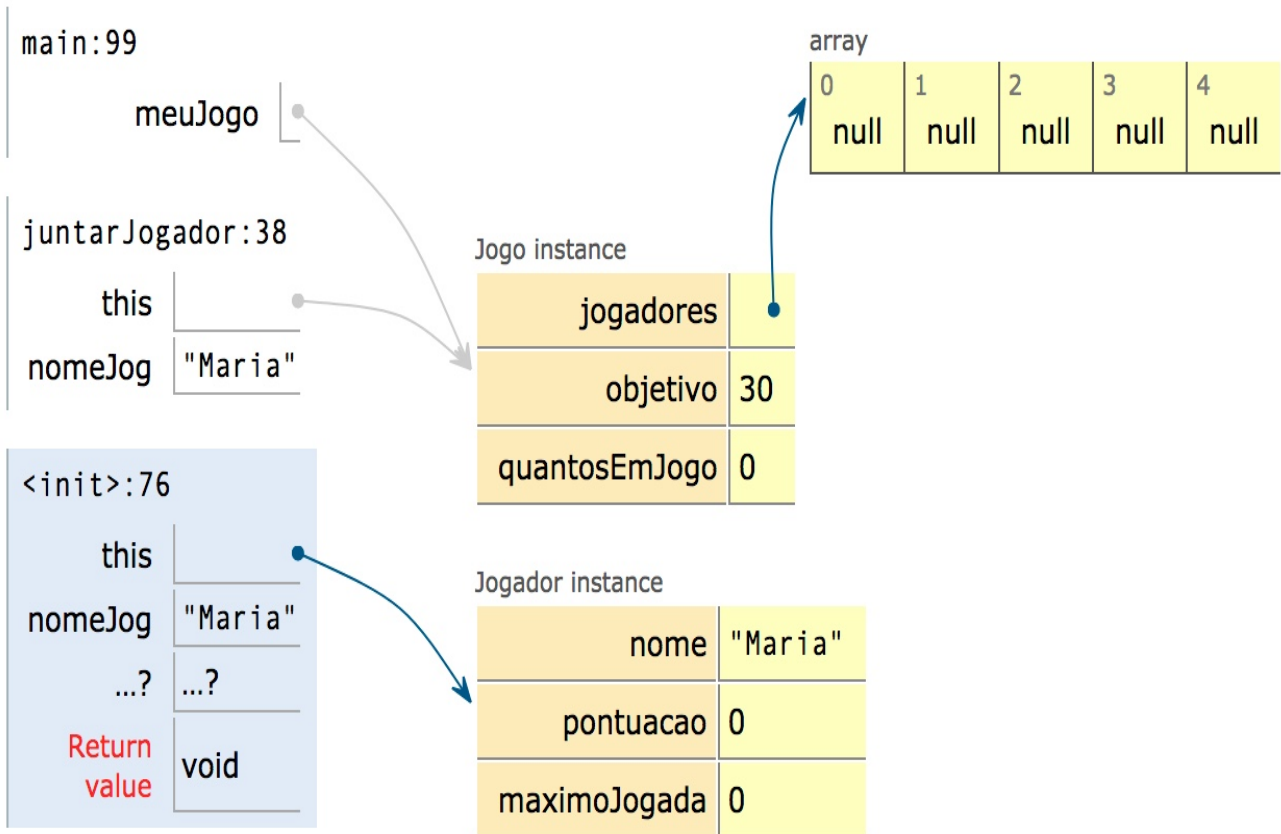
- quando o método `juntarJogador` é invocado (repare o valor de `this` no contexto deste método – é o mesmo conteúdo da variável `meuJogo`, que é precisamente o alvo da invocação):



- o método `juntarJogador`, na sua instrução `this.jogadores[this.quantosEmJogo] = new Jogador(nomeJog);`, começa por invocar o construtor da classe `Jogador` de modo a criar o jogador de nome `"Maria"`; a figura seguinte mostra o estado da memória quando esse construtor foi executado; repare no valor de `this` no contexto do construtor – é o novo objeto do tipo `Jogador`, acabado de criar e já inicializado:

Frames

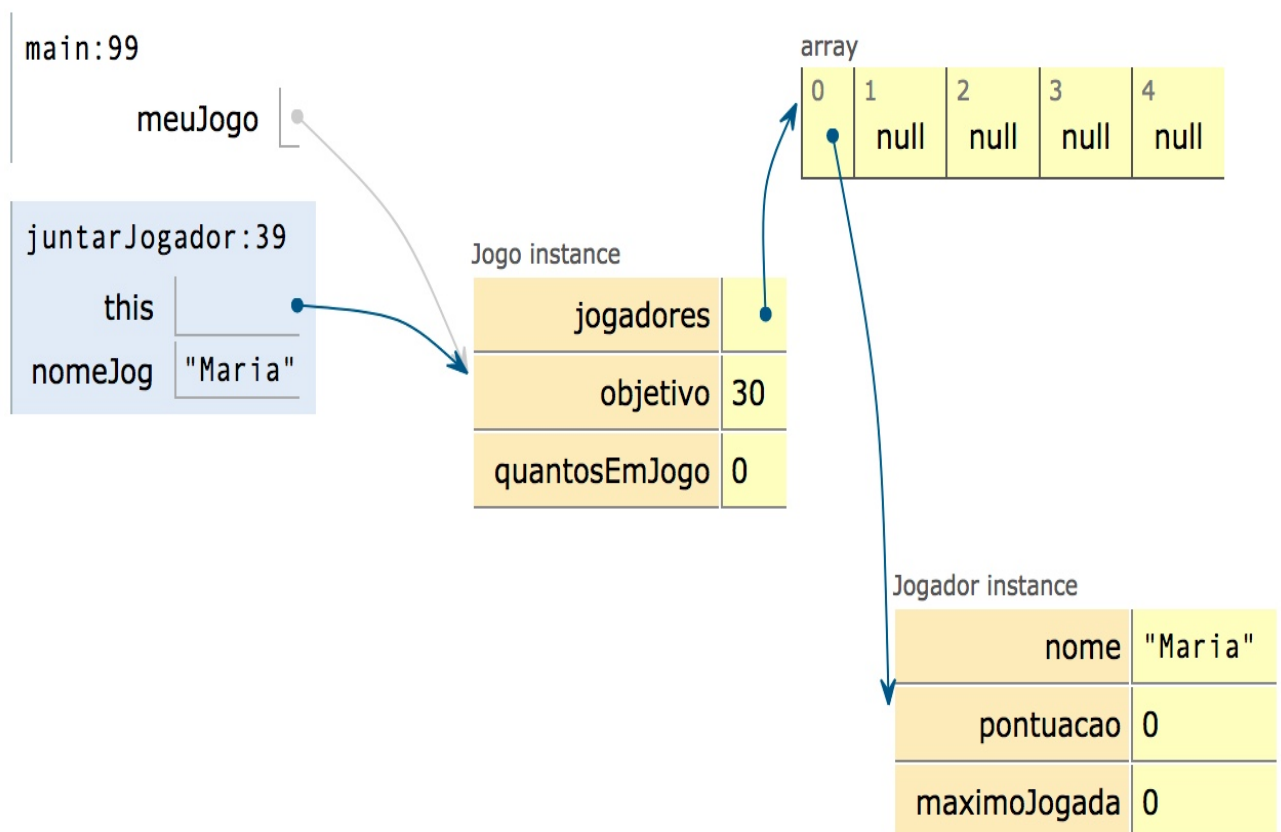
Objects



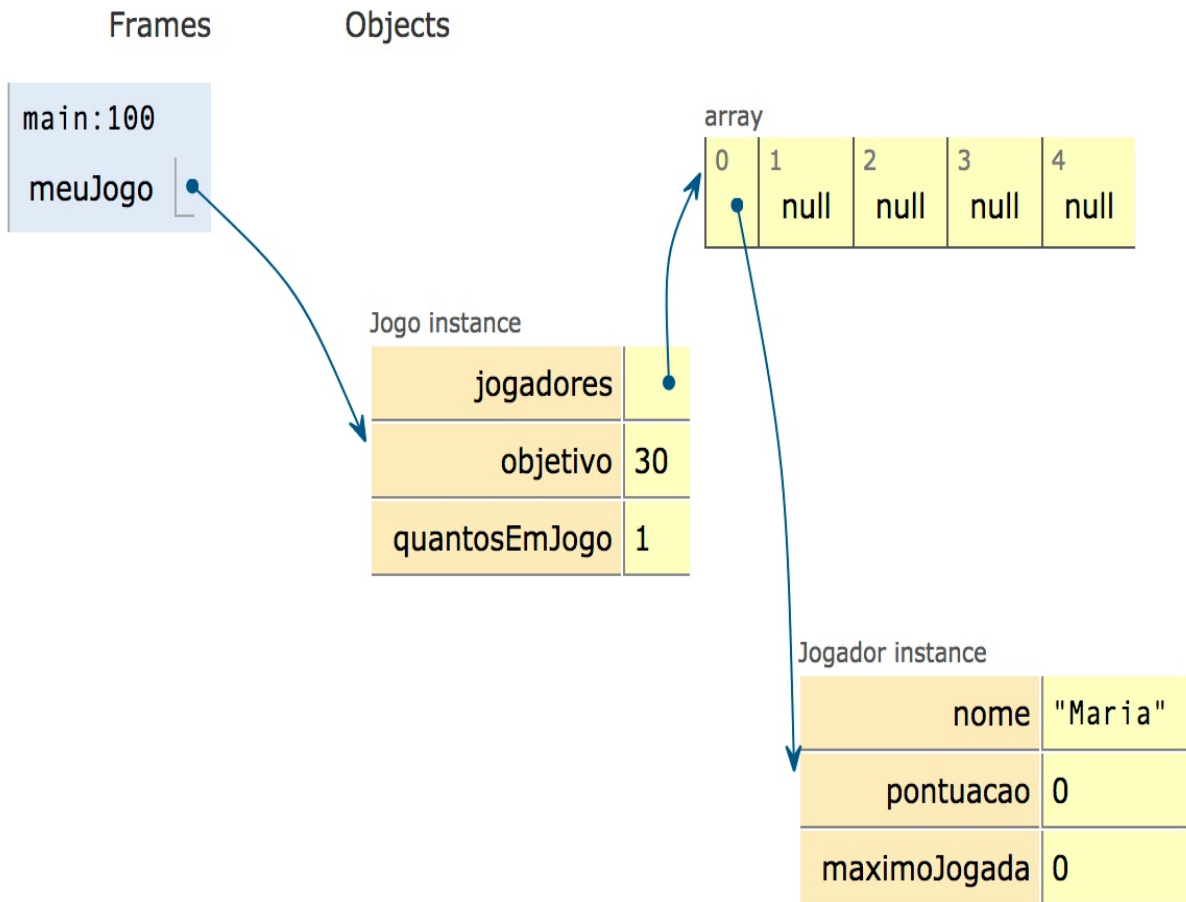
- de seguida, ainda através da instrução `this.jogadores[this.quantosEmJogo] = new Jogador (nomeJog);`, o método `juntarJogador` atribui a referência para o novo jogador à primeira posição do `array jogadores`, pois o valor que o atributo `quantosEmJogo` tem no momento é zero:

Frames

Objects



- finalmente, quando o método `juntarJogador` termina, o controle de execução volta ao `main` e o estado da memória é:



Vamos então relembrar o conteúdo da classe `ClienteDeJogo` e acrescentar ainda uma instrução que registre um segundo jogador no jogo:

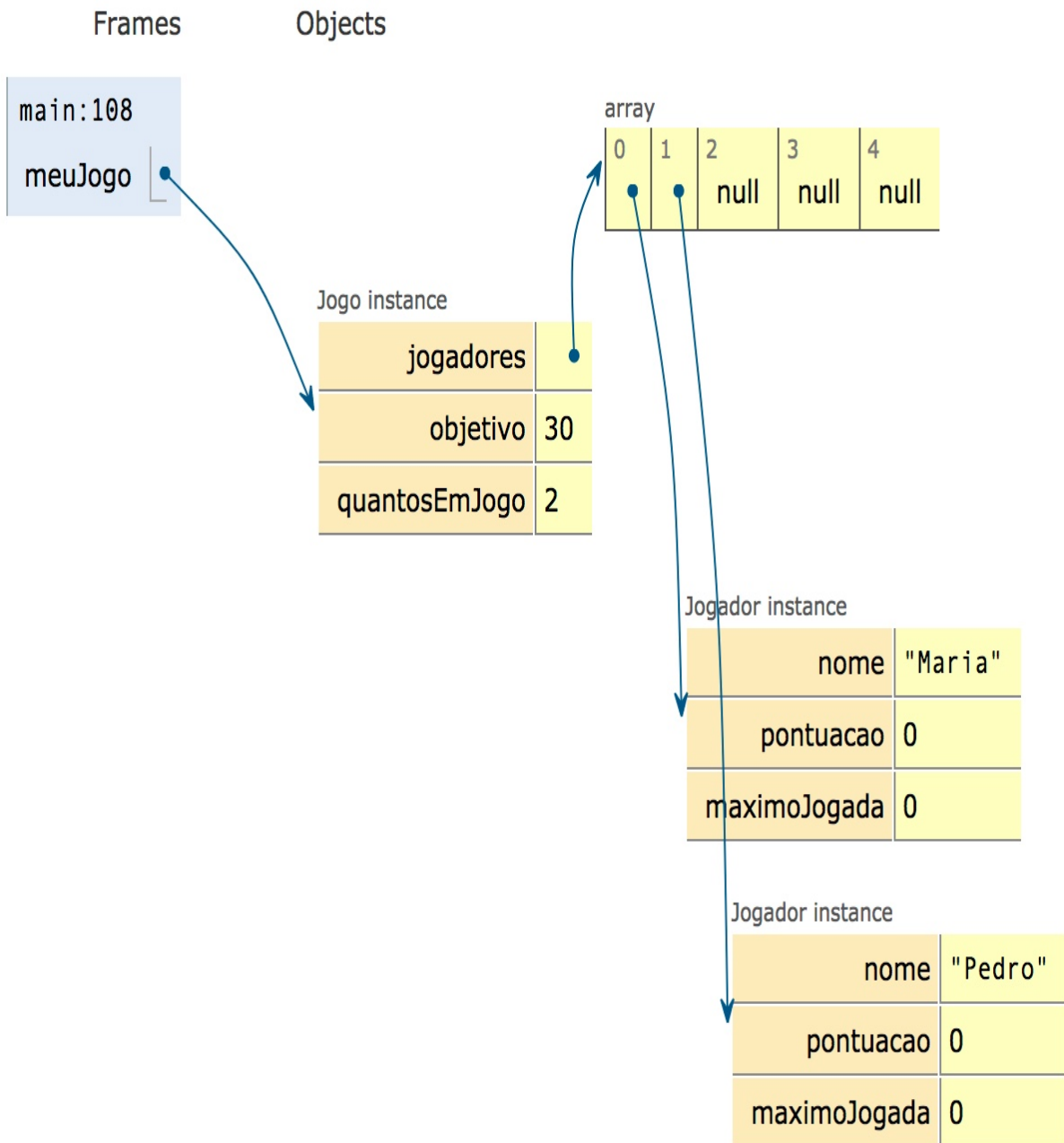
```
public class ClienteDeJogo {

    public static void main (String [] args) {

        Jogo meuJogo = new Jogo(30);
        System.out.println("O objetivo eh " + meuJogo.pontuacaoObjetivo());

        meuJogo.juntarJogador("Maria");
        meuJogo.juntarJogador("Pedro");
    }
}
```

Após a execução da última instrução do `main`, o estado da memória ficará:



11.3. Registrar pontuação de um dado jogador do jogo – 1ª versão

Voltemos agora à nossa classe `Jogo`.

Para registrar pontuações de jogadas dos jogadores do jogo, adicionamos o seguinte método de instância à classe `Jogo`:

```
/**
 * Registrar a pontuacao na jogada atual de um dado jogador.
 * @param nomeJog O nome do jogador.
 * @param pontuacao A pontuacao obtida na jogada atual.
 * @requires O jogador com o nome nomeJog estah registrado neste jogo &&
 *           pontuacao >= 0 && pontuacao <= 10
 */
public void registrarPontosJogada (String nomeJog, int pontuacao) {
```

```

        Jogador atual = null;
        for (int i = 0 ; atual == null && i < this.quantosEmJogo ; i++){
            if (nomeJog.equals(this.jogadores[i].nome())){
                atual = this.jogadores[i];
            }
        }
        atual.registarPontos(pontuacao);
    }
}

```

Este método exige que o primeiro parâmetro represente um nome de um jogador que já está registado no jogo. Primeiro encontra o jogador com o nome dado e de seguida invoca, sobre esse jogador, o método que permite adicionar a pontuação indicada.

Adicionemos agora a instrução seguinte ao método `main` da classe `ClienteDeJogo`,

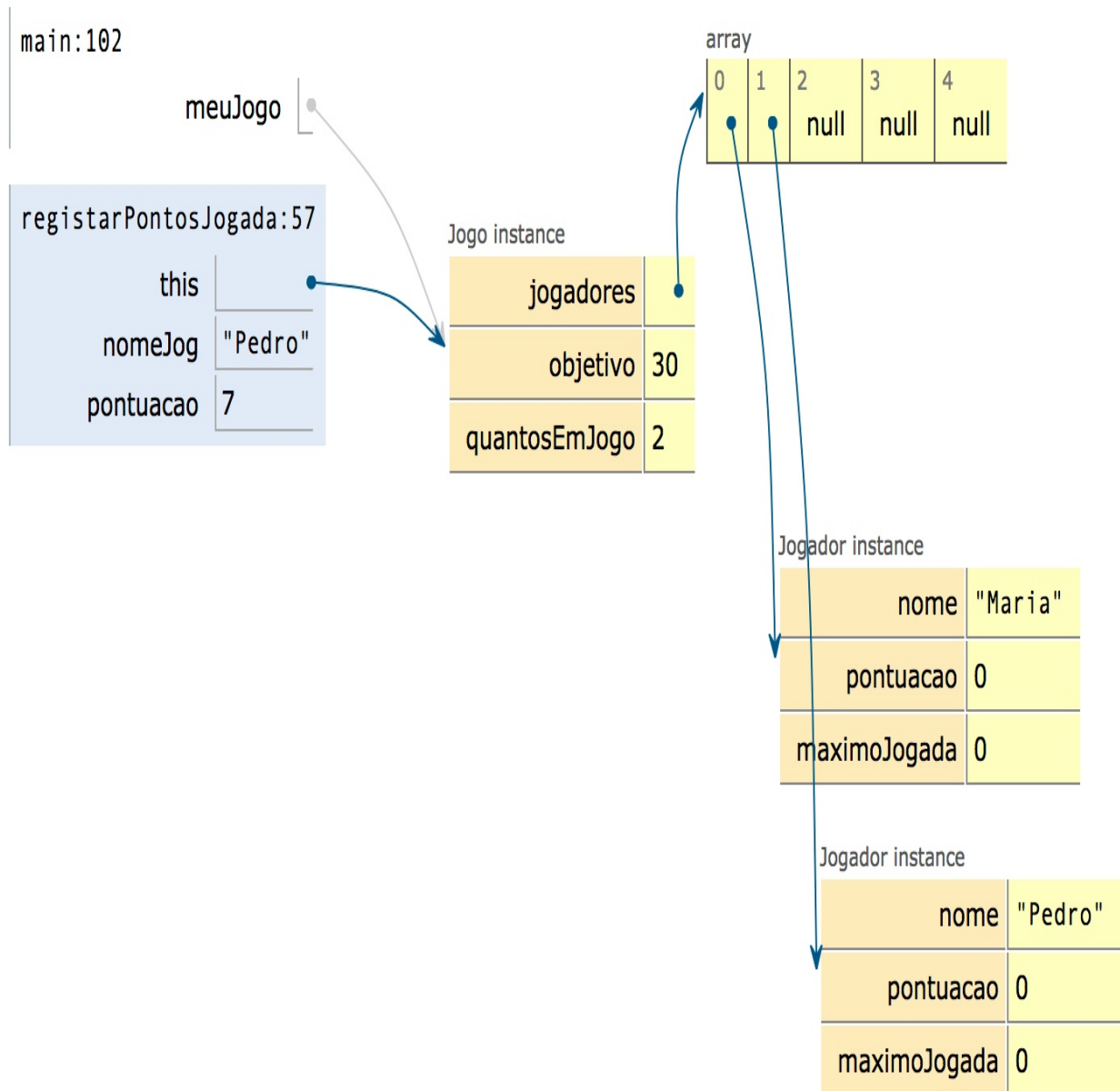
```

meuJogo.registarPontosJogada("Pedro",7);

```

As figuras seguintes ilustram o estado da memória em várias fases da execução desta instrução:

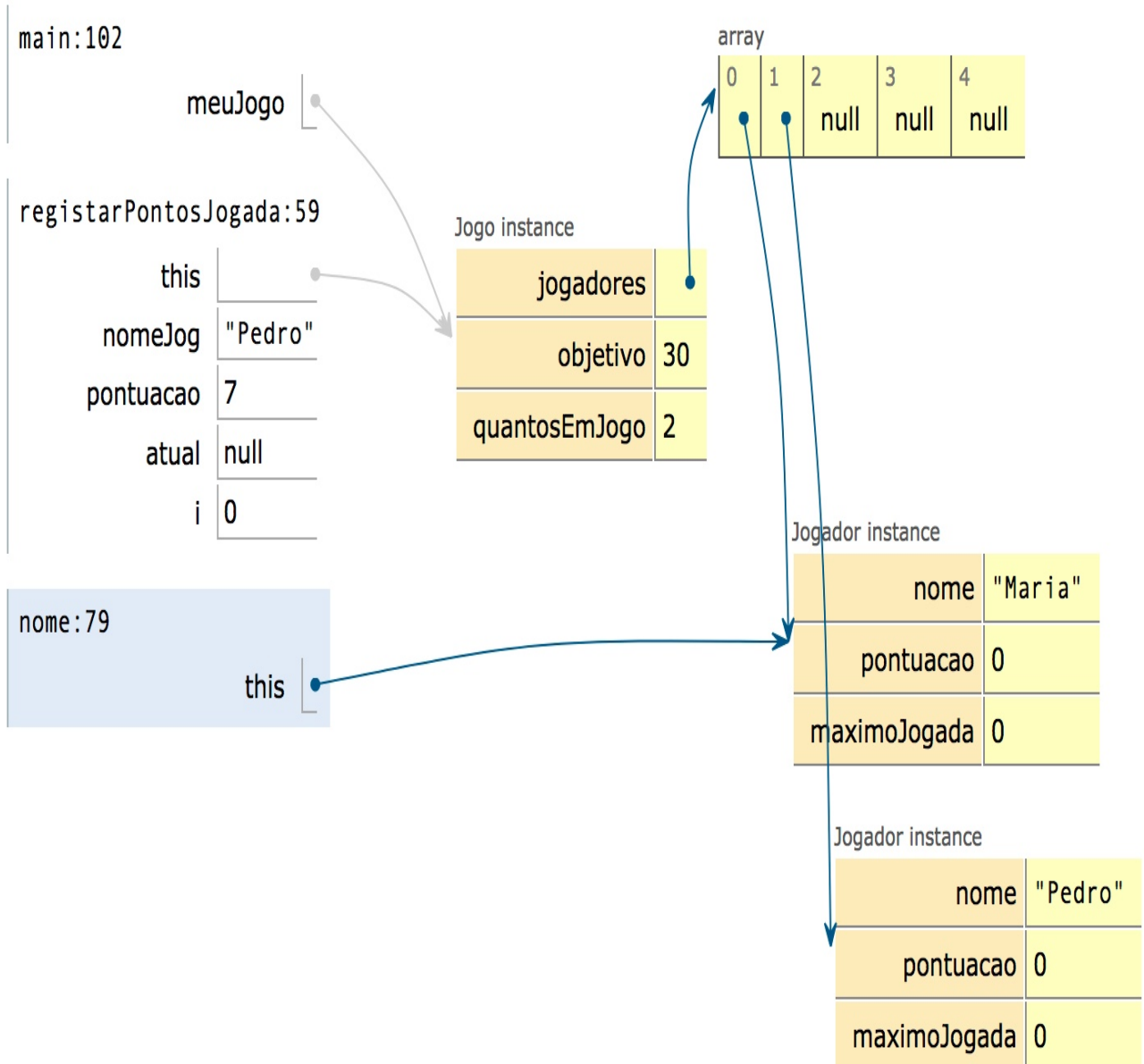
- quando o método `regarPuntosJogada` é invocado (repare o valor de `this` no contexto deste método – é o mesmo conteúdo da variável `meuJogo`, como seria de esperar):



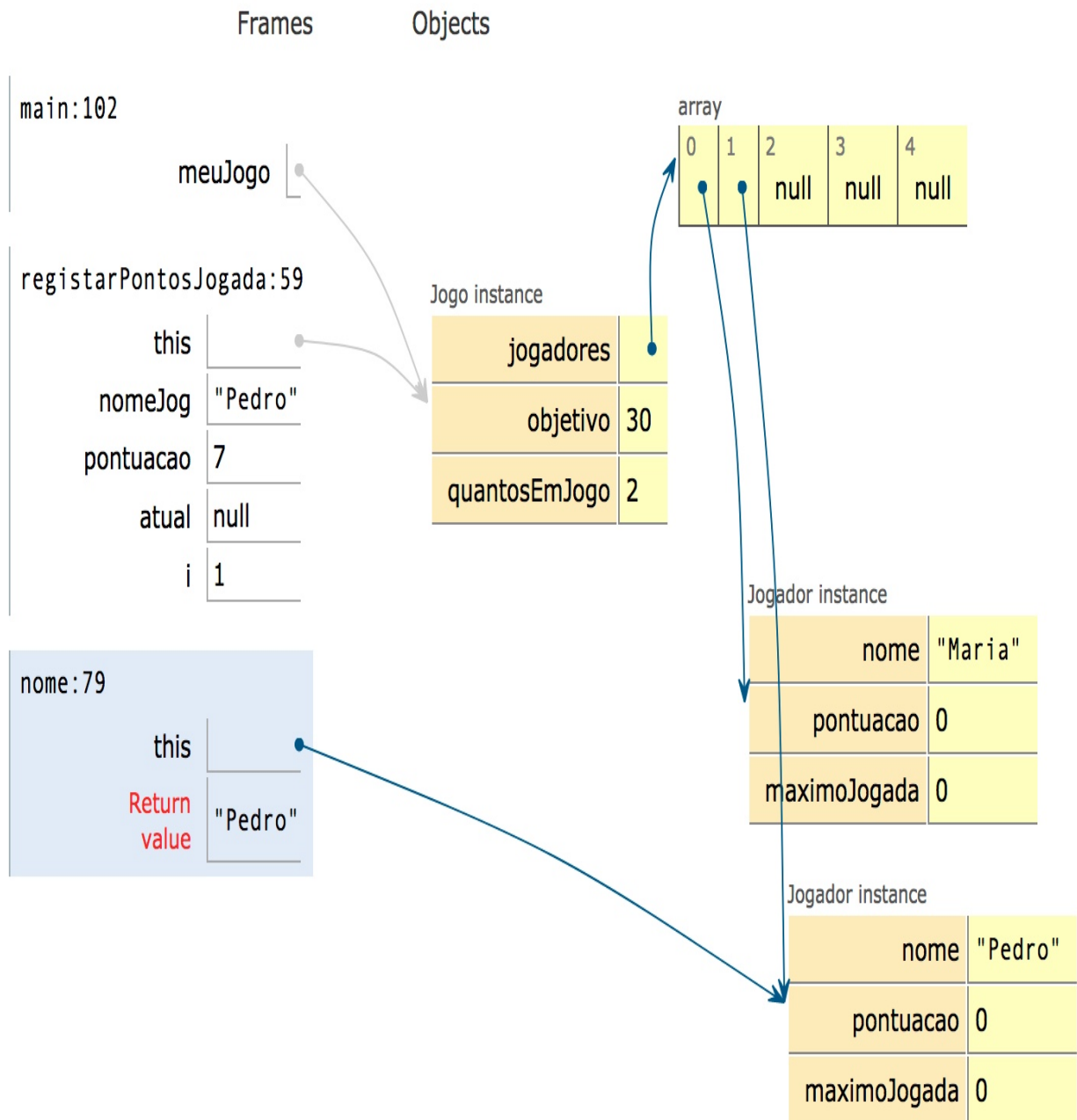
- após inicializar as variáveis locais `atual` e `i`, o método `registrarPontosJogada` invoca, através da instrução `if (nomeJog.equals(this.jogadores[i].nome()))`, o método `nome` sobre o objeto referenciado por `this.jogadores[0]`, que é o jogador de nome "Maria"; a figura seguinte mostra o estado da memória quando é feita essa invocação; repare no valor de `this` no contexto do método `nome` – é precisamente o objeto referenciado pelo primeiro elemento do array `jogadores`:

Frames

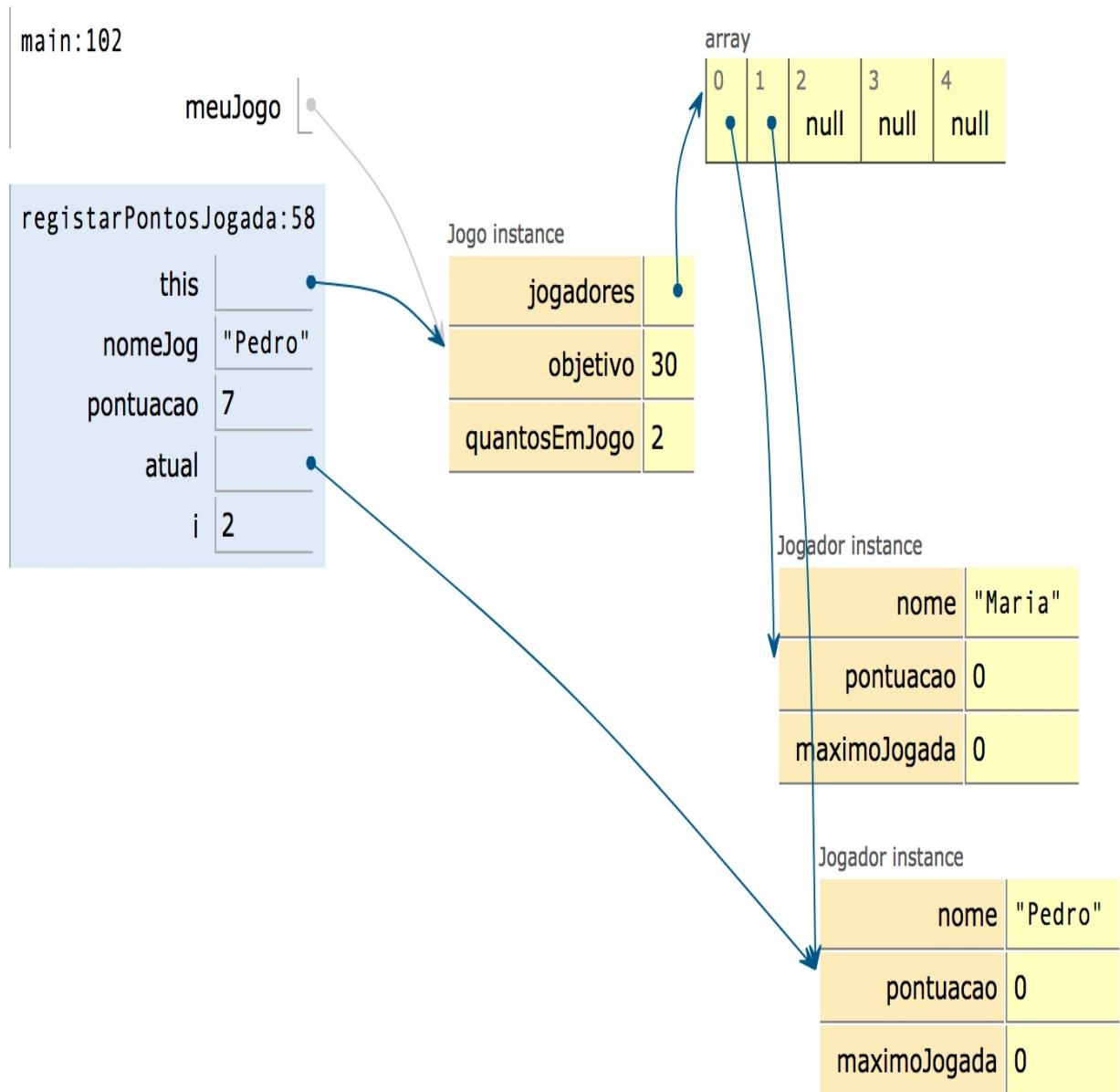
Objects



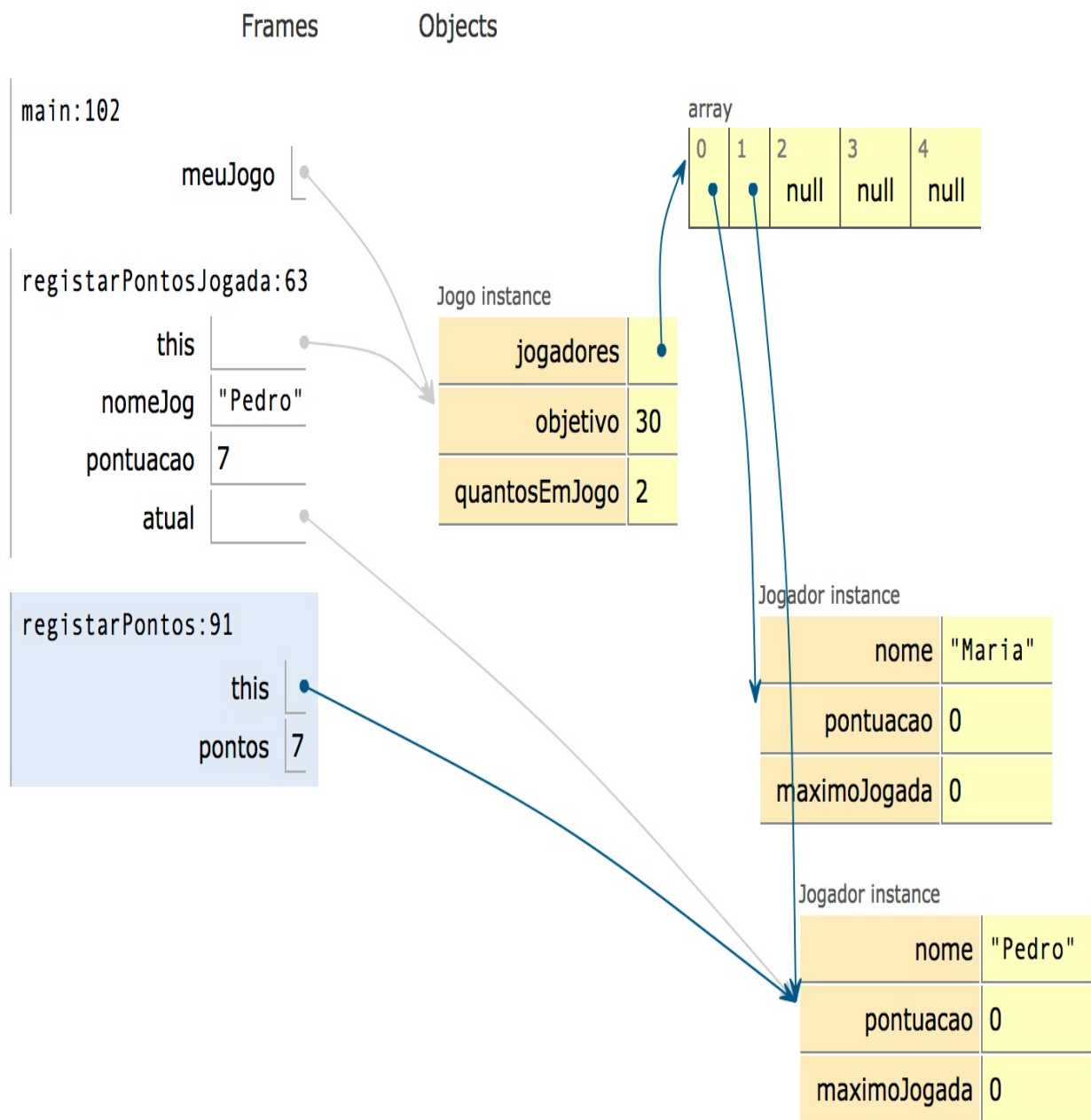
- como o nome do primeiro elemento do `array` `jogadores` não é igual ao valor do parâmetro `nomeJog` (que é "Pedro"), a variável `i` é incrementada e o método `nome` é agora invocado sobre o segundo elemento do `array` `jogadores`; a figura seguinte mostra o estado da memória quando é feita a invocação do método `nome` sobre esse segundo elemento do `array`:



- o jogador que se procurava foi encontrado e a sua referência foi guardada na variável `atual`; a guarda do ciclo passa a ser falsa:



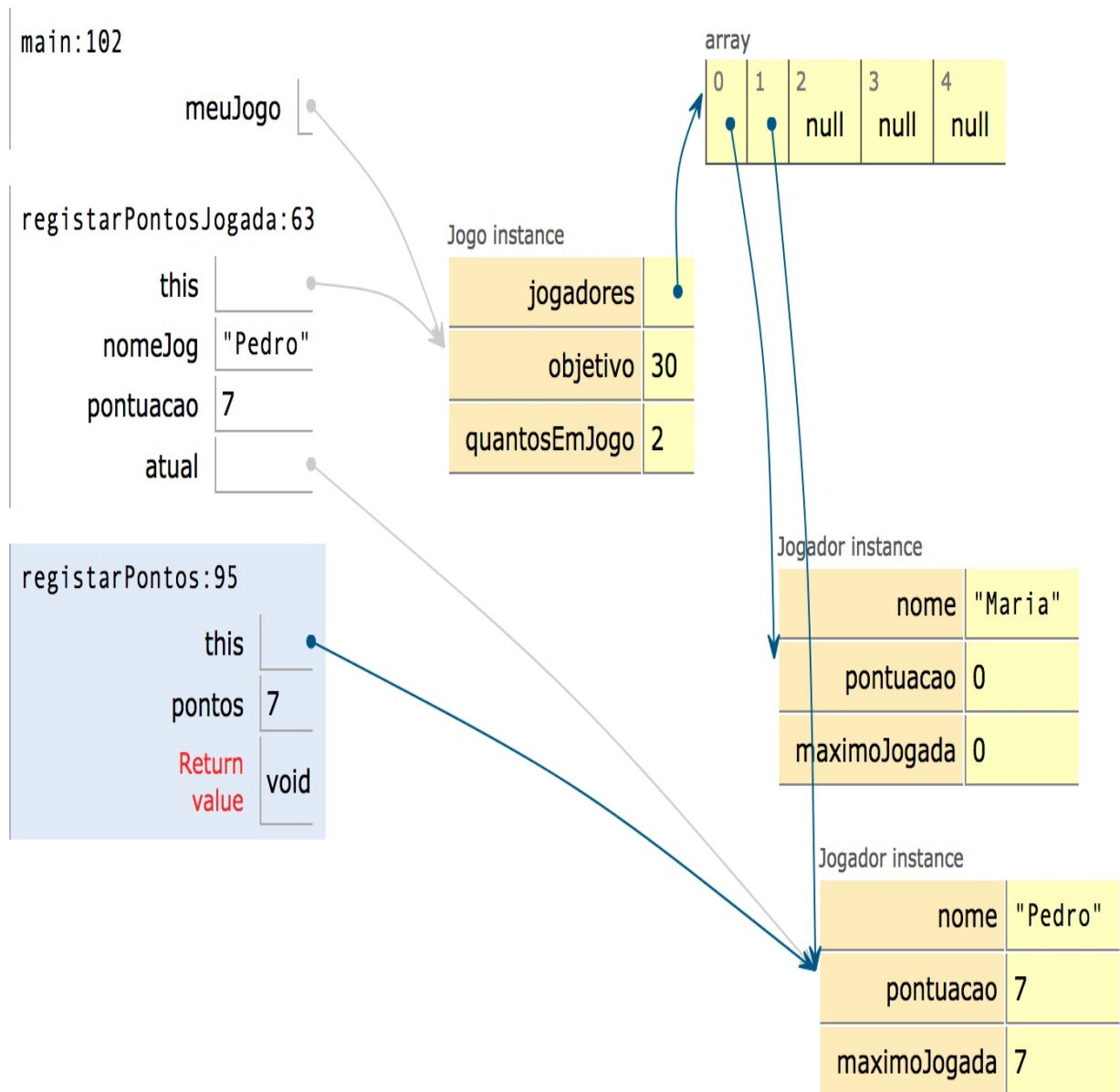
- de seguida, a última instrução do método `registrarPontosJogada` invoca o método `registrarPontos` sobre o objeto referenciado pela variável `atual`; a figura seguinte mostra o estado da memória na altura da invocação do método `registrarPontos`:



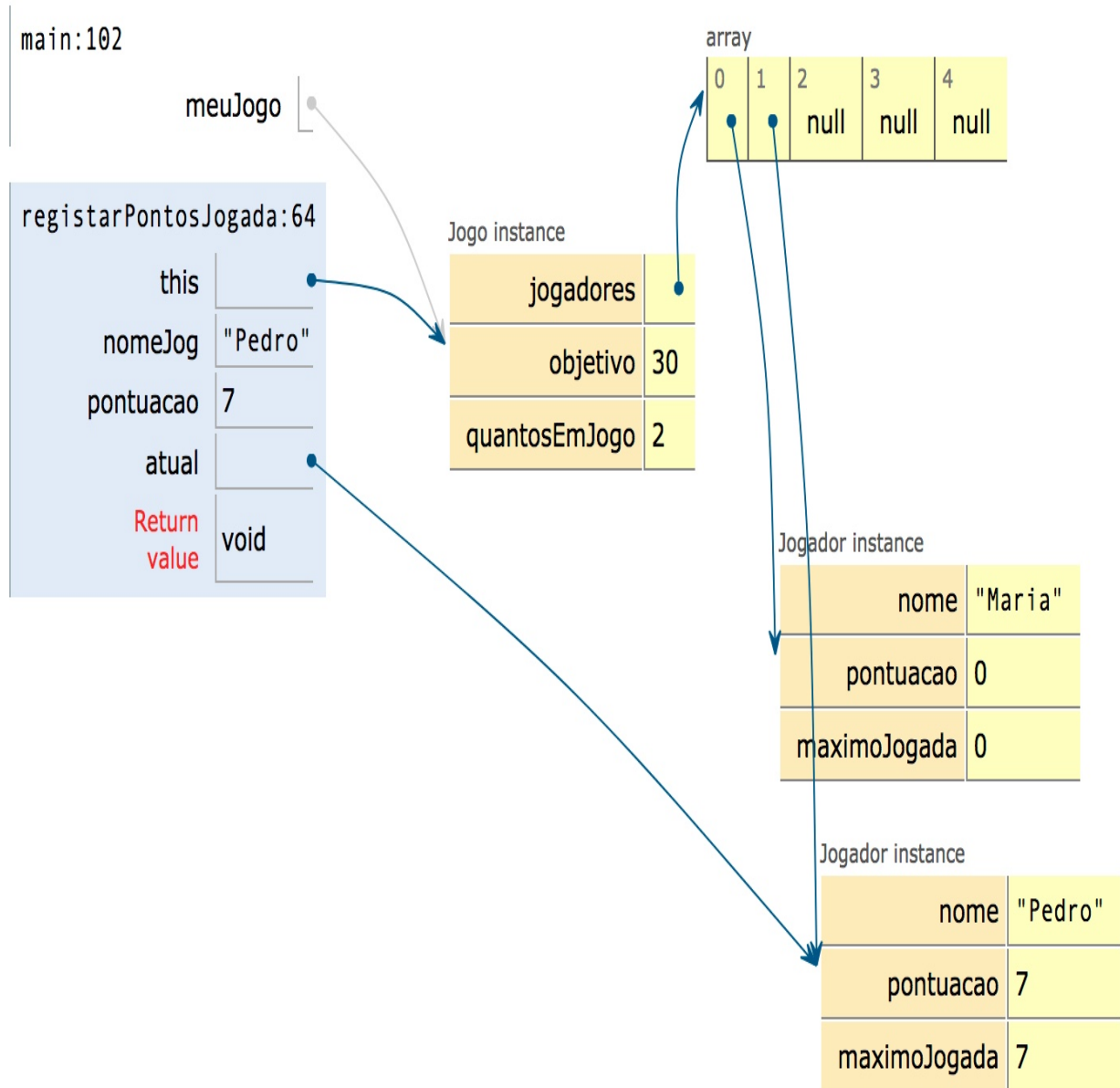
- a figura seguinte mostra o estado da memória já no fim da execução do método `registrarPontos`:

Frames

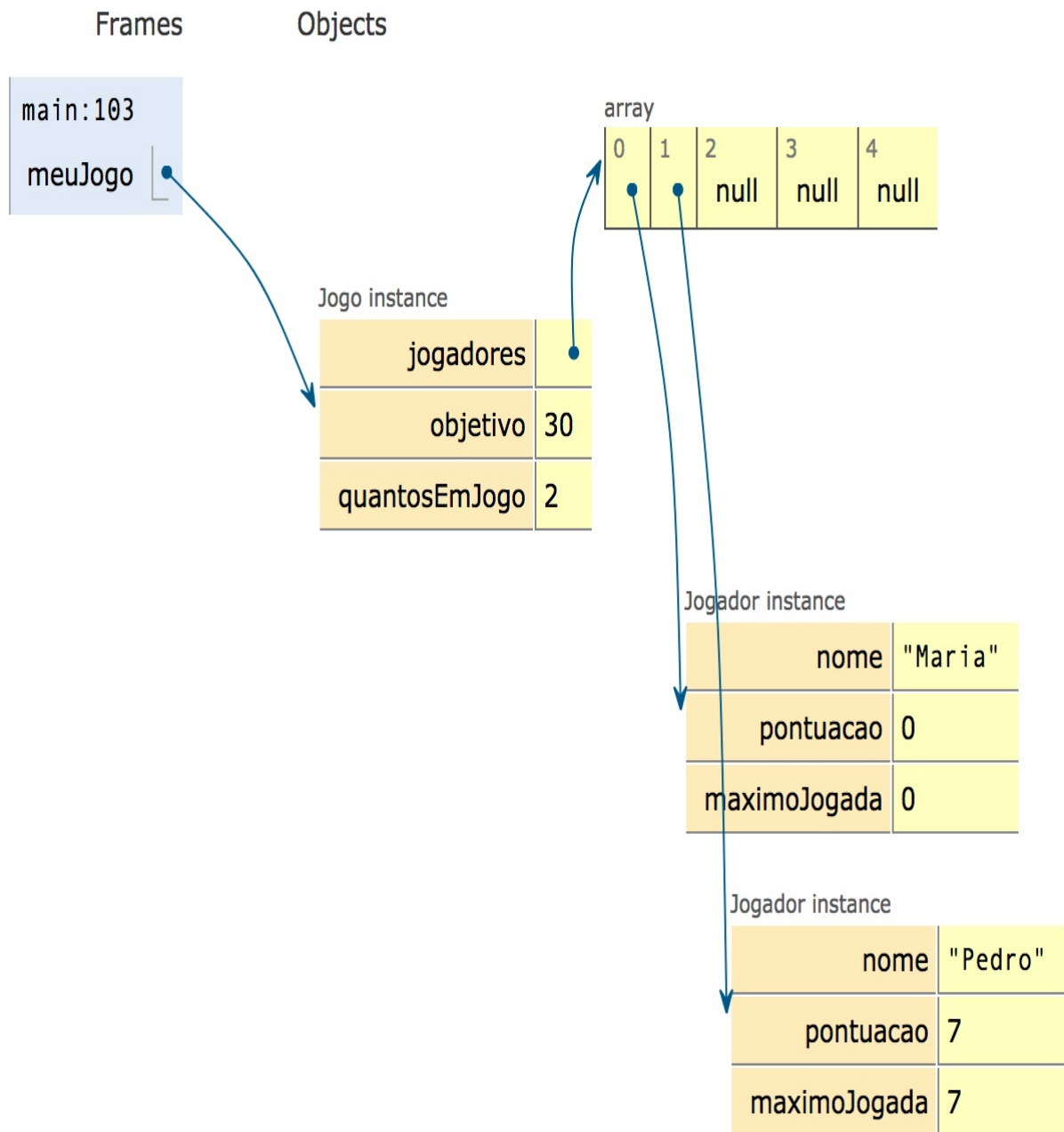
Objects



- depois do método `registrarPontos` terminar, o controle de execução volta ao método `registrarPontosJogada` e o jogador de nome "Pedro" já tem os seus atributos `pontuacao` e `maximoJogada` atualizados:



- finalmente, já depois do método `registrarPontosJogada` terminar, o objeto referenciado pela variável `meuJogo` já está atualizado pois o seu jogador de nome "Pedro" já tem os seus atributos `pontuacao` e `maximoJogada` atualizados:



Na próxima secção vamos ver uma versão melhorada deste método.

11.4. Outros métodos relativos a um dado jogador do jogo

Queremos agora construir um método para saber a pontuação de um jogador com um dado nome. O código seguinte serve:

```
/**
 * A pontuacao de um dado jogador deste jogo
 * @param nomeJog O nome do jogador.
 * @return A pontuacao do jogador com este nome
 * @requires O jogador com o nome nomeJog estah registado neste jogo
 */
public int pontuacao (String nomeJog) {
```

```

        Jogador atual = null;
        for (int i = 0 ; atual == null && i < this.quantosEmJogo ; i++){
            if (nomeJog.equals(this.jogadores[i].nome())){
                atual = this.jogadores[i];
            }
        }
        return atual.pontuacao();
    }
}

```

Repare como este método e o `registrarPontosJogada` da secção anterior são quase iguais:

- *encontrar o jogador;*
- *atuar sobre ele* (no primeiro caso registrar nova pontuação, no segundo caso perguntar-lhe qual a sua pontuação).

Outros métodos de instância de que precisaremos nesta classe seguem este padrão: por exemplo, métodos para saber a pontuação máxima numa só jogada de um jogador com dado nome, para saber se um jogador com dado nome está registado no jogo, etc.

Então, vamos definir um método privado (pois é somente para ser invocado a partir dos métodos da própria classe) que encontre e devolva um dado jogador, e vamos usá-lo a partir destes métodos.

```

/**
 * Retorna o jogador correspondente a um nome, ou null se nao houver.
 * Metodo privado, auxiliar.
 * @param nomeJog O nome do jogador.
 * @returns O jogador com este nome se existir no jogo;
 *         null caso contrario
 */
private Jogador obterJogador (String nomeJog) {
    Jogador result = null;
    for (int i = 0 ; result == null && i < this.quantosEmJogo ; i++){
        if (nomeJog.equals(this.jogadores[i].nome())){
            result = this.jogadores[i];
        }
    }
    return result;
}

```

Vamos então modificar os dois métodos anteriores para invocarem este último e acrescentar outros ainda, que também implementam o mesmo padrão.

```

/**
 * Registrar a pontuacao na jogada atual de um dado jogador.
 * @param nomeJog O nome do jogador.
 * @param pontuacao A pontuacao obtida na jogada atual.
 * @requires this.estahEmJogo (nomeJog) &&
 *         pontuacao >= 0 && pontuacao <= 10
 */
public void registrarPontosJogada (String nomeJog, int pontuacao) {
    this.obterJogador(nomeJog).registrarPontos(pontuacao);
}

/**
 * A pontuacao de um dado jogador deste jogo
 * @param nomeJog O nome do jogador.

```

```

    * @return A pontuacao do jogador com este nome
    * @requires this.estahEmJogo (nomeJog)
    */
    public int pontuacao (String nomeJog) {
        return this.obterJogador(nomeJog).pontuacao();
    }

    /**
    * Um dado jogador estah em jogo?
    * @param nomeJog O nome do jogador.
    * @return true se o jogador com este nome estah registado neste jogo
    *         false caso contrario
    */
    public boolean estahEmJogo (String nomeJog) {
        return this.obterJogador(nomeJog) != null;
    }

    /**
    * A pontuacao maxima numa soh jogada de um dado jogador deste jogo
    * @param nomeJog O nome do jogador.
    * @return A pontuacao maxima numa soh jogada do jogador com este nome
    * @requires this.estahEmJogo (nomeJog)
    */
    public int maximoNumaJogada (String nomeJog) {
        return this.obterJogador(nomeJog).maximoNumaJogada();
    }
}

```

Repare que também acrescentámos o método `estahEmJogo` que, dado o nome de um jogador, verifica se ele está registado no jogo, retornando `true` se está inscrito e `false` caso contrário. Este método permite que as classes cliente verifiquem se um jogador está registado antes de invocar um dos outros métodos, em situações em que não há a certeza de que o dado jogador está registado no jogo.

Repare que substituímos o texto `O jogador com o nome nomeJog estah registado neste jogo` que tínhamos nas pré-condições dos métodos acima por `this.estahEmJogo (nomeJog)`.

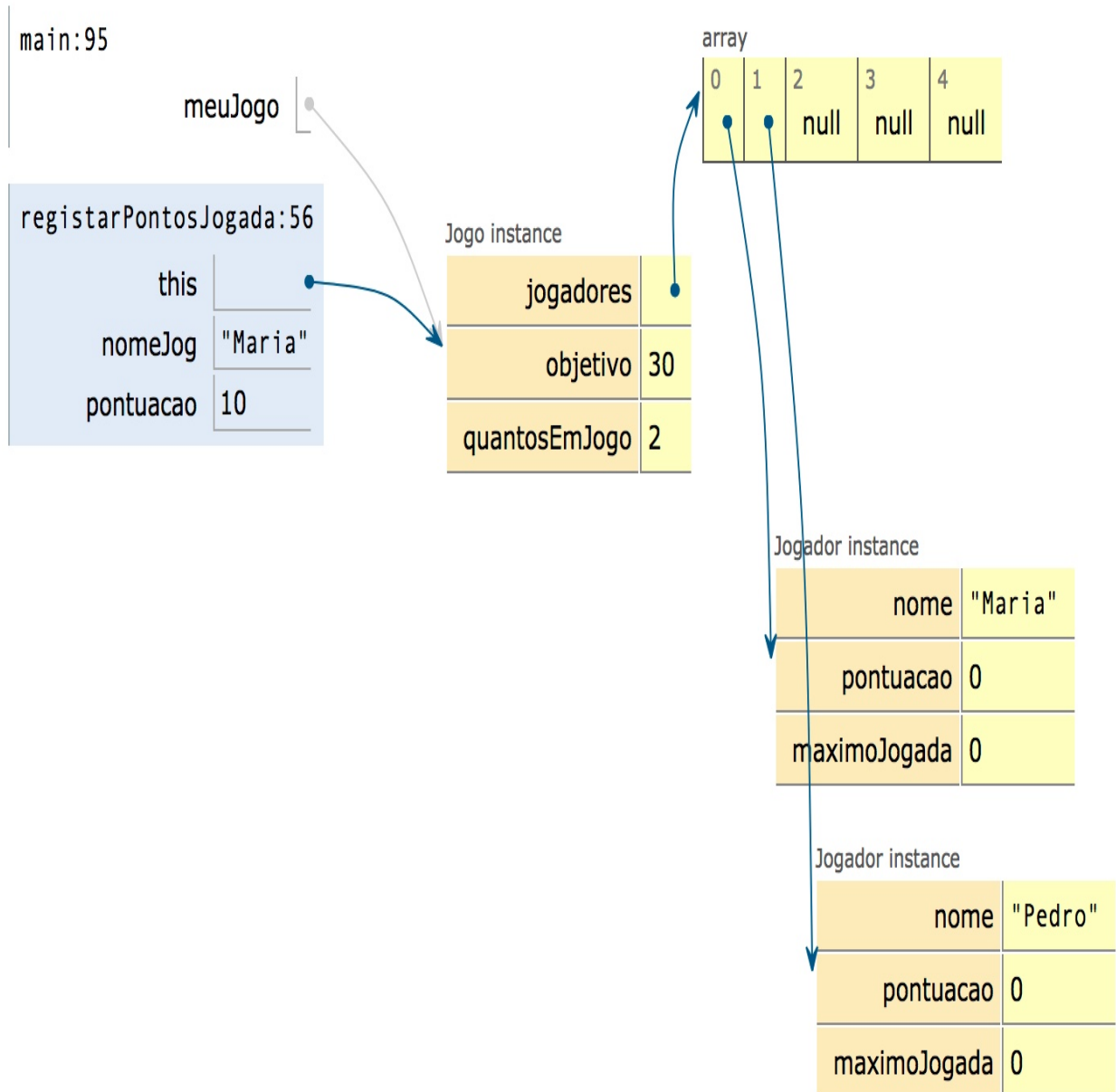
Acrescentámos também o método `maximoNumaJogada` que retorna a pontuacao máxima numa só jogada do jogador com um dado nome.

Adicionemos agora a instrução seguinte ao método `main` da classe `ClienteDeJogo`,

```
meuJogo.registarPontosJogada("Maria",10);
```

As figuras seguintes ilustram o estado da memória em várias fases da execução desta instrução:

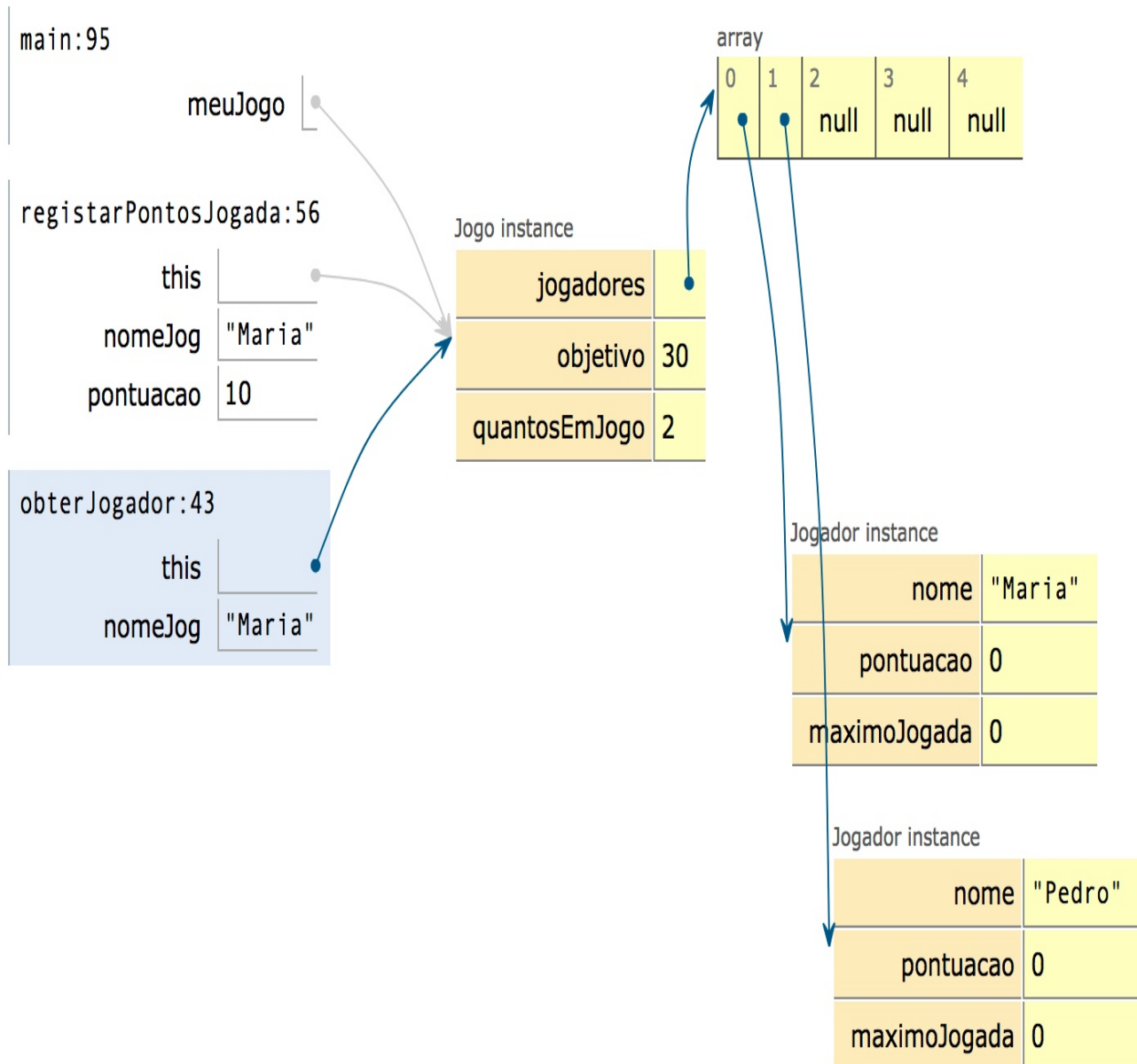
- quando o método `registarPontosJogada` é invocado (repare o valor de `this` no contexto deste método – é o mesmo conteúdo da variável `meuJogo`):



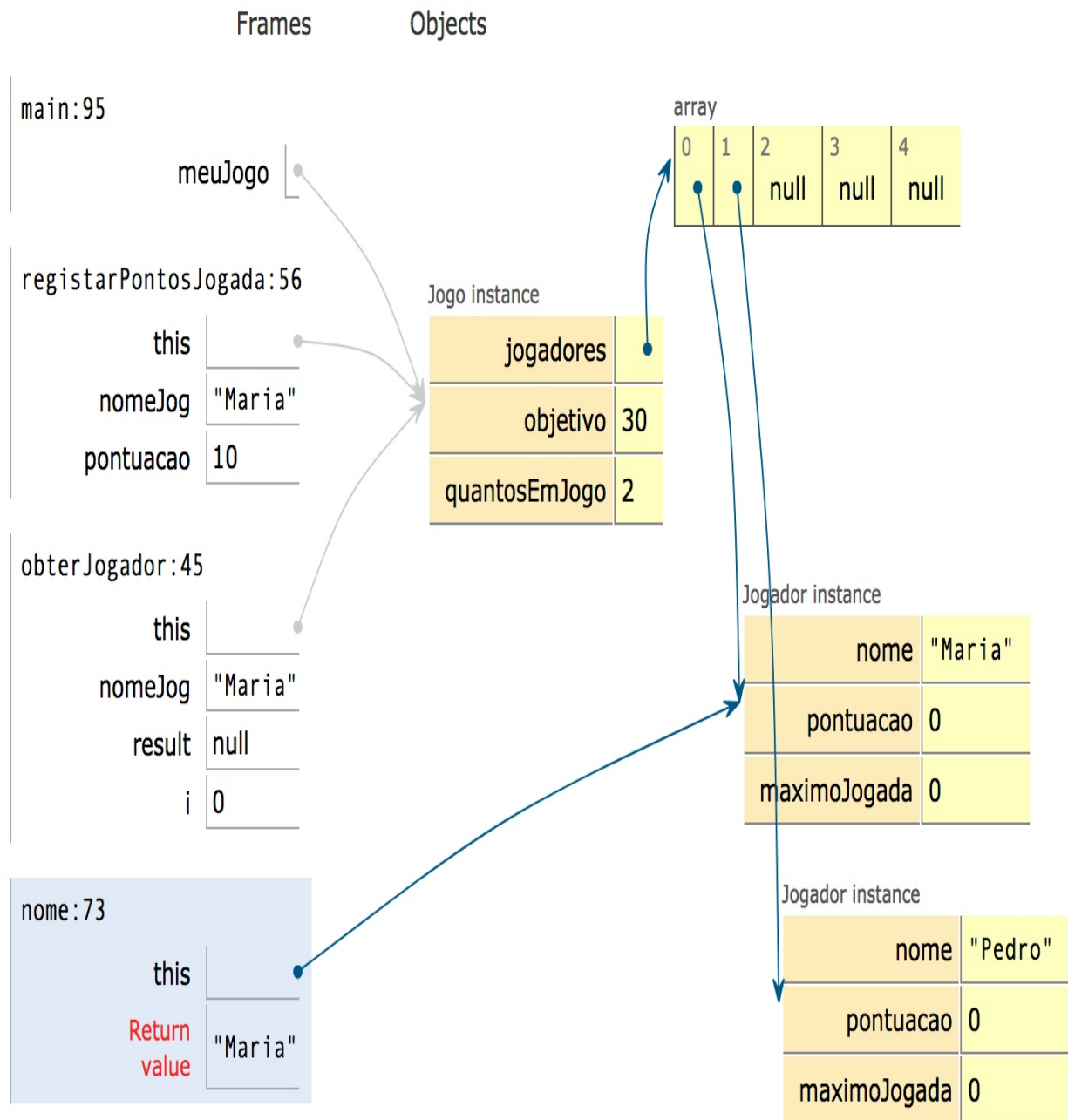
- o método `registrarPontosJogada` invoca, por sua vez, o método `obterJogador`, sobre o objeto referenciado por `this`; a figura seguinte mostra o estado da memória quando é feita essa invocação; repare no valor de `this` no contexto deste método – é o mesmo conteúdo do `this` do método `registrarPontosJogada`, como seria de esperar, ou seja, o objeto corrente continua a ser o mesmo:

Frames

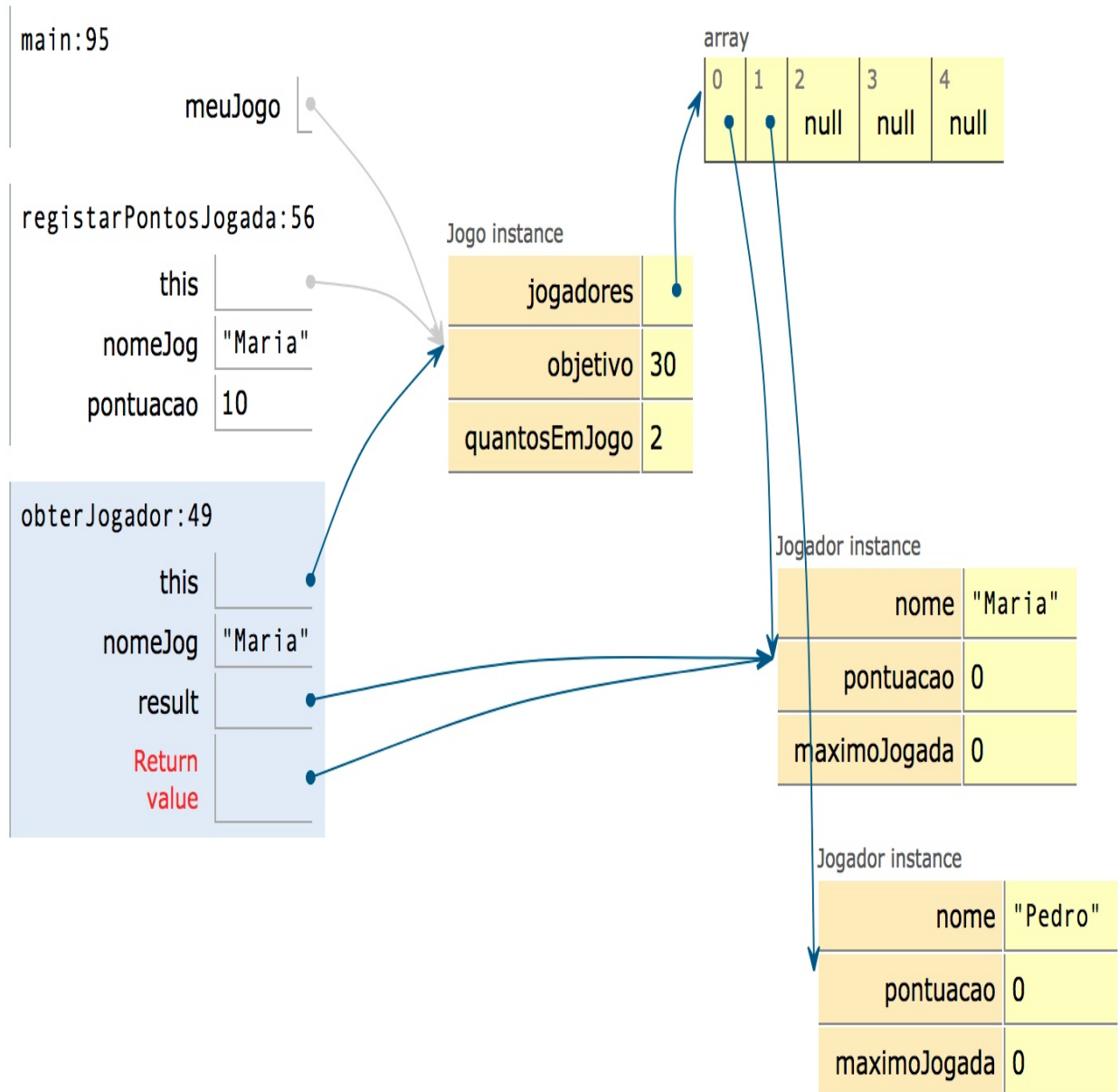
Objects



- o método `obterJogador` invoca, por sua vez, o método `nome` sobre cada um dos objetos do tipo `Jogador` que estão contidos no array `jogadores` do `this`; a figura seguinte mostra o estado da memória quando é feita a invocação do método `nome` sobre o primeiro dos elementos do array, o de índice zero, cujo nome é "Maria"; repare no valor de `this` no contexto deste método – é o objeto que representa o jogador de nome "Maria":



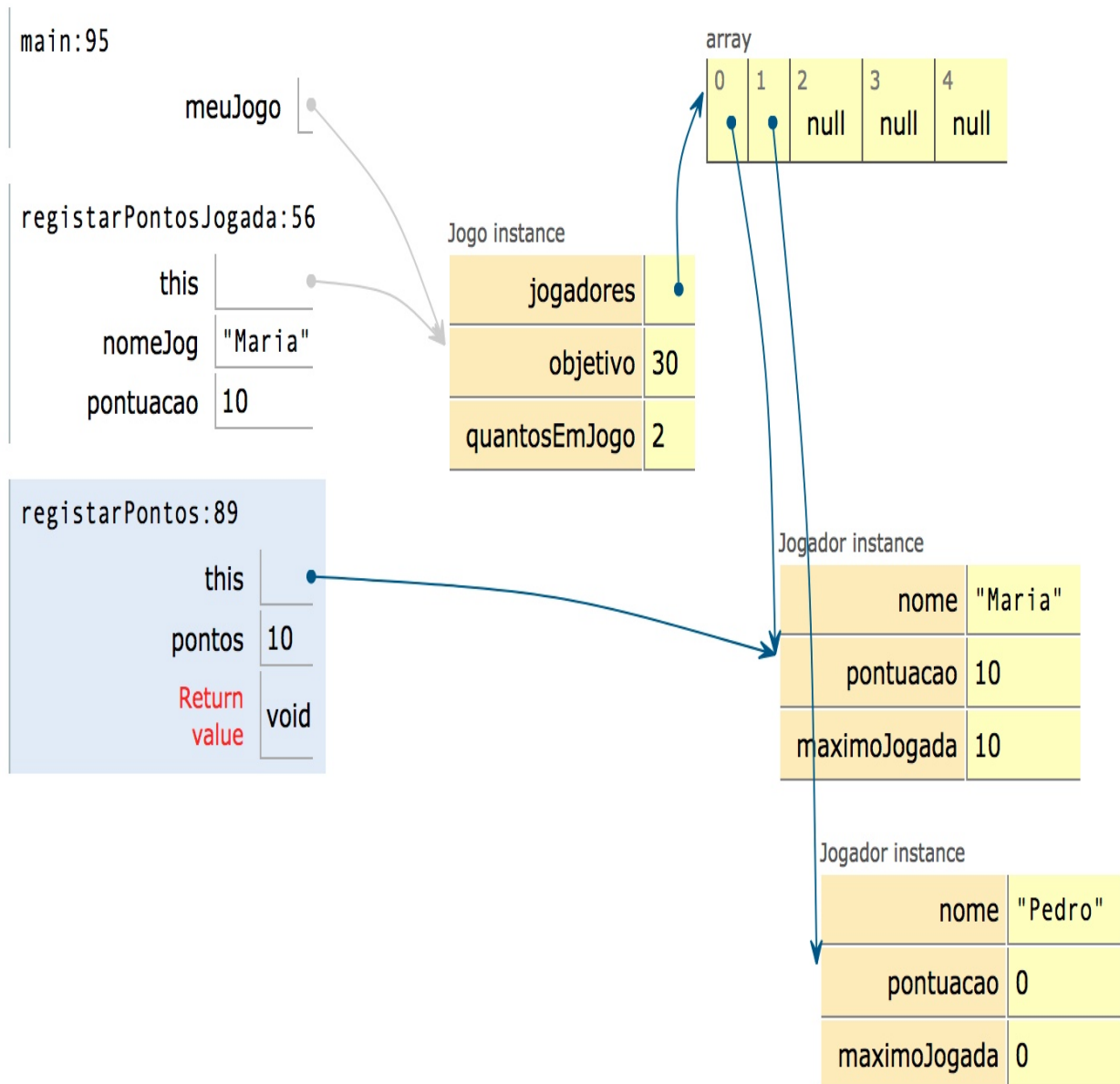
- o método `obterJogador` encontrou o jogador que procurava logo no primeiro elemento do array `jogadores`; logo, o resultado do método é uma referência a esse objeto:



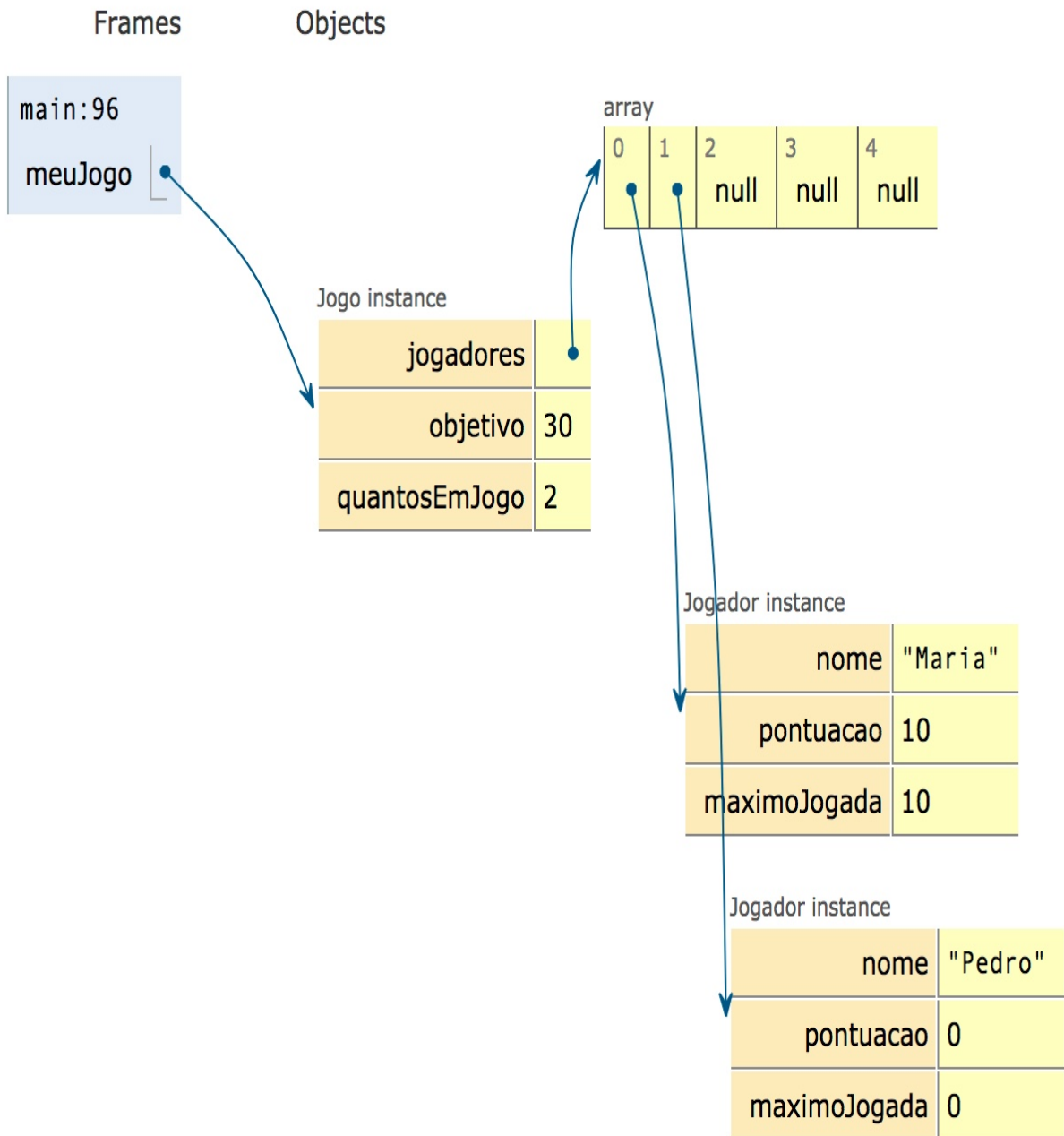
- de seguida, o método `registrarPontosJogada` invoca o método `registrarPontos` sobre o objeto que recebeu como resultado da invocação `this.obterJogador("Maria")`; a figura seguinte mostra o estado da memória já no fim da execução do método `registrarPontos`:

Frames

Objects



- finalmente, já depois do método `registrarPontosJogada` terminar, o objeto referenciado pela variável `meuJogo` já está atualizado pois o seu jogador de nome "Maria" já tem os seus atributos `pontuacao` e `maximoJogada` atualizados:



Vamos agora registrar mais pontuações de jogadas para os dois jogadores; o nosso `main` da classe `ClienteDeJogo` passa então a ser o seguinte:

```
public class ClienteDeJogo {

    public static void main (String [] args) {

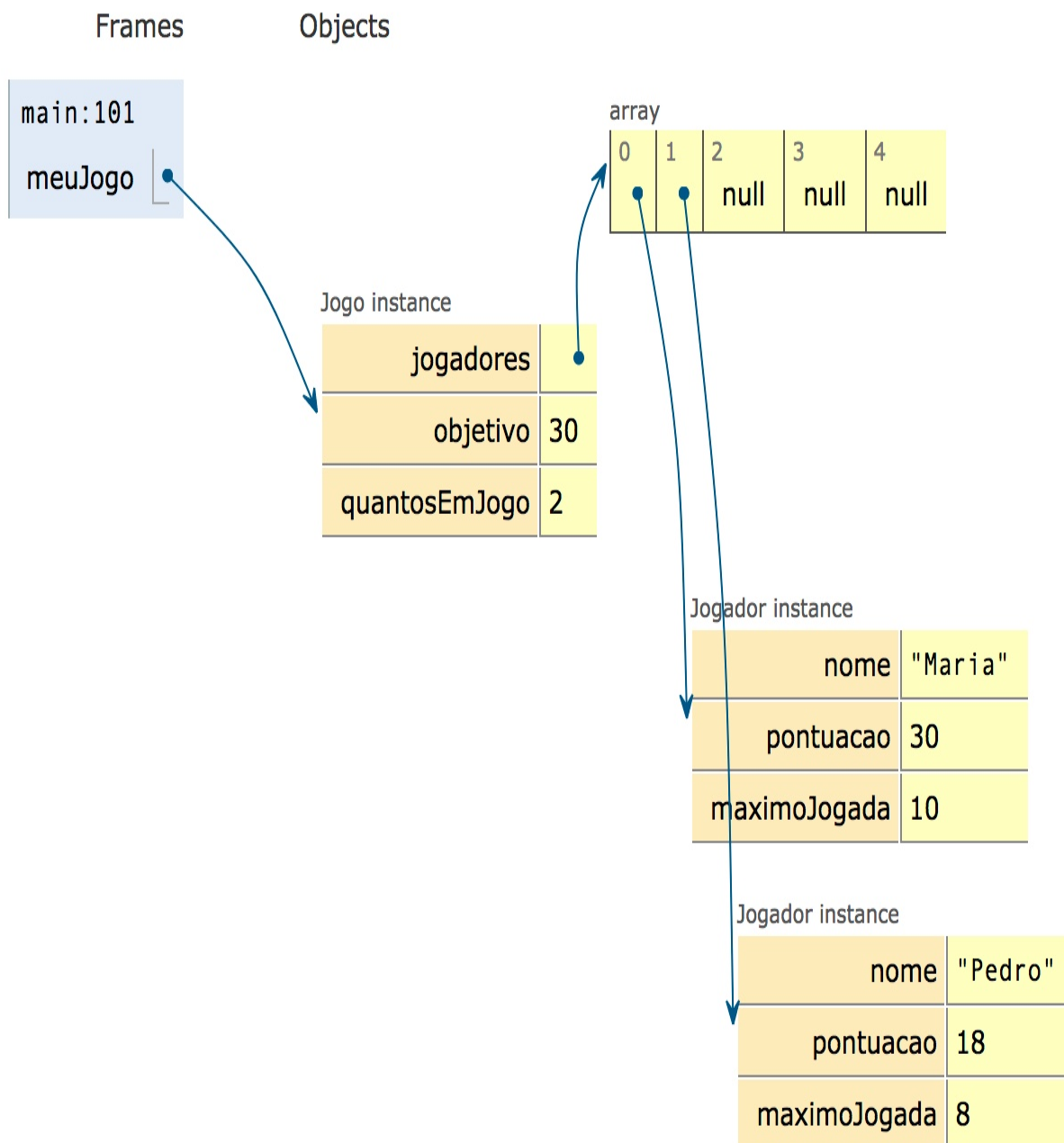
        Jogo meuJogo = new Jogo(30);
        System.out.println("O objetivo eh " + meuJogo.pontuacaoObjetivo());

        meuJogo.juntarJogador("Maria");
        meuJogo.juntarJogador("Pedro");

        meuJogo.registarPontosJogada("Maria",10);
        meuJogo.registarPontosJogada("Pedro",7);
        meuJogo.registarPontosJogada("Maria",10);
        meuJogo.registarPontosJogada("Pedro",3);
        meuJogo.registarPontosJogada("Maria",10);
        meuJogo.registarPontosJogada("Pedro",8);
    }
}
```

```
}  
}
```

Após a execução da última instrução do `main`, o estado da memória ficará:



11.5. Atributos de classe, constantes e `public`

Não queremos usar literais "espalhados" pelos métodos das nossas classes pois isso dificulta não só a compreensão (sobre o que representam) mas também uma futura alteração desses valores no texto dos programas que os usam.

No nosso caso, queremos evitar escrever diretamente os valores 5, 0 e 10 na classe `Jogo`, tanto no corpo dos métodos como na documentação. Estes são os valores do número máximo de jogadores e dos valores mínimo e máximo de pontuação por jogada, respectivamente.

11.5.1. Atributos de classe

Vamos criar mais três atributos e atribuir-lhes estes valores no construtor? Não propriamente. Se não, vejamos:

- Por nossa opção, todos os jogos usam o valor 5 como número máximo de jogadores;
- Por nossa opção, todos os jogos têm 0 e 10 como valores mínimo e máximo de pontuação por jogada;
- Então, não vale a pena guardar estes três valores como atributos de cada jogo! Estaríamos a repetir informação desnecessariamente.

Basta defini-los como *atributos de classe*

- existe um só valor para a classe, pois são iguais para todas as instâncias da classe;
- um atributo de classe define-se como `static`.

11.5.2. Constantes

Além de querermos defini-los como atributos de classe (`static`) também queremos que os seus valores se mantenham *constantes* e não possam ser alterados durante a vida de um objeto deste tipo.

Para indicar que são constantes usamos a palavra reservada `final` e atribuímos-lhes um valor inicial (que não mudará).

Por convenção, os nomes das constantes escrevem-se todos em maiúsculas e com `_` como separador de palavras.

11.5.3. Atributos `public`

Queremos ainda que as classes cliente de `Jogo` possam conhecer estes valores.

Declaramo-los então como `public` – como são constantes, não há problema em serem `public` pois nenhuma classe cliente conseguirá alterar o seu valor.

11.5.4. Exemplificando

Acrescentamos estes atributos de classe constantes à nossa classe e alteramos os pontos onde eram usados estes valores, tanto internamente como na documentação, para passarem a referir estas constantes:

```
public class Jogo {

    /**
     * O numero maximo de jogadores por jogo
     */
    public static final int MAX_JOGADORES = 5;

    /**
     * O minimo e maximo de pontos que um jogador pode ter
     * numa jogada
     */
    public static final int MIN_PONTUACAO_JOGADA = 0;
    public static final int MAX_PONTUACAO_JOGADA = 10;

    // atributos que definem este jogo:
    // A colecao dos jogadores
    private Jogador [] jogadores;
    // Pontuacao objetivo
    private int objetivo;
    // Numero de jogadores em jogo
```

```

private int quantosEmJogo;

/**
 * Construir um jogo.
 * @param objetivo A pontuacao com que se ganha este jogo
 * @requires objetivo > 0
 */
public Jogo (int objetivo) {
    this.objetivo = objetivo;
    this.jogadores = new Jogador [MAX_JOGADORES];
    this.quantosEmJogo = 0;
}

/**
 * A pontuacao objetivo deste jogo.
 */
public int pontuacaoObjetivo () {
    return this.objetivo;
}

/**
 * O numero de jogadores que estao em jogo.
 */
public int quantosEmJogo () {
    return this.quantosEmJogo;
}

/**
 * Um dado jogador estah em jogo?
 * @param nomeJog O nome do jogador.
 * @return true se o jogador com este nome estah inscrito neste jogo
 *         false cc
 */
public boolean estahEmJogo (String nomeJog) {
    return this.obterJogador(nomeJog) != null;
}

/**
 * Juntar um jogador com um dado nome a este jogo.
 * @param nomeJog O nome do jogador.
 * @requires !this.estahEmJogo(nomeJog) &&
 *           this.quantosEmJogo() < MAX_JOGADORES
 */
public void juntarJogador (String nomeJog) {
    this.jogadores[this.quantosEmJogo] = new Jogador (nomeJog);
    this.quantosEmJogo++;
}

/**
 * Juntar um jogador a este jogo.
 * @param novo O novo jogador.
 * @requires novo != null &&
 *           !this.estahEmJogo(novo.nome()) &&
 *           this.quantosEmJogo() < MAX_JOGADORES
 */
public void juntarJogador (Jogador novo) {
    this.jogadores[this.quantosEmJogo] = novo.copia();
}

```

```

        this.quantosEmJogo++;
    }

    /**
     * Retorna o jogador correspondente a um nome, ou null se nao houver.
     * Metodo privado
     * @param nomeJog O nome do jogador.
     * @returns O jogador com este nome se existir no jogo;
     *         null c.c.
     */
    private Jogador obterJogador (String nomeJog) {
        Jogador result = null;
        for (int i = 0 ; result == null && i < this.quantosEmJogo ; i++){
            if (nomeJog.equals(this.jogadores[i].nome())){
                result = this.jogadores[i];
            }
        }
        return result;
    }

    /**
     * Registrar a pontuacao na jogada atual de um dado jogador.
     * @param nomeJog O nome do jogador.
     * @param pontuacao A pontuacao obtida na jogada atual.
     * @requires this.estaEmJogo (nomeJog) &&
     *           pontuacao >= MIN_PONTUACAO_JOGADA &&
     *           pontuacao <= MAX_PONTUACAO_JOGADA
     */
    public void registrarPontosJogada (String nomeJog, int pontuacao) {
        this.obterJogador(nomeJog).registrarPontos(pontuacao);
    }

    /**
     * A pontuacao de um dado jogador deste jogo
     * @param nomeJog O nome do jogador.
     * @return A pontuacao do jogador com este nome
     * @requires this.estaEmJogo (nomeJog)
     */
    public int pontuacao (String nomeJog) {
        return this.obterJogador(nomeJog).pontuacao();
    }

    /**
     * A pontuacao maxima numa soh jogada de um dado jogador deste jogo
     * @param nomeJog O nome do jogador.
     * @return A pontuacao maxima numa soh jogada do jogador com este nome
     * @requires this.estaEmJogo (nomeJog)
     */
    public int maximoNumaJogada (String nomeJog) {
        return this.obterJogador(nomeJog).maximoNumaJogada();
    }

    // Ainda nao esta completa
}

```

Segue-se um exemplo de utilização destas constantes. As seguintes instruções numa qualquer classe cliente de `Jogo`,

```
Scanner leitor = new Scanner(System.in);
Jogo teuJogo = new Jogo(65);
teuJogo.juntarJogador("John Snow");
System.out.print("Qual a pontuacao? ");
double pontos;
do{
    System.out.println("insira valor entre " + Jogo.MIN_PONTUACAO_JOGADA +
        " e " + Jogo.MAX_PONTUACAO_JOGADA);
    nota = leitor.nextDouble();
} while(nota < Jogo.MIN_PONTUACAO_JOGADA || nota > Jogo.MAX_PONTUACAO_JOGADA);

teuJogo.registarPontosJogada("John Snow",pontos);
```

pedem ao utilizador um valor entre `Jogo.MIN_PONTUACAO_JOGADA` e `Jogo.MAX_PONTUACAO_JOGADA` e invocam o método que permite registar nova pontuação para um jogador.

Esta classe respeita a pré-condição do método `registarPontosJogada`, pois este só é invocado quando há a certeza de que o valor para o parâmetro `pontuacao` está no intervalo exigido pela pré-condição.

Enquanto o utilizador introduzir um valor fora do intervalo $[0,10]$, verá a mensagem *"insira valor entre 0 e 10"* no ecrã. Logo que introduza um valor no intervalo requerido, o ciclo termina e o método `registarPontosJogada` é invocado.

11.6. O método `toString` na classe `Jogo`

Voltemos de novo à classe `Jogo`.

Tal como fizemos para a classe `Jogador`, vamos construir o método `toString` para a classe `Jogo`, para permitir aos clientes desta classe obter uma representação textual de um jogo. Este método é útil durante o desenvolvimento da classe pois dá-nos uma forma de conhecermos o estado de um dado objeto do tipo `Jogo` em formato `String`, o qual poderemos, por exemplo, imprimir no *standard output*.

```
/**
 * Representacao textual deste jogo.
 */
public String toString () {
    StringBuilder sb = new StringBuilder();
    sb.append("Objetivo do jogo: " + this.objetivo + " pontos" + "\n");
    sb.append("Numero de jogadores: " + this.quantosEmJogo + "\n");
    for (int i = 0 ; i < this.quantosEmJogo ; i++){
        sb.append(this.jogadores[i].toString() + "\n");
    }
    return sb.toString();
}
```

Note a forma como obtemos a representação dos vários jogadores registados no jogo – invocamos sobre cada um deles o seu próprio método `toString`, definido na classe `Jogador`.

Se acrescentarmos a seguinte instrução no método `main` da classe `ClienteDeJogo`:


```
System.out.println(meuJogo.toString());
```

ela terá como efeito a escrita das seguintes linhas no *standard output*:

```
Objetivo do jogo: 30 pontos
Numero de jogadores: 2
Nome: Maria      Pontuacao: 30      Maximo numa jogada: 10
Nome: Pedro      Pontuacao: 18      Maximo numa jogada: 8
```

Repare que as duas últimas linhas são exatamente o que se obtém ao invocar o método `toString` da classe `Jogador` sobre cada um dos jogadores do jogo.

11.7. Fim de jogo, vencedores e outros métodos

A cada jogada que faz, um jogador aproxima-se do fim do jogo, ou seja, do número de pontos definido como objetivo a alcançar. Quando algum(ns) jogador(es) alcança(m) uma pontuação igual ou maior que o objetivo do jogo, este termina e o(s) vencedor(es) pode(m) ser identificado(s).

11.7.1. Fim de jogo

Se tivermos um método que calcula quantos pontos tem o jogador que tem maior pontuação, e compararmos esse valor com o objetivo do jogo, então ficamos a saber se o jogo já terminou ou não.

Vamos então criar um método que calcula a maior pontuação dos jogadores do jogo. Isto é um problema de cálculo do máximo de um conjunto de valores, tal como fizemos, de forma muito geral, na secção 9.6.2.

A diferença aqui é que o nosso *array* é o atributo de instância `jogadores` e não um parâmetro do método. Além disso, aqui não temos um *array* com os inteiros a comparar, mas sim um *array* de jogadores.

Os valores que queremos comparar não são os jogadores propriamente ditos mas as suas pontuações, ou seja, para um dado jogador referenciado por um elemento do array `jogadores`, por exemplo, `jogadores[i]`, o que queremos é a sua pontuação, ou seja, o resultado da invocação `jogadores[i].pontuacao()`.

```
/**
 * A maior pontuacao dos jogadores deste jogo
 * @return O valor da maior pontuacao
 * @requires this.quantosEmJogo() > 0
 */
public int maiorPontuacao () {
    int maiorPontuacao = 0;
    for (int i = 0 ; i < this.quantosEmJogo ; i++) {
        int pontos = this.jogadores[i].pontuacao();
        if (pontos > maiorPontuacao) {
            maiorPontuacao = pontos;
        }
    }
    return maiorPontuacao;
}
```

Então, para verificar se o jogo terminou basta comparar o resultado do método `maiorPontuacao` com o valor do atributo `objetivo`:

```

/**
 * Este jogo jah terminou?
 */
public boolean terminou () {
    return this.maiorPontuacao() >= this.objetivo;
}

```

Um exemplo de utilização deste método numa nova classe `ClienteDeJogo`:

```

public class ClienteDeJogo {

    public static void main (String [] args) {

        Jogo meuJogo = new Jogo(30);
        String[] participantes = {"Maria","Pedro","Rui","Margarida","Antonio"};
        for (int i = 0 ; i < participantes.length ; i++) {
            meuJogo.juntarJogador(participantes[i]);
        }

        Random gerador = new Random();
        int max = Jogo.MAX_PONTUACAO_JOGADA;
        int min = Jogo.MIN_PONTUACAO_JOGADA;

        // O jogo eh jogado em rondas.
        // Em cada ronda jogam todos os jogadores.
        while(!meuJogo.terminou()) {
            for (int j = 0 ; j < participantes.length ; j++) {
                // gera um valor no intervalo [min,max]
                int pontuacao = gerador.nextInt(max - min + 1) + min;
                // regista a pontuacao do proximo jogador
                meuJogo.registarPontosJogada(participantes[j], pontuacao);
                System.out.println(participantes[j] + " ganhou mais " +
                                   pontuacao + " pontos");
            }
        }
        System.out.println("Acabou o jogo!");
    }
}

```

Vamos analisar os vários passos do método `main`:

- É criado um objeto do tipo `Jogo`;
- Com o intuito de registar 5 jogadores no jogo, em vez de termos cinco instruções quase iguais `meuJogo.juntarJogador(...)`; em que a única coisa que muda é o nome do jogador, declaramos e criamos um *array* de `String`s contendo os nomes dos jogadores a registar; de seguida executamos um ciclo cujo corpo invoca o método `juntarJogador` usando, a cada iteração, um novo elemento desse *array*; no fim do ciclo todos os jogadores estão registados; o mesmo *array* `participantes` irá ser usado mais adiante, para registar os pontos que cada jogador irá ter nas várias rondas do jogo;
- O programa cria um gerador de aleatórios (objeto do tipo `Random`, como já falado na secção 8.2) para gerar valores para as várias jogadas dos jogadores; os valores a serem gerados têm que estar no intervalo `[Jogo.MAX_PONTUACAO_JOGADA , Jogo.MIN_PONTUACAO_JOGADA]`; para simplificarmos os identificadores usados no programa atribuímos esses valores a duas variáveis de nomes `min` e `max`;

- O jogo é jogado em várias rondas, até que algum(ns) jogador(es) alcance(m) uma pontuação igual ou superior ao objetivo do jogo; em cada ronda todos os jogadores jogam; então vamos ter dois ciclos encaixados, o externo para fazer as várias rondas, até o jogo terminar e o interno, para que, a cada ronda, todos os jogadores possam jogar:
 - o ciclo externo é um ciclo `while` (repetição não limitada) com a guarda `!meuJogo.terminou()` (significando "enquanto o jogo não terminar"); o corpo deste ciclo é o ciclo interno;
 - o ciclo interno regista as pontuações de todos os jogadores; em cada iteração gera um aleatório no intervalo `[min, max]` e invoca o método `registarPontosJogada` tendo como valores para os parâmetros o nome j-ésimo jogador e o valor acabado de gerar; a instrução `System.out.println` serve somente para informar o utilizador do que se vai passando no jogo;
- Quando o jogo termina, o utilizador é informado.

O *output* de uma possível execução deste programa:

```
Maria ganhou mais 9 pontos
Pedro ganhou mais 0 pontos
Rui ganhou mais 10 pontos
Margarida ganhou mais 7 pontos
Antonio ganhou mais 8 pontos
Maria ganhou mais 10 pontos
Pedro ganhou mais 1 pontos
Rui ganhou mais 3 pontos
Margarida ganhou mais 3 pontos
Antonio ganhou mais 9 pontos
Maria ganhou mais 7 pontos
Pedro ganhou mais 3 pontos
Rui ganhou mais 2 pontos
Margarida ganhou mais 2 pontos
Antonio ganhou mais 6 pontos
Maria ganhou mais 8 pontos
Pedro ganhou mais 9 pontos
Rui ganhou mais 9 pontos
Margarida ganhou mais 6 pontos
Antonio ganhou mais 10 pontos
Acabou o jogo!
```

11.7.2. Vencedor(es) – primeira tentativa

Era bom que o vencedor fosse calculado pelo próprio programa para que o utilizador não tenha que estar a fazer contas.

Quem sabe qual é o vencedor do jogo? É quem tem informação para isso, ou seja, o próprio jogo, que pode perguntar a cada um dos seus jogadores qual a sua pontuação e decidir qual o vencedor.

Vamos então agora ver como fazer isso. Se o critério para vencer o jogo fosse somente alcançar uma pontuação maior ou igual à do objetivo do jogo, então o método seguinte, que devolve o nome do vencedor, pode ser uma primeira abordagem:

```
/**
 * Vencedor deste jogo
 * @return O nome do jogador que atingiu o objetivo
 * @requires this.quantosEmJogo() > 0 && this.terminou()
 */
public String vencedor () {
```

```

        int pontos = this.jogadores[0].pontuacao();
        String resultado = this.jogadores[0].nome();
        for (int i = 1 ; i < this.jogadores.length && pontos < this.objetivo ; i++) {
            pontos = this.jogadores[i].pontuacao();
            if (pontos >= this.objetivo) {
                resultado = this.jogadores[i].nome();
            }
        }
        return resultado;
    }
}

```

Começamos por guardar nas variáveis `pontos` e `resultado` a pontuação e o nome do primeiro jogador registado no jogo (que é o primeiro elemento do *array* `jogadores`). O ciclo permite-nos aceder aos outros jogadores, por ordem de registo no jogo (a mesma ordem dos elementos no *array*), até encontrarmos algum cuja pontuação seja igual ou superior ao objetivo.

Se o primeiro jogador tiver uma pontuação igual ou superior ao objetivo do jogo, é o nome dele que resulta deste método. Caso contrário, o ciclo é executado para procurar o primeiro jogador, dos seguintes, a ter alcançado o objetivo. O ciclo pára logo que um jogador nessas condições é encontrado.

Repare na pré-condição: `this.quantosEmJogo() > 0 && this.terminou()`. Além de especificar que tem que existir pelo menos um jogador registado no jogo, requer também que o jogo já tenha terminado, ou seja, que o resultado da invocação do método `terminou` seja `true`. Isso garante que existe um vencedor, pois o jogo só termina quando pelo menos um jogador alcançou o objetivo.

Mas esta solução tem vários defeitos:

- Não garante que o resultado seja o nome do jogador que tem a maior pontuação; se o objetivo for 30 pontos e o primeiro jogador tiver 31 pontos e o segundo tiver 33 pontos, este método vai ter como resultado o nome do primeiro jogador;
- Se existir mais do que um vencedor – por exemplo, dois jogadores com 33 pontos, pontuação essa superior à dos outros jogadores – só o nome do primeiro é retornado.

11.7.3. Vencedor(es) – segunda tentativa

Numa segunda tentativa, vamos fazer as seguintes modificações:

- O tipo de retorno do método é um *array* de nomes em vez de um só nome;
- O resultado consiste dos nomes de todos os jogadores que tenham alcançado o objetivo.

```

/**
 * Vencedor(es) deste jogo
 * @return O nome do jogador que atingiu o objetivo. No caso de
 *         haver varios, retorna os seus nomes
 * @requires this.quantosEmJogo() > 0 && this.terminou()
 */
public String[] vencedores () {
    // No maximo serao this.quantosEmJogo vencedores
    String[] resultado = new String[this.quantosEmJogo];
    int k = 0;
    for (int i = 0 ; i < this.quantosEmJogo ; i++) {
        if (this.jogadores[i].pontuacao() >= this.objetivo) {
            resultado[k] = this.jogadores[i].nome();
            k++;
        }
    }
}

```

```
        return resultado;
    }
}
```

Voltando ao `main` da nossa classe `ClienteDeJogo` acima descrita, vamos fazer as seguintes alterações:

- em vez de registrar os 5 jogadores, registamos só a "Maria" e o "Pedro", para o exemplo ser mais simples;
- acrescentamos as instruções seguintes após o ciclo `while`, para imprimir no ecrã o(s) nome(s) do(s) vencedor(s),

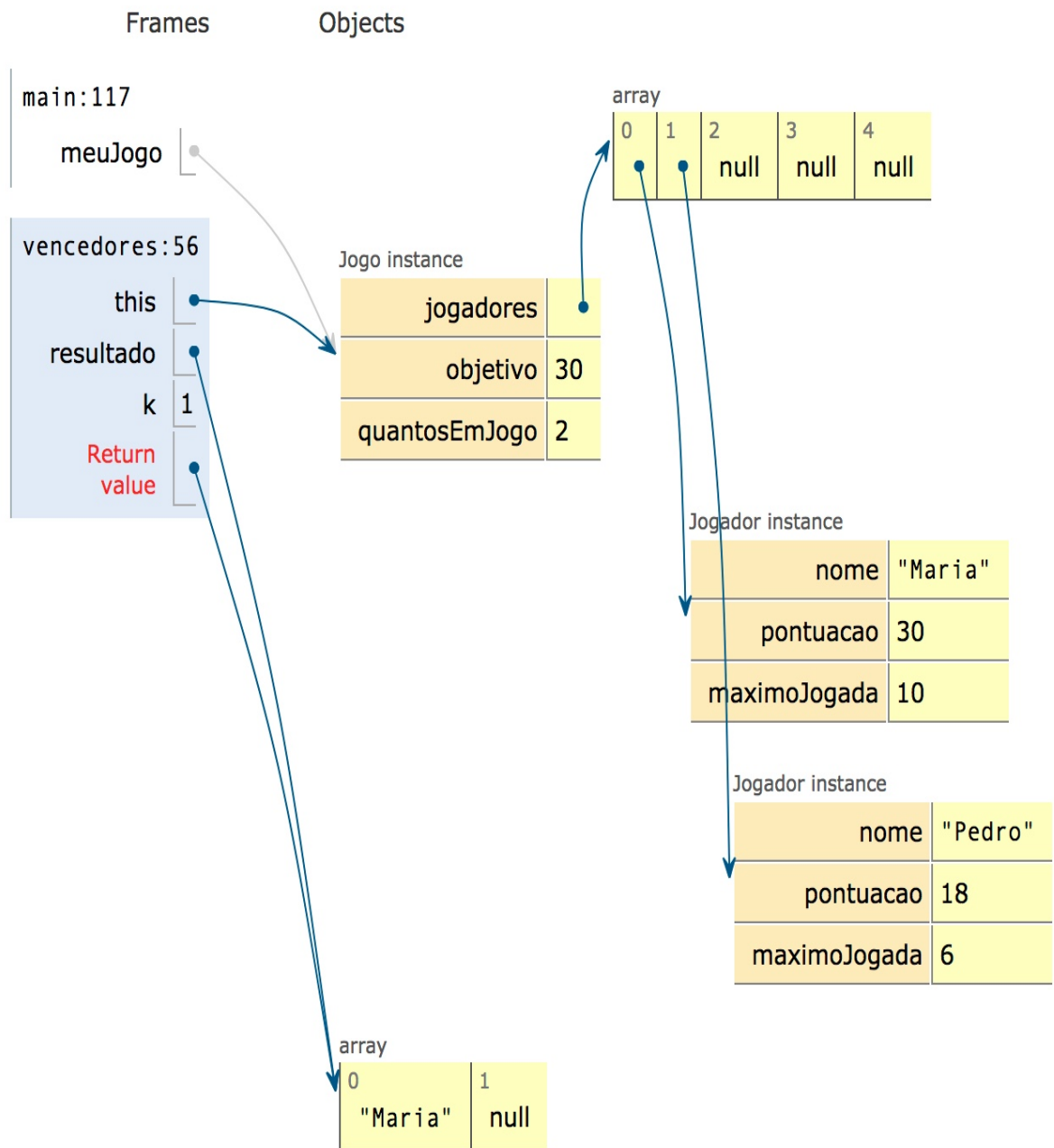
```
String[] vencedores = meuJogo.vencedores();
for (int i = 0 ; i < vencedores.length ; i++) {
    System.out.println(vencedores[i] + " venceu com " +
        meuJogo.pontuacao(vencedores[i]) + " pontos");
}
```

Ao executar o `main`, vamos obter o seguinte *output* (só apresentamos a parte produzida pelas novas instruções) seguido de uma exceção que provoca a terminação abrupta do programa:

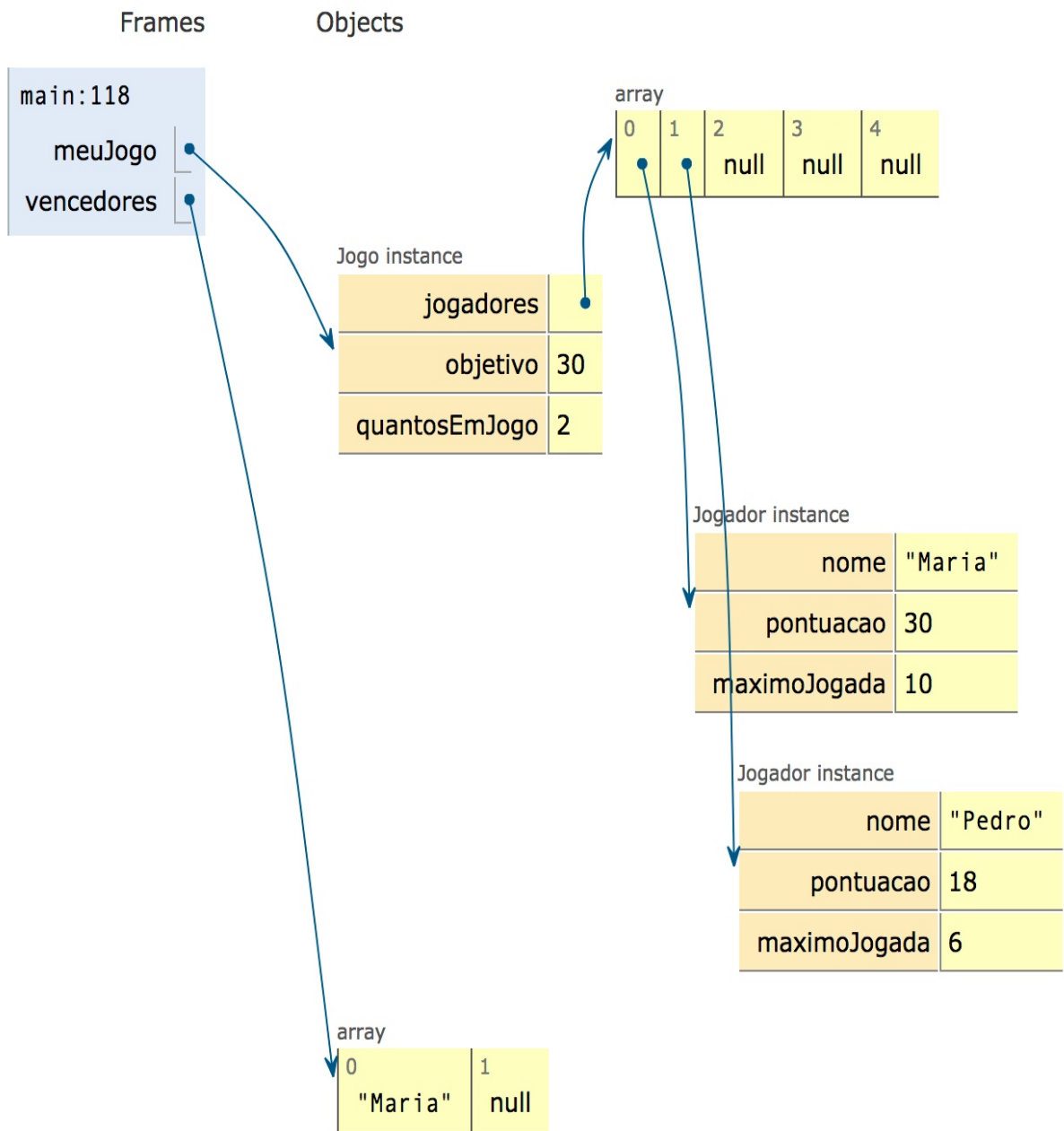
```
...
Maria venceu com 30 pontos
Exception in thread "main" java.lang.NullPointerException
    at Jogo.obterJogador(Jogo.java:88)
    at Jogo.pontuacao(Jogo.java:289)
    at ClienteDeJogo.main(ClienteDeJogo.java:48)
```

Observamos que, dos dois jogadores em jogo, só a Maria conseguiu alcançar o objetivo dos 30 pontos, o que faz com que faça parte dos vencedores. Relembre que, por gerarmos as pontuações com um objeto `Random`, o *output* será potencialmente diferente a cada execução. Mas só em casos muito particulares, nomeadamente quando todos os jogadores são vencedores, é que o programa prossegue sem exceções. Já vai perceber porquê.

- Ao imprimir os nomes dos vencedores, no `main` da classe `ClienteDeJogo`, o método `pontuação` da classe `Jogo` é invocado sobre o objeto `meuJogo` tantas vezes quantas o número de elementos do *array* `vencedores`.
 - Em cada uma dessas invocações o parâmetro do método é instanciado com a `String` contida em `vencedores[i]`.
 - A variável de progresso do ciclo, `i`, vai tomar valores desde zero até `vencedores.length`.
- Quantos elementos tem o *array* `vencedores`?
 - Tantos quantos o *array* devolvido pelo método `vencedores` da classe `Jogo`.
 - Neste caso são dois pois no método `vencedores` o *array* `resultado` é criado através da instrução `String[] resultado = new String[this.quantosEmJogo];`. Como foram registados 2 jogadores no jogo, o atributo `this.quantosEmJogo` tem o valor 2.
- Quantos vencedores há na verdade? Um!
 - No método `vencedores` da classe `Jogo`, embora o *array* `resultado` tenha 2 elementos (todos inicializados a `null`, como já sabe), a instrução `resultado[k] = jogadores[i].nome();` só é executada uma vez.
 - Então, o último elemento do *array* `resultado` fica com o valor `null`.



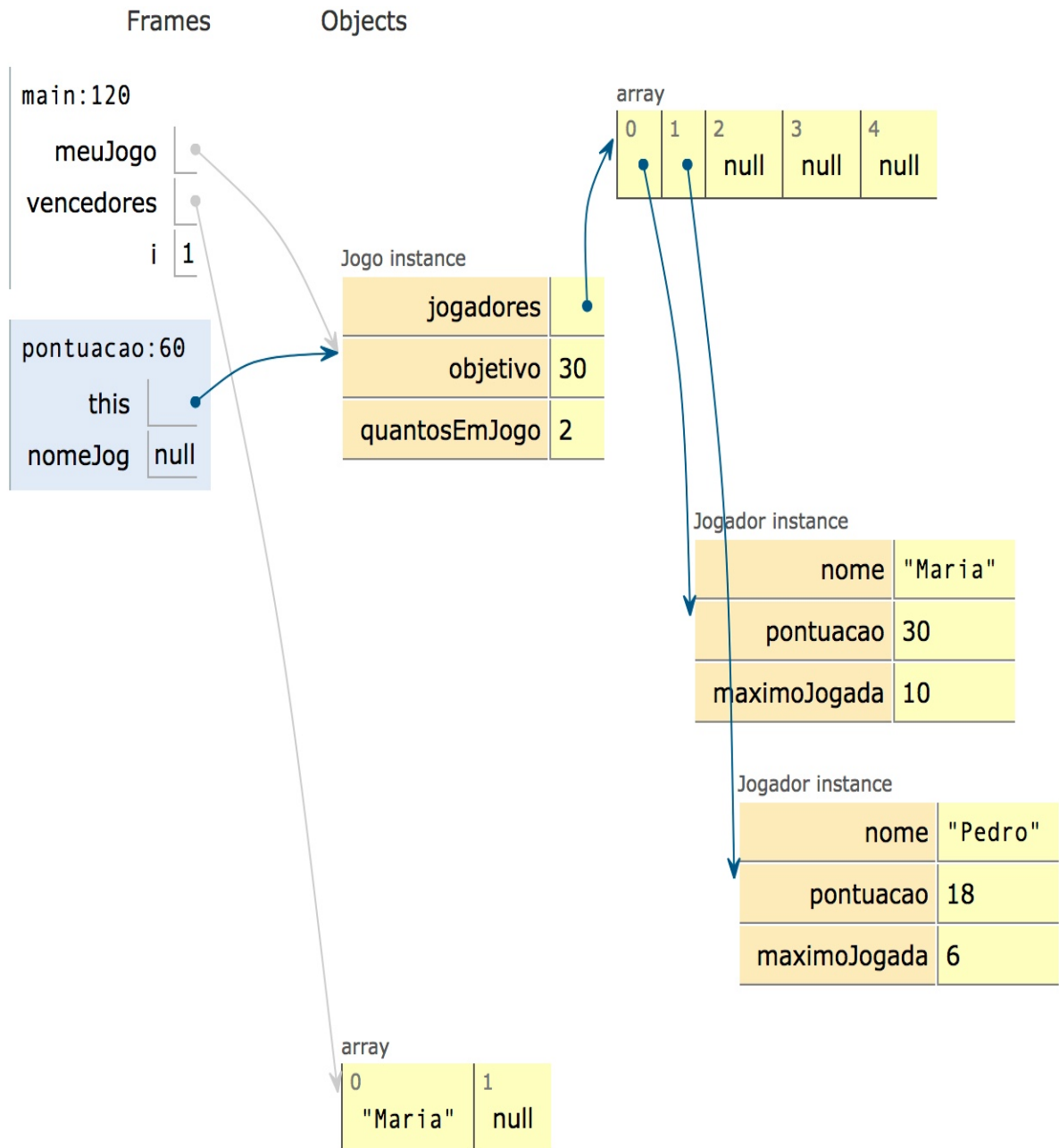
- Então, o `array` `vencedores` no `main` da classe `ClienteDeJogo` terá também dois elementos, sendo o último `null`.



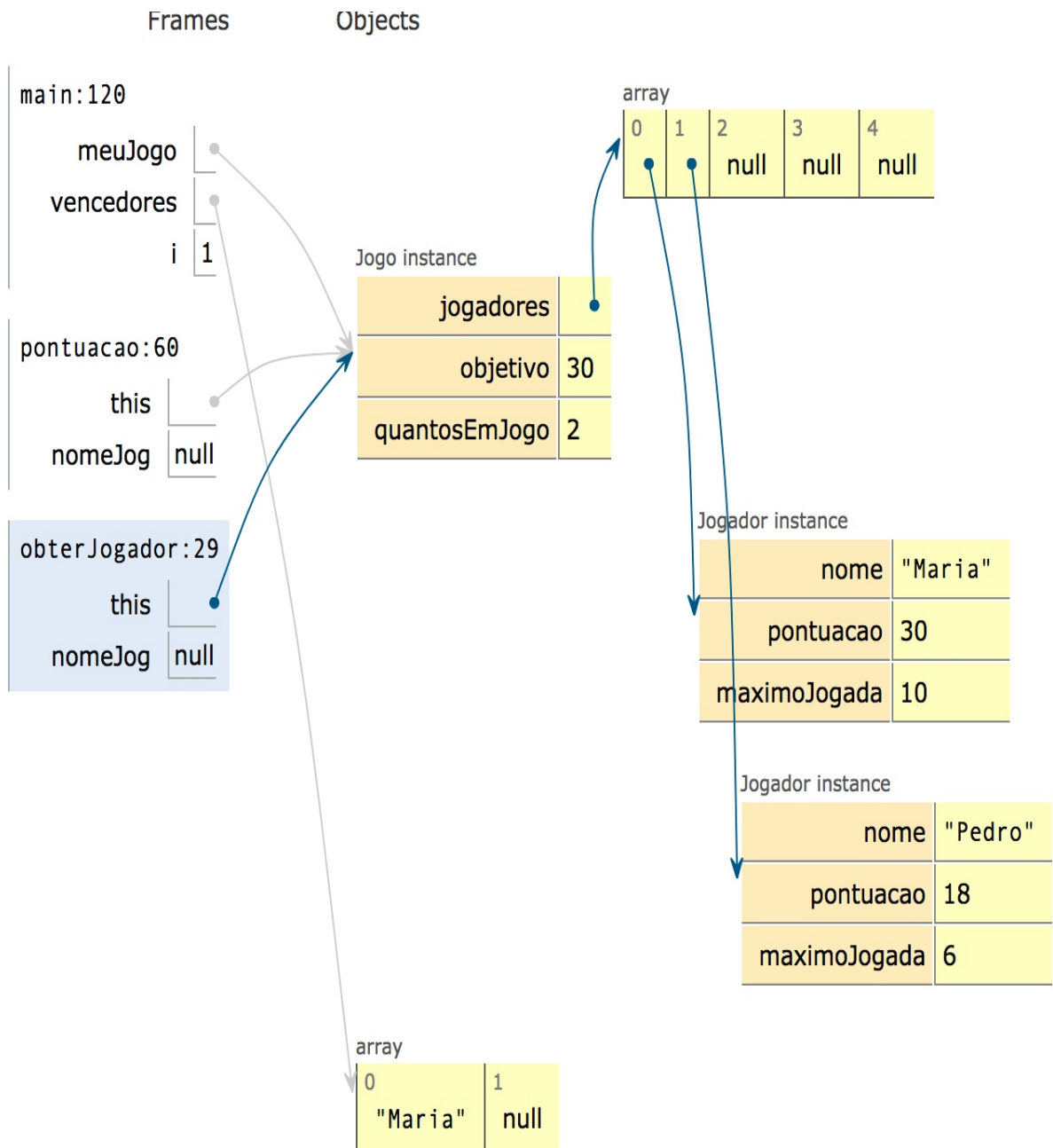
- Logo, quando a instrução

```
System.out.println(vencedores[i] + " venceu com " + meuJogo.pontuacao(vencedores[i]) + " pontos");
```

é executada para o segundo e último desses elementos (quando `i` é 1), o valor dado para o parâmetro na invocação do método `pontuacao` é `null`.



- Neste método `pontuacao`, o método `obterJogador` é invocado, usando exatamente o mesmo valor do parâmetro `nomeJog`, que neste caso é `null`.



- No método `obterJogador` o valor do parâmetro `nomeJog` é usado na instrução

```

if (nomeJog.equals(this.jogadores[i].nome())){
    result = this.jogadores[i];
}

```

Repare que na guarda do `if` é invocado o método `equals` sobre o parâmetro `nomeJog`, cujo valor é `null`! É exatamente aqui que é lançada a exceção `NullPointerException` que provoca a terminação abrupta do programa (releia o texto da exceção mais acima nesta página e verifique a origem da exceção e a forma como ela se propaga).

11.7.4. Vencedor(es) – terceira, e última, tentativa

Resumindo e concluindo:

- O método `vencedores` da classe `Jogo` não deve retornar um *array* de nomes que contenha elementos com valor `null`.
- A seguinte versão aumentada do mesmo método `vencedores` resolve esse problema:

```
/**
 * Vencedor(es) deste jogo
 * @return O nome do jogador que atingiu o objetivo. No caso de
 *         haver varios, retorna os seus nomes
 * @requires this.quantosEmJogo() > 0 && this.terminou()
 */
public String[] vencedores () {
    // No maximo serao this.quantosEmJogo vencedores
    String[] resultado = new String[this.quantosEmJogo];
    int k = 0;
    for (int i = 0 ; i < this.quantosEmJogo ; i++) {
        if (this.jogadores[i].pontuacao() >= this.objetivo) {
            resultado[k] = this.jogadores[i].nome();
            k++;
        }
    }
    // Soh interessa retornar os k primeiros elementos do vetor
    // resultado, pois os outros estao a null
    String[] resultadoFinal = new String[k];
    for (int i = 0 ; i < k ; i++) {
        resultadoFinal[i] = resultado[i];
    }
    return resultadoFinal;
}
```

11.7.5. Outra forma de definir o(s) vencedor(es)

Se quiséssemos obter somente o jogador que tem a maior pontuação, e devolver esse como vencedor do jogo, poderíamos deparar-nos com situações em que há mais que um jogador que tem a pontuação máxima. O seguinte método privado calcula e devolve os jogadores que têm a melhor pontuação. Iremos invocá-lo numa nova versão do método `vencedores`.

```
/**
 * Os jogadores que teem a maior pontuacao.
 * Metodo privado, auxiliar.
 * @return Os jogadores que teem a maior pontuacao
 * @requires this.quantosEmJogo() > 0
 */
private Jogador[] osMelhores () {
    // Qual a melhor pontuacao?
    int maiorPontuacao = this.maiorPontuacao();
    // Nao sabemos quantos serao. No maximo serao this.quantosEmJogo
    Jogador[] melhoresJogadores = new Jogador[this.quantosEmJogo];
    int k = 0;
    for (int i = 0 ; i < this.quantosEmJogo ; i++) {
        if (this.jogadores[i].pontuacao() == maiorPontuacao) {
            melhoresJogadores[k] = this.jogadores[i];
            k++;
        }
    }
}
```

```

    }
    // Soh interessa retornar os k primeiros elementos do vetor, pois
    // os outros estao a null
    Jogador[] resultado = new Jogador[k];
    for (int i = 0 ; i < k ; i++) {
        resultado[i] = melhoresJogadores[i];
    }
    return resultado;
}

```

No caso de serem mais do que um, e quisermos desempatar, podemos fazê-lo escolhendo o que tem o valor maior para o atributo que representa o "máximo numa só jogada". De novo, aqui, podem ser mais do que um. A nova versão do método `vencedores` ficaria então:

```

/**
 * Vencedor(es) deste jogo
 * @return O nome do jogador que atingiu o objetivo. No caso de
 *         haver varios, aquele que teve a pontuacao mais alta numa
 *         soh jogada. No caso de ainda haver varios, retorna os seus nomes
 * @requires this.quantosEmJogo() > 0 && this.terminou()
 */
public String[] vencedores () {
    // Os jogadores que tiveram a melhor pontuacao ex-aequo
    Jogador[] melhores = osMelhores();
    // entre esses, calcular o valor maior do atributo maximo numa soh jogada
    // o(s) jogador(es) que o obteve(iram) serah(ao) o(s) vencedor(es)
    Jogador jog = melhores[0];
    int maximo = jog.maximoNumaJogada();
    for (int i = 1 ; i < melhores.length ; i++) {
        jog = melhores[i];
        int pontos = jog.maximoNumaJogada();
        if (pontos > maximo) {
            maximo = pontos;
        }
    }
    // Selecionar, de entre os que teem pontuacao melhor, os que
    // teem o maior valor do atributo maximo numa jogada
    String[] vencedores = new String[melhores.length];
    int k = 0;
    for (int i = 0 ; i < melhores.length ; i++) {
        if (melhores[i].maximoNumaJogada() == maximo) {
            vencedores[k] = melhores[i].nome();
            k++;
        }
    }
    // Soh interessa retornar os k primeiros elementos do vetor
    String[] resultado = new String[k];
    for (int i = 0 ; i < k ; i++) {
        resultado[i] = vencedores[i];
    }
    return resultado;
}

```

11.7.6. Jogadores com pontuação superior a um dado valor

O seguinte método devolve os nomes dos jogadores cuja pontuação é superior a um valor dado:

```
/**
 * Os nomes dos jogadores com pontuacao superior a um dado valor
 * @param limiteInf O valor de comparacao.
 * @requires this.quantosEmJogo() > 0
 * @return Um vetor com os nomes dos jogadores deste jogo com
 *         pontuacao superior a limiteInf
 */
public String[] pontuacaoSuperiorA (int limiteInf) {
    // Nao sabemos quantos serao. No maximo serao
    // this.quantosEmJogo
    String [] melhores = new String [this.quantosEmJogo];
    int k = 0;
    for (int j = 0 ; j < this.quantosEmJogo ; j++){
        if (this.jogadores[j].pontuacao() >= limiteInf){
            melhores[k] = this.jogadores[j].nome();
            k++;
        }
    }
    // Soh interessa retornar os elementos do vetor que
    // nao sao null
    String [] result = new String [k];
    for (int i = 0 ; i < k ; i++){
        result[i] = melhores[i];
    }
    return result;
}
```

11.8. A classe `Jogo` completa

Juntando tudo o que fomos fazendo ao longo das várias secções deste capítulo, obtemos a classe `Jogo` completa. Acrescentámos um atributo de instância mais – a designação do jogo –, e um método para revelar o valor do atributo. Alterámos também o construtor, para receber mais um parâmetro com o valor para esse novo atributo e o método `toString`, para ficar coerente com esta alteração.

```
/**
 * Classe cujos objetos representam jogos
 * @author Isabel Nunes
 */
public class Jogo {

    /**
     * O numero maximo de jogadores por jogo
     */
    public static final int MAX_JOGADORES = 5;

    /**
     * O minimo e maximo de pontos que um jogador pode ter numa jogada
     */
    public static final int MIN_PONTUACAO_JOGADA = 0;
```

```

    public static final int MAX_PONTUACAO_JOGADA = 10;

    // atributos que definem este jogo:
    // A designacao do jogo
    private String designacao
    // A colecao dos jogadores
    private Jogador [] jogadores;
    // Pontuacao objetivo
    private int objetivo;
    // Numero de jogadores em jogo
    private int quantosEmJogo;

    /**
     * Construir um jogo.
     * @param designacao A designacao deste jogo
     * @param objetivo A pontuacao com que se ganha este jogo
     * @requires objetivo > 0
     */
    public Jogo (String designacao, int objetivo) {
        this.designacao = designacao;
        this.objetivo = objetivo;
        this.jogadores = new Jogador [MAX_JOGADORES];
        this.quantosEmJogo = 0;
    }

    /**
     * A designacao deste jogo.
     */
    public String designacao () {
        return this.designacao;
    }

    /**
     * A pontuacao objetivo deste jogo.
     */
    public int pontuacaoObjetivo () {
        return this.objetivo;
    }

    /**
     * O numero de jogadores que estao em jogo.
     */
    public int quantosEmJogo () {
        return this.quantosEmJogo;
    }

    /**
     * Juntar um jogador com um dado nome a este jogo.
     * @param nomeJog O nome do jogador.
     * @requires !this.estahEmJogo(nomeJog) &&
     *           this.quantosEmJogo() < MAX_JOGADORES
     */
    public void juntarJogador (String nomeJog) {
        this.jogadores[this.quantosEmJogo] = new Jogador (nomeJog);
        this.quantosEmJogo++;
    }
}

```

```

/**
 * Retorna o jogador correspondente a um nome, ou null se nao houver.
 * Metodo privado, auxiliar.
 * @param nomeJog O nome do jogador.
 * @returns O jogador com este nome se existir no jogo;
 *         null caso contrario
 */
private Jogador obterJogador (String nomeJog) {
    Jogador result = null;
    for (int i = 0 ; result == null && i < this.quantosEmJogo ; i++){
        if (nomeJog.equals(this.jogadores[i].nome())){
            result = this.jogadores[i];
        }
    }
    return result;
}

/**
 * Registrar a pontuacao, na jogada atual, de um dado jogador.
 * @param nomeJog O nome do jogador.
 * @param pontuacao A pontuacao obtida na jogada atual.
 * @requires this.estaEmJogo (nomeJog) &&
 *         pontuacao >= MIN_PONTUACAO_JOGADA &&
 *         pontuacao <= MAX_PONTUACAO_JOGADA
 */
public void registrarPontosJogada (String nomeJog, int pontuacao) {
    this.obterJogador(nomeJog).registrarPontos(pontuacao);
}

/**
 * Um dado jogador esta em jogo?
 * @param nomeJog O nome do jogador.
 * @return true se o jogador com este nome esta inscrito neste jogo
 *         false caso contrario
 */
public boolean estaEmJogo (String nomeJog) {
    return this.obterJogador(nomeJog) != null;
}

/**
 * A pontuacao de um dado jogador deste jogo
 * @param nomeJog O nome do jogador.
 * @return A pontuacao do jogador com este nome
 * @requires this.estaEmJogo (nomeJog)
 */
public int pontuacao (String nomeJog) {
    return this.obterJogador(nomeJog).pontuacao();
}

/**
 * A pontuacao maxima numa soh jogada de um dado jogador deste jogo
 * @param nomeJog O nome do jogador.
 * @return A pontuacao maxima numa soh jogada do jogador com este nome
 * @requires this.estaEmJogo (nomeJog)
 */
public int maximoNumaJogada (String nomeJog) {
    return this.obterJogador(nomeJog).maximoNumaJogada();
}

```

```

}

/**
 * Este jogo jah terminou?
 */
public boolean terminou () {
    return this.maiorPontuacao() >= objetivo;
}

/**
 * Vencedor(es) deste jogo
 * @return O nome do jogador que atingiu o objetivo. No caso de
 *         haver varios, aquele que teve a pontuacao mais alta numa
 *         soh jogada. No caso de haver varios, retorna os seus nomes
 * @requires this.quantosEmJogo() > 0 && this.terminou()
 */
public String[] vencedores () {
    // Os jogadores que tiveram a melhor pontuacao ex-aequo
    Jogador[] melhores = this.osMelhores();
    // calcular o maior dos maximos numa soh jogada
    // o(s) jogador(es) que o obteve(iram) serah(ao) o(s) vencedor(es)
    Jogador jog = melhores[0];
    int maximo = jog.maximoNumaJogada();
    for (int i = 1 ; i < melhores.length ; i++) {
        jog = melhores[i];
        int pontos = jog.maximoNumaJogada();
        if (pontos > maximo) {
            maximo = pontos;
        }
    }
    String[] vencedores = new String[melhores.length];
    int k = 0;
    for (int i = 0 ; i < melhores.length ; i++) {
        if (melhores[i].maximoNumaJogada() == maximo) {
            vencedores[k] = melhores[i].nome();
            k++;
        }
    }
    // Soh interessa retornar os k primeiros elementos do vetor
    // pois os outros estao a null
    String[] resultado = new String[k];
    for (int i = 0 ; i < k ; i++) {
        resultado[i] = vencedores[i];
    }
    return resultado;
}

/**
 * A maior pontuacao dos jogadores deste jogo
 * @return O valor da maior pontuacao
 * @requires this.quantosEmJogo() > 0
 */
public int maiorPontuacao () {
    int maiorPontuacao = 0;
    for (int i = 0 ; i < this.quantosEmJogo ; i++) {
        int pontos = this.jogadores[i].pontuacao();
        if (pontos > maiorPontuacao) {

```

```

        maiorPontuacao = pontos;
    }
}
return maiorPontuacao;
}

/**
 * Os jogadores que teem a maior pontuacao geral.
 * Metodo privado, auxiliar.
 * @return Os jogadores que teem a maior pontuacao
 * @requires this.quantosEmJogo() > 0
 */
private Jogador[] osMelhores () {
    // Qual a melhor pontuacao?
    int maiorPontuacao = this.maiorPontuacao();
    // Nao sabemos quantos serao. No maximo serao this.quantosEmJogo
    Jogador[] melhoresJogadores = new Jogador[this.quantosEmJogo];
    int k = 0;
    for (int i = 0 ; i < this.quantosEmJogo ; i++) {
        if (this.jogadores[i].pontuacao() == maiorPontuacao) {
            melhoresJogadores[k] = this.jogadores[i];
            k++;
        }
    }
    // Soh interessa retornar os k primeiros elementos do vetor
    // pois os outros estao a null
    Jogador[] resultado = new Jogador[k];
    for (int i = 0 ; i < k ; i++) {
        resultado[i] = melhoresJogadores[i];
    }
    return resultado;
}

/**
 * Os nomes dos jogadores com pontuacao superior a um dado valor
 * @param limiteInf O valor de comparacao.
 * @requires this.quantosEmJogo() > 0
 * @return Um vetor com os nomes dos jogadores deste jogo com
 *         pontuacao superior a limiteInf
 */
public String[] pontuacaoSuperiorA (int limiteInf) {
    // Nao sabemos quantos serao. No maximo serao this.quantosEmJogo
    String [] melhores = new String [this.quantosEmJogo];
    int k = 0;
    for (int j = 0 ; j < this.quantosEmJogo ; j++){
        if (this.jogadores[j].pontuacao() >= limiteInf){
            melhores[k] = this.jogadores[j].nome();
            k++;
        }
    }
    // Soh interessa retornar os elementos do vetor que
    // nao sao null
    String [] result = new String [k];
    for (int i = 0 ; i < k ; i++){
        result[i] = melhores[i];
    }
    return result;
}

```



```

    }

    /**
     * Representacao textual deste jogo.
     */
    public String toString () {
        StringBuilder sb = new StringBuilder();
        sb.append("Designacao do jogo: " + this.designacao + "\n");
        sb.append("Objetivo do jogo: " + this.objetivo + " pontos" + "\n");
        sb.append("Numero de jogadores: " + this.quantosEmJogo + "\n");
        for (int i = 0 ; i < this.quantosEmJogo ; i++){
            sb.append(this.jogadores[i].toString() + "\n");
        }
        return sb.toString();
    }
}

```

11.9. Representação em UML

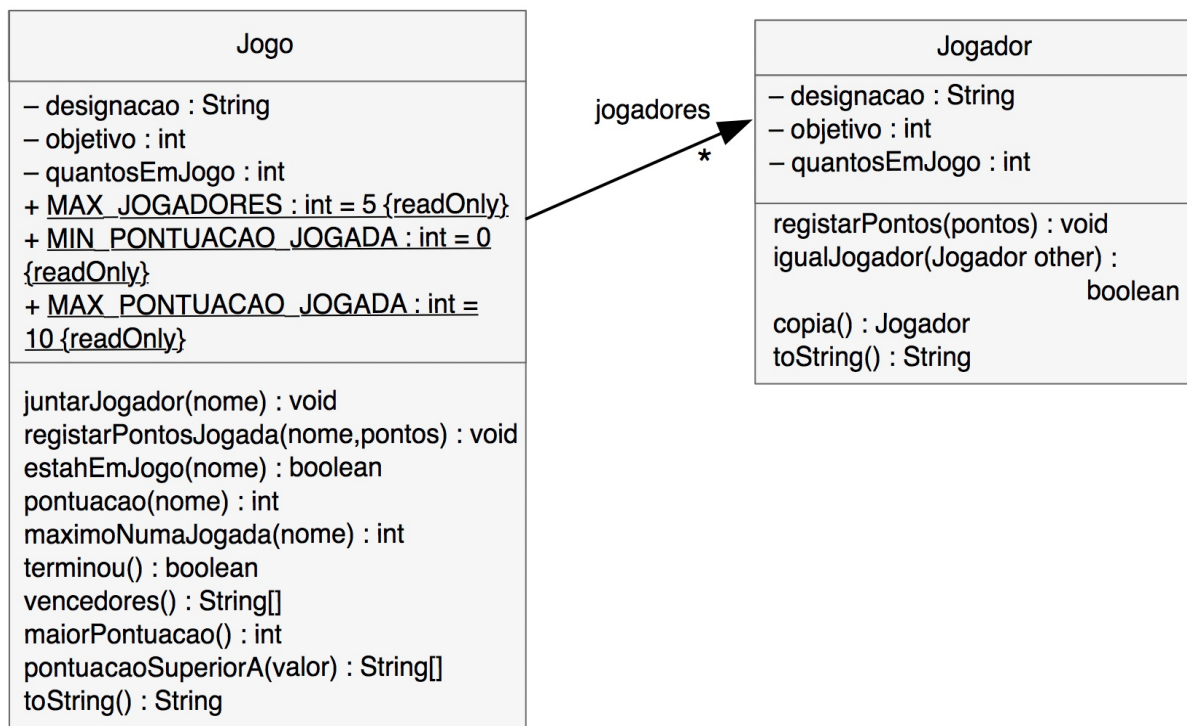
À medida que vamos acrescentando classes, e relações entre classes, ao nosso sistema, vai-se tornando mais difícil a compreensão do todo e das partes. Isto dificulta grandemente a manutenção e a evolução do nosso *software*.

A utilização de abstrações gráficas para os vários componentes do nosso sistema permite-nos representá-lo de uma forma em que sobressaem as entidades e relações que nos interessam a dado instante.

A linguagem UML – *Unified Modeling Language* – dá-nos esta capacidade, permitindo a representação dos sistemas usando variados graus de abstração/pormenor.

A forma de representar uma classe, na linguagem UML, é através de um retângulo, que pode ser dividido de modo a representar os vários elementos da classe – nome da classe, atributos, métodos. A cada elemento que constitui a classe pode-se ainda juntar informação vária – visibilidade, tipo, etc.

No nosso caso particular temos duas classes – `Jogador` e `Jogo` – cada uma definindo os seus atributos (de instância e de classe) e os seus métodos.



A primeira divisória do retângulo que representa uma dada classe, apresenta o nome da classe.

A segunda divisória apresenta os atributos, tanto os de classe (`static`), que aparecem sublinhados, como os de instância. Os sinais `-` e `+` significam que o atributo é `private` ou `public`, respetivamente.

Um atributo de tipo primitivo é representado na divisória dos atributos através do seu nome. Podemos acrescentar-lhe o tipo e outros elementos descritivos mais.

Para representar, numa classe `A`, um atributo de um tipo não primitivo `B`, usa-se uma seta com a cabeça preta, na direção da classe `A` para a classe `B`. A relação existente entre `A` e `B` diz-se *relação de composição*. Mais adiante veremos outros tipos de relações entre classes.

No nosso caso, como `Jogador` é um tipo não primitivo, o atributo `jogadores`, que é um *array* de elementos do tipo `Jogador`, é apresentado através de uma seta na direção da classe `Jogador`; etiquetamos a seta com o nome do atributo e com o carácter `*`, que significa que cada objeto do tipo `Jogo` está relacionado com uma coleção de zero ou mais elementos do tipo `Jogador`.

É usual não representar explicitamente através de elementos gráficos UML os tipos de dados não primitivos pertencentes à biblioteca do Java – como, por exemplo, `String`, `Random`, etc – que não são criados por nós. Representamo-los através dos seus nomes.

A última divisória apresenta as assinaturas dos métodos. Optámos por apresentar somente os métodos `public` e por não apresentar nem os construtores nem os métodos que revelam os valores dos atributos.

12. Tipos enumerados

Suponha que, num jogo, as jogadas que os jogadores fazem podem ter efeitos diferentes, em vez de resultarem sempre na adição dos pontos da jogada à pontuação geral do jogador. O efeito poderá ser, por exemplo, a adição, a subtração ou a multiplicação dos pontos da jogada à/da/pela pontuação do jogador. Poderá ser ainda a substituição da pontuação do jogador pelos pontos dessa jogada ou por zero.

Uma abordagem **NÃO** satisfatória para podermos controlar o tipo de efeito que as jogadas devem ter em cada execução do programa, passa por usar uma variável cujo valor represente o efeito desejado. Podemos optar por declarar uma variável `efeito`, por exemplo, e dar-lhe valores 1, 2, 3, 4 ou 5 para representar cada efeito (1: Adição, 2: Subtração, 3: Multiplicação, 4: Substituição, 5: Nulo). Ou podemos optar por declarar a variável com o tipo `char` e usar as iniciais dos nomes que demos aos efeitos – 'A', 'S', 'M', 'U' ou 'N' – para representar cada uma delas (com o inconveniente de ter que evitar letras repetidas).

Esta abordagem, em qualquer das suas formas, tem três **inconvenientes**:

1. O programador tem que relembrar a relação que definiu entre efeitos e números (ou caracteres) sempre que precisa de trabalhar com essa informação, de modo a que as coisas façam sentido;
2. Sempre que usa um valor que não está no conjunto que definiu para os valores relevantes, o programa fica incorreto, embora o compilador não se “queixe”;
3. O programa torna-se pouco legível.

A linguagem Java permite-nos trabalhar com estes tipos de informação numa forma muito mais segura e que promove a legibilidade dos programas.

Os *enumerados* no Java permitem-nos definir tipos de dados não primitivos em que

- o conjunto de valores possíveis é conhecido à partida e
- tem um número reduzido de elementos.

Definimos agora o enumerado `EfeitoJogada` criando um ficheiro de nome `EfeitoJogada.java` (tal como fazemos para as classes) onde definimos todos os valores possíveis para este tipo de dados não primitivo:

```
public enum EfeitoJogada {  
    ADICAO, SUBTRACAO, MULTIPLICACAO, SUBSTITUICAO, NULO;  
}
```

Repare na palavra reservada `enum` que é usada em substituição da palavra `class`.

À semelhança de qualquer outro tipo de dados podemos, por exemplo, declarar variáveis deste novo tipo:

```
public class ClienteDeEfeitoJogada {  
    public static void main(String[] args) {  
        EfeitoJogada e1 = EfeitoJogada.ADICAO;  
        EfeitoJogada e2 = EfeitoJogada.SUBSTITUICAO;  
        EfeitoJogada e3 = EfeitoJogada.NULO;  
    }  
}
```

Os benefícios que obtemos pelo facto de usarmos o enumerado `EfeitoJogada` são:

1. O significado e valores das variáveis ficam fáceis de entender;
2. Não podemos atribuir a estas variáveis valores que não estejam definidos no enumerado `EfeitoJogada`; se o fizermos, o compilador assinalará um erro.

Os valores de um enumerado (neste caso, desde `ADICAO` até `NULO`) são constantes, mas também são objetos pois são de um tipo não primitivo.

Cada valor do enumerado está associado a um número desde zero até ao número de elementos do enumerado – 1; esse número é o seu *número de ordem* (neste caso, os números de ordem de `ADICAO` e `NULO` são, respetivamente, 0 e 4).

Há vários métodos que são acessíveis a todos os enumerados:

- `int ordinal()`, um método de instância que devolve o número de ordem do objeto sobre o qual é invocado;
- `String name()`, um método de instância que devolve o nome do objeto sobre o qual é invocado;
- `static EfeitoJogada[] values()`, um método de classe que devolve um *array* contendo todos os valores do enumerado na ordem pela qual foram definidos (no caso do enumerado `EfeitoJogada` retorna um *array* cujo primeiro elemento é o valor `ADICAO`, o segundo é `MULTIPLICACAO`, etc),

A execução das instruções do seguinte método `main`:

```
public class ClienteDeEfeitoJogada {

    public static void main(String[] args) {
        EfeitoJogada e1 = EfeitoJogada.ADICAO;
        EfeitoJogada e2 = EfeitoJogada.SUBSTITUICAO;
        System.out.println(e1.name() + " -- " + e1.ordinal());
        System.out.println(e2.name() + " -- " + e2.ordinal());
        System.out.println("=====");

        EfeitoJogada[] todosEfeitos = EfeitoJogada.values();
        for(EfeitoJogada est : todosEfeitos){
            System.out.println(est.name() + " -- " + est.ordinal());
        }
    }
}
```

Escreve as seguintes linhas no *standard output*:

```
ADICAO -- 0
SUBSTITUICAO -- 3
=====
ADICAO -- 0
SUBTRACAO -- 1
MULTIPLICACAO -- 2
SUBSTITUICAO -- 3
NULO -- 4
```

Repare, nas últimas linhas, como os nomes dos efeitos aparecem pela mesma ordem com que foram definidos no enumerado.

Para classificarmos cada jogada com o respetivo efeito temos que alterar o método `registarPontosJogada` da classe `Jogo` e o método `registarPontos` da classe `Jogador` apresentados no capítulo anterior. É necessário adicionar um parâmetro do tipo `EfeitoJogada` a esses métodos para receber o efeito da jogada a registar. De acordo com o valor desse parâmetro, os pontos a registar vão ter o efeito correspondente ao sentido que demos a cada um dos valores do enumerado.

No método `registarPontosJogada` da classe `Jogo` a mudança no corpo do método é pequena: basta acrescentar o valor do efeito na invocação do método `registarPontos` sobre o jogador em questão.

```

/**
 * Registrar a pontuacao, na jogada atual, de um dado jogador.
 * @param nomeJog O nome do jogador.
 * @param pontuacao A pontuacao obtida na jogada atual.
 * @param efeito O efeito que a jogada vai ter na pontuacao geral do jogador.
 * @requires this.estahEmJogo (nomeJog) &&
 *             pontuacao >= MIN_PONTUACAO_JOGADA &&
 *             pontuacao <= MAX_PONTUACAO_JOGADA && efeito != null
 */
public void registrarPontosJogada (String nomeJog, int pontuacao, EfeitoJogada efeito) {
    this.obterJogador(nomeJog).registrarPontos(pontuacao,efeito);
}

```

No método `registrarPontos` da classe `Jogador`, o valor do efeito define a mudança a aplicar à pontuação atual do jogador corrente – usamos uma instrução `switch`.

Em relação ao valor máximo numa só jogada, agora fazemos o cálculo da diferença entre a pontuação que o jogador tinha antes da jogada e a que tem após a jogada. É o valor dessa diferença que é comparado com o atual máximo numa só jogada.

```

/**
 * Registrar uma nova pontuacao, com um dado efeito, para este jogador
 * @param pontos Os pontos a acumular ah pontuacao deste jogador
 * @param efeito O efeito que a jogada vai ter na pontuacao geral do jogador.
 * @requires pontos >= 0 && efeito != null
 */
public void registrarPontos (int pontos, EfeitoJogada efeito) {
    int anterior = this.pontuacao;
    int registrar = pontos;
    switch (efeito){
        case ADICAO: this.pontuacao += registrar;
                     break;
        case SUBTRACAO: this.pontuacao -= registrar;
                       break;
        case MULTIPLICACAO: this.pontuacao *= registrar;
                           break;
        case SUBSTITUICAO: this.pontuacao = registrar;
                           break;
        case NULO: this.pontuacao = 0;
                  break;
    }
    // se teve um aumento de pontuacao maior que o que obteve ateh agora,
    // passa a ser esse o aumento de pontuacao maximo numa unica jogada
    int aumento = this.pontuacao - anterior;
    if (aumento > maximoJogada) {
        maximoJogada = aumento;
    }
}

```

O seguinte método `main` está de acordo com a nova versão do método `registrarPontosJogada` da classe `Jogo`. Além de gerar aleatoriamente os valores para as pontuações das sucessivas jogadas, gera também o efeito que cada jogada vai ter.

Faz isso gerando, a cada jogada de cada jogador, um inteiro entre 0 e 4 (o array `EfeitoJogada.values()` tem 5 elementos) e usa o efeito que está nessa posição do array `EfeitoJogada.values()` como valor para o parâmetro na invocação do método `registrarPontosJogada`.

```

import java.util.Random;

public class ClienteDeJogo {

    public static void main (String [] args) {

        Jogo meuJogo = new Jogo(30);
        String[] participantes = {"Maria","Pedro"};
        for (int i = 0 ; i < participantes.length ; i++) {
            meuJogo.juntarJogador(participantes[i]);
        }

        EfeitoJogada[] todosEfeitos = EfeitoJogada.values();
        Random gerador = new Random();
        int max = Jogo.MAX_PONTUACAO_JOGADA;
        int min = Jogo.MIN_PONTUACAO_JOGADA;
        // Quantos efeitos existem?
        int maxEfeito = todosEfeitos.length;

        // O jogo eh jogado em rondas.
        // Em cada ronda jogam todos os jogadores.
        while(!meuJogo.terminou()) {
            for (int j = 0 ; j < participantes.length ; j++) {
                // gera um valor no intervalo [min,max]
                int pontuacao = gerador.nextInt(max - min + 1) + min;
                // gera um valor no intervalo [0,maxEfeito] e usa esse valor
                // para obter um efeito aleatorio
                EfeitoJogada efeito =
                    todosEfeitos[gerador.nextInt(maxEfeito)];
                // registra a pontuacao do proximo jogador
                meuJogo.registarPontosJogada(participantes[j], pontuacao, efeito);
                System.out.println(participantes[j] + " fez jogada de " +
                    pontuacao + " pontos com efeito " + efeito.name());
            }
            System.out.println("=====");
            System.out.println(meuJogo);
        }

        System.out.println("Acabou o jogo!");

        String[] vencedores = meuJogo.vencedores1();
        for (int i = 0 ; i < vencedores.length ; i++) {
            System.out.println(vencedores[i] + " venceu com " +
                meuJogo.pontuacao(vencedores[i]) + " pontos");
        }
    }
}

```

O resultado de uma possível execução deste `main`:

```

Maria fez jogada de 3 pontos com efeito NULO
Pedro fez jogada de 4 pontos com efeito ADICAO
=====
Objetivo do jogo: 30 pontos
Numero de jogadores: 2
Nome: Maria    Pontuacao: 0    Maximo numa jogada: 0

```

```

Nome: Pedro      Pontuacao: 4      Maximo numa jogada: 4

Maria fez jogada de 7 pontos com efeito SUBTRACAO
Pedro fez jogada de 4 pontos com efeito ADICAO
=====
Objetivo do jogo: 30 pontos
Numero de jogadores: 2
Nome: Maria      Pontuacao: -7      Maximo numa jogada: 0
Nome: Pedro      Pontuacao: 8      Maximo numa jogada: 4

Maria fez jogada de 9 pontos com efeito ADICAO
Pedro fez jogada de 10 pontos com efeito SUBSTITUICAO
=====
Objetivo do jogo: 30 pontos
Numero de jogadores: 2
Nome: Maria      Pontuacao: 2      Maximo numa jogada: 9
Nome: Pedro      Pontuacao: 10     Maximo numa jogada: 4

Maria fez jogada de 1 pontos com efeito ADICAO
Pedro fez jogada de 5 pontos com efeito MULTIPLICACAO
=====
Objetivo do jogo: 30 pontos
Numero de jogadores: 2
Nome: Maria      Pontuacao: 3      Maximo numa jogada: 9
Nome: Pedro      Pontuacao: 50     Maximo numa jogada: 40

Acabou o jogo!
Pedro venceu com 50 pontos

```

Em capítulos mais à frente, quando falarmos em padrões de desenho, veremos que há formas melhores de abordar este assunto.

13. Interfaces

Além das classes e dos enumerados, a linguagem Java permite a definição de novos tipos de dados, não primitivos, através de *interfaces*.

Um interface

- declara um conjunto de métodos públicos (e não só, como veremos adiante) que definem, de forma coerente, um tipo de dados abstrato;
- este tipo de dados abstrato pode depois ser total ou parcialmente implementado por várias classes;
- define comportamentos que podem ser implementados por classes que não têm qualquer outra relação entre elas;
- pode caracterizar uma "capacidade" – serve para que as classes que o implementam anunciem que sabem fazer algo (exemplos: `Cloneable`, `Comparator`, `Iterable`)

13.1. Definição e implementação

Definimos um interface `I` criando um ficheiro de nome `I.java`, contendo as assinaturas dos métodos que definem esse novo tipo de dados. Um exemplo:

```
public interface I {  
  
    /**  
     * Documentacao do metodo m1  
     */  
    int m1 ();  
  
    /**  
     * Documentacao do metodo m2  
     */  
    void m2 ();  
  
}
```

Um interface não pode ser instanciado, como é óbvio. A seguinte instrução, numa qualquer classe cliente de `I`, está **errada**:

```
I x = new I();
```

Para declarar que uma classe `MyClass` **implementa** um interface `I`, usamos a palavra reservada `implements` no cabeçalho da classe:

```
public class MyClass implements I {  
    ... // implementacao dos metodos definidos em I e,  
        // possivelmente, outros tambem  
}
```

Quando usamos o Eclipse para criar uma nova classe que implementa um interface `I`, podemos escolher a opção de geração de *stubs* para os métodos declarados em `I`,

The screenshot shows the Eclipse IDE's 'New Class' wizard. The 'Name' field contains 'MyClass'. Under 'Modifiers', the 'public' radio button is selected. The 'Superclass' field is set to 'java.lang.Object'. In the 'Interfaces' section, 'I' is listed. At the bottom, under 'Which method stubs would you like to create?', the 'Inherited abstract methods' checkbox is checked, while 'public static void main(String[] args)' and 'Constructors from superclass' are unchecked.

é-nos apresentado o seguinte código já preparado para preencher:


```

public class MyClass implements I {

    @Override
    public int m1() {
        // TODO Auto-generated method stub
        return 0;
    }

    @Override
    public void m2() {
        // TODO Auto-generated method stub
    }

}

```

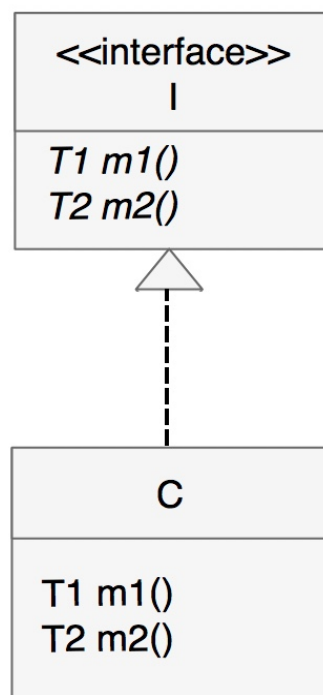
Temos então, de seguida, que declarar:

- os atributos que caracterizam o estado de objetos deste novo tipo `MyClass`,
- as implementações dos vários métodos,
- e, se necessário, poderemos definir novos métodos, não declarados no interface `I`.

13.2. Representação em UML

A forma de representar um interface é a mesma de uma classe – um retângulo – onde precedemos o nome do interface do estereótipo `<<interface>>`.

Para representar a relação de `implements` entre uma classe e um interface, usa-se uma seta a tracejado com a cabeça branca. Os nomes dos métodos não implementados no interface são escritos em itálico.



13.3. Subtipos

Já sabemos que as classes e os interfaces definem tipos.

Se uma classe `C` implementa um interface `I`, dizemos que

- `C` é *subtipo* de `I`,
- logo `I` é *supertipo* de `C`.

A noção de subtipo está na base das regras usadas pelo compilador para atribuição de valores a variáveis de tipos não primitivos (instruções de atribuição e instanciação de parâmetros aquando de invocação de métodos):

- numa instrução de atribuição `T x = expressao` o tipo de `expressao` tem que ser subtipo (que inclui ele próprio) de `T`;
- numa invocação `m(expressao)` de um método cuja assinatura é `T1 m(T2 param)`, o tipo de `expressao` tem que ser subtipo (que inclui ele próprio) de `T2`;

Logo, as instruções seguintes são corretas, pois o tipo das referências à direita das duas primeiras atribuições é subtipo do tipo das variáveis à esquerda dessas atribuições:

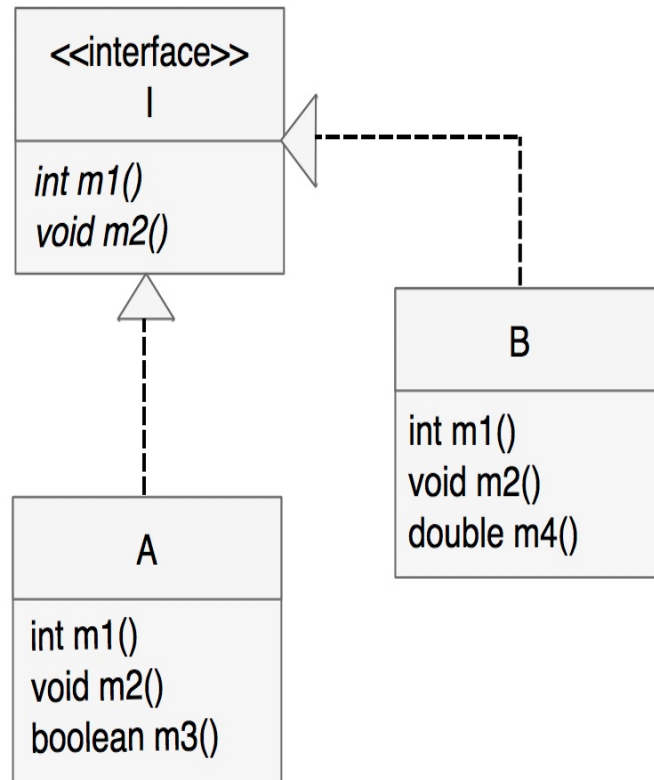
```
C x = new C();  
I y = new C();
```

Mais adiante voltaremos a este assunto para que compreenda as razões que justificam estas regras.

Considere o interface `I` definido na secção anterior. Suponhamos agora que existem

- uma classe `A` que implementa `I`, ou seja, que implementa os métodos `m1` e `m2`, e que acrescenta um outro método `m3`;
- uma classe `B` que implementa `I` e que implementa um novo método `m4`.

A figura seguinte representa um diagrama em UML das classes e interface e das suas relações.



Numa qualquer classe cliente das classes acima, podemos declarar variáveis às quais atribuímos referências para objetos dos tipos `A`, e `B` e invocar métodos sobre esses objetos:

```
public class ExemploCliente {

    public static void main (String [] args) {

        A x = new A();
        I y = new A();
        I[] vetorIs = new I[3];
        vetorIs[0] = new A();
        vetorIs[1] = new B();
        vetorIs[2] = new A();

        int soma = x.m1() + y.m1();
        x.m2();
        y.m2();
        if (x.m3()){
            System.out.println(y.toString());
        }
        for (int i = 0 ; i < vetorIs.length ; i++){
            soma += vetorIs[i].m1();
        }
        System.out.println(soma);
        vetorIs[0].m2();
        vetorIs[1].m2();
    }
}
```

Repare que:

- Como a variável `x` tem tipo declarado `A`, podemos invocar sobre ela todos os métodos acessíveis em `A`: `m1`,

`m2, m3;`

- Como a variável `y` tem tipo declarado `I`, podemos invocar sobre ela todos os métodos acessíveis em `I`: `m1`, `m2`; embora o objeto referenciado por `y` tenha o tipo real `A`, não é possível invocar sobre ele o método `m3`, pois não está definido em `I` nem em `Object`;
- Como a variável `vetorIs` tem tipo declarado `I[]`, então todos os seus elementos têm tipo declarado `I`; logo, podemos invocar sobre cada elemento de `vetorIs` todos os métodos acessíveis em `I`: `m1`, `m2`; embora o objeto referenciado por `vetorIs[0]` tenha o tipo real `A`, não é possível invocar sobre ele o método `m3`, pois não está definido em `I`; o mesmo se passa com `vetorIs[1]`: o objeto que refere tem o tipo real `B`, mas não é possível invocar sobre ele o método `m4`, pois não está definido em `I`.

13.4. O interface `Ganhavel`

Se for do nosso interesse ter programas

- que envolvam objetos de diversas naturezas,
- mas com algumas capacidades específicas em comum

e, nesses programas,

- poder aceder a essas capacidades comuns,
- independentemente da forma como são implementadas na classe de cada objeto, e
- independentemente de outras capacidades que cada um desses objetos possa ter,

então devemos

- criar um interface Java, definindo essas capacidades comuns que nos interessam,
- fazer com que as classes dos objetos que vamos usar implementem esse interface,
 - obtendo assim um supertipo comum a essas classes,
 - podendo assim manipular de forma uniforme todos esses objetos de tipos diferentes, fazendo-o através de variáveis/parâmetros cujo tipo é esse supertipo comum.

O seguinte interface define um tipo de dados – `Ganhavel` – que engloba um conjunto de capacidades que um objeto pode ter:

```
/**
 * Este interface define um tipo de dados representando uma atividade que tem
 * participantes com diferentes pontuacoes. Uma vez a atividade terminada,
 * podem ser conhecido(s) o(s) vencedor(es)
 */
public interface Ganhavel {

    /**
     * A designacao deste ganhavel.
     */
    public String designacao();

    /**
     * Os nomes dos vencedor(es) deste ganhavel
     * @return Os nomes dos participantes deste ganhavel que venceram
     * @requires this.terminou()
     */
    String [] vencedores();

    /**
     * Este ganhavel jah terminou?
     */
}
```

```

    */
    boolean terminou ();

    /**
     * Existe um participante com um dado nome neste ganhavel?
     * @param nome O nome do participante.
     * @return true se existe um participante com este nome neste ganhavel
     *         false cc
     */
    public boolean estahEmJogo (String nome);

    /**
     * A pontuacao de um dado participante deste ganhavel
     * @param nome O nome do participante.
     * @requires this.estahEmJogo(nome)
     * @return A pontuacao do participante com este nome
     */
    public int pontuacao (String nome);
}

```

Repare que a nossa classe `Jogo` da secção 11.8 já implementa todos estes métodos. No entanto, como `Jogo` não implementa `Ganhavel`, não seria possível usar um objeto do tipo `Jogo` para instanciar um parâmetro do tipo `Ganhavel`, nem atribuir a sua referência a uma variável do tipo `Ganhavel`.

Suponha que numa qualquer classe cliente de `Ganhavel`, definimos o método `imprimeInformacaoParticipante` abaixo listado, com um parâmetro do tipo `Ganhavel` e uma `String`, que invoca uma série de métodos sobre o objeto `Ganhavel` imprimindo a informação resultante no *standard output*.

```

/**
 * Imprime informacoes varias no standard ouput acerca de um ganhavel
 * e de um participante
 * @param g O ganhavel
 * @param nome O (presumido) participante
 * @requires g != null && nome != null
 */
static void imprimeInformacaoParticipante(Ganhavel g, String nome) {
    StringBuilder informacao = new StringBuilder();
    informacao.append("*** Informacao sobre " + g.designacao() +
        " e " + nome + " ***\n");
    if(g.estahEmJogo(nome)) {
        informacao.append(nome + " participa\n");
        if (g.terminou()) {
            informacao.append(g.designacao() + " jah terminou\n");
            String[] venceram = g.vencedores();
            int indiceNome = indice(venceram, nome);
            informacao.append(nome + (indiceNome == -1 ? " nao " : " "));
            informacao.append(" venceu e teve " + g.pontuacao(nome) +
                " pontos\n");
        } else {
            informacao.append(g.designacao() + " ainda nao terminou.\n");
            informacao.append(nome + " teve " + g.pontuacao(nome) +
                " pontos\n");
        }
    }
    else {
        informacao.append(nome + " nao participa\n");
    }
}

```

```

    }
    System.out.println(informacao.toString());
}

/**
 * Indice de um elemento num array
 * @param colecao O array
 * @param nome O valor a procurar
 * @return O indice do elemento no array, se existir; -1 caso contrario
 */
static int indice(String[] colecao, String nome) {

    for(int i = 0 ; i < colecao.length ; i++) {
        if (colecao[i].equals(nome)) {
            return i;
        }
    }
    return -1;
}

```

Para podermos usar um objeto do tipo `Jogo` como valor para o parâmetro `g` do tipo `Ganhavel`, teremos que declarar, na classe `Jogo`, que esta implementa o interface `Ganhavel`:

```

/**
 * Classe cujos objetos representam jogos
 * @author Isabel Nunes
 */
public class Jogo implements Ganhavel {
    ... //atributos e metodos tal como definidos na seccao 11.8
}

```

Vamos ainda supor que temos uma classe `ConcursoPublico` que também implementa o interface `Ganhavel`.

O seguinte programa trabalha com dois objetos – um do tipo `Jogo`, outro do tipo `ConcursoPublico` – que são também do tipo `Ganhavel` e, como tal, podem ser usados na invocação do método `imprimeInformacaoParticipante` para instanciar o seu primeiro parâmetro.

```

public class ClienteDeGanhavel {

    public static void main(String[] args) {

        Jogo meuJogo = new Jogo("Tiro ao Arco", 25);
        meuJogo.juntarJogador("Pedro");
        meuJogo.juntarJogador("Rita");
        meuJogo.registarPontosJogada("Pedro", 20);
        meuJogo.registarPontosJogada("Rita", 30);

        ConcursoPublico concurso = new ConcursoPublico("PR-1234");
        // algumas invocacoes de metodos sobre concurso...

        imprimeInformacaoParticipante(meuJogo, "Rui");
        imprimeInformacaoParticipante(meuJogo, "Pedro");
        imprimeInformacaoParticipante(meuJogo, "Rita");
        imprimeInformacaoParticipante(concurso, "Rita");
    }
}

```

```

    }

    /**
     * Imprime informacoes varias no standard ouput acerca de um
     * ganhavel e de um participante
     * @param g O ganhavel
     * @param nome O (presumido) participante
     */
    static void imprimeInformacaoParticipante(Ganhavel g, String nome) {
        StringBuilder informacao = new StringBuilder();
        informacao.append("*** Informacao sobre " + g.designacao() +
            " e " + nome + " ***\n");
        if(g.estaEmJogo(nome)) {
            informacao.append(nome + " participa\n");
            if (g.terminou()) {
                informacao.append(g.designacao() + " jah terminou\n");
                String[] venceram = g.vencedores();
                int indiceNome = indice(venceram, nome);
                informacao.append(nome + (indiceNome == -1 ? " nao " : " "));
                informacao.append(" venceu e teve " + g.pontuacao(nome) +
                    " pontos\n");
            } else {
                informacao.append(g.designacao() + " ainda nao terminou.\n");
                informacao.append(nome + " teve " + g.pontuacao(nome) +
                    " pontos\n");
            }
        }
        else {
            informacao.append(nome + " nao participa\n");
        }
        System.out.println(informacao.toString());
    }

    /**
     * Indice de um elemento num array
     * @param colecao O array
     * @param nome O valor a procurar
     * @return O indice do elemento no array, se existir; -1 caso contrario
     */
    static int indice(String[] colecao, String nome) {

        for(int i = 0 ; i < colecao.length ; i++) {
            if (colecao[i].equals(nome)) {
                return i;
            }
        }
        return -1;
    }
}

```

Quando executado, este programa produz o seguinte *output*:

```

*** Informacao sobre Tiro ao Arco e Rui ***
Rui nao participa

*** Informacao sobre Tiro ao Arco e Pedro ***

```

```

Pedro participa
Tiro ao Arco jah terminou
Pedro nao venceu e teve 20 pontos

*** Informacao sobre Tiro ao Arco e Rita ***
Rita participa
Tiro ao Arco jah terminou
Rita venceu e teve 30 pontos

*** Informacao sobre PR-1234 e Rita ***
Rita participa
PR-1234 ainda nao terminou.
Rita teve 23 pontos

```

Repare que, ao termos definido o parâmetro do método com tipo `Ganhavel` em vez de `Jogo`, obtivemos um método cuja utilização é mais abrangente, pois esse parâmetro pode ser instanciado por qualquer objeto cujo tipo real seja subtipo de `Ganhavel`. Se o tipo do parâmetro fosse `Jogo`, estaríamos a reduzir a aplicabilidade do método a objetos do tipo `Jogo`.

13.5. O interface genérico `List<E>`

Até agora, sempre que precisámos de representar coleções de objetos usámos `arrays`. Existem outros tipos não primitivos, definidos através de classes e interfaces na biblioteca do Java, que nos permitem representar coleções de objetos. Estas classes e interfaces compõem aquilo que usualmente se chama de *collection framework*.

Para já, vamos ver só o interface `List` e uma das suas várias implementações – a classe `ArrayList`, ambos do pacote `java.util`.

13.5.1. Porquê "genérico"?

Quando pensamos no conceito de lista, imaginamos uma coleção ordenada, uma sequência de elementos. Sobre essa coleção podemos adicionar elementos, retirar elementos, aceder a um elemento numa dada posição da lista, etc.

Se tivermos três listas – uma com elementos do tipo `Jogo`, outra com elementos do tipo `Integer` e ainda outra com elementos do tipo `Pessoa` –, percebemos que as operações atrás referidas não diferem pelo facto de os tipos dos elementos de cada lista serem diferentes, ou seja, o mecanismo das operações não depende do tipo específico dos elementos que a lista contém.

Então, ter três classes distintas que definem listas de elementos do tipo `Jogo`, do tipo `Integer` e do tipo `Pessoa`, respetivamente, seria um desperdício de tempo e espaço, pois elas seriam todas iguais excetuando no tipo de retorno ou de parâmetro de um ou outro método (como, por exemplo, do método que acede ao elemento numa dada posição da lista e do método que insere um elemento na lista).

Em vez disso, define-se o tipo genérico `List<E>` que abstrai o tipo dos elementos da lista:

Em Java, o interface `List<E>` define

- o tipo de dados *lista de elementos de um tipo `E` qualquer*, ou seja,
- as operações que qualquer classe que implemente uma coleção ordenada de elementos deverá suportar, qualquer que seja o tipo dos seus elementos.

13.5.2. Usando listas

Alguns dos métodos definidos no interface `List<E>` (<https://docs.oracle.com/javase/9/docs/api/java/util/List.html>):

- `void add(int index, E element)` que insere o objeto `element` na posição de índice `index` da lista
- `boolean add(E element)` que adiciona o elemento `element` no fim da lista
- `boolean contains(Object o)` que retorna `true` se a lista contém o elemento `o`
- `E get(int index)` que retorna o elemento que está na posição de índice `index` na lista
- `int indexOf(Object o)` que retorna o índice da 1ª ocorrência do elemento `o` na lista, ou -1 se não existe
- `boolean isEmpty()` que retorna `true` se a lista não tem elementos
- `E remove(int index)` que remove o elemento na posição de índice `index`
- `boolean remove(Object o)` que remove a primeira ocorrência do elemento `o` na lista, se presente
- `E set(int index, E element)` que substitui o elemento na posição de índice `index` por `element`
- `int size()` que retorna o número de elementos na lista

O primeiro elemento de uma lista `myList` é o que está na posição de índice zero. O último elemento está na posição de índice `myList.size()-1`

Uma das classes que implementam o interface `List<E>` é a classe `ArrayList<E>` (<https://docs.oracle.com/javase/9/docs/api/java/util/ArrayList.html>).

Um tipo genérico pode ter mais que uma variável de tipo. Um exemplo é o interface `Map<K, V>` do pacote `java.util`.

Para podermos usar uma lista num programa Java teremos que instanciar a variável de tipo `E`. Somente tipos não primitivos podem ser usados para instanciar as variáveis de tipo de um tipo genérico.

Por exemplo, se eu quiser usar uma lista de inteiros no meu programa, terei que instanciar a variável de tipo com `Integer`, que é uma classe *wrapper* (embrulho) dos inteiros.

Um parentesis:

Tipos primitivos: *autoboxing*

Cada tipo primitivo tem uma classe correspondente (chamada *wrapper*) e as conversões entre ambas são feitas de forma automática:

- Num programa com as seguintes declarações de variáveis:

```
Integer wrap;  
int i;
```

- as seguintes instruções têm o mesmo efeito:

```
wrap = 17;  
wrap = Integer.valueOf(17);
```

- e as seguintes também:

```
i = wrap;  
i = wrap.intValue();
```

Voltando agora à utilização de tipos genéricos. O seguinte programa declara, cria e usa uma lista de inteiros:

```
import java.util.ArrayList;
```

```

import java.util.Random;

public class ExemploClienteDeList {

    public static void main(String[] args) {

        ArrayList<Integer> listaInts = new ArrayList<Integer>();
        Random gerador = new Random();
        int valor = gerador.nextInt(10);
        while(valor != 9) {
            listaInts.add(valor);
            valor = gerador.nextInt(10);
        }

        for (Integer val : listaInts) {
            System.out.print(val + " ");
        }
        System.out.println();

        int indice = listaInts.indexOf(2);
        if(indice != -1) {
            System.out.println("O valor 2 foi o " + (indice + 1) +
                               "º valor a ser gerado");
        } else {
            System.out.println("O valor 2 nao foi gerado");
        }
    }
}

```

Um possível *output* deste programa:

```

7 6 0 2 3 8 6 6 2 1 2 6 4 8 1 2 0 3
O valor 2 foi o 4º valor a ser gerado

```

Agora que já conhece os tipos `List` e `ArrayList`, antes de passarmos para o próximo capítulo aproveitamos para modificar o tipo de retorno do método `vencedores` do interface `Ganhavel` para retornar uma `List<String>` em vez de um *array* de `String`. Irá notar que a nova classe `Jogo` no próximo capítulo irá beneficiar do uso mais intensivo destes dois tipos de dados.

```

/**
 * Este interface define um tipo de dados representando uma atividade que tem
 * participantes com diferentes pontuacoes. Uma vez a atividade terminada,
 * podem ser conhecido(s) o(s) vencedor(es)
 */
public interface Ganhavel {

    /**
     * A designacao deste ganhavel.
     */
    public String designacao();

    /**
     * Os nomes dos vencedor(es) deste ganhavel
     * @return Os nomes dos participantes deste ganhavel que venceram

```

```

    * @requires this.terminou()
    */
    List<String> vencedores();

    /**
     * Este ganhavel jah terminou?
     */
    boolean terminou ();

    /**
     * Existe um participante com um dado nome neste ganhavel?
     * @param nome O nome do participante.
     * @return true se existe um participante com este nome neste ganhavel
     *         false cc
     */
    public boolean estahEmJogo (String nome);

    /**
     * A pontuacao de um dado participante deste ganhavel
     * @param nome O nome do participante.
     * @requires this.estahEmJogo(nome)
     * @return A pontuacao do participante com este nome
     */
    public int pontuacao (String nome);
}

```

14. Herança

Suponha agora que pretende representar jogos cujo critério de terminação não é necessariamente o mesmo que definimos na classe `Jogo`. Por exemplo, queremos poder representar jogos que terminam de uma das seguintes formas:

- TODOS os jogadores alcançam a pontuação objetivo;
- algum jogador tem EXATAMENTE a pontuação objetivo;
- exatamente ao fim de um dado número de jogadas efetuadas;
- algum jogador alcança o objetivo OU um dado número máximo de jogadas foi feito.

Tal como podemos ter critérios de terminação diferentes também podemos ter, associado a cada um deles, uma forma específica de determinar o(s) vencedor(es).

Como podemos alcançar isto?

Uma solução de que NÃO gostamos

Uma abordagem que "funciona" mas que é fraca do ponto de vista da qualidade do *software*, é associar a cada objeto do tipo `Jogo` mais um atributo que define o tipo de jogo, no que diz respeito a terminação e cálculo de vencedor.

Podemos usar um enumerado, por exemplo, para definir os vários tipos de jogo que queremos representar:

```
public enum Terminacao {
    TODOS_ALCANCAM, EXATAMENTE, JOGADAS_FEITAS, OBJETIVO_OU_JOGADAS;
}
```

Usamos agora o enumerado `Terminacao` e modificamos a classe `Jogo` onde necessário:

```
public class Jogo {

    ...

    // atributos que definem este jogo:
    // A colecao dos jogadores
    ArrayList<Jogador> jogadores;
    // Pontuacao objetivo
    private int objetivo;
    // Tipo de terminacao
    private Terminacao termina;
    // Numero maximo de jogadas
    private int maximoJogadas;
    // Numero de jogadas jah feitas
    private int jogadas;

    /**
     * Construir um jogo.
     * @param objetivo A pontuacao com que se ganha este jogo
     * @param tipoTerm O tipo de terminacao deste jogo
     * @param maxJogadas O numero maximo de jogadas deste jogo (se se aplicar)
     * @requires tipoTerm != null
     */
    public Jogo (int objetivo, Terminacao tipoTerm, int maxJogadas) {
        this.objetivo = objetivo;
        this.jogadores = new ArrayList<Jogador>();
        this.termina = tipoTerm;
        this.maximoJogadas = maxJogadas;
        this.jogadas = 0;
    }

    ...

    /**
     * O numero de jogadores que estao em jogo.
     */
    public int quantosEmJogo () {
        return this.jogadores.size();
    }

    /**
     * Este jogo jah terminou?
     */
    public boolean terminou () {
        boolean resultado = false;
        switch (this.termina){
            case TODOS_ALCANCAM:
                resultado = this.quantosAlcancaram() == this.quantosEmJogo();
                break;
            case EXATAMENTE:
```

```

        resultado = this.algumExatamente();
        break;
    case JOGADAS_FEITAS:
        resultado = this.jogadas == this.maximoJogadas;
        break;
    case OBJETIVO_OU_JOGADAS:
        resultado = this.quantosAlcancaram() >= 1 ||
                    this.jogadas >= this.maximoJogadas;
        break;
    }

    return resultado;
}

/**
 * Quantos jogadores jah alcancaram o objetivo deste jogo?
 */
private int quantosAlcancaram() {
    int quantos = 0;
    for(Jogador jog : this.jogadores) {
        if(jog.pontuacao() >= this.objetivo) {
            quantos++;
        }
    }
    return quantos;
}

/**
 * Algum jogador tem exatamente a pontuacao objetivo deste jogo?
 */
private boolean algumExatamente() {
    for(Jogador jog : this.jogadores) {
        if(jog.pontuacao() == this.objetivo) {
            return true;
        }
    }
    return false;
}

...
}

```

Tivemos que adicionar três novos atributos: o tipo de terminação, o número máximo de jogadas e o número de jogadas já feitas. Estes dois últimos são necessários para os casos em que o critério de terminação depende (também) do número de jogadas já feitas.

No construtor tivemos que adicionar dois novos parâmetros para obter os valores para o tipo de terminação do novo jogo e o número máximo de jogadas (quando se aplica). Repare que na pré-condição não impomos qualquer restrição nem ao valor do objetivo nem ao número máximo de jogadas, pois em alguns casos os seus valores não são relevantes e podem ser quaisquer.

Finalmente, alterámos o método `terminou` para acondicionar os vários tipos de critérios de terminação. Criámos também dois métodos auxiliar `private` - `quantosAlcancaram` e `algumExatamente` - que calculam o número de jogadores em jogo que alcançaram o objetivo e se algum jogador tem o valor da pontuação igual ao objetivo, respetivamente, pois esses valores são necessários para decidir a terminação.

Pergunta: Porque é que não gostamos desta solução?

Resposta: Porque não é extensível, ou seja, é difícil de manter e fazer evoluir.

Esta solução está extremamente dependente das formas de terminação existentes. Durante a "vida" do nosso *software*, sempre que quisermos acrescentar uma nova forma de terminação temos que alterar o código da classe `Jogo` e, eventualmente, o código de classes cliente desta (por exemplo, quando temos que adicionar novos parâmetros no construtor).

Isso é negativo em vários aspetos:

- Qualquer alteração de código existente pode introduzir erros em código que não os tinha;
- Podemos nem conseguir fazer essas alterações por não termos permissão para tal;
- Qualquer alteração que obrigue também a alterações no código de classes cliente já existentes é obviamente negativa;
- A existência de várias versões da classe `Jogo`, correspondentes a cada nova alteração, é um foco de instabilidade que devemos evitar.

Além de tudo isto, temos também parâmetros do construtor e atributos de instância que só são relevantes para alguns tipos de terminação, o que não faz muito sentido.

Agora sim, vamos começar

Nos capítulos anteriores, as classes `Jogador` e `Jogo` foram sendo desenvolvidas com o intuito de ilustrar uma série de conceitos novos em Programação Orientada a Objetos.

Novos pormenores acerca desses conceitos foram sendo introduzidos a pouco e pouco, e só agora temos uma visão mais geral e completa do que pretendemos representar.

Porque é preciso saber **o que** representar **antes** de decidir **como** representar, vamos agora repensar a representação dos objetos e dos seus comportamentos, de uma forma mais estruturada.

A nova classe `Jogo`

Com o que já sabemos acerca dos jogos que queremos representar, podemos concluir que o seguinte se aplica a qualquer um:

- tem jogadores registados;
- permite registar as pontuações que os jogadores em jogo vão obtendo;
- permite a qualquer altura saber as pontuações gerais e máximas numa só jogada dos jogadores em jogo;
- termina;
- tem zero ou mais vencedores.

É a partir desta informação que vamos agora construir a classe `Jogo`; de seguida vamos aprender a representar as variantes de que falámos anteriormente (e outras que ainda não estamos a prever) como *especializações* diferentes de um mesmo conceito – o `Jogo`.

Começemos por criar a nossa classe `Jogo`, com tudo o que é comum a todos os jogos que queremos representar.

```
import java.util.ArrayList;
import java.util.List;

/**
 * Classe cujos objetos representam jogos
 * @author Isabel Nunes
 */
public class Jogo implements Ganhavel {
```

```

// atributos que definem este jogo:
// Designacao
private String designacao;
// A colecao dos jogadores
private ArrayList<Jogador> jogadores;

/**
 * Inicializar um jogo.
 * @param designacao A designacao do jogo.
 * @requires designacao != null
 */
public Jogo (String designacao) {
    this.designacao = designacao;
    this.jogadores = new ArrayList<Jogador>();
}

/**
 * A designacao deste jogo.
 */
public String designacao () {
    return this.designacao;
}

/**
 * O numero de jogadores que estao em jogo.
 */
public int quantosEmJogo () {
    return this.jogadores.size();
}

/**
 * Juntar um jogador com um dado nome a este jogo.
 * @param nomeJog O nome do jogador.
 * @requires !this.estahEmJogo(nomeJog)
 */
public void juntarJogador (String nomeJog) {
    this.jogadores.add(new Jogador (nomeJog));
}

/**
 * Retorna o jogador correspondente a um nome, ou null se nao houver.
 * Metodo privado
 * @param nomeJog O nome do jogador.
 * @returns O jogador com este nome se existir no jogo;
 *         null caso contrario
 */
private Jogador obterJogador (String nomeJog) {
    for (Jogador jog : this.jogadores){
        if (nomeJog.equals(jog.nome())){
            return jog;
        }
    }
    return null;
}

/**

```

```

    * Registrar a pontuacao, na jogada atual, de um dado jogador.
    * @param nomeJog O nome do jogador.
    * @param pontuacao A pontuacao obtida na jogada atual.
    * @requires this.estaEmJogo (nomeJog)
    */
    public void registrarPontosJogada (String nomeJog, int pontuacao) {
        this.obterJogador(nomeJog).registrarPontos(pontuacao);
    }

    /**
     * Um dado jogador esta em jogo?
     * @param nomeJog O nome do jogador.
     * @return true se o jogador com este nome esta inscrito neste jogo
     *         false caso contrario
     */
    public boolean estaEmJogo (String nomeJog) {
        return this.obterJogador(nomeJog) != null;
    }

    /**
     * A pontuacao de um dado jogador deste jogo
     * @param nomeJog O nome do jogador.
     * @return A pontuacao do jogador com este nome
     * @requires this.estaEmJogo (nomeJog)
     */
    public int pontuacao (String nomeJog) {
        return this.obterJogador(nomeJog).pontuacao();
    }

    /**
     * A pontuacao maxima numa soh jogada de um dado jogador deste jogo
     * @param nomeJog O nome do jogador.
     * @return A pontuacao maxima numa soh jogada do jogador com este nome
     * @requires this.estaEmJogo (nomeJog)
     */
    public int maximoNumaJogada (String nomeJog) {
        return this.obterJogador(nomeJog).maximoNumaJogada();
    }

    /**
     * Este jogo jah terminou?
     */
    public boolean terminou () {
        ?????
    }

    /**
     * Vencedor(es) deste jogo
     * @return O nome do jogador que teve melhor pontuacao. No caso de
     *         haver varios, aquele que teve a pontuacao mais alta numa
     *         soh jogada. No caso de haver varios, retorna os seus nomes
     * @requires this.quantosEmJogo() > 0 && this.terminou()
     */
    public List<String> vencedores () {
        // Os jogadores que tiveram a melhor pontuacao ex-aequo
        List<Jogador> melhores = this.osMelhores();
        // calcular o maior dos maximos numa soh jogada

```



```

        // o(s) jogador(es) que o obteve(iram) serah(ao) o(s) vencedor(es)
        int maximo = melhores.get(0).maximoNumaJogada();
        for (Jogador jog : melhores) {
            int pontos = jog.maximoNumaJogada();
            if (pontos > maximo) {
                maximo = pontos;
            }
        }
        List<String> vencedores = new ArrayList<String>();
        for (Jogador jog : melhores) {
            if (jog.maximoNumaJogada() == maximo) {
                vencedores.add(jog.nome());
            }
        }
        return vencedores;
    }

    /**
     * Os jogadores que teem a maior pontuacao
     * @return Os jogadores que teem a maior pontuacao
     * @requires this.quantosEmJogo() > 0
     */
    private List<Jogador> osMelhores () {
        // Qual a melhor pontuacao?
        int maiorPontuacao = this.maiorPontuacao();
        ArrayList<Jogador> melhoresJogadores = new ArrayList<Jogador>();
        for (Jogador jog : this.jogadores) {
            if (jog.pontuacao() == maiorPontuacao) {
                melhoresJogadores.add(jog);
            }
        }
        return melhoresJogadores;
    }

    /**
     * A maior pontuacao dos jogadores deste jogo
     * @return O valor da maior pontuacao
     * @requires this.quantosEmJogo() > 0
     */
    public int maiorPontuacao () {
        int maiorPontuacao = 0;
        for (Jogador jog : this.jogadores) {
            int pontos = jog.pontuacao();
            if (pontos > maiorPontuacao) {
                maiorPontuacao = pontos;
            }
        }
        return maiorPontuacao;
    }

    /**
     * Os nomes dos jogadores com pontuacao superior a um dado valor
     * @param limiteInf O valor de comparacao.
     * @requires this.quantosEmJogo() > 0
     * @return Uma lista com os nomes dos jogadores deste jogo com
     *         pontuacao superior a limiteInf
     */

```

```

    public List<String> pontuacaoSuperiorA (int limiteInf) {
        List<String> melhores = new ArrayList<String>();
        for (Jogador jog : this.jogadores){
            if (jog.pontuacao() > limiteInf){
                melhores.add(jog.nome());
            }
        }
        return melhores;
    }

    /**
     * Representacao textual deste jogo.
     */
    public String toString () {
        StringBuilder sb = new StringBuilder();
        sb.append("Designacao: " + this.designacao() + "\n");
        sb.append("Numero de jogadores: " + this.quantosEmJogo() + "\n");
        for (Jogador jog : this.jogadores){
            sb.append(jog.toString() + "\n");
        }
        return sb.toString();
    }
}

```

Tenha em atenção os seguintes pontos:

- Decidimos que não existe limite no número máximo de jogadores permitido em cada jogo; usamos uma lista, em vez de um *array*, para representarmos a coleção de jogadores em jogo;
- Como nem todos os jogos em que pensámos envolvem uma pontuação objetivo, não temos nenhum atributo correspondente;
- Os métodos que devolvem coleções de objetos têm `List` como tipo de retorno;
- A classe `Jogador` que aqui é usada é a do capítulo 10, a que não envolve variados efeitos de jogadas;
- Todos os métodos que envolvem jogadores, pontuações e jogadas, estão implementados;
- O método `vencedores` também está implementado: considerámos, como anteriormente, que vence o jogador que tem a maior pontuação geral e, em caso de haver mais do que um, os que também têm o maior valor de máximo numa só jogada;
- Finalmente:
 - Pergunta: como implementar o método `terminou`?
 - Resposta: depende! Já vimos que queremos poder variar o critério de terminação, entre os vários que elencámos na secção anterior, ou até ainda outro que não estamos a prever para já. Por isso não implementámos o método!

```

    /**
     * Este jogo jah terminou?
     */
    public boolean terminou () {
        ??????
    }

```

- Mas o compilador dará erro se deixarmos isto assim...

Antes de resolvermos este problema da definição do método `terminou` na classe `Jogo`, vamos ver como implementar os vários tipos de jogos que pretendemos – através da *Herança* – de forma a obter *software* que é extensível.

14.1. Uma subclasse de `Jogo`

Queremos poder implementar jogos que têm:

- várias facetas em comum
 - tudo o que está definido na classe `Jogo`, (exceto a terminação, claro, que ficou por definir),
- mas também algumas características e comportamentos diferentes
 - uns têm pontuação objetivo,
 - outros têm número máximo de jogadas,
 - o critério de terminação é diferente,
 - etc.

O que queremos definir são *especializações* do conceito de `Jogo`.

Vamos fazê-lo através do mecanismo de *herança*:

- permite introduzir extensões no comportamento e estado definidos por uma classe;
- permite evitar duplicação de código;
- define relações de *subclasse* e *superclasse*.

Vamos construir uma subclasse da classe `Jogo` – chamemos-lhe `JogoComObjetivo` – em que o critério de terminação é o seguinte:

- pelo menos um jogador tem uma pontuação igual ou superior à pontuação objetivo.

Começamos por apresentar uma possível implementação. Nas secções seguintes iremos abordando um a um os vários pormenores que são fundamentais para a construção de subclasses.

```
/**
 * Representa jogos com o objetivo de alcançar uma dada pontuacao
 * @author Isabel Nunes
 */
public class JogoComObjetivo extends Jogo {

    // Pontuacao objetivo
    private int objetivo;

    /**
     * Construir um jogo.
     * @param designacao A designacao do jogo.
     * @param objetivo A pontuacao com que se ganha este jogo
     * @requires designacao != null
     */
    public JogoComObjetivo (String designacao, int objetivo) {
        super(designacao);
        this.objetivo = objetivo;
    }
}
```

```

    /**
     * A pontuacao objetivo definida para este jogo.
     */
    public int pontuacaoObjetivo () {
        return this.objetivo;
    }

    /**
     * Este jogo jah terminou?
     */
    @Override
    public boolean terminou() {
        return this.maiorPontuacao() >= this.objetivo;
    }
    ...
}

```

14.2. A relação subclasse/superclasse

Repare no cabeçalho da classe:

```
public class JogoComObjetivo extends Jogo
```

A palavra `extends` diz ao compilador que esta é uma *subclasse* da classe `Jogo`, ou seja:

- todos os métodos e atributos definidos na classe `Jogo` também fazem parte, implicitamente, da classe `JogoComObjetivo`; não precisamos de voltar a escrevê-los quando definimos a subclasse;
- como consequência, as classes cliente de `JogoComObjetivo` poderão aceder:
 - a todos os métodos/atributos declarados na classe `Jogo` com acessibilidade `public`;
 - a todos os métodos/atributos declarados na classe `Jogo` com acessibilidade `package`, desde que sejam classes cliente pertencentes ao mesmo *package* que `Jogo`;
- no corpo de qualquer método definido na classe `JogoComObjetivo`, podemos aceder:
 - a todos os métodos/atributos declarados na classe `Jogo` com acessibilidade `public`;
 - a todos os métodos/atributos declarados na classe `Jogo` com acessibilidade `protected`;
 - a todos os métodos/atributos declarados na classe `Jogo` com acessibilidade `package`, desde que `JogoComObjetivo` pertença ao mesmo *package* que `Jogo`.

O seguinte método `main`, numa qualquer classe cliente de `JogoComObjetivo`, acede não só ao construtor dessa classe e ao método `pontuacaoObjetivo` nela definidos, como também aos métodos `juntarJogador`, `registarPontosJogada` e `estahEmJogo` definidos na classe `Jogo` e "herdados" pela subclasse:

```

public static void main (String [] args) {

    JogoComObjetivo meuJogo = new JogoComObjetivo("Spiral",30);
    System.out.println("O objetivo eh " + meuJogo.pontuacaoObjetivo());
    meuJogo.juntarJogador("Maria");
    meuJogo.registarPontosJogada("Maria", 13);

    System.out.println("Maria estah em jogo? " +
                       meuJogo.estahEmJogo("Maria"));
    System.out.println("Pontuacao Maria: " + meuJogo.pontuacao("Maria"));
}

```

Se o executarmos, obtemos o *output*:

```

O objetivo eh 30
Maria estah em jogo? true
Pontuacao Maria: 13

```

No Java, a relação de subclasse/superclasse é realizada através da palavra `extends`:

```

class B extends A {                // B é subclasse de A
    ...                             //      e
}                                  // A é superclasse de B

```

No Java,

- toda e qualquer classe é subclasse da classe `Object`.
- a classe `Object` é superclasse de toda e qualquer classe.

Então porque é que os cabeçalhos das classes que temos construído ao longo deste curso não contêm as palavras `extends Object`? Porque não é necessário. Quando nada é dito, o compilador assume que a classe herda diretamente da classe `Object`.

No Java, a herança é *simples*:

- uma classe só pode ter uma superclasse direta;
- mas pode ter várias subclasses diretas.

Existem outras linguagens orientadas a objetos que suportam herança múltipla mas o Java não o permite.

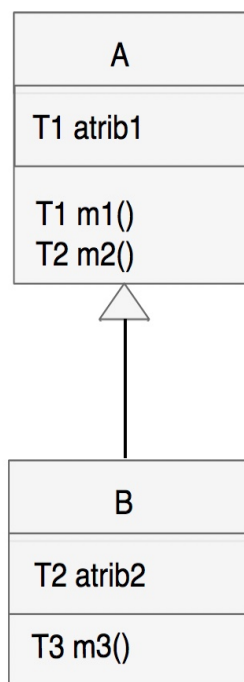
A herança é *transitiva*:

- se uma classe C é subclasse de B e se B é subclasse de A, então C é subclasse de A.

Percebe-se melhor agora a frase "Toda e qualquer classe é subclasse da classe `Object`". A classe `JogoComObjetivo` estende diretamente a classe `Jogo`; esta, por sua vez, estende diretamente a classe `Object`, pois não declara no seu cabeçalho que estende alguma outra classe. Logo, a classe `JogoComObjetivo` é subclasse de `Object`, herdando, por isso, também, os vários métodos nesta definidos (`toString`, `equals`, `hashCode`, etc), como mais adiante estudaremos.

Representação em UML

Para representar a relação de `extends` entre duas classes, usa-se uma seta com a cabeça branca.



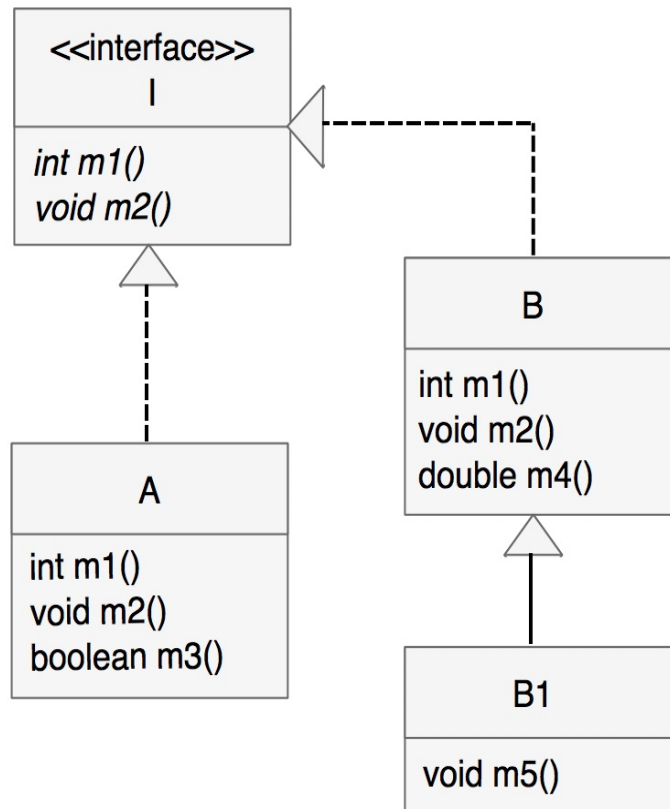
14.3. Subtipos

Já sabemos que uma classe define implicitamente um *tipo*. O tipo tem o mesmo nome da classe.

- Se a classe `B` estende a classe `A`, então o tipo `B` é um *subtipo* de `A`;
- Esta relação é transitiva, ou seja:
 - Se `B` é subtipo de `A`, e `C` é subtipo de `B`, então `C` é subtipo de `A`;
- A relação de subtipo define uma hierarquia de tipos que tem como raiz o tipo `Object`.

Repare que, pelo facto da classe `Jogo` implementar o interface `Ganhavel`, também a classe `JogoComObjetivo` o implementa, ou seja, `JogoComObjetivo` é subtipo de `Ganhavel`.

Relembremos o exemplo da secção 13.3. Se acrescentarmos uma classe `B1` que estende `B`, implementando um novo método `m5`, a representação em UML das classes e do interface é ilustrada na figura:



Já vimos na secção 13.3, que a forma como os tipos se organizam é um ingrediente fundamental na *análise estática de tipos*, executada pelo compilador

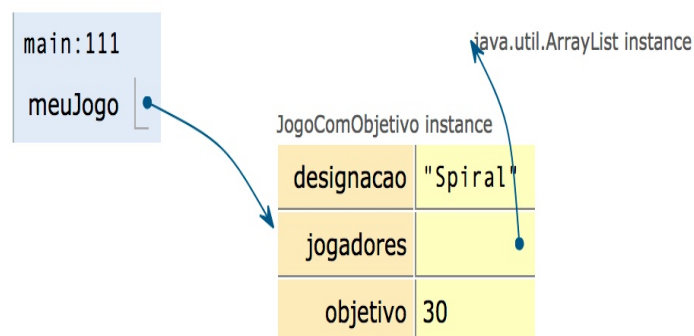
- toda a análise assenta não na relação de igualdade de tipos, mas na relação de subtipo, como analisaremos mais à frente.

14.4. Herança e construtores

De acordo com a secção anterior, um objeto do tipo `JogoComObjetivo` tem três atributos de instância:

- o inteiro `objetivo` definido na própria classe `JogoComObjetivo` e
- a String `designacao` e o array `jogadores`, herdados da classe `Jogo`.

A imagem seguinte representa o estado pretendido de um objeto do tipo `JogoComObjetivo` acabado de criar (com valor inicial de "Spiral" para a designação e para o objetivo de 30).



Relembre, da secção 10.3., o que acontece quando uma instrução de criação de um objeto é executada:

A criação de uma instância de uma classe é feita em três passos:

1. reserva e afetação de memória para o objeto (contando com o espaço a ocupar por cada um dos seus atributos);
2. inicialização desses atributos com *valores por omissão*:
 - o zero para atributos de tipos primitivos numéricos,
 - o `false` para `boolean`, e
 - o `null` para atributos cujo tipo não é primitivo;
3. invocação do construtor indicado, sobre o objeto acabado de criar.

O objetivo principal dos construtores é o de inicializar os atributos de um objeto acabado de criar com valores diferentes dos valores por omissão.

Se o papel de um construtor declarado numa classe é o de inicializar os atributos nela definidos com valores específicos, então o construtor da classe `JogoComObjetivo` deveria inicializar todos os atributos do novo objeto.

Mas os dois atributos herdados estão definidos como `private` na classe `Jogo`, ou seja, eles só estão acessíveis a partir de código da própria classe `Jogo`. Então, como fazer para inicializar esses atributos a partir do construtor `JogoComObjetivo`?

Esse é o papel da instrução `super(designacao);` no construtor da classe `JogoComObjetivo`:

```
/**
 * Construir um jogo.
 * @param designacao A designacao do jogo.
 * @param objetivo A pontuacao com que se ganha este jogo
 * @requires designacao != null
 */
public JogoComObjetivo (String designacao, int objetivo) {
    super(designacao);
    this.objetivo = objetivo;
}
```

O que esta instrução faz é invocar, sobre o objeto acabado de criar, o construtor definido na superclasse – neste caso, na classe `Jogo` – para que os atributos aí definidos possam ser inicializados adequadamente.

Neste caso, como o construtor da classe `Jogo` tem um parâmetro – a designação – a instrução `super(...)` deve fornecer um valor para esse parâmetro.

Como é suposto esse valor ser decidido, para cada jogo, por quem cria o objeto, então o construtor da subclasse também requer um parâmetro com esse significado, para além de um parâmetro que recebe o valor da pontuação objetivo.

Este construtor usa o valor do primeiro parâmetro na invocação do construtor da sua superclasse e usa o valor do segundo parâmetro para inicializar o atributo `objetivo`.

No fim da execução do construtor da classe `JogoComObjetivo`, o novo objeto já terá todos os seus atributos inicializados (tanto os herdados como aquele que a própria classe definiu).

No Java, qualquer construtor definido explicitamente numa subclasse tem como primeira instrução, obrigatoriamente, a invocação de um construtor da sua superclasse. Esta instrução pode ser explícita ou implícita (mais adiante).

Como já exemplificado, uma forma de invocar o construtor da superclasse, a partir do corpo do construtor da subclasse, é através de uma instrução da forma `super(...)`, onde `...` representa os valores para os parâmetros desse construtor:


```
public NomeClasse (...) {
    super(...);
    // Seguem-se outras instrucoes, se necessario
}
```

Deve lembrar-se de que os construtores das classes `Jogador` e `Jogo` (capítulos 10 e 11, respetivamente) não seguem esta regra... estes construtores aparentemente não definem como primeira instrução (nem nenhuma das seguintes) uma que invoque o construtor da sua superclasse. No entanto estas classes têm uma superclasse: a classe `Object`.

Quando a primeira instrução de um construtor não é uma invocação de um construtor da superclasse,

- o compilador acrescenta implicitamente a instrução `super()`, ou seja, a invocação de um construtor da superclasse que não tem argumentos.

Se fizéssemos isso no caso da classe `JogoComObjetivo`,

```
public JogoComObjetivo (int objetivo) {
    this.objetivo = objetivo;
}
```

o compilador iria acrescentar, como primeira instrução, a instrução `super();`.

Mas... na classe `Jogo` não existe nenhum construtor sem parâmetros. Só existe um construtor que tem um parâmetro do tipo `String`. Sabendo isto, o compilador daria erro.

Voltando aos construtores das classes `Jogador` e `Jogo`: como não têm uma primeira instrução em que é invocado um construtor da sua superclasse (que é `Object`, como já vimos), o compilador vai acrescentar a instrução `super();`. Aqui não há problema pois a classe `Object` tem um único construtor, o qual não tem parâmetros.

14.5. Invocação de métodos herdados

Já sabe que, durante a execução de um método, o *objeto corrente*, ou `this`, é o objeto sobre o qual o método foi invocado.

Qualquer que seja o "sítio" onde o método foi declarado – a classe da qual o objeto é instância ou uma superclasse dessa – a regra é a mesma: o `this` é o objeto sobre o qual o método foi invocado.

No seguinte `main` é criado um objeto do tipo `JogoComObjetivo`. De seguida são invocados sobre esse objeto uma série de métodos. Uns estão definidos na classe `JogoComObjetivo`, outros na sua superclasse `Jogo`. Nas execuções desses métodos, o `this` será sempre o objeto referenciado pela variável `meuJogo`.

```
public static void main (String [] args) {

    JogoComObjetivo meuJogo = new JogoComObjetivo("Spiral",30);
    System.out.println("O objetivo eh " + meuJogo.pontuacaoObjetivo());
    meuJogo.juntarJogador("Maria");
    meuJogo.registarPontosJogada("Maria", 13);

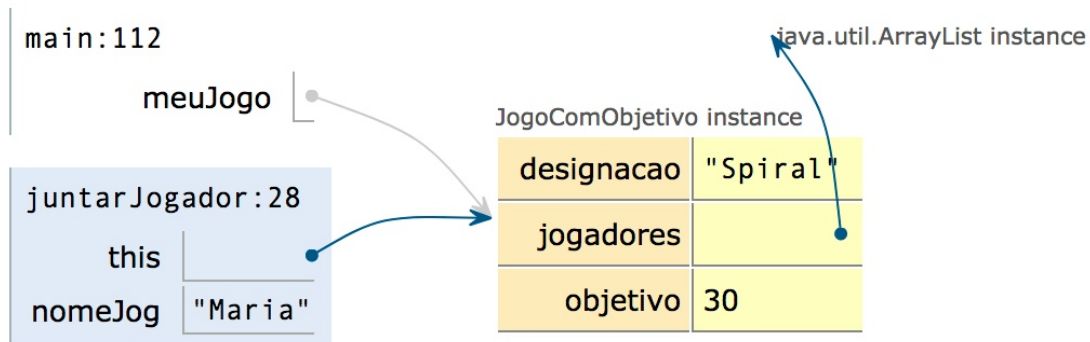
    if(meuJogo.terminou()) {
```

```

        System.out.println("Terminou");
    } else {
        System.out.println("Maria estah em jogo? " +
            meuJogo.estahEmJogo("Maria"));
    }
}

```

A figura seguinte ilustra o estado do objeto `meuJogo` aquando da invocação do método `juntarJogador`:



14.6. Redefinição de métodos

Na classe `Jogo` apresentada no início deste capítulo, deixámos o método `terminou` em aberto (mais adiante lá voltaremos), ou seja, considerámos que não há nenhum critério de terminação que seja comum à maior parte dos jogos.

O método `vencedores`, pelo contrário, foi implementado de modo a devolver o(s) jogador(s) com melhor pontuação. Ou seja, considerámos que este critério – melhor pontuação – é suficientemente geral para se aplicar à maior parte dos jogos. Há vários tipos de jogos em que o vencedor é o que tem maior pontuação, nomeadamente os jogos que terminam ao fim de um dado número de jogadas, ou ao fim de determinado tempo, etc.

Por acaso (ou de propósito, na verdade) não é isso que acontece no `JogoComObjetivo`. Neste tipo de jogo queremos que seja vencedor todo e qualquer jogador que tenha alcançado a pontuação objetivo.

Mas se já temos uma implementação do método `vencedores` herdada da classe `Jogo`, como vamos fazer isso?

- Através do que se chama *redefinição* (*overriding* em inglês);
- Vamos *redefinir* o método `vencedores` na subclasse, ou seja, vamos dar-lhe uma nova implementação;
- Assim, quando o método `vencedores` for invocado sobre um objeto do tipo `JogoComObjetivo`, é esta nova implementação que é executada e não a que está definida na classe `Jogo`;

Uma primeira tentativa (que não está correta):

Perguntar a cada jogador qual a sua pontuação e compará-la com o objetivo:

```

/**
 * Vencedor(es) deste jogo
 * @return O nome do(s) jogador(es) que atingiu(ram) o objetivo.
 * @requires this.quantosEmJogo() > 0 && this.terminou()
 */
@Override
public List<String> vencedores () {

```

```

        List<String> result = new ArrayList<String>();
        for (Jogador jog : this.jogadores){
            if(jog.pontuacao() >= this.objetivo) {
                result.add(jog.nome());
            }
        }
        return result;
    }
}

```

Qual é o problema com esta solução?

- o atributo `jogadores` foi definido como `private` na classe `Jogo`
- logo, não está acessível fora dessa classe
- o compilador dará o erro *"The field Jogo.jogadores is not visible"*

Segunda tentativa (agora sim!):

Repare que na classe `Jogo` temos um método que nos pode ser muito útil – o método `pontuacaoSuperiorA` – que, dado um inteiro, devolve os nomes dos jogadores cuja pontuação é superior a esse inteiro.

Então, podemos usar este método para obter os nomes dos nossos vencedores! Basta invocá-lo, a partir do método `vencedores`, usando o valor da pontuação objetivo -1 como limite inferior:

```

/**
 * Vencedor(es) deste jogo
 * @return O nome do(s) jogador(es) que atingiu(ram) o objetivo.
 * @requires this.quantosEmJogo() > 0 && this.terminou()
 */
@Override
public List<String> vencedores () {
    return this.pontuacaoSuperiorA(this.objetivo - 1);
}

```

Vamos agora considerar este `main`:

```

public static void main (String [] args) {

    JogoComObjetivo meuJogo = new JogoComObjetivo("Spiral",30);
    System.out.println("O objetivo eh " + meuJogo.pontuacaoObjetivo());
    meuJogo.juntarJogador("Maria");
    meuJogo.juntarJogador("Pedro");
    meuJogo.registarPontosJogada("Maria", 13);
    meuJogo.registarPontosJogada("Pedro", 25);
    meuJogo.registarPontosJogada("Maria", 17);
    meuJogo.registarPontosJogada("Pedro", 9);

    if(meuJogo.terminou()) {
        System.out.println("Os vencedores sao:");
        List<String> venceram = meuJogo.vencedores();
        for (String jog : venceram) {
            System.out.println(jog + " com " + meuJogo.pontuacao(jog));
        }
    } else {
        System.out.println("Ainda nao terminou");
    }
}

```

```
}
```

Ao executá-lo, iremos obter o seguinte *output*:

```
O objetivo eh 30
Os vencedores sao:
Maria com 30
Pedro com 34
```

Se tivesse sido executada a versão de `vencedores` implementada na classe `Jogo` o resultado daria diferente – só o Pedro seria vencedor.

Regras na redefinição de métodos em subclasses:

Seja `m` um método definido numa classe `A`. Seja `B` uma subclasse de `A`. Seguem-se algumas restrições à redefinição de `m` em `B`:

- A lista de parâmetros de `m` em `B` deve ser exatamente a mesma – em número, ordem e tipos de parâmetros – que a de `m` em `A`;
- O tipo de retorno de `m` em `B` deve ser o mesmo que, ou um subtipo de, o tipo de retorno de `m` em `A`;
- O nível de acessibilidade de `m` em `B` não pode ser mais restritivo que em `A` – por exemplo, se `m` em `A` é declarado `public`, então o nível de acessibilidade de `m` em `B` não poderá ser nenhum dos outros (`private`, `protected` ou `package`);
- Os métodos de instância só podem ser redefinidos se forem herdados pela subclasse – por exemplo, se `m` em `A` é declarado `private`, então não pode ser redefinido; se `m` em `A` tiver um nível de acessibilidade `package` e a classe `B` pertencer a um pacote diferente do de `A`, então `m` não pode ser redefinido pois não é sequer herdadado;
- Se `m` for declarado como `final` em `A`, não pode ser redefinido em `B`;
- Se `m` for declarado como `static` em `A`, não pode ser redefinido em `B` mas pode ser re-declarado;
- Uma subclasse pertencente ao mesmo pacote que a sua superclasse pode redefinir qualquer método da superclasse que não seja `private` nem `final`;
- Uma subclasse num pacote diferente só pode redefinir os métodos não `final` declarados `public` ou `protected`.

Repare na anotação `@Override` na linha imediatamente acima da assinatura do método `vencedores`: esta anotação informa o compilador que a intenção é que aquele método redefina o correspondente método na superclasse.

Embora não seja obrigatória, esta anotação é extremamente útil. Se por alguma razão o método correspondente na superclasse muda – o nome, a lista de parâmetros, etc –, deixa de haver redefinição. Nesse caso o compilador avisa-nos que a subclasse está ilegal pois aquele método devia ser redefinição de algum existente na superclasse e não é o que acontece.

A classe `JogoComObjetivo` completa:

```
/**
 * Representa jogos com o objetivo de alcancar uma dada pontuacao
 * @author Isabel Nunes
 */
public class JogoComObjetivo extends Jogo {

    // Pontuacao objetivo
    private int objetivo;
```

```

/**
 * Construir um jogo.
 * @param designacao A designacao do jogo.
 * @param objetivo A pontuacao com que se ganha este jogo
 * @requires designacao != null
 */
public JogoComObjetivo (String designacao, int objetivo) {
    super(designacao);
    this.objetivo = objetivo;
}

/**
 * A pontuacao objetivo definida para este jogo.
 */
public int pontuacaoObjetivo () {
    return this.objetivo;
}

/**
 * Este jogo jah terminou?
 */
@Override
public boolean terminou() {
    return this.maiorPontuacao() >= this.objetivo;
}

/**
 * Vencedor(es) deste jogo
 * @return O nome do(s) jogador(es) que atingiu(ram) o objetivo.
 * @requires this.quantosEmJogo() > 0 && this.terminou()
 */
@Override
public List<String> vencedores () {
    return this.pontuacaoSuperiorA(this.objetivo - 1);
}
}

```

14.7. Outra subclasse de Jogo

Vamos agora implementar uma outra subclasse de `Jogo` – uma classe que representa jogos que terminam quando tiver sido feito um dado número exato de jogadas – e em que vence quem alcançou a melhor pontuação ao fim dessas jogadas.

Não controlamos se os jogadores fizeram igual número de jogadas, nem se joga um de cada vez. A ideia é que este tipo de jogo seja suficientemente geral para representar não só jogos em que os jogadores jogam por rondas (um de cada vez em cada ronda) como também jogos em que as jogadas são registadas à medida que os jogadores as fazem, independentemente da ordem, ou de algum(ns) fazer(em) mais jogadas que outro(s). As classes cliente desta deverão escolher o número de jogadas de acordo com o pretendido (por exemplo, num jogo por rondas, o número de jogadas para terminar deve ser um valor múltiplo do número de jogadores que vão jogar).

Chamemos `JogoNumeroExatoJogadas` a esta nova subclasse.

```

/**

```

```

* Representa jogos que terminam ao fim de um dado numero de jogadas
* @author Isabel Nunes
*/
public class JogoNumeroExatoJogadas extends Jogo {

    // Numero exato de jogadas ao fim do qual este jogo termina
    private int numeroJogadasAFazer;
    // Numero de jogadas jah efetuadas neste jogo
    private int jogadasJahFeitas;
    // Este jogo jah terminou?
    private boolean jahTerminou;

    /**
     * Construir um jogo.
     * @param designacao A designacao para o jogo
     * @param numJogadas O numero de jogadas a fazer para que este jogo
     * termine
     * @requires designacao != null && numJogadas >= 0
     */
    public JogoNumeroExatoJogadas(String designacao, int numJogadas) {
        super(designacao);
        this.numeroJogadasAFazer = numJogadas;
        this.jogadasJahFeitas = 0;
        this.jahTerminou = false;
    }

    /**
     * O numero de jogadas a fazer neste jogo.
     */
    public int numeroJogadasAFazer () {
        return this.numeroJogadasAFazer;
    }

    /**
     * O numero de jogadas jah feitas neste jogo.
     */
    public int jogadasJahFeitas () {
        return this.jogadasJahFeitas;
    }

    /**
     * Registrar a pontuacao, na jogada atual, de um dado jogador.
     * @param nomeJog O nome do jogador.
     * @param pontuacao A pontuacao obtida na jogada atual.
     * @requires this.estahEmJogo (nomeJog)
     */
    @Override
    public void registrarPontosJogada (String nomeJog, int pontuacao) {
        super.registrarPontosJogada(nomeJog,pontuacao);
        this.jogadasJahFeitas++;
        if (this.jogadasJahFeitas == this.numeroJogadasAFazer) {
            this.jahTerminou = true;
        }
    }

    /**
     * Este jogo jah terminou?

```

```

    */
    public boolean terminou() {
        return this.jahTerminou;
    }

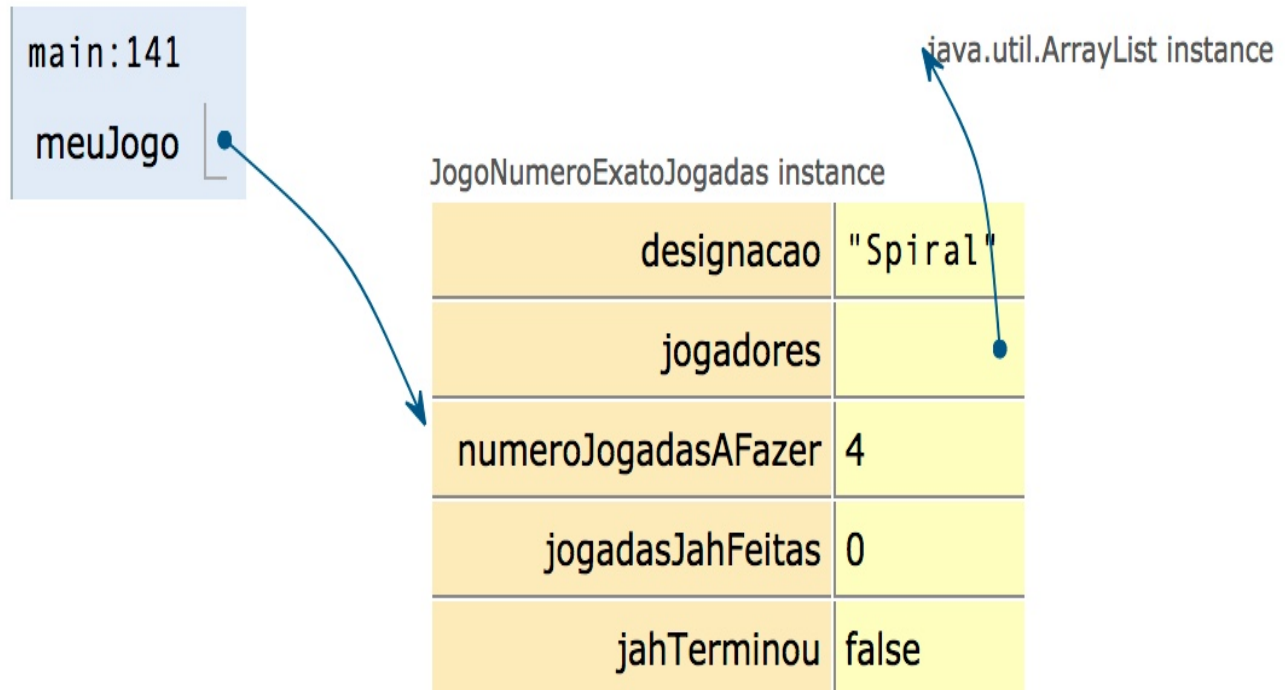
    /**
     * Representacao textual deste jogo.
     */
    @Override
    public String toString () {
        StringBuilder sb = new StringBuilder();
        sb.append(super.toString());
        sb.append("Numero de jogadas a fazer: " +
            this.numeroJogadasAFazer + "\n");
        sb.append("Numero de jogadas jah feitas: " +
            this.jogadasJahFeitas);
        return sb.toString();
    }
}

```

Então, um objeto do tipo `JogoNumeroExatoJogadas` tem cinco atributos de instância:

- os inteiros `numeroJogadasAFazer` e `jogadasJahFeitas` e o boolean `jahTerminou` definidos na própria classe `JogoNumeroExatoJogadas` e
- a `String` `designação` e o `array` `jogadores`, herdados da classe `Jogo`.

A imagem seguinte representa o estado pretendido de um objeto do tipo `JogoNumeroExatoJogadas` acabado de criar (com valor inicial para o número de jogadas a fazer de 4).



Além do valor do número de jogadas a fazer para terminar o jogo, que é dado como parâmetro do construtor, é necessário ir contabilizando cada jogada que é feita. Logo que o número de jogadas feitas iguala o número de jogadas por fazer, podemos considerar que o jogo está terminado.

Para fazer isto:

- vamos acrescentar três atributos de instância:
 - um para guardar a informação do número de jogadas a fazer – `int numeroJogadasAFazer` – que é inicializado com o valor do parâmetro correspondente no construtor e nunca mais muda o seu valor;
 - um para guardar o número de jogadas já feitas – `int jogadasJahFeitas` – que é inicializado a zero e incrementado de uma unidade sempre que o método `registrarPontosJogada` é invocado;
 - um para guardar a informação sobre se o jogo já terminou – `boolean jahTerminou` – que é inicializado a `false` e colocado a `true` no método `registrarPontosJogada` logo que o número de jogadas feitas iguala o número de jogadas a fazer;
- vamos redefinir o método que permite registrar jogadas – `registrarPontosJogada` – acrescentando-lhe:
 - uma instrução que permita contabilizar mais uma jogada:

```
this.jogadasJahFeitas++;
```

- e uma instrução que permita verificar se o jogo já pode ser considerado como terminado:

```
if (this.jogadasJahFeitas == this.numeroJogadasAFazer) {
    this.jahTerminou = true;
}
```

Mas isto não chega, como veremos de seguida.

14.8. Ainda a redefinição de métodos

No método `registrarPontosJogada` não basta fazer as duas tarefas que foram descritas; é necessário também registar os pontos no jogador a quem a jogada respeita. Ou seja, teríamos que ter também a instrução:

```
this.obterJogador(nomeJog).registrarPontos(pontuacao); // errado
```

Mas temos um problema! O método `obterJogador`, definido na classe `Jogo`, é `private`, ou seja, o compilador não aceitaria esta instrução na classe `JogoNumeroExatoJogadas`.

A solução seria invocar o próprio método `registrarPontosJogada` implementado na classe `Jogo`, que é `public`, sobre o objeto corrente, obtendo algo como:

```
public void registrarPontosJogada (String nomeJog, int puntuacao) {
    this.registrarPontosJogada(nomeJog,pontuacao); // errado!!!
    this.jogadasJahFeitas++;
    if (this.jogadasJahFeitas == this.numeroJogadasAFazer) {
        this.jahTerminou = true;
    }
}
```

Mas não é isto que nós queremos! Desta forma estaríamos a invocar o próprio método `registrarPontosJogada` implementado na subclasse, – através de *recursão* –, obtendo, neste caso em particular, um processo que não termina.

O que nós queremos mesmo é executar o corpo do método `registrarPontosJogada`, tal como definido na superclasse, sobre o objeto corrente. A forma como indicamos isso ao compilador é usando a palavra `super` em vez de `this`, para referir o objeto corrente.

Na redefinição de um método `m`, numa classe `B`, podemos invocar a versão de `m` definida na superclasse de `B` sobre o objeto corrente, referindo-nos a ele através da palavra `super`.

Finalmente, **a versão correta**:

```
@Override
public void registrarPontosJogada (String nomeJog, int pontuacao) {
    super.registrarPontosJogada(nomeJog,pontuacao);
    this.jogadasJahFeitas++;
    if (this.jogadasJahFeitas == this.numeroJogadasAFazer) {
        this.jahTerminou = true;
    }
}
```

Repare que fizemos o mesmo na redefinição do método `toString`. A representação textual de um objeto do tipo `JogoNumeroExatoJogadas` inclui não só os valores dos novos atributos definidos nesta subclasse, mas também os valores dos atributos definidos na classe `Jogo`.

A única maneira de conseguirmos aceder, a partir do método na subclasse, aos valores de todos os atributos definidos na superclasse, é através da invocação do método `toString` tal como definido na classe `Jogo`, acrescentando de seguida a informação adicionada pela subclasse.

```
/**
 * Representacao textual deste jogo.
 */
@Override
public String toString () {
    StringBuilder sb = new StringBuilder();
    sb.append(super.toString());
    sb.append("Numero de jogadas a fazer: " + this.numeroJogadasAFazer +
        "\n");
    sb.append("Numero de jogadas jah feitas: " + this.jogadasJahFeitas);
    return sb.toString();
}
```

Vamos agora considerar este novo `main`:

```
public static void main (String [] args) {

    JogoNumeroExatoJogadas meuJogo = new JogoNumeroExatoJogadas("Spiral",4);
    System.out.println("O numero de jogadas a fazer eh " +
        meuJogo.numeroJogadasAFazer());
    meuJogo.juntarJogador("Maria");
    meuJogo.juntarJogador("Pedro");
    meuJogo.registarPontosJogada("Maria", 17);
    meuJogo.registarPontosJogada("Pedro", 25);
    meuJogo.registarPontosJogada("Maria", 17);
    meuJogo.registarPontosJogada("Pedro", 9);
    System.out.println("=====");
    System.out.println(meuJogo.toString());
    System.out.println("=====");
}
```

```

        if(meuJogo.terminou()) {
            System.out.println("Os vencedores sao:");
            List<String> venceram = meuJogo.vencedores();
            for (String jog : venceram) {
                System.out.println(jog + " com " + meuJogo.pontuacao(jog));
            }
        } else {
            System.out.println("Ainda nao terminou");
        }
    }
}

```

Ao executá-lo, iremos obter o seguinte *output*:

```

O numero de jogadas a fazer eh 4
=====
Numero de jogadores: 2
Nome: Maria      Pontuacao: 34      Maximo numa jogada: 17
Nome: Pedro     Pontuacao: 34      Maximo numa jogada: 25
Numero de jogadas a fazer: 4
Numero de jogadas jah feitas: 4
=====
Os vencedores sao:
Pedro com 34

```

Como queremos que os vencedores deste novo tipo de jogo sejam calculados tal como na classe `Jogo`, então não precisamos de redefinir o método `vencedores`; pelo facto desta classe ser subclasse de `Jogo`, o método `vencedores` da classe `Jogo` é herdado tal e qual como implementado nessa classe.

14.9. Redefinição versus atributos

Ao contrário dos métodos, os atributos da superclasse não podem ser redefinidos na subclasse.

- Os atributos da superclasse só podem ser escondidos (*hiding*).
- Se existe um atributo `x` na superclasse e outro com o mesmo nome é definido na subclasse, o atributo da superclasse continua a existir.

Exemplo:

```

public class A {
    public int x = 0;

    public void m(){
        System.out.println("A" + x);
    }
}

public class B extends A {
    public int x = 5;

    public void m(){
        System.out.println("B" + x);
    }
}

```

Numa classe cliente de `A` e de `B`:

```
A oa = new A();
B ob = new B();
oa.m();           // imprime A0 no standard output
ob.m();           // imprime B5 no standard output
```

15. Polimorfismo e ligação dinâmica

Vamos agora abordar um dos mecanismos mais importantes do paradigma orientado a objetos.

Se criarmos vários jogos diferentes, não só por terem jogadores e jogadas diversas, mas também por serem de tipos diferentes – uns com pontuação objetivo, outros com número exato de jogadas a fazer, por exemplo – percebemos que há certas ações que podemos fazer com todos eles, independentemente das suas diferenças. Podemos adicionar jogadores, registrar jogadas, saber a melhor pontuação até agora, obter a representação textual, conhecer os nomes dos vencedores, etc. Ou seja, podemos fazer com todos eles exatamente aquilo que a classe `Jogo` fornece.

Um exemplo:

```
public class ExemploPolimorfismoParametros {

    public static void main(String[] args) {
        JogoComObjetivo jogo1 = new JogoComObjetivo("Argon", 25);
        JogoNumeroExatoJogadas jogo2 =
            new JogoNumeroExatoJogadas("Harvest Kingdom", 2);

        jogo1.juntarJogador("Maria");
        jogo1.juntarJogador("Pedro");
        jogo1.registarPontosJogada("Maria", 11);
        jogo1.registarPontosJogada("Pedro", 26);

        jogo2.juntarJogador("John Snow");
        jogo2.juntarJogador("Daenerys Targaryen");
        jogo2.registarPontosJogada("John Snow", 10);
        jogo2.registarPontosJogada("Daenerys Targaryen", 10);

        System.out.println("=====");
        System.out.println("Detalhes do jogo:");
        System.out.println(jogo1.toString());
        System.out.println("Melhor pontuacao ateh agora: " +
            jogo1.maiorPontuacao());

        System.out.println("=====");
        System.out.println("Detalhes do jogo:");
        System.out.println(jogo2.toString());
        System.out.println("Melhor pontuacao ateh agora: " +
            jogo2.maiorPontuacao());
    }
}
```

```

        System.out.println("=====");
        System.out.println("Vencedor(es) do jogo de tipo JogoComObjetivo:");
        List<String> venceram = jogo1.vencedores();
        for (String jog : venceram) {
            System.out.println(jog + " com " + jogo1.pontuacao(jog));
        }
        System.out.println("Parabens!");

        System.out.println("=====");
        System.out.println("Vencedor(es) do jogo de tipo
            JogoNumeroExatoJogadas:");
        venceram = jogo2.vencedores();
        for (String jog : venceram) {
            System.out.println(jog + " com " + jogo2.pontuacao(jog));
        }
        System.out.println("Parabens!");
    }
}

```

Cujo *output* é:

```

=====
Detalhes do jogo:
Designacao: Spiral
Numero de jogadores: 2
Nome: Maria      Pontuacao: 11      Maximo numa jogada: 11
Nome: Pedro      Pontuacao: 26      Maximo numa jogada: 26
Pontuacao objetivo: 25
Melhor pontuacao ateh agora: 26
=====
Detalhes do jogo:
Designacao: Harvest Kingdom
Numero de jogadores: 2
Nome: John Snow   Pontuacao: 10      Maximo numa jogada: 10
Nome: Daenerys Targaryen   Pontuacao: 10      Maximo numa jogada: 10
Numero de jogadas a fazer: 2
Numero de jogadas jah feitas: 2
Melhor pontuacao ateh agora: 10
=====
Vencedor(es) do jogo de tipo JogoComObjetivo:
Pedro com 26
Parabens!
=====
Vencedor(es) do jogo de tipo JogoNumeroExatoJogadas:
John Snow com 10
Daenerys Targaryen com 10
Parabens!

```

Seria bom poder generalizar alguns conjuntos de instruções do `main` acima, pois a única diferença que têm é o objeto sobre o qual incidem. Na próxima secção vamos ver como fazer isso.

15.1. Polimorfismo

Continuando com o exemplo da secção anterior, seria útil, por exemplo, construirmos métodos que imprimissem no *standard output*:

- os detalhes de um qualquer jogo, incluindo a melhor pontuação até ao momento;
- os nomes e pontuações dos vencedores de um qualquer jogo.

Por exemplo, algo como:

```
static void imprimeDetalhes(???? j) {
    System.out.println("=====");
    System.out.println("Detalhes do jogo:");
    System.out.println(j.toString());
    System.out.println("Melhor pontuacao ateh agora: " +
        j.maiorPontuacao());
}

static void imprimeVencedores(???? j) {
    System.out.println("=====");
    String tipoJogo = j.getClass().getName();
    System.out.println("Vencedor(es) do jogo de tipo " +
        tipoJogo + ":");
    String[] venceram = j.vencedores();
    for (String jog : venceram) {
        System.out.println(jog + " com " + j.pontuacao(jog));
    }
    System.out.println("Parabens!");
}
```

Na classe `ExemploPolimorfismoParametros`,

- incluir-se-iam estes métodos a seguir ao (ou antes do) método `main` e
- todas as instruções do `main` acima listado que fazem a impressão das informações dos jogos no ecrã, seriam reduzidas a:

```
imprimeDetalhes(jogo1);
imprimeDetalhes(jogo2);
imprimeVencedores(jogo1);
imprimeVencedores(jogo2);
```

Pergunta:

Na definição de cada método, de que tipo deve ser o parâmetro que representa o jogo sobre o qual queremos atuar?

- se o parâmetro fosse do tipo `JogoComObjetivo`, então não poderíamos invocar o método dando como valor uma referência a um objeto do tipo `JogoNumeroExatoJogadas`;
- de igual modo, se o parâmetro fosse do tipo `JogoNumeroExatoJogadas`, então não poderíamos invocar o método dando como valor uma referência a um objeto do tipo `JogoComObjetivo`.

Resposta:

O tipo do parâmetro deve ser o tipo comum a qualquer jogo, ou seja, o tipo `Jogo`.

```

static void imprimeDetalhes(Jogo j) {
    ....
}

static void imprimeVencedores(Jogo j) {
    ....
}

```

Porquê? Relembrando que:

Na análise estática de tipos, executada pelo compilador, as seguintes regras são aplicadas:

- Seja `x` uma variável de um tipo não primitivo `T`; numa atribuição `x = expressao`; o tipo de `expressao` deve ser o mesmo que o de `x` ou um subtipo deste;
- Seja `x`, de tipo não primitivo `T`, um parâmetro de um método `m`; numa invocação `m(expressao)`, o tipo de `expressao` deve ser o mesmo que o de `x` ou um subtipo deste.

percebemos então que as instruções

```

imprimeDetalhes(jogo1);
imprimeDetalhes(jogo2);
imprimeVencedores(jogo1);
imprimeVencedores(jogo2);

```

estão corretas, pois tanto o tipo de `jogo1` – `JogoComObjetivo` –, como o tipo de `jogo2` – `JogoNumeroExatoJogadas` – são subtipos de `Jogo`.

Chama-se *polimorfismo* à capacidade de tomar várias formas.

Nas invocações

```

imprimeDetalhes(jogo1); imprimeDetalhes(jogo2); imprimeVencedores(jogo1);
imprimeVencedores(jogo2);

```

dizemos que as instanciações de parâmetros são *polimórficas* pois o parâmetro dos métodos, que é do tipo `Jogo`, recebe valores de tipos que não são exatamente o tipo `Jogo`, embora sejam seus subtipos.

A razão por trás das regras aplicadas pelo compilador é intuitiva.

Suponha uma variável ou parâmetro `x` declarada com o tipo `T`:

- o compilador só permite que invoquemos sobre `x` os métodos acessíveis em `T`;
- se `x` contém uma referência a um objeto que é exatamente do tipo `T` então
 - esse objeto sabe executar todos os métodos que possam ser invocados sobre ele;
- se `x` contém uma referência a um objeto de um subtipo de `T` diferente de `T` então
 - como qualquer método acessível no tipo `T` é herdado por esse subtipo,
 - temos a certeza de que esse objeto saberá executar todos os métodos que possam ser invocados sobre ele.

No nosso exemplo, no corpo dos métodos `imprimeDetalhes` e `imprimeVencedores` os seguintes métodos são invocados sobre o parâmetro `j`:

- `toString`: método implementado na classe `Object`, redefinido na classe `Jogo` e herdado por todas as

subclasses desta;

- `getClass`: método implementado na classe `Object`, e portanto herdado pela classe `Jogo` e por todas as subclasses desta;
- `maiorPontuacao`, `vencedores` e `pontuacao`: métodos implementados na classe `Jogo` e herdados por todas as subclasses desta.

Ao parâmetro `j`, declarado como sendo do tipo `Jogo`, poderão ser atribuídas referências a objetos do tipo `Jogo` ou de qualquer dos seus subtipos (para já temos dois, mas no futuro poderão ser mais). Qualquer que seja esse subtipo, sabemos que implementará todos os métodos invocados sobre `j`, pois terão sido herdados (e possivelmente, até, redefinidos como, por exemplo, o método `vencedores`, que vamos examinar a seguir com mais pormenor).

Então, a classe `ExemploPolimorfismoParametros` ficaria assim:

```
public class ExemploPolimorfismoParametros {

    public static void main(String[] args) {
        JogoComObjetivo jogo1 = new JogoComObjetivo("Argon", 25);
        JogoNumeroExatoJogadas jogo2 =
            new JogoNumeroExatoJogadas("Harvest Kingdom", 2);

        jogo1.juntarJogador("Maria");
        jogo1.juntarJogador("Pedro");
        jogo1.registarPontosJogada("Maria", 11);
        jogo1.registarPontosJogada("Pedro", 26);

        jogo2.juntarJogador("John Snow");
        jogo2.juntarJogador("Daenerys Targaryen");
        jogo2.registarPontosJogada("John Snow", 10);
        jogo2.registarPontosJogada("Daenerys Targaryen", 10);

        imprimeDetalhes(jogo1);
        imprimeDetalhes(jogo2);
        imprimeVencedores(jogo1);
        imprimeVencedores(jogo2);
    }

    /**
     * Imprime detalhes de um dado jogo
     * @param j O jogo
     * @requires j != null
     */
    static void imprimeDetalhes(Jogo j) {
        System.out.println("=====");
        System.out.println("Detalhes do jogo:");
        System.out.println(j.toString());
        System.out.println("Melhor pontuacao ateh agora: " +
            j.maiorPontuacao());
    }

    /**
     * Imprime vencedores de um dado jogo e respetivas pontuacoes
     * @param j O jogo
     * @requires j != null
     */
    static void imprimeVencedores(Jogo j) {
```

```

        System.out.println("=====");
        String tipoJogo = j.getClass().getName();
        System.out.println("Vencedor(es) do jogo de tipo " +
                           tipoJogo + ":");
        List<String> venceram = j.vencedores();
        for (String jog : venceram) {
            System.out.println(jog + " com " + jogo2.pontuacao(jog));
        }
        System.out.println("Parabens!");
    }
}

```

15.2. Verificação estática de tipos

Também seria bom poder guardar numa mesma coleção, por exemplo um *array*, jogos de vários tipos e invocar sobre esses jogos os métodos que estão acessíveis a todos, ou seja, os métodos definidos na classe `Jogo`:

```

public class ExemploPolimorfismoArray {

    public static void main(String[] args) {
        Jogo[] meusJogos = new Jogo[2];
        meusJogos[0] = new JogoComObjetivo("Spiral", 25);
        meusJogos[1] =
            new JogoNumeroExatoJogadas("Harvest Kingdom",1);
        meusJogos[0].juntarJogador("Maria");
        meusJogos[0].juntarJogador("Pedro");
        meusJogos[1].juntarJogador("John Snow");
        meusJogos[1].juntarJogador("Daenerys Targaryen");

        meusJogos[0].registarPontosJogada("Maria", 11);
        meusJogos[0].registarPontosJogada("Pedro", 26);
        meusJogos[1].registarPontosJogada("John Snow", 10);
        meusJogos[1].registarPontosJogada("Daenerys Targaryen", 10);

        for(int i = 0 ; i < meusJogos.length ; i++) {
            List<String> venceram = meusJogos[i].vencedores();
            for (String jog : venceram) {
                System.out.println(jog);
            }
        }
    }
}

```

O *output* deste programa é:

```

Pedro
John Snow
Daenerys Targaryen

```

O *array* `meusJogos` é *polimórfico* pois contém elementos de diferentes tipos, embora ambos subtipos de `Jogo`.

Durante a compilação, como já sabemos, todas as atribuições e passagens de parâmetro são verificadas pelo compilador relativamente à correção dos tipos.

As atribuições

```
meusJogos[0] = new JogoComObjetivo("Spiral", 25);  
meusJogos[1] = new JogoNumeroExatoJogadas("Harvest Kingdom", 1);
```

estão corretas do ponto de vista das regras que o compilador aplica (relembrar secção anterior), pois atribuem a cada uma das duas variáveis do tipo `Jogo` – `meusJogos[0]` e `meusJogos[1]` – valores de um subtipo de `Jogo`.

De igual modo, as invocações de métodos também são verificadas relativamente à correção dos tipos.

A regra que temos usado:

- Os métodos invocados sobre um objeto têm que estar definidos na classe de que esse objeto é instância.

tem que ser refinada a partir do momento em que temos variáveis polimórficas.

Nestes casos podemos distinguir entre:

- o tipo da variável ou parâmetro cujo valor é uma referência a um objeto – tipo *declarado*;
- o tipo do objeto que é referenciado por essa variável ou parâmetro, ou seja, a classe de que é instância – tipo *concreto*.

O compilador permite a invocação de um método `m` sobre um objeto `o`:

- se `m` está acessível no tipo declarado de `o`.

Logo, todas as invocações do programa `ExemploPolimorfismoArray` estão corretas, pois todos os métodos invocados sobre `meusJogos[0]` e `meusJogos[1]` são métodos acessíveis no tipo `Jogo`, que é precisamente o tipo declarado dessas duas variáveis.

As seguintes invocações, pelo contrário, seriam rejeitadas pelo compilador:

```
meusJogos[0].pontuacaoObjetivo();  
meusJogos[1].jogadasJahFeitas();
```

Embora o tipo concreto de `meusJogos[0]` seja `JogoComObjetivo`, que define o método `pontuacaoObjetivo()`, no entanto esta invocação está incorreta do ponto de vista dos tipos, pois o tipo de `meusJogos[0]` é `Jogo`. O mesmo se passa com a segunda instrução, desta vez em relação ao tipo `JogoNumeroExatoJogadas`.

De igual modo, no exemplo da secção anterior, em nenhum dos métodos `imprimeVencedores(Jogo j)` e `imprimeDetalhes(Jogo j)`, se poderiam invocar, sobre o parâmetro `j`, métodos que não estivessem definidos em `Jogo`.

15.3. Ligação dinâmica

Sabemos que o método `vencedores` está implementado na classe `Jogo` de modo a devolver o(s) jogador(es) com melhor pontuação. Sabemos também que este método foi redefinido na classe `JogoComObjetivo` de modo a devolver todos os jogadores que tenham alcançado a pontuação objetivo.

Na classe `JogoNumeroExatoJogadas` não redefinimos este método, ou seja, o método `vencedores` é herdado por esta classe exatamente como foi definido na classe `Jogo`.

Qual das duas versões do método `vencedores` é usada em cada uma das duas execuções (quando `i` toma os valores zero e 1), da seguinte instrução no `main` do exemplo da secção anterior?

```
List<String> venceram = meusJogos[i].vencedores();
```

Depende!

Repare que,

- embora o tipo *declarado* dos dois elementos do array `meusJogos` seja `Jogo`,
 - o tipo *concreto* de cada um deles não é exatamente esse
 - o objeto referenciado por `meusJogos[0]` é do tipo `JogoComObjetivo`
 - o objeto referenciado por `meusJogos[1]` é do tipo `JogoNumeroExatoJogadas`.
-
- É o mecanismo de *ligação dinâmica* (*dynamic binding*) que assegura que,
 - na invocação de um método sobre um objeto, é a versão correta desse método que é executada;
 - a versão correta é a que está acessível na classe a que o objeto pertence, ou seja, na do seu tipo concreto.

No nosso caso,

- quando `i` vale zero, o alvo da invocação do método `vencedores` em `meusJogos[i].vencedores()` é do tipo `JogoComObjetivo`
 - logo, a versão do método que é executada é a que foi definida nesta classe;
- quando `i` vale um, o alvo da invocação do método `vencedores` em `meusJogos[i].vencedores()` é do tipo `JogoNumeroExatoJogadas`
 - logo, a versão do método que é executada é a que está definida na classe `Jogo` (que é herdada pela classe `JogoNumeroExatoJogadas`).

Voltemos ao programa `ExemploPolimorfismoParametros` da secção 15.1 e foquemo-nos agora na seguinte instrução do método `imprimeVencedores`, que invoca o método `vencedores` sobre o parâmetro `j`:

```
List<String> venceram = j.vencedores();
```

Qual das duas versões do método `vencedores` é executada dentro do método `imprimeVencedores` como resultado das seguintes invocações no `main`?

```
imprimeVencedores(jogo1);  
imprimeVencedores(jogo2);
```

Aqui o raciocínio é o mesmo: depende!

- na invocação `imprimeVencedores(jogo1)`; o parâmetro `j` fica a referenciar um objeto do tipo `JogoComObjetivo`
 - logo, a versão do método `vencedores` que é executada sobre o alvo `j` dentro do método `imprimeVencedores` é a que está redefinida nesta classe;
- na invocação `imprimeVencedores(jogo2)`; o parâmetro `j` fica a referenciar um objeto do tipo `JogoNumeroExatoJogadas`

- o logo, a versão do método `vencedores` que é executada sobre o alvo `j` é a que está acessível na classe `JogoNumeroExatoJogadas`, que é a versão que foi herdada da sua superclasse `Jogo`.

15.4. Invocação indireta de métodos redefinidos

O mecanismo de ligação dinâmica determina que, na invocação de um método `m`, a versão do método que é executada é a que está acessível na classe correspondente ao tipo concreto do objeto alvo dessa invocação.

No exemplo seguinte,

- o método público `m` declarado na classe `A2` é redefinido na subclasse `B2`;
- no método `n`, definido na classe `A2` e herdado pela classe `B2`, a instrução `return 2 * m();` (que é equivalente a `return 2 * this.m();`), invoca o método `m` sobre o objeto corrente;
 - logo, se o objeto corrente for do tipo `A2`, a versão de `m` que é invocada é a definida em `A2`;
 - se o objeto corrente for do tipo `B2`, é a versão redefinida em `B2` que é invocada.

```
public class A2 {
    public int m(){
        return 3;
    }
    public int n(){
        return 2 * m();
    }
}

public class B2 extends A2 {
    public int m(){
        return 0;
    }
}
```

Nas instruções seguintes, numa qualquer classe cliente das classes `A2` e `B2`, o método `n` é invocado sobre dois objetos diferentes, um do tipo `A2` e outro do tipo `B2`.

```
A2 oa = new A2();
B2 ob = new B2();
System.out.println(oa.n());           // imprime 6 no standard output
System.out.println(ob.n());           // imprime 0 no standard output
```

A mesma versão do método `n` é executada nas duas invocações pois ele não é redefinido na classe `B2`, ou seja, é herdado. Neste método `n` há uma invocação de `m` sobre o objeto corrente, por isso a diferença dos resultados é devida à versão do método `m` que é invocada – na primeira vez é a versão da classe `A2`, que retorna 3 (o qual, multiplicado por 2 dá 6), e na segunda vez é a versão da classe `B2`, que retorna zero (o qual, multiplicado por 2 dá zero).

Se o método `m` fosse privado em vez de público, como na classe `A3` abaixo, então não haveria redefinição de `m` na classe `B3`. Logo, o método `n` invoca sempre o método `m` da classe `A3`, mesmo que o tipo concreto do objeto corrente seja `B3`.

```
public class A3 {
```

```

    private int m(){
        return 3;
    }
    public int n(){
        return 2 * m();
    }
}

public class B3 extends A3 {
    public int m(){
        return 0;
    }
}

```

As instruções seguintes, numa qualquer classe cliente de `A3` e `B3`, dão os resultados indicados.

```

A3 oa = new A3();
B3 ob = new B3();
System.out.println(oa.n());           // imprime 6 no standard output
System.out.println(ob.n());           // imprime 6 no standard output

```

Acesso a atributos redefinidos

Quando um atributo é acedido diretamente por uma classe cliente (por exemplo, porque é `public`), é o tipo declarado da referência (e não o seu tipo concreto, como na invocação de métodos de instância) que define o atributo a aceder.

Exemplo de instruções numa classe cliente das classes `A` e `B` da secção 13.1.8:

```

System.out.println("AA" + oa.x);       // imprime AA0 no standard output
System.out.println("BB" + ob.x);       // imprime BB5 no standard output
System.out.println("AB" + oab.x);      // imprime AB0 no standard output

```

15.5. Herança de contratos versus redefinição

Os dois exemplos que vimos na secção anterior ilustram situações extremamente comuns em programação – a necessidade de generalizar ações semelhantes, identificando as partes comuns e as que variam:

- na classe `ExemploPolimorfismoParametros` construímos dois métodos que generalizaram dois conjuntos de instruções; cada método define uma sequência de ações sobre um dado objeto do tipo `Jogo`, parâmetro do método; este pode ser instanciado por uma referência a um objeto de qualquer subtipo de `Jogo`.
- na classe `ExemploPolimorfismoArray` temos uma mesma sequência de instruções aplicada sobre dois objetos de tipos diferentes; como estes objetos têm um supertipo comum – `Jogo` – podem ser acedidos através de variáveis desse tipo – `meusJogos[0]` e `meusJogos[1]`.

Como o tipo declarado dos objetos é `Jogo`, ainda que o seu tipo concreto possa ser qualquer subtipo de `Jogo`,

- só os métodos acessíveis na classe `Jogo` podem ser invocados sobre esses objetos
 - mesmo que as subclasses acrescentem novos métodos, eles não podem ser invocados através de uma variável do tipo `Jogo`;

- aquilo que se espera obter na invocação de métodos sobre esses objetos, em termos de contratos, deve ser o que está definido na classe `Jogo`; tomemos o exemplo do método `vencedores`:
 - se a pré-condição definida para o método na classe `Jogo` for satisfeita na altura da invocação, então o método deve poder ser invocado sem problemas, mesmo que a versão executada seja outra, por via de redefinição:
 - então, na redefinição do método, não podemos ser mais exigentes na pré-condição;
 - após a invocação do método, a pós-condição definida para ele na classe `Jogo` tem que ser garantida, mesmo que a versão executada seja outra, por via de redefinição:
 - então, na redefinição do método, temos que garantir o mesmo, ou mais, na pós-condição;

Exemplo de pré-condições

Ainda usando o exemplo do método `vencedores`, relembremos a linha de comentário que define a sua pré-condição na classe `Jogo`:

```
* @requires this.quantosEmJogo() > 0 && this.terminou()
```

Na redefinição deste método na classe `JogoComObjetivo`, a pré-condição não poderá ser mais restritiva (mais forte). Deverá ser igual ou menos restritiva (mais fraca):

- `@requires this.quantosEmJogo() > 0` estaria correto, pois é menos restritiva, mais fraca;
- `@requires this.quantosEmJogo() > 2 && this.terminou()` estaria incorreto pois é mais exigente, mais forte;
- `@requires this.quantosEmJogo() > 0 && this.terminou() && this.pontuacaoObjetivo() > this.maiorPontuacao()` também estaria incorreto pois é mais exigente.

Exemplo de pós-condições

Vamos agora ver o exemplo do método `void registrarPontosJogada(String nome, int pontos)`. Embora não tenhamos apresentado pós-condição para este método na classe `Jogo`, ela poderia ser:

```
* @ensures this.pontuacao(nome) == \old this.pontuacao(nome) + pontos &&
           pontos > \old this.maximoNumaJogada(nome)
           implies
           this.maximoNumaJogada(nome) == pontos
```

Na redefinição deste método na classe `JogNumeroExatoJogadas`, a pós-condição deverá garantir esta condição ou outra mais forte:

- A seguinte pós-condição estaria correta pois é mais forte:

```
* @ensures this.pontuacao(nome) == \old this.pontuacao(nome) + pontos
    &&
    pontos > \old this.maximoNumaJogada(nome)
    implies
    this.maximoNumaJogada(nome) == pontos
    &&
    this.jogadasJahFeitas() == \old this.jogadasJahFeitas() + 1
    &&
    this.jogadasJahFeitas() == this.numeroJogadasAFazer()
    implies
    this.terminou()
```

- A seguinte pós-condição estaria incorreta pois assegura menos, é mais fraca:

```
@ensures this.pontuacao(nome) == \old this.pontuacao(nome) + pontos;
```

No geral,

para duas classes `A` e `B`, em que `B` é subclasse de `A`, temos que, para todo o método `m` acessível aos clientes de `A`:

- `m` está também acessível em `B` – pelo mecanismo de herança;
- a pré-condição de `m` em `B` não pode ser mais forte do que a declarada em `A`;
- a pós-condição de `m` em `B` não pode ser mais fraca do que a declarada em `A`.

Ou seja, `B` não pode exigir mais nem prometer menos!

Isto leva-nos ao princípio conhecido por **Liskov substitution principle**:

Só fazendo com que a subclasse honre os contratos da superclasse conseguimos garantir que:

- o raciocínio feito em termos da superclasse é válido para as subclasses e, por isso,
- um objeto com tipo `S`, subtipo de `T`, pode ser usado em qualquer sítio em que é esperado um objeto com o tipo `T` (sendo o comportamento consistente com o especificado por `T`).

15.6. Tipos genéricos e sub-tipos

Como já vimos anteriormente, a relação de sub-tipo é extremamente importante em programação por objetos, tanto ao nível conceptual (lembrar *Liskov substitution principle*) como de implementação.

Quando trabalhamos com classes e interfaces não genéricos, a definição dessa relação é algo simples de determinar. `T2` é sub-tipo de `T1` se:

- `T1` e `T2` são classes e `T2` é sub-classe (*extends*) de `T1`;
- `T2` é uma classe e `T1` é um interface e `T2` implementa (*implements*) `T1`;
- `T1` e `T2` são interfaces e `T2` é sub-interface (*extends*) de `T1`;

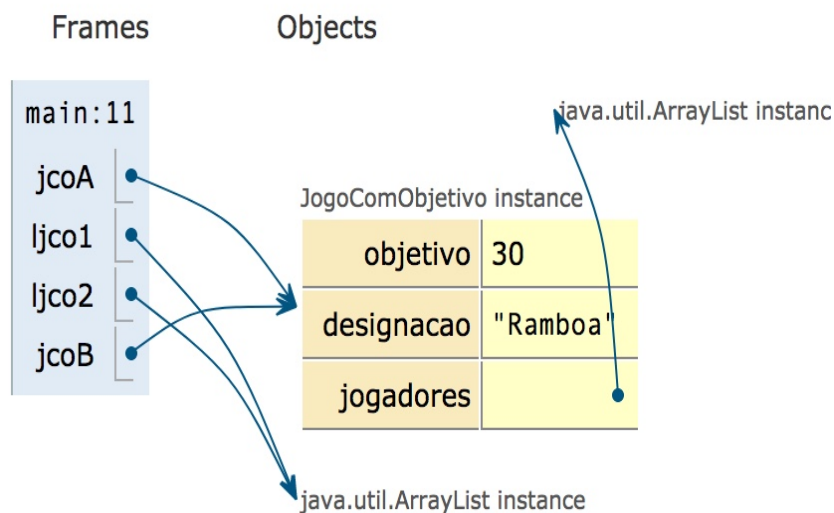
Relembre que a relação de sub-tipo é transitiva, ou seja, se `T3` é sub-tipo de `T2` e `T2` é sub-tipo de `T1`, então `T3` é sub-tipo de `T1`.

Quando trabalhamos com tipos genéricos, a relação de sub-tipo não é tão fácil de detectar pois não vai de encontro à nossa intuição imediata.

Vamos ver uma sucessão de exemplos que permita perceber as regras do compilador Java em relação a genéricos e subtipos. Considere as seguintes instruções num método `main`:

```
ArrayList<JogoComObjetivo> ljco1 = new ArrayList<JogoComObjetivo>();
ArrayList<JogoComObjetivo> ljco2 = ljco1;
JogoComObjetivo jcoA = new JogoComObjetivo("Ramboa", 30);
ljco2.add(jcoA);
JogoComObjetivo jcoB = ljco1.get(0);
```

Se executarmos estas instruções, os objetos em memória estarão organizados e ligados da seguinte forma:



Repare que, como seria de esperar, as variáveis `ljco1` e `ljco2` contêm ambas uma referência à mesma lista. Então, as duas últimas instruções adicionam um objeto a essa lista (através da variável `ljco1`) e acedem a esse mesmo objeto logo de seguida (através da variável `ljco2`). Por isso é que as variáveis `jcoA` e `jcoB` apontam ambas para o mesmo objeto do tipo `JogoComObjetivo`.

Considere agora este novo excerto em que temos uma lista `lj` do tipo `ArrayList<Jogo>`:

```
ArrayList<JogoComObjetivo> ljco1 = new ArrayList<JogoComObjetivo>();
ArrayList<Jogo> lj = ljco1;
lj.add(new JogoNumeroExatoJogadas("Covix", 8));
```

Relembre a assinatura do método `add` do interface genérico `List<E>` que a classe `ArrayList<E>` implementa – `boolean add(E element)`. Relembre também que numa invocação de um método, podemos instanciar um parâmetro de um dado tipo com um objeto de qualquer subtipo desse tipo. Então, como o tipo `JogoNumeroExatoJogadas` é subtipo de `Jogo`, e `lj` é do tipo `ArrayList<Jogo>`, a instrução `lj.add(new JogoNumeroExatoJogadas("Covix", 8));` está correta.

Suponha agora que a seguir a essas três instruções tínhamos ainda uma quarta instrução:

```
ArrayList<JogoComObjetivo> ljco1 = new ArrayList<JogoComObjetivo>();
ArrayList<Jogo> lj = ljco1;
lj.add(new JogoNumeroExatoJogadas("Covix", 8));
JogoComObjetivo jcoB = ljco1.get(0);
```

Do ponto de vista de compatibilidade de tipos, a instrução `JogoComObjetivo jcoB = ljco1.get(0);` está correta e o compilador aceitá-la-ia. No entanto, isso seria desastroso pois o objeto devolvido por `ljco1.get(0)` é verdadeiramente do tipo `JogoNumeroExatoJogadas`, o qual é incompatível com `JogoComObjetivo` para efeitos de atribuição. Estes dois tipos são ambos subtipos de `Jogo` mas não têm qualquer relação de subtipo entre eles.

Então o que é que está mal aqui?

O problema está na atribuição feita na segunda instrução – `ArrayList<Jogo> lj = ljco1;` – pois o tipo de `ljco1` não é subtipo de `ArrayList<Jogo>`, ou seja,

Apesar de `JogoComObjetivo` ser subtipo de `Jogo`,

`ArrayList<JogoComObjetivo>` NÃO é subtipo de `ArrayList<Jogo>`

Se pensarmos um pouco, nem teria sentido que fosse:

- Uma `ArrayList<Jogo>` é uma lista que pode conter objetos do tipo `Jogo` e de qualquer dos seus subtipos (como `JogoComObjetivo` e `JogoNumeroExatoJogadas`);
- Uma `ArrayList<JogoComObjetivo>` é uma lista que pode conter objetos do tipo `JogoComObjetivo` e de qualquer dos seus subtipos (para já não está nenhum definido);
- Quando fazemos uma atribuição estamos a dizer que o objeto à direita da atribuição consegue fazer tudo o que se espera do tipo da variável que está à esquerda.
 - neste caso, com a instrução `ArrayList<Jogo> lj = ljco1` estaríamos a dizer que o objeto referenciado por `ljco1` consegue fazer tudo o que uma lista de `Jogo` consegue, como por exemplo, conter objetos dos tipos `Jogo`, `JogoComObjetivo` e `JogoNumeroExatoJogadas`.
 - Mas sabemos que isso não é verdade, pois `ljco1` só pode conter objetos do tipo `JogoComObjetivo` ...

Mais à frente neste texto veremos quais são exatamente as regras a aplicar para determinar que um tipo genérico é subtipo de outro.

15.6.1. O wildcard ?

Vamos agora supor que queremos um método de nome `imprimeLista`, que recebe uma lista qualquer e escreve no *standard output* a representação textual de todos os elementos dessa lista.

A ideia é poder invocar esse método para escrever o conteúdo de várias listas de elementos de tipos diversos:

```
public static void main( String[] args) {

    ArrayList<String> l1 = new ArrayList<String>();
    l1.add("Primeiro");
    l1.add("Segundo");

    LinkedList<JogoComObjetivo> l2 = new LinkedList<JogoComObjetivo>();
    l2.add(new JogoComObjetivo("Ramboa", 30));
    l2.add(new JogoComObjetivo("Limbo", 50));

    imprimeLista(l1);
    imprimeLista(l2);
}
```



```
}
```

A classe `LinkedList`, tal como a classe `ArrayList`, é uma implementação do interface `List`, ou seja, são ambas subtipos de `List`.

Vamos então fazer uma primeira tentativa de construção do método `imprimeLista`. Como queremos tratar listas de elementos de quaisquer tipos, será que a seguinte versão serve?

```
static void imprimeLista (List<Object> lista) {  
    for( Object o: lista) {  
        System.out.println(o);  
    }  
}
```

NÃO.

De acordo com o que vimos acima, embora `String` seja subtipo de `Object`, `ArrayList<String>` não é subtipo de `List<Object>`. O mesmo para `LinkedList<JogoComObjetivo>`. Então, o `main` acima teria erros de compilação nas duas instruções de invocação do método `imprimeLista`.

Esta versão do método só pode ser invocada sobre listas de `Object`! O seguinte estaria correto:

```
public static void main( String[] args) {  
  
    ArrayList<Object> l1 = new ArrayList<Object>();  
    l1.add("Primeiro");  
    l1.add("Segundo");  
    l1.add(new JogoComObjetivo("Ramboa", 30));  
    l1.add(new JogoComObjetivo("Limbo", 50));  
  
    imprimeLista(l1);  
  
}
```

O tipo `List<Object>` representa listas contendo elementos do tipo `Object` ou de subtipos deste. Ora, as listas do tipo `List<String>` só podem conter elementos do tipo `String`.

O que nós queremos é representar listas quaisquer, seja qual for o tipo declarado para os seus elementos.

O tipo que representa qualquer lista denota-se por `List<?>`.

Neste contexto, o `?` é chamado de *wildcard*.

A versão desejada para o método `imprimeLista` é então:

```
static void imprimeLista (List<?> lista) {  
    for( Object o: lista) {  
        System.out.println(o);  
    }  
}
```

A utilização do tipo `Object` no ciclo *for-each* tem todo o sentido: qualquer que seja o tipo dos elementos da lista que instancia o parâmetro `lista`, ele será sempre um subtipo de `Object`, pois todos os tipos Java são subtipos de `Object`.

Esta nova versão do método já pode ser invocado dando como parâmetro uma lista de elementos declarados com qualquer tipo.

```
public static void main( String[] args) {

    ArrayList<String> l1 = new ArrayList<String>();
    l1.add("Primeiro");
    l1.add("Segundo");

    LinkedList<JogoComObjetivo> l2 = new LinkedList<JogoComObjetivo>();
    l2.add(new JogoComObjetivo("Ramboa", 30));
    l2.add(new JogoComObjetivo("Limbo", 50));

    ArrayList<Object> l3 = new ArrayList<Object>();
    l3.add("Outra");
    l3.add(new JogoComObjetivo("X-Ray", 600));
    l3.add(new Jogador("Pedro"));

    imprimeLista(l1);
    imprimeLista(l2);
    imprimeLista(l3);

}
```

Como se torna óbvio, a instrução `List<?> l = new ArrayList<String>();` é perfeitamente correta também.

No seguinte exemplo, verificamos que é possível fazer `get` numa lista da qual desconhecemos o tipo dos elementos:

```
static Object primeiroElementoOuNull (List<?> lista) {
    if(!lista.isEmpty()){
        return lista.get(0);
    } else {
        return null;
    }
}
```

No entanto, não podemos adicionar nada a uma lista assim definida, ou seja, o seguinte método tem um erro de compilação na invocação do método `add`:

```
static void adicionaObjeto (List<?> lista, Object novo) {
    if(novo != null) {
        lista.add(novo);
    }
}
```

Se pensarmos bem, esta proibição tem todo o sentido:

O compilador só sabe que o parâmetro `lista` é uma lista; não conhece o tipo dos elementos dessa lista. O tipo real desses elementos só será conhecido em tempo de execução, e irá variar de invocação para invocação.

Então, podíamos estar a adicionar a `lista` um objeto cujo tipo é incompatível com o tipo real dos seus elementos. Isso daria inevitavelmente origem a um erro de execução.

Para evitar este tipo de erros, o compilador proíbe a adição de elementos a uma lista declarada com o tipo `List<?>`. Na verdade, é mais geral que isso: proíbe a invocação sobre métodos que tenham algum parâmetro do tipo dos elementos da lista.

Por exemplo, o compilador não deixa que sejam invocados os seguintes métodos sobre uma lista declarada com o tipo `List<?>`:

- `E set (int index, E element)`, que substitui o elemento na posição `index` por `element`;
- `boolean add (E e)`, que adiciona `e` no fim da lista;
- os vários métodos `add` e `addAll`

em que `E` é a variável de tipo usada na API do tipo `List<E>`. São precisamente os métodos que têm algum parâmetro do tipo `E`.

15.6.2. Bounded wildcards

Vamos agora supor que queremos um método de nome `somaDeLista`, que recebe uma lista de valores numéricos e devolve a soma dos elementos dessa lista.

A ideia é poder invocar esse método para obter a soma de várias listas valores numéricos:

```
public static void main( String[] args) {

    List<Integer> l1 = new ArrayList<Integer>();
    ... // adicionar valores a l1

    LinkedList<Double> l2 = new LinkedList<Double>();
    ... // adicionar valores a l2

    List<Number> l3 = new ArrayList<Number>();
    ... // adicionar valores a l3

    System.out.println(somaDeLista(l1));
    System.out.println(somaDeLista(l2));
    System.out.println(somaDeLista(l3));
}
```

Relembre que `Integer` e `Double` são as classes *wrapper* dos tipos primitivos `int` e `double`, respetivamente. `Number` é uma classe abstrata que é superclasse de `Integer`, `Double`, `Float`, etc.

Vamos então fazer uma primeira tentativa de construção do método `somaDeLista`. Como queremos tratar listas de elementos de quaisquer subtipos de `Number`, será que a seguinte versão serve?

```
static double somaDeLista(List<Number> lista) {
    double soma = 0.0;
    for( Number n: lista) {
        soma += n.doubleValue();
    }
    return soma;
}
```

Nesta fase já deve adivinhar a resposta: **NÃO**.

As duas primeiras invocações de `somaDeLista` no `main` acima provocam erro de compilação. Já sabemos porquê: `List<Integer>` e `LinkedList<Double>` não são subtipos de `List<Number>` (embora `Integer` e `Double` sejam subtipos de `Number`).

Segunda tentativa. Vamos generalizar o tipo dos elementos da lista parâmetro usando o *wildcard* `?`. Será que a seguinte versão já é o que queremos?

```
static double somaDeLista(List<?> lista) {
    double soma = 0.0;
    for( Number n: lista) {
        soma += n.doubleValue();
    }
    return soma;
}
```

Com esta versão, as instruções de invocação do `main` já não têm erros de compilação, pois `List<Integer>` e `LinkedList<Double>` são subtipos de `List<?>`.

Mas... a própria função `somaDeLista` tem um erro na instrução `for( Number n: lista) {`.

Tem sentido que isso aconteça! O compilador não sabe qual é o tipo dos elementos de `lista`... por isso não pode aceitar que sejam usados como `Number`. Na verdade, de acordo com o tipo do parâmetro, esta função pode receber uma lista de elementos de qualquer tipo aquando da invocação.

Para que o compilador aceite que os elementos de lista sejam encarados como `Number`, há que garantir que, embora o seu tipo seja desconhecido, ele é de certeza algum subtipo de `Number`.

Usamos *bounded wildcards* para representar um qualquer subtipo de um tipo dado.

O bounded wildcard `? extends T` representa todos os tipos de dados que sejam subtipos de `T` (que, como já sabe, incluem `T`).

Por exemplo, `List<? extends Number>` representa qualquer lista de elementos cujo tipo seja subtipo de `Number`.

Isto leva-nos à nossa terceira, e última, tentativa. Generalizamos o tipo dos elementos da lista parâmetro, ao mesmo tempo que limitamos a um sub-conjunto de todos os tipos existentes:

```
static double somaDeLista(List<? extends Number> lista) {
    double soma = 0.0;
    for( Number n: lista) {
        soma += n.doubleValue();
    }
    return soma;
}
```

Desta forma, o compilador, embora não saiba qual o tipo exato dos elementos de `lista`, tem a certeza que é um subtipo de `Number` e que, por isso, pode usar o tipo `Number` na instrução *for-each* e invocar sobre cada elemento o método `doubleValue` definido na classe `Number`.

Agora finalmente o método `main` acima já está livre de erros de compilação, bem como o próprio método `somaDeLista`.

Tal como anteriormente, não podemos adicionar elementos a uma lista definida com *bounded wildcards* pois não há forma de o compilador saber qual o tipo real dos elementos da lista parâmetro (só em tempo de execução).

Repare que `List<?>` é o mesmo que `List<? extends Object>` já que todos os tipos Java são subtipos de `Object`, incluindo o próprio `Object`.

Além da relação `extends`, podemos usar a relação `super` para definição de `bounded wildcards`.

O bounded wildcard `? super T` representa todos os tipos de dados que sejam supertipos de `T` (que, como já sabe, incluem `T`).

Por exemplo, `List<? super Integer>` representa qualquer lista de elementos cujo tipo seja supertipo de `Integer` como, por exemplo, listas de `Integer`, listas de `Number` e listas de `Object`.

Suponha que quer criar um método para copiar valores numéricos de uma lista para outra, seja qual for o seu tipo (`Integer`, `Double`, `Float` etc).

- O parâmetro que representa a lista origem será então do tipo `List<? extends Number>`
- Qual o tipo para o parâmetro que representa a lista destino? Este deverá ser tal que possa receber, sem qualquer problema, os valores da lista origem. Vamos pôr várias hipóteses e verificar que apenas uma serve o nosso objetivo:
 - `List<Object>` : desta forma, estaríamos a limitar a utilização do método a listas destino cujos elementos fossem exatamente do tipo `Object` (por exemplo `List<Object>`, `ArrayList<Object>`, etc) que não é o que queremos;
 - `List<?>` : este não serve porque haveria o perigo de ser instanciado com uma lista de elementos de tipo incompatível com numéricos (por exemplo, `List<Jogo>`, `List<String>`, etc);
 - `List<Number>` : aqui o problema é o mesmo que na primeira opção – o parâmetro só poderia ser instanciado por listas cujos elementos fossem exatamente do tipo `Number`;
 - `List<? extends Number>` : com esta hipótese, a seguinte situação indesejável seria possível do ponto de vista dos tipos: a lista origem ser instanciada por uma lista de `Double` e a lista destino por uma lista de `Integer`, o que claramente é errado;
 - `List<? super Number>` : Finalmente, esta hipótese é a única correta pois qualquer lista cujos elementos sejam de um supertipo de `Number` (que inclui `Number`) pode receber os elementos de qualquer lista cujos elementos sejam de um subtipo de `Number` (que inclui `Number`).

Obtemos então o seguinte método, do uso muito geral como desejávamos, que faz uso dos dois tipos de *bounded wildcards*:

```
static void copiarLista( List<? extends Number> origem,
                       List<? super Number> destino) {

    destino.clear();
    for (Number elem: origem){
        destino.add(elem);
    }
}
```

Verifique que é possível adicionar elementos do tipo `Number` a uma lista cujos elementos são de um supertipo de `Number`.

15.6.3. Regras para determinar se um tipo genérico é subtipo de outro

Várias classes genéricas muito úteis da biblioteca do Java definem métodos onde são usados *wildcards* e *bounded wildcards*. Para os podermos usar nos nossos programas, é muito importante que saibamos invocá-los e, por isso, convém saber se os valores com que queremos instanciar os seus parâmetros são legais do ponto de vista de concordância de tipos.

Para sabermos se um tipo genérico é subtipo de outro, aplicamos as seguintes regras:

`A1<T1>` é subtipo de `A2<T2>` se se verificar:

- `A1` é subtipo de `A2` e
- uma das seguintes, consoante a forma de `T2`:
 - `T1 = T2` (se `T2` não contém *bounded wildcards*)
 - `T1` é subtipo de `U` (se `T2` é da forma `? extends U`)
 - `U` é subtipo de `T1` (se `T2` é da forma `? super U`)

Apliquemos agora estas regras a alguns casos de exemplo, como exercício.

- `ArrayList<String>` é subtipo de `List<String>` porque
 - `ArrayList` é subtipo de `List` e
 - `String = String`
- `ArrayList<Integer>` não é subtipo de `List<Number>` porque
 - embora `ArrayList` seja subtipo de `List`,
 - `Integer` é diferente de `Number`
- `ArrayList<JogoComObjetivo>` é subtipo de `List<? extends Jogo>` porque
 - `ArrayList` é subtipo de `List` e
 - `JogoComObjetivo` é subtipo de `Jogo`
- `List<List<Integer>>` não é subtipo de `Collection<List<Number>>` porque
 - embora `List` seja subtipo de `Collection`,
 - `List<Integer>` é diferente de `List<Number>`
- `List<List<Integer>>` não é subtipo de `Collection<? extends List<Number>>` porque
 - embora `List` seja subtipo de `Collection`,
 - `List<Integer>` não é subtipo de `List<Number>` porque
 - embora `List` seja subtipo de `List`,
 - `Integer` é diferente de `Number`
- `List<List<Integer>>` é subtipo de `Collection<? extends List<? extends Number>>` porque
 - `List` é subtipo de `Collection` e
 - `List<Integer>` é subtipo de `List<? extends Number>` porque
 - `List` é subtipo de `List` e
 - `Integer` é subtipo de `Number`

15.7. Métodos genéricos

Um tipo diz-se genérico quando a sua definição envolve uma ou mais variáveis de tipo, que são instanciadas quando o tipo é usado na declaração de variáveis e na criação de objetos.

Por exemplo, `ArrayList<E>` é um tipo genérico pois quando criamos um objeto deste tipo temos que instanciar a variável `E` (variável de tipo).

Isto permite que se use uma só definição para a representação de vários tipos – tantos quantos os valores possíveis para as variáveis de tipo.

Esta noção de genérico também se aplica a métodos individualmente – podemos definir um método que deixa em aberto um ou mais tipos através da utilização de variáveis de tipo.

Vamos então perceber a utilidade deste conceito aplicado a métodos.

Queremos um método para ver se duas listas são iguais.

1ª tentativa:

```
static boolean listasIguais(List<Object> a, List<Object> b) {

    boolean result = a.size() == b.size();
    for (int i = 0 ; i < a.size() && result ; i++) {
        if(!a.get(i).equals(b.get(i))) {
            result = false;
        }
    }
    return result;
}
```

Este método não é muito útil pois, como já deve adivinhar, exige que se instanciem os parâmetros com listas de `Object`. Das invocações no método `main` que se segue, só a terceira está correta:

```
public static void main( String[] args) {

    ArrayList<String> l1 = new ArrayList<String>();
    ... // adicionar elementos a l1

    List<String> l2 = new LinkedList<String>();
    ... // adicionar elementos a l2

    ArrayList<Object> l3 = new ArrayList<Object>();
    ... // adicionar elementos a l3
    ArrayList<Object> l4 = new ArrayList<Object>();
    ... // adicionar elementos a l4

    boolean iguaisA = listasIguais(l1, l2);
    boolean iguaisB = listasIguais(l1, l3);
    boolean iguaisC = listasIguais(l3, l4);
}
```

Gostaríamos que o método aceitasse também listas de elementos de quaisquer tipos.

2ª tentativa:

```
static boolean listasIguais(List<?> a, List<?> b) {

    boolean result = a.size() == b.size();
    for (int i = 0 ; i < a.size() && result ; i++) {
        if(!a.get(i).equals(b.get(i))) {
            result = false;
        }
    }
    return result;
}
```

Esta versão já aceita listas de elementos de qualquer tipo. Até aceita duas listas cujos elementos são de tipos incomparáveis (por exemplo, `Integer` e `Jogo`).

15.7.1. Parâmetros de tipo

Se quisermos garantir que os parâmetros `a` e `b` representam listas de elementos do mesmo tipo e que esse tipo pode ser qualquer, teremos que usar um *parâmetro de tipo*, tornando o método genérico.

Última tentativa:

```
static <T> boolean listasIguais(List<T> a, List<T> b) {

    boolean result = a.size() == b.size();
    for (int i = 0 ; i < a.size() && result ; i++) {
        if(!a.get(i).equals(b.get(i))) {
            result = false;
        }
    }
    return result;
}
```

Indicamos que o método é genérico colocando `< T >` antes do tipo de retorno do método.

Por ser um parâmetro de tipo, `T` pode representar qualquer tipo; usamo-lo nos dois parâmetros `a` e `b` para obrigar a que os elementos dessas listas tenham exatamente o mesmo tipo.

Ao invocar um método genérico, não explicitamos qual o tipo que instancia `T`. O compilador faz isso por nós.

Um método genérico pode ter um ou mais parâmetros de tipo. Um exemplo com dois parâmetros de tipo:

```
static <P,Q> Par<P,Q> emparelhaOuNull(P prim, Q seg) {

    Par<P,Q> result = null;

    if(prim != null && seg != null) {
        result = new Par<P,Q>(prim, seg);
    }
    return result;
}
```


Exemplos de invocação:

```
public static void main( String[] args) {

    List<Object> lo1 = new ArrayList<Object>();
    List<Object> lo2 = new ArrayList<Object>();
    boolean b = listasIguais(lo1,lo2); // compilador infere que T e' Object

    List<String> ls1 = new ArrayList<String>();
    List<String> ls2 = new ArrayList<String>();
    b = listasIguais(ls1,ls2);        // compilador infere que T e' String

    b = listasIguais(ls1,lo1);        // compile-time error

    List<Number> ln1 = new ArrayList<Number>();
    List<Integer> li1 = new ArrayList<Integer>();
    b = listasIguais(li1,ln1);        // compile-time error

    // compilador infere que P e' Integer e Q e' Double
    Par<Integer, Double> par = emparelhaOuNull(23, 2.3);

}
```

15.7.2. Parâmetros de tipo *bounded*

Podemos restringir os parâmetros de tipo de um método genérico a um subconjunto dos tipos Java. Fazemos isso usando parâmetros de tipo *bounded*.

Método genérico para ver se os elementos de uma lista são o dobro dos de outra:

```
static <T extends Number> boolean dobro(List<T> a, List<T> b) {
    boolean result = a.size() == b.size();
    for (int i = 0 ; i < a.size() && result ; i++){
        double ad = a.get(i).doubleValue();
        double bd = b.get(i).doubleValue();
        if(!(Math.abs(ad - 2 * bd) < 0.0001)){
            result = false;
        }
    }
    return result;
}
```

Aqui o parâmetro de tipo é *bounded* pois queremos restringir o tipo dos elementos das listas parâmetro – têm que ser números.

Neste último método, as listas `a` e `b` têm que ter exatamente o mesmo tipo de elementos. Se quisermos que possam ser tipos diferentes, embora ambos subtipos de `Number`, poderíamos fazer assim:

```

static <U extends Number, V extends Number> boolean dobro(List<U> a, List<V> b) {
    boolean result = a.size() == b.size();
    for (int i = 0 ; i < a.size() && result ; i++){
        double ad = a.get(i).doubleValue();
        double bd = b.get(i).doubleValue();
        if(!(Math.abs(ad - 2 * bd) < 0.0001)){
            result = false;
        }
    }
    return result;
}

```

15.7.3. Exemplos de métodos genéricos na biblioteca Java

No interface `Collection`, do pacote `java.util`, temos, por exemplo:

- `static void shuffle(List<?> list)` que baralha uma lista de elementos de qualquer tipo;
- `static <T extends Comparable <? super T>> sort (List<T> list)` que ordena uma lista de elementos que se conseguem comparar com outros do mesmo tipo, ou seja, que implementam o método `int compare(Q obj)` definido no interface `Comparable<Q>`. A razão para que `T` tenha que ser subtipo de `Comparable <? super T>` é para ter a certeza que implementa o método `compare` com um parâmetro do próprio tipo `T` ou acima na hierarquia. Por exemplo, para que `T` possa ser instanciado com o tipo `JogoComObjetivo`, então `JogoComObjetivo` tem que implementar o método `compare` com um parâmetro do mesmo tipo ou do tipo `Jogo`, por exemplo;
- `static <T> sort(List<T> list, Comparator<? Super T> comp)` que faz o mesmo que o anterior, mas em vez de exigir que os elementos da lista se saibam comparar com outros do mesmo tipo, usa um comparador para os comparar;
- `static <T> int binarySearch(List<? extends Comparable<? super T>> list, T key)` que faz uma pesquisa binária numa lista já ordenada;
- `static <T> T min(Collection<? extends T> coll, Comparator<? super T> comp)` que devolve o valor mínimo de uma lista, com o auxílio de um comparador.

16. Classes abstratas

Ainda se lembra do método `terminou`, na classe `Jogo`? Não lhe demos a atenção devida pois considerámos que não há nenhuma maneira comum e geral de terminar um jogo, contrariamente ao cálculo do(s) vencedor(es), em que considerámos que o mais comum era que vencesse o jogador com o maior número de pontos.

Definimos uma implementação específica para o método `terminou` em cada uma das subclasses de `Jogo` e "esquecemos" esse método na classe `Jogo`.

Implementámos assim na subclasse `JogoComObjetivo`:

```

/**
 * Este jogo jah terminou?
 */
public boolean terminou() {
    return this.maiorPontuacao() >= objetivo;
}

```

E da seguinte forma na subclasse `JogoNumeroExatoJogadas`:

```
/**
 * Este jogo jah terminou?
 */
public boolean terminou() {
    return this.jahTerminou;
}
```

Se na classe `Jogo` tivermos

```
/**
 * Este jogo jah terminou?
 */
public boolean terminou () {
    ??????
}
```

já sabemos que o compilador dá erro. Mas a verdade é que não sabemos como implementar este método na classe `Jogo` pois ao nível desta classe não há informação suficiente para decidir quando é que um jogo termina.

Vamos agora examinar o que acontece se a classe `Jogo` **não** declarar nenhum método `terminou`.

No seguinte programa, o método `terminou` é invocado sobre dois objetos cujo tipo declarado é `Jogo`, mas cujos tipos concretos são `JogoComObjetivo` e `JogoNumeroExatoJogadas`:

```
public class ExemploNaoExisteMetodo {

    public static void main(String[] args) {
        Jogo[] meusJogos = new Jogo[2];
        meusJogos[0] = new JogoComObjetivo("Spiral", 25);;
        meusJogos[1] = new JogoNumeroExatoJogadas("Harvest Kingdom",1);;
        meusJogos[0].juntarJogador("Maria");
        meusJogos[0].juntarJogador("Pedro");
        meusJogos[1].juntarJogador("John Snow");
        meusJogos[1].juntarJogador("Daenerys Targaryen");

        meusJogos[0].registarPontosJogada("Maria", 11);
        meusJogos[0].registarPontosJogada("Pedro", 26);
        meusJogos[1].registarPontosJogada("John Snow", 10);
        meusJogos[1].registarPontosJogada("Daenerys Targaryen", 10);

        for(int i = 0 ; i < meusJogos.length ; i++) {
            // metodo terminou eh invocado
            if(meusJogos[i].terminou()){
                System.out.println("Jogo " + meusJogos[i].designacao() +
                                   " terminou");
            }
        }
    }
}
```

Como o método `terminou` não está definido na classe `Jogo`, que é o tipo declarado da variável `meusJogos`, a compilação produz o seguinte erro nesta classe: *The method terminou() is undefined for the type Jogo.*

Na verdade, este erro só aconteceria se a classe `Jogo` não implementasse o interface `Ganhavel` (ver secção 14). Como a classe `Jogo` implementa o interface `Ganhavel`, então o efeito de não implementar o método `boolean terminou()` seria o erro de compilação: *The type Jogo must implement the inherited abstract method Ganhavel.terminou()*.

Na próxima secção vamos ver como o conceito de *classe abstrata* nos permite resolver este problema.

16.1. A classe abstrata `Jogo`

Queremos então:

- poder ter estruturas de dados contendo vários jogos, independentemente do tipo real de cada um, e
- poder invocar sobre eles o método `terminou`.

Para isso:

- temos que definir o método `terminou` na classe `Jogo`,
- embora não saibamos dar-lhe uma implementação.

Então, temos que:

- definir o método `terminou` como `abstract` na classe `Jogo`,
- e, por isso, definir a classe `Jogo` como `abstract`, também.

Na definição de uma classe, a palavra `abstract` imediatamente antes da palavra `class` indica que a classe é *abstrata*. Na definição de um método, a palavra `abstract` imediatamente antes do tipo de retorno do método indica que o método é abstrato, ou seja, não tem implementação nessa classe.

```
import java.util.ArrayList;
import java.util.List;

/**
 * Classe abstrata cujos objetos representam jogos
 * @author Isabel Nunes
 */
public abstract class Jogo implements Ganhavel {

    // atributos que definem este jogo:
    // Designacao
    private String designacao
    // A colecao dos jogadores
    private ArrayList<Jogador> jogadores;

    /**
     * Inicializar um jogo.
     * @param designacao A designacao do jogo.
     * @requires designacao != null
     */
    public Jogo (String designacao) {
        this.designacao = designacao;
        this.jogadores = new ArrayList<Jogador>();
    }
}
```

```
... // Faltam aqui metodos. Ver atras.

    /**
     * Este jogo jah terminou?
     */
    public abstract boolean terminou ();

... // Faltam aqui metodos. Ver atras.

}
```

Se uma classe `A` é declarada como abstrata,

- `A` não pode ter instâncias;
- `A` pode e deve ter subclasses concretas, que implementarão o que ficou por implementar em `A`;
- `A` pode definir atributos, construtores (que têm como papel inicializar esses atributos e são chamados pelos construtores das subclasses) e métodos não abstratos;
- `A` pode estender uma classe, que tanto pode ser concreta como abstrata;
- `A` pode implementar um interface.

Se um método `m` é definido como `abstract` numa classe `A`,

- o método `m` representa um serviço que todos os objetos do tipo `A` têm que oferecer (lembre que qualquer subtipo de `A` é também do tipo `A`);
- logo, todas as subclasses concretas (não abstratas) de `A` têm que implementar o método `m` porque qualquer subclasse de `A` define um subtipo de `A`.

Com a classe `Jogo` definida como abstrata e o seu método `terminou` também, o programa

`ExemploNaoExisteMetodo` acima já compila bem e a sua execução imprime o seguinte no *standard output*:

```
Jogo Spiral terminou
Jogo Harvest Kingdom terminou
```

16.2. Métodos *template*

Suponhamos agora que queremos ter um método, acessível a qualquer tipo de jogo, que devolva os nomes dos jogadores do jogo. Mas queremos que este resultado seja:

- os nomes de todos os jogadores registados no jogo, se o jogo ainda não terminou;
- os nomes dos jogadores "ativos", ou seja, aquelas com pontuação maior que zero, se o jogo já terminou.

Uma implementação deste método poderia ser:

```
/**
 * Metodo template (ou implementacao esqueleto)
 * Os nomes dos jogadores registados ou que teem pontuacao > 0
 * @return Os jogadores que registados se este jogo ainda
 *         nao terminou
 *         Os nomes dos jogadores com pontuacao > 0 se este
```

```

        *           jogo jah terminou
        */
private List<String> jogadores () {
    ArrayList<String> result = new ArrayList<String>();
    for (Jogador jog : this.jogadores) {
        if(this.terminou()) {
            if (jog.pontuacao() > 0) {
                result.add(jog.nome());
            }
        } else {
            result.add(jog.nome());
        }
    }
    return result;
}
}

```

Em que classe devemos incluir este método? Se queremos que todos os subtipos de `Jogo` tenham exatamente este método, devemos inclui-lo na classe `Jogo`, para que seja herdado por todas as suas subclasses.

Mas na instrução `if(this.terminou())` invocamos o método `terminou()` que é `abstract` na classe `Jogo`! **Não há problema:**

- como o método `terminou` está declarado na classe `Jogo` como *abstract*,
- o compilador sabe que todas as subclasses concretas de `Jogo` têm que implementar este método (se não o fizerem, terão que ser declaradas como *abstract*);
- então, é perfeitamente natural invocar `terminou()` dentro de um método da classe `Jogo`.

Diz-se que este é um *método template* ou *implementação esqueleto*